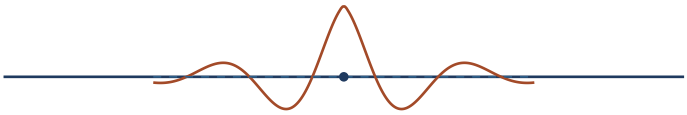
Mathematik von Grund auf — Band 5

Die Kunst der Näherung

Eine vollständige Einführung in die Numerische Mathematik



Von der Rundung bis zum Mehrgitterverfahren

Jeder Schritt erklärt. Mit vollständig durchgerechneten Übungsaufgaben.

Stand der Forschung: August 2026

Mathematik von Grund auf — Band 5
Die Kunst der Näherung · Numerische Mathematik

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Simulationen, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/mathematik-von-grund-auf`
Dauerhaft archiviert bei Zenodo: `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Mathematik von Grund auf, Band 5: Die Kunst der Näherung*. Fassung 1.0.1.
`https://doi.org/10.5281/zenodo.23698547`

Satz: LaTeX mit Palatino. Fassung 1.0.1, Oktober 2026. 224 Seiten.

Vorwort: Wenn Mathematik auf Maschinen trifft

In den vier vorangegangenen Bänden ging es um Wahrheit. Ein Satz war bewiesen oder nicht; eine Zahl war gleich einer anderen oder nicht; ein Grenzwert existierte oder nicht.

In diesem Band geht es um etwas anderes: um *Genauigkeit*, *Aufwand* und *Vertrauen*.

Die Numerische Mathematik beantwortet die Frage, die in allen vorigen Bänden offen blieb: *Wie rechnet man das aus?* Nicht im Prinzip, sondern tatsächlich – auf einer Maschine, in endlicher Zeit, mit endlich vielen Ziffern.

Ein Beispiel gleich zu Beginn

Berechnen Sie

$$x = \frac{1}{3}, \qquad y = 3x, \qquad z = y - 1.$$

Mathematisch ist $z = 0$. Auf jedem Taschenrechner und in jeder Programmiersprache ist z meist ebenfalls 0 – aber nicht immer, und wenn nicht, dann aus einem Grund, den man verstehen muss.

Härter: Berechnen Sie

$$f(x) = \frac{1 - \cos x}{x^2} \qquad \text{für } x = 10^{-8}.$$

Der wahre Wert ist etwa 0,5. Ein Rechner mit doppelter Genauigkeit liefert typischerweise 0. Nicht 0,4999, nicht 0,51 – **glatt null**, ein Fehler von 100 %.

Der Rechner ist nicht defekt. Die Formel ist es. Warum, und wie man es besser macht, steht in Kapitel 1.

Was dieses Buch will

Sie sollen drei Dinge lernen.

Erstens: Algorithmen. Wie löst man ein lineares Gleichungssystem mit einer Million Unbekannten? Wie findet man Nullstellen, wie integriert man, wie löst man eine Differentialgleichung? Für jedes Problem gibt es Verfahren, und die guten unterscheiden sich von den naheliegenden oft dramatisch.

Zweitens: Fehleranalyse. Jedes numerische Ergebnis ist falsch. Die Frage ist nur, *wie* falsch – und ob man es weiß. Der Unterschied zwischen einem brauchbaren und einem unbrauchbaren Verfahren liegt fast nie in der Formel, sondern in seinem Verhalten gegenüber Rundungsfehlern.

Drittens: Aufwandsdenken. Ein Verfahren, das $\mathcal{O}(n^3)$ Operationen braucht, ist für $n = 1000$ machbar und für $n = 10^6$ hoffnungslos. Numerik ist die Disziplin, in der Mathematiker über Laufzeiten nachdenken müssen.

> **Die drei Illusionen**
>
> Wer aus der reinen Mathematik kommt, bringt drei falsche Erwartungen mit:
> **Illusion 1: „Der Rechner rechnet genau."** Er rechnet mit etwa 16 Dezimalstellen und rundet nach jeder einzelnen Operation. Bei 10^{12} Operationen – eine Sekunde moderner Rechenleistung – können sich die Fehler aufschaukeln.
> **Illusion 2: „Wenn die Formel stimmt, stimmt das Programm."** Zwei mathematisch identische Formeln können auf dem Rechner völlig verschiedene Ergebnisse liefern. Die Umformung
> $$\sqrt{x+1} - \sqrt{x} = \frac{1}{\sqrt{x+1} + \sqrt{x}}$$
> ist in der Analysis eine Trivialität und in der Numerik der Unterschied zwischen Unsinn und Präzision.
> **Illusion 3: „Konvergenz genügt."** Ein Verfahren, das nach 10^{20} Schritten konvergiert, konvergiert nicht. Die *Geschwindigkeit* ist keine Nebensache, sondern der Kern der Sache.

Die drei Regeln, wie in den vorigen Bänden

Erstens: Es gibt kein „man sieht leicht". Jede Fehlerabschätzung wird vollständig hergeleitet, jeder Algorithmus Schritt für Schritt an einem Zahlenbeispiel vorgeführt.

Zweitens: Motivation vor Definition. Warum pivotisiert man? Weil ich Ihnen vorher ein 2×2-System zeige, bei dem die naive Gauß-Elimination ein Ergebnis mit 100 % Fehler liefert. Warum braucht man Tschebyschow-Knoten? Weil ich Ihnen vorher das Runge-Phänomen mit Zahlen zeige.

Drittens: Aufgaben mit vollständigen Lösungen – mit ausgeschriebener Arithmetik, nicht mit „man erhält".

Zum Aufbau

- **Teil I** (Kap. 1–2): Gleitkommazahlen, IEEE 754, Auslöschung; Kondition eines Problems und Stabilität eines Algorithmus. Dieser Teil ist das Fundament – wer ihn überspringt, versteht den Rest nicht.
- **Teil II** (Kap. 3–6): Numerische lineare Algebra. LR- und Cholesky-Zerlegung, QR und Ausgleichsrechnung, iterative Verfahren bis zum CG-Verfahren, Eigenwerte und die Singulärwertzerlegung.
- **Teil III** (Kap. 7): Nichtlineare Gleichungen – Bisektion, Newton, Konvergenzordnung, das mehrdimensionale Newton-Verfahren.

- **Teil IV** (Kap. 8–10): Interpolation und Approximation. Polynome, das Runge-Phänomen, Splines, trigonometrische Interpolation und die schnelle Fourier-Transformation.
- **Teil V** (Kap. 11): Numerische Integration und Differentiation – von der Trapezregel bis zur Gauß-Quadratur.
- **Teil VI** (Kap. 12–13): Differentialgleichungen. Runge-Kutta-Verfahren, Schrittweitensteuerung, Steifheit, Randwertprobleme, Finite Differenzen und Finite Elemente, Mehrgitterverfahren.
- **Teil VII** (Kap. 14–15): Optimierung und Monte-Carlo (hier trifft man Band 4 wieder), dann der aktuelle Forschungsstand – gemischte Präzision, randomisierte lineare Algebra, wissenschaftliches maschinelles Lernen, probabilistische und verifizierte Numerik. Stand August 2026.

Ein Fach, das älter ist als die Rechner

Man hält Numerik gern für ein Kind des Computerzeitalters. Das ist falsch.
Die Babylonier berechneten um 1800 v. Chr. Quadratwurzeln mit einem Iterationsverfahren, das identisch ist mit dem Newton-Verfahren, angewandt auf $x^2 - a$. Die Tontafel YBC 7289 gibt $\sqrt{2}$ auf sechs Dezimalstellen genau an – ein Wert, der erst mit dem Taschenrechner wieder alltäglich wurde.
ARCHIMEDES bestimmte um 250 v. Chr. die Kreiszahl durch ein- und umbeschriebene Vielecke und erhielt $\frac{223}{71} < \pi < \frac{22}{7}$ – eine Intervalleinschließung, also verifizierte Numerik im heutigen Sinn.
NEWTON entwickelte sein Verfahren um 1669, EULER 1768 sein Polygonzugverfahren für Differentialgleichungen, GAUSS 1809 die Methode der kleinsten Quadrate – zur Bahnbestimmung des Kleinplaneten Ceres, mit Papier und Feder. GAUSS erfand 1805 auch die schnelle Fourier-Transformation, notierte sie in einem Heft und veröffentlichte sie nie; sie wurde 1965 von COOLEY und TUKEY wiederentdeckt und ist heute vermutlich der meistausgeführte Algorithmus der Welt.
Bis in die 1940er Jahre war „Computer“ eine *Berufsbezeichnung*: Menschen, meist Frauen, die in großen Räumen an Tischrechnern Zahlenkolonnen abarbeiteten. Die Numerische Mathematik war die Wissenschaft davon, wie man ihnen die Arbeit einteilt.
Als die Maschinen kamen, änderte sich nicht die Mathematik, sondern der Maßstab. JAMES WILKINSON, der bei der Konstruktion des britischen ACE mitarbeitete, entwickelte in den 1950er und 60er Jahren die Rückwärtsfehleranalyse – die vielleicht wichtigste begriffliche Erfindung des Fachs. Wir lernen sie in Kapitel 2.

Der Zweck des Rechnens ist Einsicht, nicht Zahlen.
— Richard W. Hamming, 1962

(Hamming fügte später hinzu, manchmal sei der Zweck des Rechnens doch die Zahl – etwa wenn ein Flugzeug damit fliegen soll.)

Inhaltsverzeichnis

Vorwort **iii**

I Rechnen mit endlicher Genauigkeit **1**

1 Zahlen im Rechner **3**
1.1 Warum es überhaupt ein Problem gibt 3
1.2 Das Gleitkommaformat 3
1.3 Der Standard IEEE 754 4
1.4 Runden und Maschinengenauigkeit 6
1.5 Sonderwerte: Null, Unendlich und NaN 8
1.6 Auslöschung – der gefährlichste Effekt 9
1.7 Summation: ein unterschätztes Problem 12

2 Kondition und Stabilität **17**
2.1 Zwei verschiedene Fragen 17
2.2 Die Kondition eines Problems 17
2.3 Die Kondition linearer Gleichungssysteme 20
2.4 Stabilität: Vorwärts- und Rückwärtsfehler 24

II Numerische lineare Algebra **29**

3 Lineare Gleichungssysteme und die LR-Zerlegung **31**
3.1 Warum das das wichtigste Problem der Numerik ist 31
3.2 Dreieckssysteme – der einfache Fall 32
3.3 Die Gauß-Elimination als Matrixzerlegung 32
3.4 Pivotisierung: warum man tauschen muss 36
3.5 Cholesky-Zerlegung 39

4 QR-Zerlegung und Ausgleichsrechnung **43**
4.1 Das Problem der kleinsten Quadrate 43
4.2 Orthogonale Matrizen und warum sie ideal sind 45
4.3 Householder-Spiegelungen 47

5 Iterative Verfahren für große Systeme **55**
5.1 Warum direkte Verfahren irgendwann versagen 55
5.2 Splitting-Verfahren: Jacobi und Gauß–Seidel 56

5.3 Das Verfahren der konjugierten Gradienten 61

6 Eigenwerte und Singulärwerte 67
6.1 Warum man das charakteristische Polynom nicht benutzt 67
6.2 Die Potenzmethode 67
6.3 Der QR-Algorithmus 71
6.4 Die Singulärwertzerlegung 73

III Nichtlineare Gleichungen 79

7 Nullstellen und Fixpunkte 81
7.1 Das Problem und die Konvergenzordnung 81
7.2 Bisektion – langsam, aber unfehlbar 82
7.3 Das Newton-Verfahren 83
7.4 Verfahren ohne Ableitung 87
7.5 Newton im Mehrdimensionalen 91

IV Interpolation und Approximation 95

8 Polynominterpolation 97
8.1 Das Problem 97
8.2 Der Interpolationsfehler 100
8.3 Das Runge-Phänomen 101
8.4 Tschebyschow-Knoten 102

9 Splines und trigonometrische Interpolation 109
9.1 Die Idee der stückweisen Polynome 109
9.2 Trigonometrische Interpolation und die FFT 114

10 Approximation 119
10.1 Drei Arten, „nahe“ zu sagen 119
10.2 Approximation in der Quadratmittelnorm 119
10.3 Beste gleichmäßige Approximation 123

V Integration und Differentiation 127

11 Numerische Quadratur 129
11.1 Warum man numerisch integriert 129
11.2 Newton–Cotes-Formeln 129
11.3 Romberg-Extrapolation 134
11.4 Gauß-Quadratur 137
11.5 Numerische Differentiation 141

VI Differentialgleichungen **145**

12 Anfangswertprobleme **147**
12.1 Das Problem . . . 147
12.2 Das Euler-Verfahren . . . 148
12.3 Runge–Kutta-Verfahren . . . 149
12.4 Schrittweitensteuerung . . . 153
12.5 Steife Differentialgleichungen . . . 154

13 Randwertprobleme und partielle Differentialgleichungen **159**
13.1 Der Unterschied zum Anfangswertproblem . . . 159
13.2 Finite Differenzen . . . 159
13.3 Die Poisson-Gleichung in zwei Dimensionen . . . 163
13.4 Mehrgitterverfahren . . . 165
13.5 Finite Elemente . . . 168

VII Optimierung und Zufall **173**

14 Numerische Optimierung und Monte-Carlo **175**
14.1 Das Optimierungsproblem . . . 175
14.2 Abstiegsverfahren . . . 176
14.3 Monte-Carlo-Verfahren . . . 181

VIIIAktueller Stand **187**

15 Numerik heute **189**
15.1 Gemischte Präzision . . . 189
15.2 Randomisierte numerische lineare Algebra . . . 191
15.3 Automatisches Differenzieren . . . 192
15.4 Wissenschaftliches maschinelles Lernen . . . 194
15.5 Probabilistische und verifizierte Numerik . . . 195
15.6 Reproduzierbarkeit . . . 196
15.7 Was offen ist . . . 197

IX Anhänge **199**

A Formelsammlung **201**

B Verfahren im Überblick **205**

C Zeitleiste der Numerischen Mathematik **207**

D Glossar **209**

E Kommentierte Literatur **211**

Part I

Rechnen mit endlicher Genauigkeit

1 Zahlen im Rechner

1.1 Warum es überhaupt ein Problem gibt

Die reellen Zahlen sind überabzählbar (Band 3, Kapitel 1). Ein Rechner mit 64 Bit Speicher pro Zahl kann höchstens 2^{64} verschiedene Zahlen darstellen – eine endliche Menge.

> **Die Grundtatsache**
>
> **Fast jede reelle Zahl ist im Rechner nicht darstellbar.**
> Die darstellbaren Zahlen bilden eine endliche Teilmenge $\mathbb{F} \subset \mathbb{R}$. Jede Rechnung muss ihr Ergebnis auf diese Menge zurückrunden. Das ist keine Unvollkommenheit heutiger Technik, sondern eine logische Notwendigkeit für jede endliche Maschine.
> **Die zentrale Frage der Numerik lautet daher nicht:** „Wie vermeide ich Fehler?“ – das geht nicht –, **sondern:** „Wie sorge ich dafür, dass die unvermeidlichen Fehler klein bleiben und sich nicht aufschaukeln?“

1.2 Das Gleitkommaformat

> **Die Idee: wissenschaftliche Notation im Binärsystem**
>
> Wir schreiben Zahlen wie im Physikunterricht:
>
> $$6{,}022 \cdot 10^{23}, \qquad -1{,}602 \cdot 10^{-19}.$$
>
> Man merkt sich getrennt: das *Vorzeichen*, die *Ziffernfolge* (Mantisse) und die *Größenordnung* (Exponent).
> Der Rechner macht dasselbe zur Basis 2:
>
> $$x = (-1)^s \cdot m \cdot 2^e.$$

Definition 1.1 (Gleitkommazahlen). Sei $\beta \geq 2$ die **Basis**, $t \in \mathbb{N}$ die **Mantissenlänge** und $e_{\min} \leq e \leq e_{\max}$ der Exponentenbereich. Die Menge der **normalisierten Gleitkommazahlen** ist

$$\mathbb{F} = \left\{ (-1)^s \cdot \left(d_0 + \frac{d_1}{\beta} + \cdots + \frac{d_{t-1}}{\beta^{t-1}} \right) \cdot \beta^e \right\}$$

mit $s \in \{0, 1\}$, $d_i \in \{0, \ldots, \beta - 1\}$ und $d_0 \neq 0$ (Normalisierung), zusammen mit der Zahl 0.

Warum Normalisierung?

Ohne die Bedingung $d_0 \neq 0$ hätte dieselbe Zahl mehrere Darstellungen:

$$1{,}5 \cdot 10^0 = 0{,}15 \cdot 10^1 = 0{,}015 \cdot 10^2 = \ldots$$

Die Normalisierung macht die Darstellung *eindeutig* und nutzt außerdem alle Mantissenstellen für signifikante Ziffern aus.
Ein Geschenk der Basis 2: Ist $\beta = 2$ und $d_0 \neq 0$, so muss $d_0 = 1$ sein – es gibt keine andere Möglichkeit. Man braucht die führende Ziffer also gar nicht zu speichern. Dieses **implizite Bit** liefert eine Mantissenstelle geschenkt.

1.3 Der Standard IEEE 754

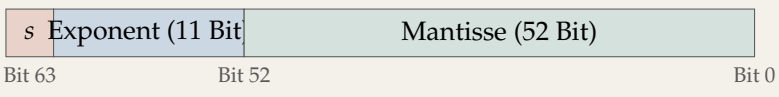
Bis 1985 hatte jeder Hersteller sein eigenes Zahlenformat. Dieselbe Rechnung lieferte auf einer CDC-Maschine ein anderes Ergebnis als auf einer IBM; portable numerische Software war praktisch unmöglich.

WILLIAM KAHAN von der Universität Berkeley trieb daraufhin einen Industriestandard voran, der 1985 als **IEEE 754** verabschiedet wurde[17]; Kahan erhielt dafür 1989 den Turing-Preis. Die aktuelle Fassung stammt von 2019.

Die beiden wichtigsten Formate

	single (32 Bit)	**double** (64 Bit)
Vorzeichen	1 Bit	1 Bit
Exponent	8 Bit	11 Bit
Mantisse (gespeichert)	23 Bit	52 Bit
Mantisse (effektiv)	24 Bit	53 Bit
Dezimalstellen (etwa)	7	16
Kleinste positive Zahl	$\approx 1{,}2 \cdot 10^{-38}$	$\approx 2{,}2 \cdot 10^{-308}$
Größte Zahl	$\approx 3{,}4 \cdot 10^{38}$	$\approx 1{,}8 \cdot 10^{308}$
Maschinengenauigkeit ε_M	$\approx 1{,}19 \cdot 10^{-7}$	$\approx 2{,}22 \cdot 10^{-16}$

Wie die 64 Bit aufgeteilt werden



Der Exponent wird mit einem **Bias** von 1023 gespeichert: Steht dort die Bitfolge für 1030, so ist der tatsächliche Exponent $1030 - 1023 = 7$. So kommt man ohne Vorzeichenbit für den Exponenten aus, und der Größenvergleich zweier positiver Gleitkommazahlen ist derselbe wie der Vergleich ihrer Bitmuster als ganze Zahlen – ein Trick, den Sortieralgorithmen ausnutzen.

Aufgabe 1.1. Stellen Sie die Zahl $x = -6{,}25$ im Format `double` dar.

Lösung Schritt für Schritt.

Schritt 1 – Vorzeichen. $x < 0$, also $s = 1$.
Schritt 2 – Betrag ins Binärsystem.
Ganzzahliger Teil: $6 = 4 + 2 = 2^2 + 2^1 = (110)_2$.
Nachkommateil: $0{,}25 = \frac{1}{4} = 2^{-2} = (0{,}01)_2$.
Zusammen:

$$6{,}25 = (110{,}01)_2.$$

Kontrolle: $(110{,}01)_2 = 4 + 2 + 0 + 0{,}25 = 6{,}25$. ✓
Schritt 3 – Normalisieren. Das Komma wandert nach links, bis genau eine Eins davorsteht:

$$(110{,}01)_2 = (1{,}1001)_2 \cdot 2^2.$$

Das Komma wanderte um 2 Stellen, also $e = 2$.
Schritt 4 – Mantisse. Die führende 1 wird nicht gespeichert (implizites Bit). Gespeichert wird

$$1001\,\underbrace{00\ldots0}_{48\text{ Nullen}}.$$

Schritt 5 – Exponent mit Bias.

$$e + 1023 = 2 + 1023 = 1025 = (10000000001)_2.$$

Kontrolle: $(10000000001)_2 = 2^{10} + 2^0 = 1024 + 1 = 1025$. ✓
Schritt 6 – Zusammensetzen.

$$\underbrace{1}_{s}\ \underbrace{10000000001}_{\text{Exponent}}\ \underbrace{1001000\ldots0}_{\text{Mantisse}}$$

In Hexadezimalschreibweise: `C019 0000 0000 0000`.
Bemerkung. Diese Zahl ist *exakt* darstellbar, weil 6,25 eine endliche Binärentwicklung hat. Das ist der Ausnahmefall.

Warum 0,1 nicht exakt darstellbar ist

Im Dezimalsystem ist $\frac{1}{3} = 0{,}333\ldots$ nicht endlich darstellbar, weil 3 kein Teiler von 10 ist.

Im Binärsystem ist eine Zahl genau dann endlich darstellbar, wenn ihr Nenner (vollständig gekürzt) eine Zweierpotenz ist. Und

$$0{,}1 = \frac{1}{10} = \frac{1}{2 \cdot 5}$$

hat den Primfaktor 5 im Nenner. Folglich:

$$(0{,}1)_{10} = (0{,}\overline{0011})_2 = 0{,}00011001100110011\ldots$$

– periodisch, also unendlich.
Die Konsequenz, die jeder Programmierer kennt:

$$0{,}1 + 0{,}2 \neq 0{,}3$$

In doppelter Genauigkeit ist

$$0{,}1 + 0{,}2 = 0{,}30000000000000004440892098500626 16\ldots,$$

und der Vergleich mit 0,3 schlägt fehl.
Die Regel: Vergleichen Sie Gleitkommazahlen nie mit ==. Prüfen Sie stattdessen

$$|a - b| \leq \tau \cdot \max(|a|, |b|, 1)$$

mit einer Toleranz τ in der Größenordnung einiger ε_{M}.
Und ein warnendes Beispiel aus der Praxis: An der Vancouver Stock Exchange lief 1982 ein Index mit Startwert 1000, der bei jeder Transaktion neu berechnet und dabei stets *abgeschnitten* statt gerundet wurde. Nach 22 Monaten stand er bei 524, während der korrekt gerechnete Wert bei 1098 lag. Der Fehler betrug also mehr als das Doppelte – allein durch systematisches Abrunden.

1.4 Runden und Maschinengenauigkeit

Definition 1.2. Die **Rundungsabbildung** $\mathrm{fl} : \mathbb{R} \to \mathbb{F}$ ordnet jeder reellen Zahl (im darstellbaren Bereich) die nächstgelegene Gleitkommazahl zu. Bei Gleichstand wird nach IEEE 754 zu derjenigen mit gerader letzter Mantissenziffer gerundet (**round half to even**).

Warum „round half to even“ und nicht „kaufmännisch“?

Beim kaufmännischen Runden wird bei genau 0,5 immer aufgerundet. Das erzeugt einen *systematischen* Aufwärtsdrift: Über viele Rechnungen summiert sich ein Bias. Beim Runden zur geraden Ziffer wird in der Hälfte der Gleichstände auf, in der anderen abgerundet. Der Bias verschwindet. Genau das ist im Fall Vancouver schiefgegangen.

Satz 1.3 (Rundungsfehlerabschätzung). *Für jedes x im normalisierten Bereich gilt*

$$\mathrm{fl}(x) = x(1+\delta) \qquad \textit{mit} \qquad |\delta| \le \varepsilon_{\mathrm{M}},$$

wobei

$$\varepsilon_{\mathrm{M}} := \frac{1}{2}\beta^{1-t}$$

*die **Maschinengenauigkeit** heißt.*

Beweis. **Schritt 1 – Der Abstand benachbarter Gleitkommazahlen.** Sei x normalisiert mit Exponent e, also $\beta^e \le |x| < \beta^{e+1}$. Die Mantisse hat t Stellen; die letzte Stelle hat den Wert β^{e-t+1}. Der Abstand zweier benachbarter Gleitkommazahlen in diesem Bereich ist also genau

$$\Delta = \beta^{e-t+1}.$$

Schritt 2 – Der Rundungsfehler. Beim Runden zur nächstgelegenen Zahl ist der absolute Fehler höchstens die Hälfte des Abstands:

$$|\mathrm{fl}(x) - x| \le \frac{\Delta}{2} = \frac{1}{2}\beta^{e-t+1}.$$

Schritt 3 – Relativ machen. Wegen $|x| \ge \beta^e$:

$$\frac{|\mathrm{fl}(x) - x|}{|x|} \le \frac{\frac{1}{2}\beta^{e-t+1}}{\beta^e} = \frac{1}{2}\beta^{1-t} = \varepsilon_{\mathrm{M}}.$$

Schritt 4 – Umschreiben. Setze

$$\delta := \frac{\mathrm{fl}(x) - x}{x}.$$

Dann ist $|\delta| \le \varepsilon_{\mathrm{M}}$ nach Schritt 3, und Auflösen nach $\mathrm{fl}(x)$ ergibt $\mathrm{fl}(x) = x(1+\delta)$. ■

Was ε_{M} anschaulich bedeutet

ε_{M} ist die kleinste Zahl, für die der Rechner $1 + \varepsilon_{\mathrm{M}}$ noch von 1 unterscheidet – genauer: Es ist der halbe Abstand von 1 zur nächstgrößeren Gleitkommazahl.
Für `double`: $t = 53$, $\beta = 2$, also

$$\varepsilon_{\mathrm{M}} = \frac{1}{2} \cdot 2^{-52} = 2^{-53} \approx 1{,}11 \cdot 10^{-16}.$$

(Manche Quellen definieren $\varepsilon_{\mathrm{M}} = 2^{-52} \approx 2{,}22 \cdot 10^{-16}$ als vollen Abstand. Beide Konventionen sind gebräuchlich; wichtig ist nur, konsistent zu bleiben.)
Merkregel: `double` hat etwa 16 signifikante Dezimalstellen, `single` etwa 7.
Ein wichtiger Punkt: Die Schranke gilt *relativ*, nicht absolut. Bei $x \approx 10^{20}$ beträgt der absolute Rundungsfehler etwa 10^4 – Gleitkommazahlen sind bei großen Werten sehr grob. Bei $x \approx 10^{-20}$ ist er etwa 10^{-36}. Die Gleitkommazahlen liegen logarithmisch verteilt.

Das Standardmodell der Gleitkommaarithmetik

IEEE 754 verlangt, dass jede der Grundoperationen $+, -, \cdot, /$ und $\sqrt{}$ *exakt gerundet* ist: Das Ergebnis ist die korrekt gerundete Fassung des exakten Resultats. Formal, mit $\circ \in \{+, -, \cdot, /\}$:

$$\mathrm{fl}(a \circ b) = (a \circ b)(1 + \delta), \qquad |\delta| \leq \varepsilon_{\mathrm{M}}.$$

Das ist eine bemerkenswert starke Garantie. Sie bedeutet: *Jede einzelne Operation* ist so gut, wie sie sein kann. Alle Fehler, die in einem längeren Algorithmus entstehen, kommen ausschließlich aus der *Verkettung* vieler solcher optimaler Einzelschritte.
Alle Fehleranalysen dieses Buches beruhen auf dieser Gleichung.

1.5 Sonderwerte: Null, Unendlich und NaN

Die reservierten Bitmuster

IEEE 754 reserviert zwei Exponentenwerte für Sonderfälle:
Exponent ganz null:

- Mantisse null $\Longrightarrow$ die Zahl ± 0. (Ja, es gibt zwei Nullen. $1/+0 = +\infty$, $1/-0 = -\infty$.)
- Mantisse ungleich null $\Longrightarrow$ **denormalisierte Zahlen**. Sie füllen die Lücke zwischen 0 und der kleinsten normalisierten Zahl und erlauben *allmählichen Unterlauf*: $a \neq b \Longrightarrow a - b \neq 0$ bleibt gültig. Ohne sie könnte $a - b = 0$ herauskommen, obwohl $a \neq b$ – eine Falle in jeder Abbruchbedingung.

Exponent ganz eins:

- Mantisse null $\Longrightarrow \pm\infty$. Entsteht bei Überlauf und bei $1/0$.
- Mantisse ungleich null $\Longrightarrow$ **NaN** (*not a number*). Entsteht bei $0/0$, $\infty - \infty$, $\sqrt{-1}$.

Die wichtigste Eigenschaft von NaN: Es ist mit *nichts* gleich, auch nicht mit sich selbst. Der Test

```
x != x
```

ist die standardkonforme Prüfung auf NaN. Und NaN *propagiert*: Jede Operation mit NaN liefert NaN. Ein einziger fehlerhafter Eintrag verseucht so eine ganze Rechnung – was ein Vorzug ist, denn er macht den Fehler sichtbar, statt ihn zu verstecken.

1.6 Auslöschung – der gefährlichste Effekt

Das Phänomen

Subtraktion fast gleicher Zahlen ist katastrophal

Werden zwei fast gleiche Zahlen subtrahiert, so löschen sich die führenden – also die *sicheren* – Ziffern aus. Übrig bleiben die hinteren Ziffern, die bereits mit Rundungsfehlern behaftet waren. Der *relative* Fehler explodiert.

Ein Zahlenbeispiel mit 6 Dezimalstellen. Sei

$$a = 1{,}234567\ldots, \qquad b = 1{,}234558\ldots$$

Gerundet auf 6 Stellen: $\mathrm{fl}(a) = 1{,}23457$, $\mathrm{fl}(b) = 1{,}23456$.
Exakte Differenz: $a - b = 0{,}000009 = 9{,}0 \cdot 10^{-6}$.
Berechnete Differenz: $1{,}23457 - 1{,}23456 = 0{,}00001 = 1{,}0 \cdot 10^{-5}$.
Relativer Fehler:

$$\frac{\left|10^{-5} - 9 \cdot 10^{-6}\right|}{9 \cdot 10^{-6}} = \frac{10^{-6}}{9 \cdot 10^{-6}} = 11{,}1\,\%.$$

Aus zwei Zahlen, die auf 10^{-6} relativ genau waren, wurde ein Ergebnis mit über 11 % Fehler. **Die Subtraktion selbst war exakt** – der Fehler steckte schon vorher in den Eingaben und wurde nur *sichtbar gemacht*.

Aufgabe 1.2. Berechnen Sie die Nullstellen von

$$x^2 - 10^8 x + 1 = 0$$

mit der Mitternachtsformel in doppelter Genauigkeit. Was geht schief, und wie behebt man es?

Lösung Schritt für Schritt.

Schritt 1 – Die exakten Werte. Mit $p = -10^8$, $q = 1$:

$$x_{1,2} = \frac{10^8 \pm \sqrt{10^{16} - 4}}{2}.$$

Die exakten Werte sind (auf 17 Stellen)

$$x_1 = 99\,999\,999{,}99999999, \qquad x_2 = 1{,}0000000000000000 \cdot 10^{-8}.$$

Wie man x_2 nachprüft: Nach dem Satz von Vieta ist $x_1 x_2 = q = 1$, also $x_2 = 1/x_1 \approx 10^{-8}$.

Schritt 2 – Die Wurzel im Rechner.

$$\sqrt{10^{16} - 4} = \sqrt{9999999999999996}.$$

Der exakte Wert ist 99 999 999,99999998. In `double` stehen etwa 16 signifikante Stellen zur Verfügung; die Wurzel wird zu

$$99\,999\,999{,}99999998$$

gerundet – so gut es geht, aber die letzten Stellen sind bereits unsicher.
Schritt 3 – Die große Wurzel.

$$x_1 = \frac{10^8 + 99\,999\,999{,}99999998}{2} = \frac{199\,999\,999{,}99999998}{2} = 99\,999\,999{,}99999999.$$

Hier wird *addiert* – kein Problem. Das Ergebnis ist auf volle Genauigkeit richtig. ✓
Schritt 4 – Die kleine Wurzel: hier passiert das Unglück.

$$x_2 = \frac{10^8 - 99\,999\,999{,}99999998}{2}.$$

Der Zähler ist die Differenz zweier Zahlen, die in den ersten 16 Stellen übereinstimmen. Vom Ergebnis

$$0{,}00000002$$

ist *keine einzige* Ziffer verlässlich – alle stammen aus dem Rundungsrest. In der Praxis erhält man Werte wie $7{,}45 \cdot 10^{-9}$ oder $1{,}49 \cdot 10^{-8}$ statt $1{,}0 \cdot 10^{-8}$; der relative Fehler liegt bei 25 bis 50 %.
Schritt 5 – Die Rettung durch Umformung. Nach Vieta gilt $x_1 x_2 = q$. Also berechnet man die kleine Wurzel *nicht* über die Differenz, sondern über

$$x_2 = \frac{q}{x_1} = \frac{1}{99\,999\,999{,}99999999} = 1{,}0000000000000000 \cdot 10^{-8}.\checkmark$$

Eine Division – keine Auslöschung, volle Genauigkeit.
Schritt 6 – Der allgemeine Algorithmus. Für $ax^2 + bx + c = 0$ mit reeller Diskriminante:

1. Setze $\sigma := \operatorname{sign}(b) \quad$ (mit $\operatorname{sign}(0) := 1$).
2. Berechne $Q := -\frac{1}{2}\left(b + \sigma\sqrt{b^2 - 4ac}\right)$.
3. Dann $x_1 = Q/a, \quad x_2 = c/Q$.

Warum das funktioniert: Durch das Vorzeichen σ werden in Schritt 2 immer zwei Zahlen *gleichen* Vorzeichens addiert. Auslöschung ist damit ausgeschlossen. Die zweite Wurzel folgt aus Vieta.
Merke: Dieser Algorithmus steht in jeder ernsthaften Bibliothek. Die Mitternachtsformel aus der Schule ist mathematisch richtig und numerisch unbrauchbar.

Aufgabe 1.3. Warum liefert $\dfrac{1-\cos x}{x^2}$ für $x = 10^{-8}$ den Wert 0, und wie rechnet man richtig?

Lösung Schritt für Schritt.

Schritt 1 – Der wahre Wert. Nach der Taylorreihe (Band 3, Kapitel 8):

$$\cos x = 1 - \frac{x^2}{2} + \frac{x^4}{24} - \dots$$

Also

$$\frac{1-\cos x}{x^2} = \frac{\frac{x^2}{2} - \frac{x^4}{24} + \dots}{x^2} = \frac{1}{2} - \frac{x^2}{24} + \dots \longrightarrow \frac{1}{2}.$$

Für $x = 10^{-8}$ ist der wahre Wert 0,5 bis auf 10^{-17}.

Schritt 2 – Was der Rechner tut. Für $x = 10^{-8}$ ist $x^2 = 10^{-16}$, also

$$\cos x = 1 - 5 \cdot 10^{-17} + \dots$$

Nun ist $5 \cdot 10^{-17}$ *kleiner* als $\varepsilon_{\mathrm{M}} \approx 1{,}11 \cdot 10^{-16}$. Die nächstgelegene Gleitkommazahl zu $1 - 5 \cdot 10^{-17}$ ist daher exakt 1:

$$\mathrm{fl}(\cos(10^{-8})) = 1.$$

Folglich

$$1 - \cos x = 1 - 1 = 0 \Longrightarrow \frac{0}{10^{-16}} = 0.$$

Relativer Fehler: 100 %. Und das Ergebnis sieht harmlos aus – keine Warnung, kein NaN.

Schritt 3 – Erste Rettung: trigonometrische Identität. Es gilt

$$1 - \cos x = 2\sin^2\left(\frac{x}{2}\right).$$

Herleitung: Aus $\cos(2u) = 1 - 2\sin^2 u$ mit $u = x/2$ folgt $\cos x = 1 - 2\sin^2(x/2)$, also die Behauptung.

Damit:

$$\frac{1-\cos x}{x^2} = \frac{2\sin^2(x/2)}{x^2} = \frac{1}{2}\left(\frac{\sin(x/2)}{x/2}\right)^2.$$

Für $x = 10^{-8}$ ist $\sin(5 \cdot 10^{-9}) = 5 \cdot 10^{-9}$ auf volle Genauigkeit (die Sinusfunktion ist bei kleinen Argumenten hervorragend konditioniert), und man erhält

$$\frac{1}{2} \cdot 1^2 = 0{,}5. \checkmark$$

Schritt 4 – Zweite Rettung: Taylorentwicklung verwenden. Für $|x| < 10^{-4}$ rechnet man einfach

$$\frac{1}{2} - \frac{x^2}{24}$$

und ist auf etwa 10^{-17} genau.

Die allgemeine Lehre. Wo Auslöschung droht, hilft eine der beiden Techniken:

- **Algebraisch umformen**, so dass die Differenz verschwindet (Erweitern mit dem konjugierten Ausdruck, trigonometrische Identitäten, Vieta).
- **Reihenentwicklung** für den kritischen Bereich benutzen.

Weitere klassische Fälle:

Gefährlich	Stabil
$\sqrt{x+1}-\sqrt{x}$	$\dfrac{1}{\sqrt{x+1}+\sqrt{x}}$
e^x-1	`expm1(x)` (Bibliotheksfunktion)
$\ln(1+x)$	`log1p(x)` (Bibliotheksfunktion)
$\dfrac{e^x-1}{x}$	`expm1(x)/x`
$\sin(x)-x$	Taylor: $-\dfrac{x^3}{6}+\dfrac{x^5}{120}$

Die Funktionen `expm1` und `log1p` gibt es in jeder ernstzunehmenden Standardbibliothek genau aus diesem Grund. Wer sie nicht kennt, schreibt unnötig ungenauen Code.

1.7 Summation: ein unterschätztes Problem

Aufgabe 1.4. Man summiere 10^7-mal die Zahl 0,1 in einfacher Genauigkeit. Welchen Fehler erwartet man?

Lösung Schritt für Schritt.

Schritt 1 – Der exakte Wert. $10^7 \cdot 0{,}1 = 10^6$.

Schritt 2 – Fehleranalyse der naiven Summation. Sei S_k die berechnete Teilsumme. Bei jedem Schritt kommt ein relativer Fehler von höchstens ε_{M} hinzu:

$$S_k = \mathrm{fl}(S_{k-1} + x_k) = (S_{k-1} + x_k)(1+\delta_k).$$

Führt man das aus, so erhält man für die Gesamtsumme die Schranke

$$\left| S_n - \sum_i x_i \right| \le (n-1)\,\varepsilon_{\mathrm{M}} \sum_i |x_i| + \mathcal{O}(\varepsilon_{\mathrm{M}}^2).$$

Der Fehler wächst also *linear* in der Zahl der Summanden.

Schritt 3 – Zahlen einsetzen. Für `single` ist $\varepsilon_{\mathrm{M}} \approx 6 \cdot 10^{-8}$, und $\sum |x_i| = 10^6$:

$$\text{Fehlerschranke} \approx 10^7 \cdot 6 \cdot 10^{-8} \cdot 10^6 = 6 \cdot 10^5.$$

Das ist die Größenordnung des Ergebnisses selbst – die Schranke ist also nutzlos pessimistisch, aber sie warnt zu Recht.

Schritt 4 – Was tatsächlich passiert. In `single` liefert die naive Summation etwa **90 000** statt 1 000 000 – ein Fehler von 91 %.
Der Grund: Sobald die Teilsumme etwa $2^{24} \approx 1{,}7 \cdot 10^7$ erreicht, ist der Abstand benachbarter Gleitkommazahlen größer als 1. Die Addition von 0,1 ändert dann gar nichts mehr – die Summe *stagniert*. Genauer: Ab einer Teilsumme von etwa $2{,}1 \cdot 10^6$ wird 0,1 vollständig weggerundet.
Schritt 5 – Abhilfe 1: paarweise Summation. Statt sequenziell summiert man rekursiv in Blöcken:

$$\mathrm{sum}(x_1, \ldots, x_n) = \mathrm{sum}(x_1, \ldots, x_{n/2}) + \mathrm{sum}(x_{n/2+1}, \ldots, x_n).$$

Die Fehlerschranke verbessert sich von $\mathcal{O}(n\varepsilon_\mathrm{M})$ auf $\mathcal{O}(\varepsilon_\mathrm{M} \log_2 n)$ – für $n = 10^7$ ist $\log_2 n \approx 23$, also eine Verbesserung um den Faktor 400 000. Numpy und die meisten Bibliotheken tun dies standardmäßig.
Schritt 6 – Abhilfe 2: Kahan-Summation. WILLIAM KAHAN veröffentlichte 1965 einen Algorithmus, der den jeweiligen Rundungsrest mitführt und im nächsten Schritt zurückgibt:

$s := 0, \quad c := 0$
für $i = 1, \ldots, n$:
  $y := x_i - c$ (Rest aus dem letzten Schritt abziehen)
  $t := s + y$ (die eigentliche Addition)
  $c := (t - s) - y$ (addiert minus zu addieren)
  $s := t$

Warum das funktioniert: Die Zeile $c := (t - s) - y$ berechnet den bei der Addition *verlorenen* Anteil. Da t und s nah beieinanderliegen, ist $t - s$ exakt darstellbar (Satz von Sterbenz); der Rest wird korrekt erfasst und im nächsten Durchgang nachgereicht.
Die Fehlerschranke wird zu

$$|S_n - \sum x_i| \leq \left(2\varepsilon_\mathrm{M} + \mathcal{O}(n\varepsilon_\mathrm{M}^2)\right) \sum |x_i|$$

– praktisch *unabhängig von n*. Mit Kahan-Summation liefert unser Beispiel 1 000 000,0 auf volle einfache Genauigkeit.
Der Preis: vier statt einer Operation pro Summand.

Vorsicht beim Optimieren

Ein optimierender Compiler „sieht“, dass $(t - s) - y$ mathematisch gleich 0 ist, und streicht die Zeile. Der Algorithmus wird dadurch zur naiven Summation. Man muss dem Compiler die Umformung verbieten – in C etwa mit `volatile`, oder indem man aggressive Gleitkomma-Optimierung (`-ffast-math`) abschaltet.
Das ist ein allgemeines Problem: `-ffast-math` erlaubt dem Compiler, Gleitkommaarithmetik als assoziativ zu behandeln. Sie ist es nicht.

Gleitkommaaddition ist nicht assoziativ

$$(a + b) + c \neq a + (b + c) \qquad \text{im Allgemeinen.}$$

Beispiel in einer Arithmetik mit 3 Dezimalstellen:

$$a = 1{,}00, \quad b = 0{,}00100, \quad c = -1{,}00.$$

$$(a + b) + c = \mathrm{fl}(1{,}00100) + (-1{,}00) = 1{,}00 - 1{,}00 = 0$$

$$a + (b + c) = 1{,}00 + (-0{,}999) = 0{,}00100 \checkmark$$

Das zweite Ergebnis ist richtig, das erste nicht.
Konsequenzen in der Praxis:

- Parallele Summation liefert je nach Aufteilung verschiedene Ergebnisse. Zwei Läufe desselben Programms auf verschieden vielen Prozessorkernen können abweichen – ein bekanntes Reproduzierbarkeitsproblem in der Klimasimulation und im maschinellen Lernen.
- Der Compiler darf Summanden nicht umordnen, es sei denn, man erlaubt es ausdrücklich.
- **Faustregel für die Reihenfolge:** Bei gleichem Vorzeichen zuerst die betragskleinen Summanden addieren. So bleibt die Teilsumme möglichst lange klein, und weniger wird weggerundet.

Der Patriot-Zwischenfall von 1991

Am 25. Februar 1991 verfehlte eine Patriot-Abwehrrakete in Dhahran, Saudi-Arabien, eine anfliegende Scud-Rakete. Der Einschlag tötete 28 amerikanische Soldaten.
Die Ursache war ein Rundungsfehler. Die interne Uhr des Systems zählte Zehntelsekunden seit dem Einschalten. Zur Umrechnung in Sekunden wurde mit 0,1 multipliziert – gespeichert in einem 24-Bit-Festkommaformat.

Die Binärentwicklung von 0,1 ist, wie wir gesehen haben, periodisch. Nach 24 Bit abgeschnitten, ergibt sich ein relativer Fehler von etwa $9{,}5 \cdot 10^{-8}$.
Die Batterie in Dhahran lief zum Zeitpunkt des Angriffs seit etwa 100 Stunden ohne Neustart. Der absolute Zeitfehler betrug damit

$$100 \cdot 3600\ \text{s} \cdot 9{,}5 \cdot 10^{-8} \approx 0{,}34\ \text{Sekunden}.$$

Eine Scud-Rakete fliegt mit etwa 1676 m/s. In 0,34 Sekunden legt sie

$$1676 \cdot 0{,}34 \approx 570\ \text{Meter}$$

zurück. Das Suchfenster des Radars verfehlte das Ziel um mehr als einen halben Kilometer.
Das Problem war bekannt; eine korrigierte Software wurde am 26. Februar ausgeliefert – einen Tag nach dem Einschlag. Als vorläufige Abhilfe hatte der Hersteller empfohlen, die Systeme regelmäßig neu zu starten. Wie regelmäßig, stand nicht dabei.
Die Lehre: Ein relativer Fehler von 10^{-7} klingt vernachlässigbar. Ob er es ist, entscheidet nicht die Zahl, sondern die *Verstärkung* auf dem Weg zum Ergebnis. Genau davon handelt das nächste Kapitel.

2 Kondition und Stabilität

2.1 Zwei verschiedene Fragen

Die begriffliche Trennung, auf der alles beruht

Kondition ist eine Eigenschaft des *Problems*. Sie fragt: Wie stark ändert sich die Lösung, wenn ich die Eingabedaten leicht störe?
Stabilität ist eine Eigenschaft des *Algorithmus*. Sie fragt: Liefert dieses Rechenverfahren ein Ergebnis, das so gut ist, wie es die Kondition erlaubt?
Diese Trennung ist die wichtigste Idee der Numerik. Sie stammt im Wesentlichen von JAMES HARDY WILKINSON und erlaubt es, zwei ganz verschiedene Fehlerquellen getrennt zu behandeln:

- Ein *schlecht konditioniertes* Problem ist von Natur aus empfindlich. Kein Algorithmus der Welt kann daran etwas ändern.
- Ein *instabiler* Algorithmus verschlechtert das Ergebnis über das hinaus, was die Kondition erzwingt. Ihn kann man ersetzen.

Die Merkformel, die wir in diesem Kapitel begründen:

$$\text{Fehler im Ergebnis} \lesssim \text{Kondition} \times \text{Rückwärtsfehler des Algorithmus.}$$

2.2 Die Kondition eines Problems

Ein Problem ist eine Abbildung $f : X \to Y$ von den Eingabedaten zum Ergebnis. Wir stören die Eingabe von x auf $\tilde{x}$ und fragen, was mit $f(x)$ geschieht.

Definition 2.1 (Konditionszahlen). Sei $f : \mathbb{R} \to \mathbb{R}$ differenzierbar, $x \neq 0$, $f(x) \neq 0$. Die **absolute Konditionszahl** ist

$$\kappa_{\text{abs}}(x) = \left|f'(x)\right|,$$

die **relative Konditionszahl**

$$\kappa_{\text{rel}}(x) = \frac{|f'(x)| \cdot |x|}{|f(x)|}.$$

Woher die relative Konditionszahl kommt

Wir stören x zu $\tilde{x} = x(1+\varepsilon)$ mit kleinem ε und fragen nach dem relativen Fehler des Ergebnisses.

Schritt 1 – Taylor, erste Ordnung (Band 3, Kapitel 8):

$$f(\tilde{x}) = f(x + x\varepsilon) \approx f(x) + f'(x) \cdot x\varepsilon.$$

Schritt 2 – Relativer Fehler des Ergebnisses:

$$\frac{|f(\tilde{x}) - f(x)|}{|f(x)|} \approx \frac{|f'(x)|\,|x|\,|\varepsilon|}{|f(x)|} = \underbrace{\frac{|f'(x)|\,|x|}{|f(x)|}}_{=\,\kappa_{\text{rel}}} \cdot |\varepsilon|\,.$$

Also:

$$\boxed{\text{relativer Ausgabefehler} \approx \kappa_{\text{rel}} \cdot \text{relativer Eingabefehler}}$$

κ_{rel} ist der *Verstärkungsfaktor* für relative Fehler.
Sprechweise:

- $\kappa_{\text{rel}} \approx 1$: das Problem ist **gut konditioniert**.
- $\kappa_{\text{rel}} \gg 1$: das Problem ist **schlecht konditioniert**.

Die praktische Faustregel: Ist $\kappa_{\text{rel}} \approx 10^k$, so verliert man etwa k signifikante Dezimalstellen – unvermeidlich, unabhängig vom Algorithmus.

Beispiel 2.2 (Die Kondition der Grundoperationen). **Multiplikation**, $f(x) = cx$ mit festem $c \neq 0$:

$$\kappa_{\text{rel}} = \frac{|c| \cdot |x|}{|cx|} = 1.$$

Perfekt konditioniert.

Wurzel, $f(x) = \sqrt{x}$ für $x > 0$:

$$f'(x) = \frac{1}{2\sqrt{x}} \Longrightarrow \kappa_{\text{rel}} = \frac{\frac{1}{2\sqrt{x}} \cdot x}{\sqrt{x}} = \frac{x}{2x} = \frac{1}{2}.$$

Die Wurzel *dämpft* relative Fehler sogar.

Exponentialfunktion, $f(x) = \mathrm{e}^x$:

$$\kappa_{\text{rel}} = \frac{\mathrm{e}^x \cdot |x|}{\mathrm{e}^x} = |x|\,.$$

Für $x = 100$ ist $\kappa_{\text{rel}} = 100$: Ein relativer Eingabefehler von 10^{-16} wird zu 10^{-14}. Noch harmlos. Für $x = 10^6$ verliert man sechs Stellen.

Subtraktion, $f(a, b) = a - b$. Hier braucht man die mehrdimensionale Fassung; für Störungen $a(1 + \varepsilon_a)$, $b(1 + \varepsilon_b)$ ist

$$\frac{|(a\varepsilon_a - b\varepsilon_b)|}{|a - b|} \leq \frac{|a| + |b|}{|a - b|} \cdot \max(|\varepsilon_a|\,, |\varepsilon_b|),$$

also

$$\kappa_{\text{rel}} = \frac{|a| + |b|}{|a - b|}.$$

Für $a \approx b$ geht der Nenner gegen null und $\kappa_{\text{rel}} \to \infty$.

Das ist die eigentliche Erklärung der Auslöschung: Sie ist kein Fehler des Algorithmus. Die Subtraktion fast gleicher Zahlen ist ein *schlecht konditioniertes Problem*. Der Rechner tut nichts falsch – die Aufgabe selbst ist empfindlich.

Zahlenbeispiel: $a = 1{,}0000001$, $b = 1{,}0000000$. Dann

$$\kappa_{\text{rel}} = \frac{2{,}0000001}{10^{-7}} = 2 \cdot 10^{7}.$$

Man verliert etwa 7 Stellen. In `double` bleiben von 16 also 9 – genau das, was man beobachtet.

Aufgabe 2.1. Bestimmen Sie die relative Kondition der Nullstellen des Polynoms

$$p(x) = x^2 - 2x + (1 - \delta^2)$$

in Abhängigkeit vom Parameter δ, und deuten Sie das Ergebnis.

Lösung Schritt für Schritt.

Schritt 1 – Die Nullstellen.

$$x_{1,2} = 1 \pm \sqrt{1 - (1 - \delta^2)} = 1 \pm \delta.$$

Schritt 2 – Als Funktion des Koeffizienten $c := 1 - \delta^2$. Wir stören c und fragen, wie $x_1 = 1 + \sqrt{1 - c}$ reagiert:

$$\frac{\mathrm{d}x_1}{\mathrm{d}c} = \frac{-1}{2\sqrt{1-c}} = \frac{-1}{2\delta}.$$

Schritt 3 – Relative Kondition.

$$\kappa_{\text{rel}} = \frac{|\,\mathrm{d}x_1 / \,\mathrm{d}c| \cdot |c|}{|x_1|} = \frac{\frac{1}{2\delta} \cdot |1 - \delta^2|}{1 + \delta} \approx \frac{1}{2\delta} \quad \text{für kleines } \delta.$$

Schritt 4 – Deutung. Für $\delta \to 0$ divergiert die Kondition. Bei $\delta = 10^{-6}$ ist $\kappa_{\text{rel}} = 5 \cdot 10^{5}$: Eine Störung des Koeffizienten in der 16. Stelle verschiebt die Nullstelle in der 10.

Der Grund ist geometrisch. Für $\delta \to 0$ fallen die beiden Nullstellen zusammen; das Polynom berührt die x-Achse, statt sie zu schneiden. Bei einer Berührung genügt eine winzige vertikale Verschiebung, um die Schnittpunkte weit zu verschieben – oder ganz verschwinden zu lassen.

Merke: Mehrfache Nullstellen sind grundsätzlich schlecht konditioniert. Eine m-fache Nullstelle reagiert auf Störungen der Größe ε mit Verschiebungen der Größe $\varepsilon^{1/m}$. Bei einer dreifachen Nullstelle und $\varepsilon = 10^{-16}$ sind das $10^{-5{,}3}$ – man verliert zwei Drittel aller Stellen.

Wilkinsons Polynom

JAMES H. WILKINSON (1919–1986) demonstrierte 1963 mit einem Beispiel, wie extrem schlecht Nullstellenprobleme konditioniert sein können – und zwar auch dann, wenn die Nullstellen gut getrennt sind.
Er betrachtete

$$p(x) = (x-1)(x-2)\cdots(x-20) = x^{20} - 210x^{19} + \dots$$

Die Nullstellen sind $1, 2, \dots, 20$ – alle einfach, alle mit Abstand 1.
Nun störte er einen einzigen Koeffizienten: Er ersetzte -210 durch

$$-210 - 2^{-23} \approx -210{,}000000119.$$

Eine relative Störung von etwa $5{,}7 \cdot 10^{-10}$ – weniger, als `single` auflösen kann.
Das Ergebnis: Die Nullstellen 16 und 17 wurden zu

$$16{,}73 \;\pm\; 2{,}81\,\text{ß},$$

also *komplex*. Andere Nullstellen verschoben sich um mehrere Zehntel. Die relative Verschiebung betrug bis zu 10^9-mal die Eingangsstörung.
Wilkinson schrieb später, dies sei die traumatischste Erfahrung seiner Laufbahn als Numeriker gewesen – er hätte nie vermutet, dass ein so harmlos aussehendes Problem so empfindlich sein könne.
Die Lehre für die Praxis: Man sollte Polynomnullstellen *nie* über die Koeffizienten des ausmultiplizierten Polynoms bestimmen. Alle guten Verfahren arbeiten stattdessen mit den Eigenwerten der Begleitmatrix (Kapitel 6) – was numerisch etwas ganz anderes ist, obwohl es mathematisch dasselbe Problem löst.
Wilkinsons Buch *Rounding Errors in Algebraic Processes* (1963) und *The Algebraic Eigenvalue Problem* (1965)[28] begründeten die moderne Fehleranalyse. Er war zugleich Praktiker: Im Zweiten Weltkrieg berechnete er Ballistiktabellen, danach arbeitete er mit ALAN TURING am Pilot ACE.

2.3 Die Kondition linearer Gleichungssysteme

Vorbereitung: Matrixnormen

Zu einer Vektornorm $\|\cdot\|$ gehört die **induzierte Matrixnorm**

$$\|A\| := \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}.$$

Sie misst, um wie viel A einen Vektor höchstens verlängert.
Die wichtigsten Fälle:

Norm	Formel	Merkregel
$\|A\|_1$	$\max_j \sum_i \|a_{ij}\|$	größte Spaltensumme
$\|A\|_\infty$	$\max_i \sum_j \|a_{ij}\|$	größte Zeilensumme
$\|A\|_2$	$\sigma_{\max}(A)$	größter Singulärwert

Alle induzierten Normen sind **submultiplikativ**:

$$\|AB\| \le \|A\|\,\|B\|, \qquad \|Ax\| \le \|A\|\,\|x\|.$$

Beweis der zweiten Ungleichung: Nach Definition des Maximums ist $\frac{\|Ax\|}{\|x\|} \le \|A\|$ für alle $x \neq 0$; Multiplikation mit $\|x\|$ gibt die Behauptung.

Definition 2.3. Für eine reguläre Matrix A heißt

$$\operatorname{cond}(A) := \|A\| \cdot \left\|A^{-1}\right\|$$

die **Konditionszahl** von A.

Satz 2.4 (Störungssatz für lineare Systeme). *Sei $Ax = b$ mit regulärem A, und sei $\tilde{x}$ die Lösung des gestörten Systems $A\tilde{x} = b + \Delta b$. Dann gilt*

$$\frac{\|\tilde{x} - x\|}{\|x\|} \le \operatorname{cond}(A) \cdot \frac{\|\Delta b\|}{\|b\|}.$$

Beweis. **Schritt 1 – Die Störung isolieren.** Aus $A\tilde{x} = b + \Delta b$ und $Ax = b$ folgt durch Subtraktion

$$A(\tilde{x} - x) = \Delta b \Longrightarrow \tilde{x} - x = A^{-1}\Delta b.$$

Schritt 2 – Norm abschätzen.

$$\|\tilde{x} - x\| = \left\|A^{-1}\Delta b\right\| \le \left\|A^{-1}\right\| \cdot \|\Delta b\|.$$

Schritt 3 – Den Nenner abschätzen. Aus $b = Ax$ folgt

$$\|b\| \le \|A\| \cdot \|x\| \Longrightarrow \frac{1}{\|x\|} \le \frac{\|A\|}{\|b\|}.$$

Schritt 4 – Zusammensetzen.

$$\frac{\|\tilde{x} - x\|}{\|x\|} \le \left\|A^{-1}\right\| \|\Delta b\| \cdot \frac{\|A\|}{\|b\|} = \operatorname{cond}(A) \cdot \frac{\|\Delta b\|}{\|b\|}.$$

■

Eigenschaften der Konditionszahl

(a) $\operatorname{cond}(A) \geq 1$ für jede reguläre Matrix.
Beweis: $1 = \|I\| = \left\|AA^{-1}\right\| \leq \|A\| \left\|A^{-1}\right\| = \operatorname{cond}(A)$.

(b) $\operatorname{cond}(cA) = \operatorname{cond}(A)$ für $c \neq 0$. Die Kondition ist skalierungsinvariant – eine gleichmäßige Streckung der Matrix ändert nichts.
Beweis: $\|cA\| \left\|(cA)^{-1}\right\| = |c| \, \|A\| \cdot \frac{1}{|c|} \left\|A^{-1}\right\|$.

(c) In der Spektralnorm gilt

$$\operatorname{cond}_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}$$

mit den Singulärwerten aus Kapitel 6. Für symmetrisch positiv definite A ist das $\lambda_{\max}/\lambda_{\min}$.

Die geometrische Deutung. $\operatorname{cond}_2(A)$ misst, wie stark A die Einheitskugel zu einem Ellipsoid *verzerrt*: Es ist das Verhältnis von längster zu kürzester Halbachse. Eine Matrix mit cond $= 1$ (etwa eine Drehung) verzerrt nicht; eine Matrix mit cond $= 10^{10}$ drückt die Kugel in einer Richtung fast flach.
Und genau deshalb ist Rückrechnen schwer: Wo das Bild fast flach ist, liegen weit auseinander liegende Urbilder fast aufeinander. Die Rückabbildung reißt sie wieder auseinander – und mit ihnen die Fehler.

Aufgabe 2.2. Gegeben sei

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 1{,}0001 \end{pmatrix}, \qquad b = \begin{pmatrix} 2 \\ 2{,}0001 \end{pmatrix}.$$

(a) Lösen Sie $Ax = b$ exakt.

(b) Stören Sie b zu $\tilde{b} = (2;\ 2{,}0002)^{\mathsf{T}}$ und lösen Sie erneut.

(c) Berechnen Sie $\operatorname{cond}_\infty(A)$ und prüfen Sie die Störungsabschätzung.

Lösung Schritt für Schritt.

(a) Exakte Lösung. Zweite Zeile minus erste:

$$0{,}0001\, x_2 = 0{,}0001 \Longrightarrow x_2 = 1.$$

Einsetzen in die erste Zeile: $x_1 + 1 = 2$, also $x_1 = 1$.

$$x = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

(b) Gestörte Lösung. Zweite Zeile minus erste:

$$0{,}0001\, \tilde{x}_2 = 0{,}0002 \Longrightarrow \tilde{x}_2 = 2.$$

Erste Zeile: $\tilde{x}_1 + 2 = 2$, also $\tilde{x}_1 = 0$.

$$\tilde{x} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}.$$

Beobachtung. Die relative Störung der rechten Seite betrug

$$\frac{\|\Delta b\|_\infty}{\|b\|_\infty} = \frac{0{,}0001}{2{,}0001} \approx 5 \cdot 10^{-5}.$$

Die relative Änderung der Lösung:

$$\frac{\|\tilde{x} - x\|_\infty}{\|x\|_\infty} = \frac{\max(1,1)}{1} = 1 = 100\,\%.$$

Der Fehler wurde um den Faktor $20\,000$ **verstärkt.**
(c) Konditionszahl.
Inverse berechnen. Für 2×2-Matrizen:

$$A^{-1} = \frac{1}{\det A} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}, \qquad \det A = 1 \cdot 1{,}0001 - 1 \cdot 1 = 0{,}0001.$$

$$A^{-1} = \frac{1}{0{,}0001} \begin{pmatrix} 1{,}0001 & -1 \\ -1 & 1 \end{pmatrix} = \begin{pmatrix} 10001 & -10000 \\ -10000 & 10000 \end{pmatrix}.$$

Kontrolle: $A \cdot A^{-1}$, erste Zeile mal erste Spalte: $1 \cdot 10001 + 1 \cdot (-10000) = 1$. ✓
Zeilensummennormen.

$$\|A\|_\infty = \max(1+1,\ 1+1{,}0001) = 2{,}0001,$$

$$\left\|A^{-1}\right\|_\infty = \max(10001 + 10000,\ 10000 + 10000) = 20001.$$

$$\operatorname{cond}_\infty(A) = 2{,}0001 \cdot 20001 = 40\,004.$$

Prüfung der Abschätzung:

$$1 \le 40\,004 \cdot 5 \cdot 10^{-5} = 2{,}0 \quad \checkmark$$

Die Schranke ist um den Faktor 2 pessimistisch – also fast scharf.
Interpretation. cond $\approx 4 \cdot 10^4$ heißt: Man verliert etwa $\log_{10}(4 \cdot 10^4) \approx 4{,}6$, also rund 5 Dezimalstellen. In `double` bleiben von 16 noch 11 – meist unproblematisch. Bei cond $= 10^{16}$ dagegen bleibt *keine* Stelle übrig.
Geometrisch: Die beiden Zeilen von A beschreiben zwei Geraden, die fast parallel sind (Richtungen $(1,1)$ und $(1;1{,}0001)$). Ihr Schnittpunkt ist schlecht bestimmt – eine winzige Parallelverschiebung einer Geraden verschiebt den Schnittpunkt weit.

2.4 Stabilität: Vorwärts- und Rückwärtsfehler

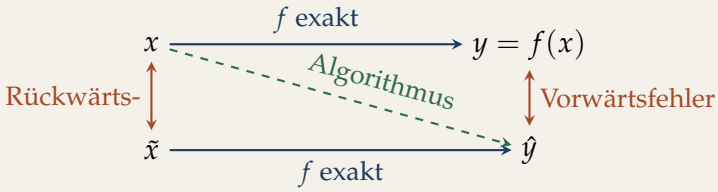
Wilkinsons entscheidende Idee

Man will wissen, wie gut ein berechnetes $\hat{y}$ die exakte Lösung $y = f(x)$ trifft. Der naheliegende Weg – den **Vorwärtsfehler** $\|\hat{y} - y\|$ direkt abzuschätzen – ist meist hoffnungslos kompliziert.

Wilkinsons Umkehrung der Fragestellung: Statt zu fragen „Wie falsch ist meine Antwort?“, fragt man

Zu welcher leicht gestörten Eingabe wäre meine Antwort die exakte?

Also: Man sucht ein $\tilde{x}$ mit $f(\tilde{x}) = \hat{y}$ *exakt* und misst $\|\tilde{x} - x\|$. Das ist der **Rückwärtsfehler**.



Ein Algorithmus heißt rückwärtsstabil, wenn der Rückwärtsfehler stets von der Größenordnung ε_M ist.

Warum das die richtige Frage ist: Die Eingabedaten sind ohnehin nur auf ε_M genau bekannt – sie stammen aus Messungen oder wurden beim Einlesen gerundet. Ein rückwärtsstabiler Algorithmus liefert also die *exakte* Antwort auf eine Frage, die von der gestellten nicht unterscheidbar ist. Mehr kann man nicht verlangen.

Satz 2.5 (Die Grundgleichung der Numerik)**.** *Ist ein Algorithmus rückwärtsstabil mit Rückwärtsfehler η, so gilt für den relativen Vorwärtsfehler*

$$\frac{\|\hat{y} - y\|}{\|y\|} \lesssim \kappa_{\text{rel}}(x) \cdot \eta.$$

Beweis. Nach Voraussetzung ist $\hat{y} = f(\tilde{x})$ mit $\|\tilde{x} - x\| / \|x\| = \eta$. Nach Definition der relativen Kondition ist

$$\frac{\|f(\tilde{x}) - f(x)\|}{\|f(x)\|} \approx \kappa_{\text{rel}}(x) \cdot \frac{\|\tilde{x} - x\|}{\|x\|} = \kappa_{\text{rel}}(x) \cdot \eta. \qquad \blacksquare$$

Was diese eine Zeile bedeutet

$$\boxed{\text{Vorwärtsfehler} \lesssim \text{Kondition} \times \text{Rückwärtsfehler}}$$

Sie zerlegt jeden numerischen Fehler in zwei unabhängige Anteile:

- Die **Kondition** gehört zum Problem. Sie ist gegeben und nicht verhandelbar. Wer ein Problem mit cond $= 10^{12}$ lösen will, verliert 12 Stellen – Punkt.
- Der **Rückwärtsfehler** gehört zum Algorithmus. Bei einem guten Verfahren ist er von der Größe ε_{M}; bei einem schlechten beliebig groß.

Daraus folgt die Arbeitsteilung der Numerik:

1. Prüfe die Kondition. Ist sie schlecht, formuliere das *Problem* um – ein anderer Algorithmus hilft nicht.
2. Ist die Kondition gut, wähle einen rückwärtsstabilen Algorithmus. Dann ist das Ergebnis so gut, wie es überhaupt sein kann.

Und eine tröstliche Einsicht: Ein großer Vorwärtsfehler ist kein Beweis für einen schlechten Algorithmus. Er kann allein von der Kondition kommen.

Aufgabe 2.3. Zeigen Sie, dass die Berechnung des Skalarprodukts

$$s = \sum_{i=1}^{n} x_i y_i$$

in der üblichen sequenziellen Form rückwärtsstabil ist.

Lösung Schritt für Schritt.

Schritt 1 – Das Standardmodell anwenden. Die Produkte werden gerundet:

$$\mathrm{fl}(x_i y_i) = x_i y_i (1+\delta_i), \qquad |\delta_i| \le \varepsilon_{\mathrm{M}}.$$

Die Teilsummen ebenfalls; mit $s_1 = \mathrm{fl}(x_1 y_1)$ und

$$s_k = \mathrm{fl}(s_{k-1} + \mathrm{fl}(x_k y_k)) = \big(s_{k-1} + x_k y_k (1+\delta_k)\big)(1+\eta_k), \quad |\eta_k| \le \varepsilon_{\mathrm{M}}.$$

Schritt 2 – Ausmultiplizieren. Der Term $x_i y_i$ wird von allen Rundungen ab Schritt i betroffen. Man erhält

$$\hat{s} = \sum_{i=1}^{n} x_i y_i (1+\delta_i) \prod_{k=\max(i,2)}^{n} (1+\eta_k).$$

Schritt 3 – Die Produkte abschätzen. Wir brauchen ein Hilfsresultat: Sind $|\theta_j| \le \varepsilon_{\mathrm{M}}$ und $m\varepsilon_{\mathrm{M}} < 1$, so gilt

$$\prod_{j=1}^{m}(1+\theta_j) = 1+\theta \qquad \text{mit} \qquad |\theta| \le \frac{m\varepsilon_{\mathrm{M}}}{1-m\varepsilon_{\mathrm{M}}} =: \gamma_m.$$

Begründung: Nach oben ist $\prod(1+\theta_j) \le (1+\varepsilon_{\mathrm{M}})^m$. Die Bernoulli-artige Abschätzung $(1+\varepsilon_{\mathrm{M}})^m \le \frac{1}{1-m\varepsilon_{\mathrm{M}}}$ folgt aus der geometrischen Reihe. Nach unten analog.

Für praktische Zwecke ist $\gamma_m \approx m\varepsilon_M$.
Schritt 4 – Rückwärtsdeutung. Setze

$$\tilde{x}_i := x_i(1+\theta_i), \qquad |\theta_i| \le \gamma_n.$$

Dann ist

$$\hat{s} = \sum_i \tilde{x}_i y_i \qquad \textbf{exakt}.$$

Interpretation: Das berechnete Skalarprodukt ist das *exakte* Skalarprodukt eines leicht gestörten Vektors $\tilde{x}$, wobei jede Komponente relativ um höchstens $n\varepsilon_M$ gestört ist.
Für $n = 10^6$ und `double` ist $n\varepsilon_M \approx 10^6 \cdot 10^{-16} = 10^{-10}$ – immer noch weit unterhalb typischer Messgenauigkeit. **Der Algorithmus ist rückwärtsstabil.**
Schritt 5 – Und der Vorwärtsfehler? Nach der Grundgleichung

$$\frac{|\hat{s}-s|}{|s|} \lesssim \kappa \cdot n\varepsilon_M \qquad \text{mit} \qquad \kappa = \frac{\sum_i |x_i y_i|}{|\sum_i x_i y_i|}.$$

Sind alle Summanden positiv, so ist $\kappa = 1$ – alles bestens.
Heben sich die Summanden dagegen weitgehend auf (*Auslöschung im Großen*), so wird κ riesig. Beispiel: Stehen zwei fast gleich große Beträge mit verschiedenem Vorzeichen gegenüber, kann $\kappa = 10^{12}$ sein – und dann ist auch das rückwärtsstabile Verfahren vorwärts unbrauchbar.
Das ist die Grundgleichung in Aktion: guter Algorithmus, schlechtes Problem, schlechtes Ergebnis. Die Schuld liegt nicht beim Verfahren.

Zusammenfassung des Kapitels

	Kondition	**Stabilität**
Eigenschaft von	dem *Problem*	dem *Algorithmus*
Frage	Wie empfindlich ist die Lösung gegen Datenstörungen?	Wie viel schlechter als nötig ist mein Verfahren?
Symbol	κ, $\mathrm{cond}(A)$	Rückwärtsfehler η
Verbesserbar durch	Umformulierung des Problems	Wahl eines besseren Verfahrens
Beispiel schlecht	Wilkinson-Polynom, fast singuläre Matrix	Mitternachtsformel, Gauß ohne Pivotisierung

Die drei Diagnosefragen bei jedem unerwarteten Ergebnis:

1. Wie groß ist die Kondition? (Wenn groß: Problem umformulieren.)
2. Ist der Algorithmus rückwärtsstabil? (Wenn nein: ersetzen.)
3. Gibt es eine algebraisch äquivalente, aber besser konditionierte Formulierung? (Fast immer ja.)

Part II

Numerische lineare Algebra

3 Lineare Gleichungssysteme und die LR-Zerlegung

3.1 Warum das das wichtigste Problem der Numerik ist

Die zentrale Stellung von $Ax = b$

Schätzungen zufolge entfällt der größte Teil aller wissenschaftlichen Rechenzeit weltweit auf das Lösen linearer Gleichungssysteme. Das liegt nicht daran, dass die Anwendungen linear wären, sondern daran, dass *jedes* Verfahren für ein nichtlineares Problem es im Kern linearisiert:

- Das Newton-Verfahren löst in jedem Schritt ein lineares System (Kapitel 7).
- Finite Elemente und Finite Differenzen führen partielle Differentialgleichungen auf große lineare Systeme zurück (Kapitel 13).
- Die Ausgleichsrechnung, die Ausgangspunkt fast aller Datenanalyse ist, führt auf Normalgleichungen (Kapitel 4).
- Implizite Verfahren für Differentialgleichungen lösen pro Zeitschritt ein System (Kapitel 12).

Wer $Ax = b$ schnell und stabil lösen kann, kann fast alles.

Zwei Verfahren, die man nie benutzt

Erstens: die Cramersche Regel. Sie ist in der linearen Algebra ein hübscher Satz und in der Numerik unbrauchbar. Für ein $n \times n$-System braucht man $n + 1$ Determinanten. Berechnet man sie über die Leibniz-Formel, so kostet das $\mathcal{O}((n + 1)!)$ Operationen.
Zahlenbeispiel: Für $n = 20$ sind das etwa $5 \cdot 10^{19}$ Operationen. Ein Rechner mit 10^{12} Operationen pro Sekunde bräuchte dafür rund **1,6 Millionen Jahre**. Die Gauß-Elimination erledigt dasselbe System in etwa 5000 Operationen – in Nanosekunden.
Zweitens: die Inverse berechnen. Man ist versucht, $x = A^{-1}b$ zu schreiben und die Inverse auszurechnen. Das ist

- etwa dreimal so teuer wie die direkte Lösung,
- numerisch weniger genau,
- und bei dünn besetzten Matrizen katastrophal, weil A^{-1} in aller Regel voll besetzt ist, auch wenn A es nicht war.

Merke: In numerischem Code steht praktisch nie eine Matrixinverse. Wo sie in einer Formel auftaucht, löst man stattdessen ein System. Die Schreibweise $A^{-1}b$ ist eine *Notation*, keine Rechenanweisung.

3.2 Dreieckssysteme – der einfache Fall

Bevor wir eliminieren, klären wir, worauf wir hinauswollen.

Rückwärtseinsetzen

Sei R eine obere Dreiecksmatrix mit $r_{ii} \neq 0$:

$$\begin{pmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ 0 & r_{22} & \cdots & r_{2n} \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & r_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

Die letzte Zeile lautet $r_{nn}x_n = b_n$, also $x_n = b_n / r_{nn}$. Damit geht man in die vorletzte Zeile, und so weiter von unten nach oben:

$$x_i = \frac{1}{r_{ii}} \left(b_i - \sum_{j=i+1}^{n} r_{ij} x_j \right), \qquad i = n, n-1, \ldots, 1.$$

Aufwand. In Zeile i sind $n - i$ Multiplikationen und ebenso viele Subtraktionen nötig, dazu eine Division:

$$\sum_{i=1}^{n} \bigl(2(n-i)+1\bigr) = 2 \cdot \frac{n(n-1)}{2} + n = n^2.$$

Also $\mathcal{O}(n^2)$ Operationen. Für $n = 1000$ sind das eine Million – Bruchteile einer Sekunde.
Analog geht **Vorwärtseinsetzen** bei einer unteren Dreiecksmatrix, von oben nach unten.

3.3 Die Gauß-Elimination als Matrixzerlegung

Die Idee

Die Gauß-Elimination formt A durch Zeilenoperationen in eine obere Dreiecksmatrix um. Die entscheidende Beobachtung der Numerik ist:

Jede dieser Zeilenoperationen ist eine Multiplikation von links mit einer einfachen Matrix.

Sammelt man diese Matrizen ein, so erhält man eine *Zerlegung*

$$A = L \cdot R$$

in eine untere Dreiecksmatrix L (mit Einsen auf der Diagonale) und eine obere Dreiecksmatrix R.
Warum das der springende Punkt ist: Hat man $A = LR$ einmal berechnet, so löst man

$$Ax = b \iff LRx = b$$

in zwei billigen Schritten:

1. $Ly = b$ durch Vorwärtseinsetzen $\mathcal{O}(n^2)$
2. $Rx = y$ durch Rückwärtseinsetzen $\mathcal{O}(n^2)$

Die teure Zerlegung ($\mathcal{O}(n^3)$) fällt *einmal* an. Für weitere rechte Seiten b kostet jede Lösung nur noch $\mathcal{O}(n^2)$.
Das ist praktisch enorm wichtig. In der Strukturmechanik etwa löst man dasselbe System für hunderte Lastfälle; in impliziten Zeitschrittverfahren für tausende Zeitschritte. Wer stattdessen jedes Mal neu eliminiert, verschwendet einen Faktor $n/3$.

Definition 3.1 (Frobenius-Matrizen). Für $k < n$ und $\ell_{ik} \in \mathbb{R}$ sei

$$L_k = I - \ell_k e_k^{\mathsf{T}}, \qquad \ell_k = (0, \dots, 0, \ell_{k+1,k}, \dots, \ell_{nk})^{\mathsf{T}}.$$

Ausgeschrieben ist L_k die Einheitsmatrix mit zusätzlichen Einträgen $-\ell_{ik}$ unterhalb der Diagonale in Spalte k.

Satz 3.2 (Eigenschaften der Frobenius-Matrizen). *(a)* $L_k^{-1} = I + \ell_k e_k^{\mathsf{T}}$ *– man kehrt einfach die Vorzeichen um.*
(b) $L_1^{-1} L_2^{-1} \cdots L_{n-1}^{-1} = I + \sum_k \ell_k e_k^{\mathsf{T}}$ *– die Faktoren setzen sich einfach nebeneinander.*

Beweis. **(a).** Wir rechnen nach:

$$(I - \ell_k e_k^{\mathsf{T}})(I + \ell_k e_k^{\mathsf{T}}) = I + \ell_k e_k^{\mathsf{T}} - \ell_k e_k^{\mathsf{T}} - \ell_k \underbrace{(e_k^{\mathsf{T}} \ell_k)}_{=0} e_k^{\mathsf{T}} = I.$$

Der markierte Term verschwindet, weil ℓ_k an der k-ten Stelle eine Null hat – ℓ_k beginnt erst unterhalb der Diagonale.

(b). Für $j > k$ gilt ebenso $e_k^{\mathsf{T}} \ell_j = 0$, da ℓ_j erst ab Position $j+1 > k$ von null verschieden ist. Also

$$(I + \ell_k e_k^{\mathsf{T}})(I + \ell_j e_j^{\mathsf{T}}) = I + \ell_k e_k^{\mathsf{T}} + \ell_j e_j^{\mathsf{T}} + \ell_k \underbrace{(e_k^{\mathsf{T}} \ell_j)}_{=0} e_j^{\mathsf{T}} = I + \ell_k e_k^{\mathsf{T}} + \ell_j e_j^{\mathsf{T}}.$$

Induktiv folgt die Behauptung. ∎

Was (b) praktisch bedeutet

Die Multiplikatoren ℓ_{ik}, die man während der Elimination ohnehin berechnet, *sind* bereits die Einträge von L. Man muss nichts zusätzlich tun – man schreibt sie einfach an die Stellen, die durch die Elimination frei geworden sind.
Deshalb kommt die LR-Zerlegung ohne zusätzlichen Speicher aus: L und R passen gemeinsam in das Feld von A.

Aufgabe 3.1. Berechnen Sie die LR-Zerlegung von

$$A = \begin{pmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{pmatrix}$$

und lösen Sie damit $Ax = (8, -11, -3)^{\mathsf{T}}$.

Lösung Schritt für Schritt.

Schritt 1 – Erste Spalte eliminieren. Pivot ist $a_{11} = 2$.
Multiplikatoren:

$$\ell_{21} = \frac{-3}{2} = -1{,}5, \qquad \ell_{31} = \frac{-2}{2} = -1.$$

Zeilenoperationen: $Z_2 \leftarrow Z_2 - \ell_{21} Z_1$, also $Z_2 + 1{,}5\, Z_1$; und $Z_3 \leftarrow Z_3 + 1 \cdot Z_1$.
Zeile 2:

$$(-3, -1, 2) + 1{,}5 \cdot (2, 1, -1) = (-3 + 3,\ -1 + 1{,}5,\ 2 - 1{,}5) = (0,\ 0{,}5,\ 0{,}5).$$

Zeile 3:

$$(-2, 1, 2) + 1 \cdot (2, 1, -1) = (0,\ 2,\ 1).$$

Zwischenstand:

$$A^{(1)} = \begin{pmatrix} 2 & 1 & -1 \\ 0 & 0{,}5 & 0{,}5 \\ 0 & 2 & 1 \end{pmatrix}$$

Schritt 2 – Zweite Spalte eliminieren. Pivot ist $a_{22}^{(1)} = 0{,}5$.

$$\ell_{32} = \frac{2}{0{,}5} = 4.$$

$Z_3 \leftarrow Z_3 - 4\, Z_2$:

$$(0, 2, 1) - 4 \cdot (0;\ 0{,}5;\ 0{,}5) = (0,\ 0,\ 1 - 2) = (0, 0, -1).$$

Schritt 3 – Ergebnis.

$$R = \begin{pmatrix} 2 & 1 & -1 \\ 0 & 0{,}5 & 0{,}5 \\ 0 & 0 & -1 \end{pmatrix}, \qquad L = \begin{pmatrix} 1 & 0 & 0 \\ -1{,}5 & 1 & 0 \\ -1 & 4 & 1 \end{pmatrix}.$$

Schritt 4 – Probe. Wir multiplizieren LR aus, zweite Zeile:

$$(-1{,}5)\cdot(2,1,-1)+1\cdot(0;0{,}5;0{,}5)+0=(-3;\ -1{,}5+0{,}5;\ 1{,}5+0{,}5)=(-3,-1,2).\checkmark$$

Dritte Zeile:

$$(-1)(2,1,-1)+4(0;0{,}5;0{,}5)+1\cdot(0,0,-1)=(-2;\ -1+2;\ 1+2-1)=(-2,1,2).\checkmark$$

Schritt 5 – Vorwärtseinsetzen: $Ly=b$.

$$\begin{aligned} y_1 &= 8 \\ -1{,}5\cdot 8+y_2 &= -11 \quad &\Longrightarrow y_2=-11+12=1 \\ -1\cdot 8+4\cdot 1+y_3 &= -3 \quad &\Longrightarrow y_3=-3+8-4=1 \end{aligned}$$

Also $y=(8,1,1)^{\mathsf{T}}$.
Schritt 6 – Rückwärtseinsetzen: $Rx=y$.

$$\begin{aligned} -x_3 &= 1 \quad &\Longrightarrow x_3=-1 \\ 0{,}5x_2+0{,}5\cdot(-1) &= 1 \quad &\Longrightarrow 0{,}5x_2=1{,}5\Longrightarrow x_2=3 \\ 2x_1+1\cdot 3-1\cdot(-1) &= 8 \quad &\Longrightarrow 2x_1=8-3-1=4\Longrightarrow x_1=2 \end{aligned}$$

$$x=\begin{pmatrix}2\\3\\-1\end{pmatrix}.$$

Schritt 7 – Endprobe an der Originalmatrix.

$$2\cdot 2+1\cdot 3+(-1)(-1)=4+3+1=8\checkmark$$

$$-3\cdot 2+(-1)\cdot 3+2\cdot(-1)=-6-3-2=-11\checkmark$$

$$-2\cdot 2+1\cdot 3+2\cdot(-1)=-4+3-2=-3\checkmark$$

Satz 3.3 (Aufwand der LR-Zerlegung)**.** *Die LR-Zerlegung einer vollbesetzten $n\times n$-Matrix erfordert*

$$\frac{2}{3}n^3+\mathcal{O}(n^2)$$

Gleitkommaoperationen.

Beweis. **Schritt 1 – Zählen im k-ten Eliminationsschritt.** Es sind $n-k$ Multiplikatoren zu berechnen (je eine Division). Anschließend wird eine Untermatrix der Größe $(n-k)\times(n-k)$ aktualisiert; für jeden ihrer Einträge eine Multiplikation und eine Subtraktion:

$$(n-k)+2(n-k)^2 \quad \text{Operationen.}$$

Schritt 2 – Summieren. Mit der Substitution $m := n - k$, also $m = n-1, \dots, 1$:

$$\sum_{k=1}^{n-1} \big(2(n-k)^2 + (n-k)\big) = \sum_{m=1}^{n-1} (2m^2 + m).$$

Schritt 3 – Summenformeln einsetzen. Es gilt

$$\sum_{m=1}^{N} m^2 = \frac{N(N+1)(2N+1)}{6}, \qquad \sum_{m=1}^{N} m = \frac{N(N+1)}{2}.$$

Mit $N = n-1$:

$$2 \cdot \frac{(n-1)n(2n-1)}{6} + \frac{(n-1)n}{2} = \frac{(n-1)n(2n-1)}{3} + \mathcal{O}(n^2).$$

Schritt 4 – Führenden Term isolieren.

$$\frac{(n-1)n(2n-1)}{3} = \frac{2n^3 - 3n^2 + n}{3} = \frac{2}{3}n^3 - n^2 + \frac{n}{3}.$$

Also $\frac{2}{3}n^3 + \mathcal{O}(n^2)$. ■

Was $\frac{2}{3}n^3$ konkret bedeutet

n	Operationen	Zeit bei 10^{11} Op./s
100	$6{,}7 \cdot 10^5$	7 Mikrosekunden
1 000	$6{,}7 \cdot 10^8$	7 Millisekunden
10 000	$6{,}7 \cdot 10^{11}$	7 Sekunden
100 000	$6{,}7 \cdot 10^{14}$	2 Stunden
1 000 000	$6{,}7 \cdot 10^{17}$	77 Tage

Die kubische Skalierung ist unerbittlich. Verzehnfacht man n, so vertausendfacht sich die Rechenzeit. Zugleich wächst der Speicherbedarf mit n^2: Eine Million Unbekannte bräuchten $8 \cdot 10^{12}$ Byte, also 8 Terabyte, allein für die Matrix.
Daraus folgt: Für große Systeme sind direkte Verfahren aussichtslos. Man braucht entweder Struktur (Bandmatrizen, dünne Besetzung) oder *iterative* Verfahren – Kapitel 5.
Ein Wort zum Ausblick: Es gibt asymptotisch schnellere Verfahren. VOLKER STRASSEN zeigte 1969, dass Matrixmultiplikation in $\mathcal{O}(n^{2{,}807})$ möglich ist; der aktuelle theoretische Rekord liegt bei etwa $\mathcal{O}(n^{2{,}371})$. In der Praxis lohnt sich Strassen erst ab etwa $n > 1000$, und die exotischeren Verfahren nie – ihre Konstanten sind astronomisch.

3.4 Pivotisierung: warum man tauschen muss

Ein Beispiel, das alles zeigt

Betrachten Sie

$$\begin{pmatrix} 10^{-20} & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

Die exakte Lösung ist (auf 16 Stellen)

$$x_1 = \frac{1}{1 - 10^{-20}} \approx 1{,}0000000000000000, \qquad x_2 = 1 - 10^{-20} \approx 1{,}0000000000000000.$$

Naive Elimination ohne Tausch. Multiplikator:

$$\ell_{21} = \frac{1}{10^{-20}} = 10^{20}.$$

Zeile 2 wird zu

$$(1, 1) - 10^{20} \cdot (10^{-20}, 1) = (0,\ 1 - 10^{20}).$$

Rechte Seite: $2 - 10^{20} \cdot 1 = 2 - 10^{20}$.
Und jetzt der Rundungsfehler: In `double` ist

$$\mathrm{fl}(1 - 10^{20}) = -10^{20}, \qquad \mathrm{fl}(2 - 10^{20}) = -10^{20}.$$

Die 1 und die 2 verschwinden vollständig – sie liegen 20 Größenordnungen unter dem Hauptterm.
Rückwärtseinsetzen:

$$x_2 = \frac{-10^{20}}{-10^{20}} = 1 \quad \checkmark$$

$$10^{-20} x_1 + 1 \cdot 1 = 1 \Longrightarrow 10^{-20} x_1 = 0 \Longrightarrow x_1 = \mathbf{0}.$$

Ergebnis: $x_1 = 0$ **statt** 1**. Relativer Fehler:** $100\,\%$**.**
Mit Zeilentausch. Wir tauschen zuerst:

$$\begin{pmatrix} 1 & 1 \\ 10^{-20} & 1 \end{pmatrix}, \qquad b = \begin{pmatrix} 2 \\ 1 \end{pmatrix}.$$

Multiplikator $\ell_{21} = 10^{-20}$. Zeile 2:

$$(10^{-20}, 1) - 10^{-20}(1, 1) = (0,\ 1 - 10^{-20}) \ \to\ \mathrm{fl} = (0, 1).$$

Rechte Seite: $1 - 10^{-20} \cdot 2 \to \mathrm{fl} = 1$.

$$x_2 = 1, \qquad x_1 + 1 = 2 \Longrightarrow x_1 = 1. \quad \checkmark$$

Volle Genauigkeit.
Und jetzt die Pointe: Die Konditionszahl dieser Matrix ist

$$\mathrm{cond}_\infty(A) \approx 4.$$

Das Problem ist bestens konditioniert! Der Fehler kam ausschließlich vom *Algorithmus*. Gauß-Elimination ohne Pivotisierung ist **instabil** – genau die Situation, die Kapitel 2 von der schlechten Kondition unterscheidet.

Spaltenpivotisierung

Vor dem k-ten Eliminationsschritt sucht man in der aktuellen Spalte k (ab Zeile k abwärts) das betragsgrößte Element und tauscht die zugehörige Zeile nach oben:

$$\left|a_{pk}^{(k)}\right| = \max_{i \geq k} \left|a_{ik}^{(k)}\right| \Longrightarrow \text{tausche Zeile } k \text{ und } p.$$

Warum das hilft: Nach dem Tausch ist $|\ell_{ik}| = |a_{ik}/a_{kk}| \leq 1$ für alle i. Die Multiplikatoren können nichts mehr verstärken – genau darin lag das Problem oben, wo $\ell_{21} = 10^{20}$ war.
Die resultierende Zerlegung lautet

$$PA = LR$$

mit einer **Permutationsmatrix** P, die die Zeilentausche protokolliert. Da $P^{-1} = P^{\mathsf{T}}$, löst man

$$Ax = b \iff PAx = Pb \iff LRx = Pb.$$

Man vertauscht also nur die Einträge der rechten Seite entsprechend.
Kosten: $\mathcal{O}(n^2)$ Vergleiche insgesamt – verschwindend gegenüber $\frac{2}{3}n^3$. **Pivotisierung ist praktisch gratis.**
Weitere Varianten. Bei der *vollständigen* Pivotisierung sucht man das Maximum in der gesamten Restmatrix und tauscht auch Spalten. Das ist beweisbar stabiler, kostet aber $\mathcal{O}(n^3)$ Vergleiche und wird deshalb kaum verwendet. In der Praxis genügt Spaltenpivotisierung fast immer.

Der Wachstumsfaktor – ein offenes Problem

Wilkinsons Analyse zeigt: Die Gauß-Elimination mit Spaltenpivotisierung hat den Rückwärtsfehler

$$\eta \lesssim 2n^3 \, \rho \, \varepsilon_{\mathrm{M}},$$

wobei

$$\rho = \frac{\max_{i,j,k} \left|a_{ij}^{(k)}\right|}{\max_{i,j} \left|a_{ij}\right|}$$

der **Wachstumsfaktor** ist – er misst, wie stark die Einträge während der Elimination anwachsen.
Die schlechte Nachricht: Wilkinson konstruierte Matrizen mit $\rho = 2^{n-1}$. Für $n = 60$ ist das $5 \cdot 10^{17}$ – der Algorithmus wäre unbrauchbar.
Die gute Nachricht: Diese Matrizen kommen in der Praxis nie vor. Empirisch ist ρ fast immer kleiner als n, oft im einstelligen Bereich.
Der offene Punkt: Eine befriedigende theoretische Erklärung dafür gibt es bis heute nicht – NICK TREFETHEN und ROBERT SCHREIBER lieferten 1990 statistische Argumente, und Arbeiten zur *Glättungsanalyse* (smoothed analysis) von SANKAR,

SPIELMAN und TENG ab 2006 zeigen, dass zufällig gestörte Matrizen kleines Wachstum haben. Eine vollständige Erklärung, warum die Praxis so viel besser ist als die Theorie, steht aus.
Es ist eines der bekanntesten ungelösten Rätsel der numerischen linearen Algebra[15] – und ein schönes Beispiel dafür, dass ein Verfahren jahrzehntelang zuverlässig funktionieren kann, ohne dass man ganz versteht, warum.

3.5 Cholesky-Zerlegung

Wenn die Matrix symmetrisch positiv definit ist

Viele Matrizen aus Anwendungen sind **symmetrisch positiv definit** (s.p.d.):

$$A = A^{\mathsf{T}} \quad \text{und} \quad x^{\mathsf{T}} A x > 0 \ \text{ für alle } x \neq 0.$$

Das trifft zu auf Steifigkeitsmatrizen der Strukturmechanik, Kovarianzmatrizen der Statistik (Band 4, Kapitel 8), Gram-Matrizen und die diskretisierte Laplace-Gleichung.
Für sie gibt es eine bessere Zerlegung.

Satz 3.4 (Cholesky). *Ist A symmetrisch positiv definit, so existiert genau eine untere Dreiecksmatrix L mit positiven Diagonaleinträgen und*

$$A = LL^{\mathsf{T}}.$$

Die Rechenvorschrift und woher sie kommt

Man schreibt $A = LL^{\mathsf{T}}$ komponentenweise aus. Der Eintrag (i, j) mit $i \geq j$ lautet

$$a_{ij} = \sum_{k=1}^{j} \ell_{ik} \ell_{jk}.$$

(Die Summe endet bei j, weil $\ell_{jk} = 0$ für $k > j$.)
Für $i = j$ (Diagonale):

$$a_{jj} = \sum_{k=1}^{j} \ell_{jk}^2 = \sum_{k=1}^{j-1} \ell_{jk}^2 + \ell_{jj}^2$$

$$\Longrightarrow \quad \ell_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} \ell_{jk}^2}.$$

Für $i > j$:

$$a_{ij} = \sum_{k=1}^{j-1} \ell_{ik} \ell_{jk} + \ell_{ij} \ell_{jj}$$

$$\Longrightarrow \quad \ell_{ij} = \frac{1}{\ell_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} \ell_{ik}\ell_{jk} \right).$$

Man arbeitet spaltenweise von links nach rechts; bei jedem Eintrag sind nur schon berechnete Werte nötig.

Drei Vorzüge gegenüber der LR-Zerlegung:

1. **Halber Aufwand:** $\frac{1}{3}n^3$ statt $\frac{2}{3}n^3$ Operationen – man nutzt die Symmetrie.
2. **Halber Speicher:** nur die untere Hälfte.
3. **Keine Pivotisierung nötig.** Das Verfahren ist ohne jeden Zeilentausch rückwärtsstabil. Das ist bemerkenswert und liegt daran, dass wegen $a_{jj} = \sum_k \ell_{jk}^2$ alle Einträge von L durch $\sqrt{a_{jj}}$ beschränkt sind – der Wachstumsfaktor ist automatisch 1.

Ein nützlicher Nebeneffekt: Das Verfahren ist ein *Test* auf positive Definitheit. Wird unter einer Wurzel eine negative Zahl verlangt, so war A nicht positiv definit. Das ist der schnellste bekannte Test – schneller als die Berechnung aller Eigenwerte.

Aufgabe 3.2. Bestimmen Sie die Cholesky-Zerlegung von

$$A = \begin{pmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{pmatrix}.$$

Lösung Schritt für Schritt.

Spalte 1.

$$\ell_{11} = \sqrt{a_{11}} = \sqrt{4} = 2.$$

$$\ell_{21} = \frac{a_{21}}{\ell_{11}} = \frac{12}{2} = 6, \qquad \ell_{31} = \frac{-16}{2} = -8.$$

Spalte 2.

$$\ell_{22} = \sqrt{a_{22} - \ell_{21}^2} = \sqrt{37 - 36} = \sqrt{1} = 1.$$

$$\ell_{32} = \frac{1}{\ell_{22}}\left(a_{32} - \ell_{31}\ell_{21}\right) = \frac{1}{1}\left(-43 - (-8)(6)\right) = -43 + 48 = 5.$$

Spalte 3.

$$\ell_{33} = \sqrt{a_{33} - \ell_{31}^2 - \ell_{32}^2} = \sqrt{98 - 64 - 25} = \sqrt{9} = 3.$$

Ergebnis.

$$L = \begin{pmatrix} 2 & 0 & 0 \\ 6 & 1 & 0 \\ -8 & 5 & 3 \end{pmatrix}.$$

Probe. Wir berechnen LL^T, Eintrag $(3,3)$:

$$(-8)^2 + 5^2 + 3^2 = 64 + 25 + 9 = 98\checkmark$$

Eintrag $(3,2)$:

$$(-8)(6) + 5 \cdot 1 + 3 \cdot 0 = -48 + 5 = -43\checkmark$$

Eintrag $(2,2)$: $6^2 + 1^2 = 37$ ✓
Nebenergebnis. Alle Wurzeln waren aus positiven Zahlen zu ziehen – A ist also positiv definit. Außerdem:

$$\det A = (\det L)^2 = (2 \cdot 1 \cdot 3)^2 = 36.$$

Die Determinante fällt gratis ab, und zwar in einer numerisch weit besseren Form als über die Leibniz-Formel.

Struktur ausnutzen – Bandmatrizen

Viele Matrizen aus Differentialgleichungen sind **Bandmatrizen**: $a_{ij} = 0$ für $|i-j| > m$ mit kleiner Bandbreite m.
Der entscheidende Satz: Bei der LR-Zerlegung ohne Pivotisierung bleibt die Bandstruktur erhalten – L und R haben dieselbe Bandbreite. Der Aufwand sinkt von $\frac{2}{3}n^3$ auf

$$\mathcal{O}(nm^2).$$

Beispiel: Für eine Tridiagonalmatrix ($m = 1$) ist der Aufwand $\mathcal{O}(n)$ – **linear**. Das zugehörige Verfahren heißt **Thomas-Algorithmus** und ist die Grundlage jeder Spline-Berechnung (Kapitel 9) und jedes eindimensionalen Differenzenverfahrens.
Zahlen: Für $n = 10^6$ und $m = 1$ sind das etwa $8 \cdot 10^6$ Operationen – Millisekunden, statt der 77 Tage der vollbesetzten Rechnung.

Pivotisierung zerstört die Bandstruktur teilweise

Zeilentausche vergrößern die Bandbreite von R auf bis zu $2m$. Deshalb ist es wertvoll, dass s.p.d.-Matrizen keine Pivotisierung brauchen: Bei ihnen bleibt die Struktur exakt erhalten.
Bei allgemein dünn besetzten (nicht bandförmigen) Matrizen ist das Problem größer: Die Elimination erzeugt **Fill-in** – neue Einträge an vorher leeren Stellen. Die Wahl einer Zeilen- und Spaltenordnung, die Fill-in minimiert, ist ein kombinatorisches Problem (es ist NP-schwer); praktisch verwendet man Heuristiken wie *minimum degree* oder *nested dissection*. Bibliotheken wie UMFPACK und MUMPS tun dies automatisch.

André-Louis Cholesky

ANDRÉ-LOUIS CHOLESKY (1875–1918) war französischer Artillerieoffizier und Geodät, kein Mathematiker im akademischen Sinn.
Er arbeitete für den *Service géographique de l'armée* an der Triangulation Frankreichs, Algeriens und Tunesiens. Dabei entstehen Ausgleichsprobleme mit vielen Unbekannten – Normalgleichungssysteme, deren Matrizen symmetrisch positiv definit sind. Cholesky entwickelte um 1910 sein Verfahren, um sie mit Handrechnung effizient zu lösen.
Er veröffentlichte es nie. Am 31. August 1918 fiel er, 43 Jahre alt, an der Westfront bei einem Angriff in Nordfrankreich.
Sein Kollege COMMANDANT BENOÎT fand die Notizen und veröffentlichte das Verfahren 1924 posthum im *Bulletin géodésique*, ausdrücklich unter Choleskys Namen. Es blieb zunächst weitgehend unbeachtet und wurde erst nach 1945 – vor allem durch JOHN TODD und die aufkommende Rechentechnik – allgemein bekannt.
Ein Manuskript Choleskys von 1910 tauchte 2004 in seinem Nachlass auf; es zeigt, dass er die Methode vollständig ausgearbeitet und ihre Vorzüge klar erkannt hatte.
Heute ist die Cholesky-Zerlegung eines der meistgenutzten Verfahren der Numerik – unter anderem in jeder Monte-Carlo-Simulation korrelierter Zufallsvariablen: Um Vektoren mit vorgegebener Kovarianzmatrix Σ zu erzeugen, berechnet man $\Sigma = LL^{\mathsf{T}}$ und setzt $x = Lz$ mit unabhängigen standardnormalverteilten z (Band 4, Kapitel 8).

4 QR-Zerlegung und Ausgleichsrechnung

4.1 Das Problem der kleinsten Quadrate

Überbestimmte Systeme

In der Praxis hat man meist *mehr Messungen als Unbekannte*. Ein System

$$Ax = b, \qquad A \in \mathbb{R}^{m \times n}, \quad m > n$$

hat dann im Allgemeinen *keine* Lösung: b liegt nicht im Bild von A, das nur ein n-dimensionaler Unterraum des $\mathbb{R}^m$ ist.
Man ändert deshalb die Frage. Statt $Ax = b$ zu fordern, sucht man dasjenige x, für das der Rest möglichst klein wird:

$$\min_{x \in \mathbb{R}^n} \|Ax - b\|_2^2 = \min_{x} \sum_{i=1}^{m} \left(\sum_{j} a_{ij} x_j - b_i \right)^2 .$$

Das ist die **Methode der kleinsten Quadrate**.

Warum ausgerechnet Quadrate?

Man könnte auch $\sum |r_i|$ minimieren oder $\max_i |r_i|$. Für Quadrate sprechen vier Gründe:

1. **Differenzierbarkeit.** Die Zielfunktion ist glatt; das Minimum findet man durch Nullsetzen des Gradienten. Bei Beträgen ist die Funktion an Knicken nicht differenzierbar.
2. **Lineare Lösung.** Die Bedingung erster Ordnung ist ein *lineares* System. Bei anderen Normen braucht man Optimierungsverfahren.
3. **Geometrie.** Die Quadratsumme ist die euklidische Norm; die Lösung ist eine *Orthogonalprojektion*. Das gibt dem Verfahren eine klare Anschauung.
4. **Statistik.** Sind die Messfehler unabhängig und normalverteilt mit gleicher Varianz, so ist die Kleinste-Quadrate-Lösung genau der *Maximum-Likelihood-Schätzer* (Band 4, Kapitel 14). Und nach dem Satz von Gauß–Markow ist sie unter allen linearen erwartungstreuen Schätzern die mit der kleinsten Varianz.

Der Preis: Empfindlichkeit gegen Ausreißer

Weil Abweichungen *quadratisch* eingehen, dominiert ein einzelner grober Messfehler die gesamte Anpassung. Ein Ausreißer mit dem zehnfachen Rest trägt hundertmal so viel bei.

Wo Ausreißer zu erwarten sind, verwendet man **robuste** Verfahren: die ℓ^1-Regression (minimiert Beträge), Hubers Verlustfunktion (quadratisch nahe null, linear außen) oder RANSAC. Das ist ein aktives Forschungsgebiet und in der Bildverarbeitung Standard.

Satz 4.1 (Normalgleichungen). *x^* minimiert $\|Ax - b\|_2$ genau dann, wenn*

$$A^\mathsf{T} A\, x^* = A^\mathsf{T} b.$$

Hat A vollen Spaltenrang, so ist $A^\mathsf{T} A$ symmetrisch positiv definit und x^* eindeutig.

Beweis. **Schritt 1 – Die Zielfunktion ausschreiben.**

$$\varphi(x) := \|Ax - b\|_2^2 = (Ax - b)^\mathsf{T}(Ax - b) = x^\mathsf{T} A^\mathsf{T} A x - 2b^\mathsf{T} A x + b^\mathsf{T} b.$$

(Der mittlere Term: $x^\mathsf{T} A^\mathsf{T} b$ und $b^\mathsf{T} A x$ sind Skalare und transponiert zueinander, also gleich.)

Schritt 2 – Gradient bilden. Mit den Regeln $\nabla(x^\mathsf{T} M x) = 2Mx$ für symmetrisches M und $\nabla(c^\mathsf{T} x) = c$:

$$\nabla\varphi(x) = 2A^\mathsf{T} A x - 2A^\mathsf{T} b.$$

Schritt 3 – Nullsetzen.

$$\nabla\varphi(x) = 0 \iff A^\mathsf{T} A x = A^\mathsf{T} b.$$

Schritt 4 – Es ist wirklich ein Minimum. Die Hesse-Matrix ist $2A^\mathsf{T} A$. Für beliebiges $y \neq 0$:

$$y^\mathsf{T} A^\mathsf{T} A y = (Ay)^\mathsf{T}(Ay) = \|Ay\|_2^2 \geq 0,$$

und bei vollem Spaltenrang ist $Ay \neq 0$, also > 0. Die Zielfunktion ist strikt konvex; der kritische Punkt ist das globale Minimum.

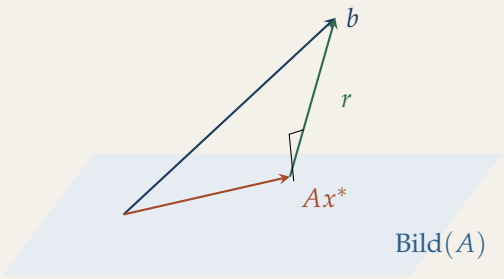
Schritt 5 – Die geometrische Deutung. Die Normalgleichungen lassen sich umschreiben als

$$A^\mathsf{T}(Ax^* - b) = 0 \iff A^\mathsf{T} r = 0,$$

mit dem Residuum $r = Ax^* - b$. Das heißt: **Das Residuum steht senkrecht auf allen Spalten von** A, also senkrecht auf dem Bildraum.

Das ist genau die Charakterisierung der Orthogonalprojektion: Ax^* ist der Punkt im Bildraum von A, der b am nächsten liegt. ∎

Die geometrische Anschauung



Die Kleinste-Quadrate-Lösung ist die *Orthogonalprojektion* von b auf den von den Spalten von A aufgespannten Unterraum. Der Residuenvektor steht senkrecht darauf – das ist die anschauliche Bedeutung der Normalgleichungen.

Warum man die Normalgleichungen trotzdem nicht benutzt

Sie sind theoretisch richtig und numerisch schlecht. Der Grund ist eine einzige Formel:

$$\operatorname{cond}_2(A^{\mathsf{T}}A) = \operatorname{cond}_2(A)^2.$$

Die Kondition wird quadriert.
Begründung: Die Singulärwerte von $A^{\mathsf{T}}A$ sind die Quadrate derer von A (Kapitel 6). Also

$$\operatorname{cond}_2(A^{\mathsf{T}}A) = \frac{\sigma_{\max}^2}{\sigma_{\min}^2} = \operatorname{cond}_2(A)^2.$$

Zahlenbeispiel: Ist $\operatorname{cond}_2(A) = 10^8$ – für Ausgleichsprobleme keineswegs ungewöhnlich –, so ist $\operatorname{cond}_2(A^{\mathsf{T}}A) = 10^{16}$. In `double` bleibt **keine einzige** signifikante Stelle übrig.
Ein extremes Beispiel (Läuchli): Sei

$$A = \begin{pmatrix} 1 & 1 \\ \varepsilon & 0 \\ 0 & \varepsilon \end{pmatrix} \Longrightarrow A^{\mathsf{T}}A = \begin{pmatrix} 1+\varepsilon^2 & 1 \\ 1 & 1+\varepsilon^2 \end{pmatrix}.$$

Ist $\varepsilon < \sqrt{\varepsilon_{\mathrm{M}}}$, so wird $\mathrm{fl}(1+\varepsilon^2) = 1$, und die Matrix wird singulär – obwohl A vollen Rang hat und gut konditioniert ist.
Die Lösung: nie $A^{\mathsf{T}}A$ bilden. Stattdessen arbeitet man direkt mit A – über die QR-Zerlegung.

4.2 Orthogonale Matrizen und warum sie ideal sind

Definition 4.2. $Q \in \mathbb{R}^{n\times n}$ heißt **orthogonal**, wenn $Q^{\mathsf{T}}Q = I$, also $Q^{-1} = Q^{\mathsf{T}}$.

Satz 4.3 (Normerhaltung). *Für orthogonales Q gilt*

$$\|Qx\|_2 = \|x\|_2 \quad \textit{für alle } x, \qquad \|QA\|_2 = \|A\|_2, \qquad \operatorname{cond}_2(QA) = \operatorname{cond}_2(A).$$

Beweis.

$$\|Qx\|_2^2 = (Qx)^\mathsf{T}(Qx) = x^\mathsf{T}Q^\mathsf{T}Qx = x^\mathsf{T}Ix = \|x\|_2^2.$$

Daraus folgen die übrigen Aussagen unmittelbar: Da Q keine Länge ändert, ändert sich auch das Verhältnis von größter zu kleinster Streckung nicht. ■

Warum das der Schlüssel zur Stabilität ist

Orthogonale Matrizen sind *Drehungen und Spiegelungen*. Sie verzerren nicht – sie haben $\operatorname{cond}_2 = 1$.
Die Konsequenz: Multipliziert man mit orthogonalen Matrizen, so werden vorhandene Fehler *nicht verstärkt*. Ein Algorithmus, der nur aus orthogonalen Transformationen besteht, ist automatisch rückwärtsstabil.
Das ist der Grund, warum QR-Zerlegung, Householder-Verfahren, Givens-Rotationen, der QR-Algorithmus für Eigenwerte und die Singulärwertzerlegung alle auf orthogonalen Matrizen aufbauen. **Es ist das Leitmotiv der gesamten numerischen linearen Algebra.**

Satz 4.4 (QR-Zerlegung). *Jede Matrix $A \in \mathbb{R}^{m\times n}$ mit $m \geq n$ lässt sich schreiben als*

$$A = QR$$

mit orthogonalem $Q \in \mathbb{R}^{m\times m}$ und

$$R = \begin{pmatrix} \hat{R} \\ 0 \end{pmatrix}, \qquad \hat{R} \in \mathbb{R}^{n\times n} \textit{ obere Dreiecksmatrix.}$$

Wie man damit das Ausgleichsproblem löst

Schritt 1. Nach Satz 4.3 ändert Q^T keine Norm:

$$\|Ax - b\|_2 = \left\|Q^\mathsf{T}(Ax - b)\right\|_2 = \left\|Rx - Q^\mathsf{T}b\right\|_2.$$

Schritt 2. Wir zerlegen $Q^\mathsf{T}b = \binom{c_1}{c_2}$ mit $c_1 \in \mathbb{R}^n$, $c_2 \in \mathbb{R}^{m-n}$. Dann

$$\left\|Rx - Q^\mathsf{T}b\right\|_2^2 = \|\hat{R}x - c_1\|_2^2 + \|c_2\|_2^2.$$

Schritt 3. Der zweite Summand hängt *nicht von x* ab – er ist unvermeidlich. Der erste lässt sich zu null machen:

$$\hat{R}x = c_1.$$

Ein Dreieckssystem, gelöst durch Rückwärtseinsetzen.
Schritt 4. Das minimale Residuum ist $\|c_2\|_2$ – man erhält es gratis mitgeliefert.

Und der entscheidende Punkt: Nirgends wurde $A^\mathsf{T}A$ gebildet. Die Kondition bleibt $\mathrm{cond}_2(A)$, nicht ihr Quadrat.

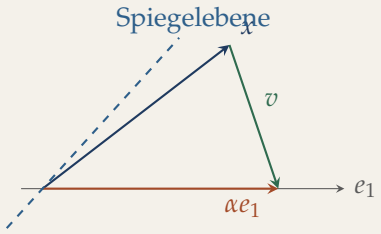
4.3 Householder-Spiegelungen

Die Idee

Man will einen Vektor x durch eine *Spiegelung* auf ein Vielfaches des ersten Einheitsvektors abbilden:

$$Hx = \alpha e_1.$$

Da Spiegelungen längentreu sind, muss $|\alpha| = \|x\|_2$ sein.



Die Spiegelebene halbiert den Winkel; ihr Normalenvektor ist $v = x - \alpha e_1$.

Definition 4.5. Für $v \neq 0$ heißt

$$H = I - 2\frac{vv^\mathsf{T}}{v^\mathsf{T}v}$$

die **Householder-Matrix** zu v.

Satz 4.6. *H ist symmetrisch und orthogonal, und $Hv = -v$, $Hw = w$ für alle $w \perp v$.*

Beweis. **Symmetrie:** vv^T ist symmetrisch, also auch H.

Orthogonalität: Mit $\beta := v^\mathsf{T}v$:

$$H^\mathsf{T}H = H^2 = \left(I - \frac{2vv^\mathsf{T}}{\beta}\right)^2 = I - \frac{4vv^\mathsf{T}}{\beta} + \frac{4v\overbrace{(v^\mathsf{T}v)}^{=\beta}v^\mathsf{T}}{\beta^2} = I - \frac{4vv^\mathsf{T}}{\beta} + \frac{4vv^\mathsf{T}}{\beta} = I.$$

Wirkung auf v:

$$Hv = v - \frac{2v(v^\mathsf{T}v)}{v^\mathsf{T}v} = v - 2v = -v.$$

Wirkung auf $w \perp v$: Da $v^\mathsf{T}w = 0$, ist $Hw = w - 0 = w$.

Das ist genau die geometrische Beschreibung einer Spiegelung an der Hyperebene $v^\perp$. ∎

Die richtige Wahl von v – und ein Vorzeichentrick

Man setzt

$$v = x + \operatorname{sign}(x_1)\,\|x\|_2\, e_1,$$

also

$$v = \begin{pmatrix} x_1 + \operatorname{sign}(x_1)\,\|x\|_2 \\ x_2 \\ \vdots \\ x_m \end{pmatrix}.$$

Warum das Vorzeichen so und nicht anders? Mathematisch funktionieren beide Vorzeichen. Numerisch gilt:

- Bei $v = x - \|x\|\,e_1$ und $x \approx \|x\|\,e_1$ (also wenn x schon fast in Richtung e_1 zeigt) entsteht in der ersten Komponente **Auslöschung**: $x_1 - \|x\| \approx 0$.
- Mit $\operatorname{sign}(x_1)$ werden stets zwei Zahlen *gleichen* Vorzeichens addiert. Keine Auslöschung.

Dieser winzige Vorzeichentrick ist der Unterschied zwischen einem stabilen und einem instabilen Verfahren – ein Musterbeispiel dafür, wie Numerik funktioniert.
Ein zweiter Kunstgriff: Man berechnet H nie explizit. Die Multiplikation HA schreibt man als

$$HA = A - \frac{2}{v^\mathsf{T} v}\, v\,(v^\mathsf{T} A),$$

also: erst das Zeilenvektor-Produkt $v^\mathsf{T} A$ ($\mathcal{O}(mn)$), dann ein Rang-1-Update ($\mathcal{O}(mn)$). Der Aufwand ist $\mathcal{O}(mn)$ statt $\mathcal{O}(m^2 n)$ für die explizite Matrixmultiplikation.

Aufgabe 4.1. Bestimmen Sie die QR-Zerlegung von

$$A = \begin{pmatrix} 3 & 1 \\ 4 & 2 \\ 0 & 5 \end{pmatrix}$$

mit einer Householder-Spiegelung für die erste Spalte.

Lösung Schritt für Schritt.

Schritt 1 – Erste Spalte und ihre Norm.

$$x = \begin{pmatrix} 3 \\ 4 \\ 0 \end{pmatrix}, \qquad \|x\|_2 = \sqrt{9 + 16 + 0} = 5.$$

Schritt 2 – Vektor v bestimmen. Da $x_1 = 3 > 0$, ist $\operatorname{sign}(x_1) = +1$:

$$v = x + 5e_1 = \begin{pmatrix} 3+5 \\ 4 \\ 0 \end{pmatrix} = \begin{pmatrix} 8 \\ 4 \\ 0 \end{pmatrix}.$$

$$v^\mathsf{T} v = 64 + 16 + 0 = 80.$$

Schritt 3 – Erste Spalte transformieren. Wir prüfen die Formel:

$$Hx = x - \frac{2(v^\mathsf{T} x)}{v^\mathsf{T} v} v.$$

$$v^\mathsf{T} x = 8 \cdot 3 + 4 \cdot 4 + 0 = 24 + 16 = 40.$$

$$\frac{2 \cdot 40}{80} = 1.$$

$$Hx = \begin{pmatrix} 3 \\ 4 \\ 0 \end{pmatrix} - 1 \cdot \begin{pmatrix} 8 \\ 4 \\ 0 \end{pmatrix} = \begin{pmatrix} -5 \\ 0 \\ 0 \end{pmatrix}. \quad \checkmark$$

Die Länge stimmt ($|-5| = 5$), und die unteren Einträge sind null – genau wie gewünscht.

Schritt 4 – Zweite Spalte transformieren.

$$a_2 = \begin{pmatrix} 1 \\ 2 \\ 5 \end{pmatrix}, \qquad v^\mathsf{T} a_2 = 8 \cdot 1 + 4 \cdot 2 + 0 \cdot 5 = 8 + 8 = 16.$$

$$\frac{2 \cdot 16}{80} = 0{,}4.$$

$$Ha_2 = \begin{pmatrix} 1 \\ 2 \\ 5 \end{pmatrix} - 0{,}4 \begin{pmatrix} 8 \\ 4 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 - 3{,}2 \\ 2 - 1{,}6 \\ 5 \end{pmatrix} = \begin{pmatrix} -2{,}2 \\ 0{,}4 \\ 5 \end{pmatrix}.$$

Schritt 5 – Zwischenstand.

$$H_1 A = \begin{pmatrix} -5 & -2{,}2 \\ 0 & 0{,}4 \\ 0 & 5 \end{pmatrix}.$$

Schritt 6 – Zweite Householder-Spiegelung. Jetzt ist der Teilvektor

$$x^{(2)} = \begin{pmatrix} 0{,}4 \\ 5 \end{pmatrix}, \qquad \left\| x^{(2)} \right\|_2 = \sqrt{0{,}16 + 25} = \sqrt{25{,}16} = 5{,}0160.$$

$$v^{(2)} = \begin{pmatrix} 0{,}4 + 5{,}0160 \\ 5 \end{pmatrix} = \begin{pmatrix} 5{,}4160 \\ 5 \end{pmatrix}, \qquad (v^{(2)})^\mathsf{T} v^{(2)} = 29{,}333 + 25 = 54{,}333.$$

$$(v^{(2)})^{\mathsf{T}} x^{(2)} = 5{,}4160 \cdot 0{,}4 + 5 \cdot 5 = 2{,}1664 + 25 = 27{,}166.$$

$$\frac{2 \cdot 27{,}166}{54{,}333} = 1{,}0000.$$

$$H_2 x^{(2)} = \begin{pmatrix} 0{,}4 \\ 5 \end{pmatrix} - \begin{pmatrix} 5{,}4160 \\ 5 \end{pmatrix} = \begin{pmatrix} -5{,}0160 \\ 0 \end{pmatrix}. \checkmark$$

Schritt 7 – Ergebnis.

$$\hat{R} = \begin{pmatrix} -5 & -2{,}2 \\ 0 & -5{,}0160 \end{pmatrix}, \qquad Q = H_1 H_2 \text{ (Produkt zweier Spiegelungen).}$$

Kontrolle über die Determinanten. Es ist $\left|\det \hat{R}\right| = 5 \cdot 5{,}0160 = 25{,}08$. Andererseits:

$$\det(A^{\mathsf{T}} A) = \det \begin{pmatrix} 25 & 11 \\ 11 & 30 \end{pmatrix} = 750 - 121 = 629,$$

und $\sqrt{629} = 25{,}08$. ✓

Bemerkung. Die Vorzeichen von R sind nicht eindeutig – man kann jede Zeile von R und die zugehörige Spalte von Q gleichzeitig mit -1 multiplizieren. Manche Bibliotheken normieren auf positive Diagonale, LAPACK tut es nicht.

Die drei Wege zur QR-Zerlegung im Vergleich

Verfahren	Aufwand	Stabilität	Einsatz
Gram–Schmidt (klassisch)	$2mn^2$	schlecht	nie – verliert Orthogonalität
Gram–Schmidt (modifiziert)	$2mn^2$	brauchbar	wenn Q spaltenweise gebraucht wird
Householder	$2mn^2 - \frac{2}{3}n^3$	sehr gut	**Standard** für volle Matrizen
Givens-Rotationen	$\approx \frac{4}{3}$-mal Householder	sehr gut	dünn besetzte Matrizen, Updates

Zum klassischen Gram–Schmidt. Das aus der linearen Algebra bekannte Verfahren

$$q_k = a_k - \sum_{j<k} (q_j^{\mathsf{T}} a_k) q_j, \qquad q_k \leftarrow q_k / \left\| q_k \right\|$$

ist mathematisch korrekt und numerisch unbrauchbar: Die berechneten q_k verlieren rapide ihre Orthogonalität. Bei $\operatorname{cond}_2(A) = 10^8$ kann $\|Q^\mathsf{T}Q - I\|$ die Größenordnung 1 erreichen – Q ist dann keine orthogonale Matrix mehr.
Die modifizierte Fassung zieht die Projektionen *nacheinander* ab, statt sie erst zu summieren:

$$\text{für } j = 1, \dots, k-1: \quad a_k \leftarrow a_k - (q_j^\mathsf{T} a_k) q_j.$$

Mathematisch identisch, numerisch deutlich besser: Der Orthogonalitätsverlust ist nur noch $\mathcal{O}(\varepsilon_\mathrm{M} \cdot \operatorname{cond}_2(A))$ statt $\mathcal{O}(\varepsilon_\mathrm{M} \cdot \operatorname{cond}_2(A)^2)$.
Householder dagegen liefert Orthogonalität bis auf $\mathcal{O}(\varepsilon_\mathrm{M})$ – unabhängig von der Kondition. Deshalb ist es der Standard.
Givens-Rotationen drehen jeweils in einer Koordinatenebene und annullieren gezielt einen einzelnen Eintrag:

$$G = \begin{pmatrix} c & s \\ -s & c \end{pmatrix}, \qquad c = \frac{a}{\sqrt{a^2+b^2}}, \quad s = \frac{b}{\sqrt{a^2+b^2}}.$$

Sie sind teurer als Householder, aber *lokal*: Man kann einzelne Einträge behandeln, ohne die ganze Matrix anzufassen. Deshalb sind sie bei dünn besetzten Matrizen und bei laufenden Aktualisierungen (etwa im Kalman-Filter) die Methode der Wahl.

Aufgabe 4.2. Passen Sie eine Gerade $y = \alpha + \beta t$ an die Messpunkte

$$(1;\ 2{,}1), \quad (2;\ 3{,}9), \quad (3;\ 6{,}2), \quad (4;\ 7{,}8)$$

im Sinne der kleinsten Quadrate an.

Lösung Schritt für Schritt.

Schritt 1 – Modell aufstellen.

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix}, \qquad x = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}, \qquad b = \begin{pmatrix} 2{,}1 \\ 3{,}9 \\ 6{,}2 \\ 7{,}8 \end{pmatrix}.$$

Schritt 2 – Normalgleichungen (hier zulässig, weil 2×2 und gut konditioniert).

$$A^\mathsf{T}A = \begin{pmatrix} 4 & \sum t_i \\ \sum t_i & \sum t_i^2 \end{pmatrix} = \begin{pmatrix} 4 & 10 \\ 10 & 30 \end{pmatrix}.$$

Zwischenrechnung: $\sum t_i = 1+2+3+4 = 10$; $\sum t_i^2 = 1+4+9+16 = 30$.

$$A^\mathsf{T}b = \begin{pmatrix} \sum y_i \\ \sum t_i y_i \end{pmatrix}.$$

$$\sum y_i = 2{,}1 + 3{,}9 + 6{,}2 + 7{,}8 = 20{,}0.$$

$$\sum t_i y_i = 1 \cdot 2{,}1 + 2 \cdot 3{,}9 + 3 \cdot 6{,}2 + 4 \cdot 7{,}8 = 2{,}1 + 7{,}8 + 18{,}6 + 31{,}2 = 59{,}7.$$

Schritt 3 – System lösen.

$$\begin{pmatrix} 4 & 10 \\ 10 & 30 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} 20{,}0 \\ 59{,}7 \end{pmatrix}.$$

Determinante: $4 \cdot 30 - 10 \cdot 10 = 120 - 100 = 20$.

$$\alpha = \frac{1}{20}(30 \cdot 20{,}0 - 10 \cdot 59{,}7) = \frac{600 - 597}{20} = \frac{3}{20} = 0{,}15.$$

$$\beta = \frac{1}{20}(4 \cdot 59{,}7 - 10 \cdot 20{,}0) = \frac{238{,}8 - 200}{20} = \frac{38{,}8}{20} = 1{,}94.$$

Ergebnis:

$$y = 0{,}15 + 1{,}94\, t.$$

Schritt 4 – Residuen berechnen.

t	y gemessen	$\hat{y}$ berechnet	$r = y - \hat{y}$
1	2,10	2,09	+0,01
2	3,90	4,03	−0,13
3	6,20	5,97	+0,23
4	7,80	7,91	−0,11

Schritt 5 – Zwei Kontrollen.
Erstens: Die Residuen müssen sich zu null summieren (weil die erste Spalte von A aus Einsen besteht und $A^{\mathsf{T}} r = 0$ gilt):

$$0{,}01 - 0{,}13 + 0{,}23 - 0{,}11 = 0{,}00. \checkmark$$

Zweitens: Auch $\sum t_i r_i$ muss null sein (zweite Spalte):

$$1(0{,}01) + 2(-0{,}13) + 3(0{,}23) + 4(-0{,}11) = 0{,}01 - 0{,}26 + 0{,}69 - 0{,}44 = 0{,}00. \checkmark$$

Diese beiden Proben decken fast jeden Rechenfehler auf.
Schritt 6 – Residuenquadratsumme.

$$\|r\|_2^2 = 0{,}0001 + 0{,}0169 + 0{,}0529 + 0{,}0121 = 0{,}0820.$$

Die Standardabweichung der Residuen ist

$$s = \sqrt{\frac{0{,}0820}{4 - 2}} = \sqrt{0{,}0410} = 0{,}2025$$

(mit $n - p = 4 - 2 = 2$ Freiheitsgraden – zwei gingen für die geschätzten Parameter verloren).

Gauß, Ceres und die kleinsten Quadrate

Am 1. Januar 1801 entdeckte GIUSEPPE PIAZZI in Palermo einen neuen Himmelskörper – den Kleinplaneten Ceres. Er konnte ihn 41 Tage lang beobachten, dann verschwand er hinter der Sonne.
Die Frage war: Wo würde er wieder auftauchen? Aus 41 Tagen Beobachtung – weniger als 3 % eines Umlaufs – eine Bahn zu bestimmen, galt als unmöglich. Die etablierten Astronomen scheiterten.
CARL FRIEDRICH GAUSS, damals 24 Jahre alt, entwickelte ein Verfahren, das alle Beobachtungen gleichzeitig ausnutzte und die Summe der quadrierten Abweichungen minimierte. Im Dezember 1801 fand FRANZ XAVER VON ZACH Ceres genau dort wieder, wo Gauß es vorhergesagt hatte – rund einen halben Grad von den Vorhersagen der anderen entfernt.
Der Erfolg machte Gauß über Nacht berühmt und begründete seinen Ruf als führender Mathematiker Europas.
Ein Prioritätsstreit. ADRIEN-MARIE LEGENDRE veröffentlichte die Methode 1805 zuerst. Gauß behauptete 1809, sie schon seit 1795 zu verwenden – was durch seine Tagebücher gestützt wird, aber Legendre tief kränkte. Der Streit blieb ungelöst; heute nennt man die Methode nach beiden.
Der bleibende Beitrag von Gauß war ohnehin ein anderer: Er lieferte 1809 die *statistische* Begründung. Er zeigte, dass die Methode genau dann optimal ist, wenn die Messfehler normalverteilt sind – und er führte die Normalverteilung überhaupt erst zu diesem Zweck ein. Der Satz von Gauß–Markow, der die Optimalität ohne Normalverteilungsannahme sichert, folgte 1821.
Eine kaum bekannte Pointe: Gauß löste die entstehenden Normalgleichungen nicht mit der „Gauß-Elimination“, wie man vermuten würde, sondern entwickelte für die geodätischen Anwendungen bereits ein *iteratives* Verfahren – das, was wir heute Gauß–Seidel-Verfahren nennen. In einem Brief von 1823 empfahl er es mit den Worten, man könne die Arbeit jederzeit unterbrechen und später fortsetzen, was bei Handrechnung ein erheblicher Vorzug war.

5 Iterative Verfahren für große Systeme

5.1 Warum direkte Verfahren irgendwann versagen

Ein Problem aus der Praxis

Die Wärmeleitungsgleichung auf einem Würfel, diskretisiert mit 100 Gitterpunkten je Richtung, hat

$$n = 100^3 = 10^6$$

Unbekannte. Die zugehörige Matrix ist *dünn besetzt*: Jede Zeile hat höchstens 7 von einer Million Einträgen ungleich null.

Direkte Lösung:

- Speicher für die volle Matrix: 10^{12} Zahlen $\cdot$ 8 Byte $=$ 8 Terabyte.
- Aufwand: $\frac{2}{3} \cdot 10^{18}$ Operationen $\approx$ 77 Tage.
- Und selbst bei Ausnutzung der Bandstruktur (Bandbreite $\approx 10^4$) noch $\mathcal{O}(nm^2) = 10^{14}$ Operationen.

Iterative Lösung mit dem CG-Verfahren:

- Speicher: nur die $7 \cdot 10^6$ Einträge ungleich null, plus einige Vektoren – unter 1 Gigabyte.
- Aufwand: einige hundert Iterationen zu je $\mathcal{O}(n)$ Operationen – Sekunden.

Das ist der Grund für dieses Kapitel. Für große dünn besetzte Systeme sind iterative Verfahren nicht schneller, sondern der einzige gangbare Weg.

Der grundlegende Unterschied

	Direkte Verfahren	**Iterative Verfahren**
Ergebnis nach	festem Aufwand	wählbarer Genauigkeit
Exaktheit	exakt (bis auf Rundung)	beliebig genau, nie exakt
Matrix wird	verändert	nur multipliziert
Braucht	$\mathcal{O}(n^3)$, $\mathcal{O}(n^2)$ Speicher	$\mathcal{O}(n)$ pro Schritt
Abbruch	am Ende	jederzeit möglich

Der letzte Punkt ist praktisch wichtig: Wenn die Eingabedaten ohnehin nur auf 10^{-3} genau sind, ist es Verschwendung, das System auf 10^{-16} zu lösen. Iterative Verfahren erlauben es, genau so genau zu rechnen, wie nötig.

5.2 Splitting-Verfahren: Jacobi und Gauß–Seidel

Das allgemeine Prinzip

Man zerlegt

$$A = M - N$$

mit einem *leicht invertierbaren* M. Dann ist

$$Ax = b \iff Mx = Nx + b \iff x = M^{-1}(Nx + b),$$

und man iteriert:

$$x^{(k+1)} = M^{-1}\big(Nx^{(k)} + b\big).$$

Das ist eine **Fixpunktiteration** – die Lösung ist ein Fixpunkt der Abbildung.
Die Standardzerlegung schreibt $A = D + L + U$ (Diagonale, strikt untere, strikt obere Dreiecksmatrix). Die klassischen Verfahren unterscheiden sich nur in der Wahl von M:

Verfahren	M	Auflösen kostet
Jacobi	D	$\mathcal{O}(n)$ – nur Divisionen
Gauß–Seidel	$D + L$	$\mathcal{O}(\text{nnz})$ – Vorwärtseinsetzen
SOR	$\frac{1}{\omega}D + L$	$\mathcal{O}(\text{nnz})$

(„nnz“ = Anzahl der Einträge ungleich null.)

Die Verfahren komponentenweise

Jacobi-Verfahren. Man löst die i-te Gleichung nach x_i auf und benutzt für alle anderen die *alten* Werte:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right).$$

Gauß–Seidel. Dasselbe, aber man benutzt die bereits *neu berechneten* Werte sofort:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right).$$

Der Vergleich:

- Jacobi braucht *zwei* Vektoren (alt und neu), Gauß–Seidel nur *einen* – man überschreibt.
- Jacobi ist perfekt parallelisierbar (alle Komponenten unabhängig), Gauß–Seidel inhärent sequenziell.
- Gauß–Seidel konvergiert meist etwa *doppelt so schnell* – es braucht ungefähr halb so viele Iterationen.

Auf modernen Parallelrechnern kehrt sich der zweite Punkt oft in einen Vorteil für Jacobi um. Es gibt Kompromisse – etwa Rot-Schwarz-Gauß–Seidel, bei dem die Gitterpunkte wie ein Schachbrett gefärbt und in zwei parallelen Durchgängen behandelt werden.

Satz 5.1 (Konvergenzkriterium). *Die Iteration $x^{(k+1)} = Tx^{(k)} + c$ mit $T = M^{-1}N$ konvergiert für jeden Startwert genau dann, wenn der* ***Spektralradius***

$$\rho(T) := \max_i |\lambda_i(T)|$$

kleiner als 1 ist. Die Konvergenz ist dann linear mit dem Faktor $\rho(T)$ pro Schritt.

Beweis. **Schritt 1 – Die Fehlergleichung.** Die exakte Lösung erfüllt $x = Tx + c$. Subtraktion von der Iterationsvorschrift ergibt für den Fehler $e^{(k)} := x^{(k)} - x$:

$$e^{(k+1)} = Te^{(k)} \Longrightarrow e^{(k)} = T^k e^{(0)}.$$

Schritt 2 – Wann geht $T^k \to 0$? Sei zunächst T diagonalisierbar, $T = S\Lambda S^{-1}$. Dann

$$T^k = S\Lambda^k S^{-1}, \qquad \Lambda^k = \operatorname{diag}(\lambda_1^k, \ldots, \lambda_n^k).$$

Es gilt $\lambda_i^k \to 0$ genau dann, wenn $|\lambda_i| < 1$. Also $T^k \to 0 \iff \rho(T) < 1$.

Schritt 3 – Der allgemeine Fall. Ist T nicht diagonalisierbar, verwendet man die Jordan-Normalform. Für einen Jordanblock der Größe m zum Eigenwert λ gilt

$$\left\| J^k \right\| \leq C k^{m-1} |\lambda|^k,$$

und für $|\lambda| < 1$ dominiert die geometrische Abnahme das polynomiale Wachstum. Also wieder $T^k \to 0$.

Schritt 4 – Notwendigkeit. Ist $|\lambda_j| \geq 1$ für ein j, so wählt man $e^{(0)}$ als zugehörigen Eigenvektor: Dann ist $\left\|e^{(k)}\right\| = |\lambda_j|^k \left\|e^{(0)}\right\| \not\to 0$. ■

Ein praktikables hinreichendes Kriterium

Der Spektralradius ist teuer zu berechnen. Ein einfaches hinreichendes Kriterium: **Ist A strikt diagonaldominant**, das heißt

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \qquad \text{für alle } i,$$

so konvergieren Jacobi- und Gauß–Seidel-Verfahren.
Beweisskizze für Jacobi: Die Iterationsmatrix ist $T = -D^{-1}(L+U)$ mit den Einträgen $t_{ij} = -a_{ij}/a_{ii}$ für $j \neq i$. Die Zeilensummennorm ist

$$\|T\|_\infty = \max_i \sum_{j \neq i} \frac{|a_{ij}|}{|a_{ii}|} < 1$$

nach Voraussetzung. Da stets $\rho(T) \leq \|T\|$ für jede induzierte Norm gilt, folgt $\rho(T) < 1$.
Ein zweites Kriterium: Ist A symmetrisch positiv definit, so konvergiert Gauß–Seidel immer (Satz von Ostrowski–Reich) – auch ohne Diagonaldominanz.

Aufgabe 5.1. Lösen Sie

$$\begin{pmatrix} 4 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 4 \end{pmatrix} x = \begin{pmatrix} 2 \\ 4 \\ 10 \end{pmatrix}$$

mit dem Jacobi- und dem Gauß–Seidel-Verfahren, Startwert $x^{(0)} = 0$. Vergleichen Sie.

Lösung Schritt für Schritt.

Vorprüfung – Diagonaldominanz.

$$\text{Zeile 1: } 4 > |-1| + 0 = 1\checkmark, \quad \text{Zeile 2: } 4 > 1 + 1 = 2\checkmark, \quad \text{Zeile 3: } 4 > 1\checkmark$$

Beide Verfahren konvergieren.
Die exakte Lösung (zur Kontrolle, per Elimination): $x = (1,\ 2,\ 3)^{\mathsf{T}}$.
Probe: $4 \cdot 1 - 1 \cdot 2 = 2\ \checkmark$; $-1 + 8 - 3 = 4\ \checkmark$; $-2 + 12 = 10\ \checkmark$

Teil 1 – Jacobi. Die Iterationsvorschrift lautet

$$x_1^{(k+1)} = \frac{2 + x_2^{(k)}}{4}, \qquad x_2^{(k+1)} = \frac{4 + x_1^{(k)} + x_3^{(k)}}{4}, \qquad x_3^{(k+1)} = \frac{10 + x_2^{(k)}}{4}.$$

Iteration 1 (aus $x^{(0)} = (0,0,0)$):

$$x_1 = \frac{2+0}{4} = 0{,}5, \quad x_2 = \frac{4+0+0}{4} = 1{,}0, \quad x_3 = \frac{10+0}{4} = 2{,}5.$$

Iteration 2 (aus $(0{,}5;\ 1{,}0;\ 2{,}5)$):

$$x_1 = \frac{2+1{,}0}{4} = 0{,}75, \quad x_2 = \frac{4+0{,}5+2{,}5}{4} = \frac{7}{4} = 1{,}75, \quad x_3 = \frac{10+1{,}0}{4} = 2{,}75.$$

Iteration 3 (aus $(0{,}75;\ 1{,}75;\ 2{,}75)$):

$$x_1 = \frac{3{,}75}{4} = 0{,}9375, \quad x_2 = \frac{4+0{,}75+2{,}75}{4} = \frac{7{,}5}{4} = 1{,}875, \quad x_3 = \frac{11{,}75}{4} = 2{,}9375.$$

Teil 2 – Gauß–Seidel. Jetzt werden neue Werte sofort verwendet:

$$x_1^{(k+1)} = \frac{2 + x_2^{(k)}}{4}, \qquad x_2^{(k+1)} = \frac{4 + x_1^{(k+1)} + x_3^{(k)}}{4}, \qquad x_3^{(k+1)} = \frac{10 + x_2^{(k+1)}}{4}.$$

Iteration 1:

$$x_1 = \frac{2+0}{4} = 0{,}5$$

$$x_2 = \frac{4 + \mathbf{0{,}5} + 0}{4} = \frac{4{,}5}{4} = 1{,}125 \qquad \text{(benutzt das neue } x_1\text{)}$$

$$x_3 = \frac{10 + \mathbf{1{,}125}}{4} = \frac{11{,}125}{4} = 2{,}78125.$$

Iteration 2:

$$x_1 = \frac{2+1{,}125}{4} = 0{,}78125$$

$$x_2 = \frac{4+0{,}78125+2{,}78125}{4} = \frac{7{,}5625}{4} = 1{,}890625$$

$$x_3 = \frac{10+1{,}890625}{4} = 2{,}97265625.$$

Iteration 3:

$$x_1 = \frac{3{,}890625}{4} = 0{,}97265625$$

$$x_2 = \frac{4+0{,}97265625+2{,}97265625}{4} = \frac{7{,}9453125}{4} = 1{,}986328$$

$$x_3 = \frac{11{,}986328}{4} = 2{,}996582.$$

Vergleich der Fehler (in der Maximumnorm, gegen $x = (1,2,3)$):

Iteration	Jacobi	Gauß–Seidel
1	1,00	0,875
2	0,25	0,219
3	0,125	0,0273
5	0,0156	0,00043
10	$1{,}2 \cdot 10^{-4}$	$2{,}7 \cdot 10^{-8}$

Auswertung. Der Fehler fällt bei Jacobi etwa mit dem Faktor 0,354 pro Schritt, bei Gauß–Seidel mit etwa 0,125. Es gilt $0{,}125 \approx 0{,}354^2$ – Gauß–Seidel braucht *halb so viele* Iterationen.

Theoretische Bestätigung. Für diese Tridiagonalmatrix ist der Spektralradius der Jacobi-Iterationsmatrix

$$\rho(T_J) = \frac{1}{2}\cos\frac{\pi}{4} = \frac{\sqrt{2}}{4} \approx 0{,}3536,$$

und für konsistent geordnete Matrizen (wozu Tridiagonalmatrizen zählen) gilt der Satz von YOUNG:

$$\rho(T_{GS}) = \rho(T_J)^2 = 0{,}125. \checkmark$$

Die beobachteten Werte stimmen exakt.

SOR – Überrelaxation

Die Idee: Gauß–Seidel bewegt sich in die richtige Richtung, aber zu vorsichtig. Man geht deshalb *weiter* als berechnet:

$$x_i^{(k+1)} = (1-\omega)x_i^{(k)} + \omega \cdot x_i^{(\mathrm{GS})}.$$

Für $\omega = 1$ ist das Gauß–Seidel; für $\omega > 1$ spricht man von **Überrelaxation** (SOR, successive over-relaxation).

Der Gewinn kann dramatisch sein. Für die diskretisierte Laplace-Gleichung auf einem $N \times N$-Gitter:

Verfahren	Iterationen für Fehler 10^{-6}	Skalierung
Jacobi	$\mathcal{O}(N^2)$	bei $N = 100$: $\approx 46\,000$
Gauß–Seidel	$\mathcal{O}(N^2)$, halb so viele	$\approx 23\,000$
SOR (optimal)	$\mathcal{O}(N)$	≈ 220

Ein Faktor 100 – allein durch einen richtig gewählten Parameter.

Das optimale ω ist für konsistent geordnete Matrizen

$$\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - \rho(T_J)^2}},$$

was stets zwischen 1 und 2 liegt.

> **Die Kurve ist unsymmetrisch**
>
> Die Konvergenzrate als Funktion von ω hat bei ω_{opt} einen *Knick*: Links davon fällt sie sanft ab, rechts steigt sie steil an. Ein zu großes ω ist also deutlich schlimmer als ein zu kleines.
> **Praktische Regel:** Im Zweifel ω etwas kleiner wählen als geschätzt. Und für $\omega \geq 2$ divergiert das Verfahren immer (Satz von Kahan).

Historisches. DAVID YOUNG entwickelte die Theorie 1950 in seiner Dissertation in Harvard, betreut von GARRETT BIRKHOFF. SOR war bis in die 1970er Jahre das Standardverfahren für partielle Differentialgleichungen; heute ist es meist durch Krylow-Verfahren und Mehrgittermethoden verdrängt, dient aber weiterhin als Glätter innerhalb dieser Verfahren (Kapitel 13).

5.3 Das Verfahren der konjugierten Gradienten

Ein völlig anderer Zugang: Minimierung

Sei A symmetrisch positiv definit. Dann ist die Lösung von $Ax = b$ äquivalent zur Minimierung von

$$\varphi(x) := \frac{1}{2} x^{\mathsf{T}} A x - b^{\mathsf{T}} x.$$

Warum: $\nabla\varphi(x) = Ax - b$. Nullsetzen gibt $Ax = b$. Und weil A positiv definit ist, ist φ strikt konvex – der kritische Punkt ist das globale Minimum.
Die geometrische Vorstellung: φ ist ein Paraboloid; seine Niveaumengen sind Ellipsoide mit Achsenverhältnis $\sqrt{\text{cond}_2(A)}$. Bei gut konditioniertem A sind es fast Kugeln, bei schlecht konditioniertem lange schmale Täler.
Der naive Ansatz – steilster Abstieg – geht in jedem Schritt in Richtung $-\nabla\varphi = r$ (dem Residuum). In einem schmalen Tal führt das zu einem *Zickzackkurs*: Man springt von Wand zu Wand und kommt nur langsam voran. Die Konvergenzrate ist

$$\frac{\text{cond}_2(A) - 1}{\text{cond}_2(A) + 1},$$

also für cond $= 10^4$ etwa 0,9998 pro Schritt – praktisch Stillstand.

Die Idee des CG-Verfahrens

Der Fehler des steilsten Abstiegs ist, dass aufeinanderfolgende Schritte einander teilweise *rückgängig* machen.
Abhilfe: Man wählt Suchrichtungen $p_0, p_1, \ldots$, die **A-konjugiert** (oder A-orthogonal) sind:

$$p_i^\mathsf{T} A p_j = 0 \qquad \text{für } i \neq j.$$

Was das bewirkt: Bezüglich des Skalarprodukts $\langle u, v \rangle_A := u^\mathsf{T} A v$ stehen die Richtungen senkrecht aufeinander. In diesen Koordinaten ist das Ellipsoid eine Kugel, und die Minimierung zerfällt in n *unabhängige* eindimensionale Probleme. Ein Schritt in Richtung p_i verdirbt nichts, was in Richtung p_j schon erreicht wurde.
Folgerung: Nach höchstens n Schritten ist man exakt am Minimum. *CG ist also im Prinzip ein direktes Verfahren* – man benutzt es aber iterativ, weil es meist schon nach viel weniger Schritten genau genug ist.

Der CG-Algorithmus

Initialisierung: x_0 beliebig (meist 0), $r_0 = b - Ax_0$, $p_0 = r_0$.
Für $k = 0, 1, 2, \ldots$**:**

$$\begin{aligned}
\alpha_k &= \frac{r_k^\mathsf{T} r_k}{p_k^\mathsf{T} A p_k} && \text{Schrittweite (exakte Liniensuche)} \\
x_{k+1} &= x_k + \alpha_k p_k && \text{Schritt ausführen} \\
r_{k+1} &= r_k - \alpha_k A p_k && \text{Residuum aktualisieren} \\
\beta_k &= \frac{r_{k+1}^\mathsf{T} r_{k+1}}{r_k^\mathsf{T} r_k} && \text{Korrekturfaktor} \\
p_{k+1} &= r_{k+1} + \beta_k p_k && \text{neue Suchrichtung}
\end{aligned}$$

Das Bemerkenswerte an dieser Vorschrift:

- Pro Iteration ist *genau eine* Matrix-Vektor-Multiplikation nötig (Ap_k).
- Man speichert nur vier Vektoren – unabhängig von k.
- Die A-Konjugiertheit *aller* bisherigen Richtungen ergibt sich automatisch aus der einfachen Zweitermrekursion. Das ist ein kleines Wunder und beruht darauf, dass die Krylow-Räume ineinander geschachtelt sind.
- Die Matrix A muss nicht explizit vorliegen – es genügt eine Routine, die $v \mapsto Av$ berechnet. Das nennt man **matrixfrei** und ist bei sehr großen Problemen entscheidend.

Satz 5.2 (Konvergenz von CG). *Für den Fehler in der A-Norm $\|e\|_A := \sqrt{e^\mathsf{T} A e}$ gilt*

$$\frac{\|e_k\|_A}{\|e_0\|_A} \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k, \qquad \kappa := \operatorname{cond}_2(A).$$

Warum die Wurzel den Unterschied macht

Vergleichen wir mit dem steilsten Abstieg:

κ	Steilster Abstieg	CG
10^2	0,980	0,818
10^4	0,9998	0,980
10^6	0,999998	0,998

Die Zahl der Iterationen für eine feste Genauigkeit skaliert

- beim steilsten Abstieg mit $\mathcal{O}(\kappa)$,
- bei CG mit $\mathcal{O}(\sqrt{\kappa})$.

Bei $\kappa = 10^6$ ist das der Unterschied zwischen einer Million und tausend Iterationen.
Und noch besser: Die Schranke ist oft pessimistisch. CG konvergiert besonders schnell, wenn die Eigenwerte *geclustert* sind – liegen sie in wenigen Häufungen, so genügen fast so viele Iterationen, wie es Cluster gibt. Genau das ist der Ansatzpunkt der Vorkonditionierung.

Vorkonditionierung – der wichtigste Hebel

Statt $Ax = b$ löst man

$$M^{-1}Ax = M^{-1}b$$

mit einer Matrix $M \approx A$, die leicht zu invertieren ist. Ist $M^{-1}A$ besser konditioniert als A, so konvergiert das Verfahren schneller.
Der Zielkonflikt:

- $M = I$: nichts gewonnen, nichts gekostet.
- $M = A$: perfekte Kondition ($\kappa = 1$), aber man müsste A invertieren – das ist das ursprüngliche Problem.

Man sucht also einen Kompromiss: M nah genug an A, um zu helfen, und einfach genug, um billig zu sein.
Gebräuchliche Vorkonditionierer:

Verfahren	Beschreibung
Jacobi (diagonal)	$M = \operatorname{diag}(A)$. Trivial, hilft bei stark unterschiedlichen Skalen.
Unvollständige Cholesky	$A \approx \tilde{L}\tilde{L}^\mathsf{T}$, wobei Fill-in verworfen wird. Sehr verbreitet.
Unvollständige LR (ILU)	Analog für unsymmetrische Matrizen.
Algebraisches Mehrgitter	Für elliptische Probleme oft optimal, $\mathcal{O}(n)$.
Gebietszerlegung	Für Parallelrechner; das Gebiet wird aufgeteilt, lokal exakt gelöst.

Die praktische Erfahrung: Bei realen Problemen entscheidet fast immer der Vorkonditionierer, nicht das Krylow-Verfahren. Der Unterschied zwischen CG und GMRES ist oft ein Faktor 2, der Unterschied zwischen gutem und schlechtem Vorkonditionierer ein Faktor 1000.

Krylow-Räume – der gemeinsame Rahmen

CG und seine Verwandten gehören zur Familie der **Krylow-Unterraumverfahren**. Der k-te Krylow-Raum ist

$$\mathcal{K}_k(A, r_0) = \operatorname{span}\{r_0,\ Ar_0,\ A^2 r_0,\ \ldots,\ A^{k-1} r_0\}.$$

Alle Verfahren suchen ihre Näherung in $x_0 + \mathcal{K}_k$; sie unterscheiden sich darin, *welche* Näherung sie daraus wählen.

Verfahren	Für Matrizen	Optimalitätseigenschaft
CG	s.p.d.	minimiert $\|e_k\|_A$; kurze Rekursion
MINRES	symmetrisch	minimiert $\|r_k\|_2$; kurze Rekursion
GMRES	beliebig	minimiert $\|r_k\|_2$; lange Rekursion
BiCGStab	beliebig	kurze Rekursion, keine Optimalität

Das Dilemma bei unsymmetrischen Matrizen. GMRES ist optimal, muss aber *alle* bisherigen Suchrichtungen speichern – Speicher und Aufwand wachsen mit k. In

der Praxis startet man deshalb periodisch neu („GMRES(m)“), was die Optimalität und manchmal die Konvergenz zerstört.
BiCGStab hingegen hat eine kurze Rekursion wie CG, kann aber unregelmäßig konvergieren oder zusammenbrechen.
FABER und MANTEUFFEL bewiesen 1984, dass dieses Dilemma *unvermeidlich* ist: Für allgemeine Matrizen gibt es kein Verfahren, das zugleich optimal ist und mit kurzer Rekursion auskommt. Nur die Symmetrie von A macht CG möglich.
Historisches. MAGNUS HESTENES und EDUARD STIEFEL entwickelten CG 1952 unabhängig voneinander und veröffentlichten gemeinsam. Sie hielten es zunächst für ein direktes Verfahren; dass es als iteratives Verfahren mit Vorkonditionierung seine eigentliche Stärke entfaltet, erkannte man erst in den 1970er Jahren durch JOHN REID. Heute steht CG mit Vorkonditionierung im Kern praktisch jedes Simulationsprogramms[24].

Aufgabe 5.2. Führen Sie zwei CG-Schritte für

$$A = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix}, \qquad b = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad x_0 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

aus und prüfen Sie, dass nach zwei Schritten die exakte Lösung erreicht ist.

Lösung Schritt für Schritt.

Vorbereitung – exakte Lösung. $\det A = 12 - 1 = 11$.

$$x^* = \frac{1}{11}\begin{pmatrix} 3 & -1 \\ -1 & 4 \end{pmatrix}\begin{pmatrix} 1 \\ 2 \end{pmatrix} = \frac{1}{11}\begin{pmatrix} 3-2 \\ -1+8 \end{pmatrix} = \begin{pmatrix} 1/11 \\ 7/11 \end{pmatrix} = \begin{pmatrix} 0{,}09091 \\ 0{,}63636 \end{pmatrix}.$$

Schritt 0 – Initialisierung.

$$Ax_0 = \begin{pmatrix} 4\cdot 2 + 1\cdot 1 \\ 1\cdot 2 + 3\cdot 1 \end{pmatrix} = \begin{pmatrix} 9 \\ 5 \end{pmatrix}$$

$$r_0 = b - Ax_0 = \begin{pmatrix} 1-9 \\ 2-5 \end{pmatrix} = \begin{pmatrix} -8 \\ -3 \end{pmatrix} = p_0.$$

$$r_0^{\mathsf{T}} r_0 = 64 + 9 = 73.$$

Schritt 1.

$$Ap_0 = \begin{pmatrix} 4(-8) + 1(-3) \\ 1(-8) + 3(-3) \end{pmatrix} = \begin{pmatrix} -35 \\ -17 \end{pmatrix}.$$

$$p_0^{\mathsf{T}} A p_0 = (-8)(-35) + (-3)(-17) = 280 + 51 = 331.$$

$$\alpha_0 = \frac{73}{331} = 0{,}220544.$$

$$x_1 = \begin{pmatrix}2\\1\end{pmatrix} + 0{,}220544\begin{pmatrix}-8\\-3\end{pmatrix} = \begin{pmatrix}2-1{,}764350\\1-0{,}661631\end{pmatrix} = \begin{pmatrix}0{,}235650\\0{,}338369\end{pmatrix}.$$

$$r_1 = r_0 - \alpha_0 A p_0 = \begin{pmatrix}-8\\-3\end{pmatrix} - 0{,}220544\begin{pmatrix}-35\\-17\end{pmatrix} = \begin{pmatrix}-8+7{,}719033\\-3+3{,}749245\end{pmatrix} = \begin{pmatrix}-0{,}280967\\0{,}749245\end{pmatrix}.$$

Kontrolle der Orthogonalität: $r_1^{\mathsf{T}} r_0 = (-0{,}280967)(-8) + (0{,}749245)(-3) = 2{,}24774 - 2{,}24774 = 0$ ✓
Das ist eine wesentliche Eigenschaft von CG: Die Residuen sind paarweise orthogonal.
Schritt 2.

$$r_1^{\mathsf{T}} r_1 = 0{,}078943 + 0{,}561368 = 0{,}640311.$$

$$\beta_0 = \frac{0{,}640311}{73} = 0{,}0087714.$$

$$p_1 = r_1 + \beta_0 p_0 = \begin{pmatrix}-0{,}280967\\0{,}749245\end{pmatrix} + 0{,}0087714\begin{pmatrix}-8\\-3\end{pmatrix} = \begin{pmatrix}-0{,}351138\\0{,}722931\end{pmatrix}.$$

Kontrolle der A-Konjugiertheit:

$$Ap_1 = \begin{pmatrix}4(-0{,}351138)+0{,}722931\\-0{,}351138+3(0{,}722931)\end{pmatrix} = \begin{pmatrix}-0{,}681621\\1{,}817655\end{pmatrix}.$$

$$p_0^{\mathsf{T}} A p_1 = (-8)(-0{,}681621) + (-3)(1{,}817655) = 5{,}452968 - 5{,}452965 \approx 0.\checkmark$$

$$\begin{aligned} p_1^{\mathsf{T}} A p_1 &= (-0{,}351138)(-0{,}681621) + (0{,}722931)(1{,}817655)\\ &= 0{,}239344 + 1{,}313944 = 1{,}553288.\end{aligned}$$

$$\alpha_1 = \frac{0{,}640311}{1{,}553288} = 0{,}412229.$$

$$\begin{aligned} x_2 &= \begin{pmatrix}0{,}235650\\0{,}338369\end{pmatrix} + 0{,}412229\begin{pmatrix}-0{,}351138\\0{,}722931\end{pmatrix}\\ &= \begin{pmatrix}0{,}235650-0{,}144749\\0{,}338369+0{,}298000\end{pmatrix} = \begin{pmatrix}0{,}090901\\0{,}636369\end{pmatrix}.\end{aligned}$$

Vergleich mit der exakten Lösung:

$$x^* = \begin{pmatrix}0{,}090909\\0{,}636364\end{pmatrix}.$$

Übereinstimmung auf fünf Stellen – die Abweichung stammt allein aus der Rundung meiner Zwischenrechnung auf sechs Stellen. **Nach $n = 2$ Schritten ist CG exakt.** ✓

6 Eigenwerte und Singulärwerte

6.1 Warum man das charakteristische Polynom nicht benutzt

Der Weg aus der Vorlesung ist numerisch verboten

In der linearen Algebra lernt man: Eigenwerte sind die Nullstellen von

$$p(\lambda) = \det(A - \lambda I).$$

Numerisch ist das aus drei Gründen unbrauchbar:
Erstens: schlechte Kondition. Wir haben in Kapitel 2 gesehen, dass Polynomnullstellen extrem empfindlich gegen Koeffizientenstörungen sind (Wilkinsons Polynom). Der Umweg über die Koeffizienten wirft Information weg und macht ein gut konditioniertes Problem schlecht konditioniert.
Zweitens: Aufwand. Die Determinante symbolisch zu entwickeln kostet $\mathcal{O}(n!)$.
Drittens: es gibt keine Formel. Nach dem Satz von ABEL und GALOIS gibt es für Polynome vom Grad ≥ 5 keine Lösungsformel in Radikalen (Band 1). Da jede Matrix Begleitmatrix eines beliebigen Polynoms ist, folgt: **Jedes Eigenwertverfahren muss iterativ sein.** Das ist kein technischer Mangel, sondern eine algebraische Notwendigkeit.
Eine Ironie am Rande. Der übliche Weg, Polynomnullstellen numerisch zu bestimmen – etwa in `numpy.roots` –, ist genau die Umkehrung: Man baut die Begleitmatrix und berechnet ihre Eigenwerte mit dem QR-Algorithmus. Man geht also *vom Polynom zur Matrix*, nicht umgekehrt.

6.2 Die Potenzmethode

Die Idee

Man multipliziert einen beliebigen Startvektor immer wieder mit A. Der Eigenvektor zum betragsgrößten Eigenwert setzt sich durch.
Warum: Sei A diagonalisierbar mit Eigenvektoren $v_1, \ldots, v_n$ und

$$|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_n|.$$

Man entwickelt den Startvektor:

$$x_0 = c_1 v_1 + c_2 v_2 + \cdots + c_n v_n.$$

Dann ist

$$A^k x_0 = c_1 \lambda_1^k v_1 + c_2 \lambda_2^k v_2 + \cdots = \lambda_1^k \left(c_1 v_1 + c_2 \left(\frac{\lambda_2}{\lambda_1} \right)^k v_2 + \dots \right).$$

Da $|\lambda_2/\lambda_1| < 1$, verschwinden alle Terme außer dem ersten – geometrisch mit der Rate $|\lambda_2/\lambda_1|$.

Der Algorithmus

Wähle x_0 mit $\|x_0\| = 1$.
Für $k = 0, 1, 2, \cdots$:

$$y_{k+1} = A x_k \qquad \text{(eine Matrix-Vektor-Multiplikation)}$$
$$x_{k+1} = \frac{y_{k+1}}{\|y_{k+1}\|} \qquad \text{(normieren!)}$$
$$\mu_{k+1} = x_{k+1}^{\mathsf{T}} A x_{k+1} \qquad \text{(Rayleigh-Quotient)}$$

Warum man normieren muss: Ohne Normierung wächst $\|A^k x_0\|$ wie $|\lambda_1|^k$ – bei $\lambda_1 = 10$ und $k = 400$ ist das 10^{400}, also Überlauf. Bei $|\lambda_1| < 1$ droht Unterlauf. Die Normierung hält alles im darstellbaren Bereich, ohne die Richtung zu ändern.
Der Rayleigh-Quotient ist die beste Schätzung des Eigenwerts zu einem gegebenen Vektor: Er minimiert $\|Ax - \mu x\|_2$ über μ. Bei symmetrischen Matrizen ist er sogar *quadratisch* genau – ein Richtungsfehler ε im Eigenvektor ergibt nur einen Eigenwertfehler $\mathcal{O}(\varepsilon^2)$.

Aufgabe 6.1. Bestimmen Sie den betragsgrößten Eigenwert von

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$$

mit der Potenzmethode, Startvektor $x_0 = (1, 0)^{\mathsf{T}}$.

Lösung Schritt für Schritt.

Vorbereitung – die exakten Werte.

$$\det(A - \lambda I) = (2 - \lambda)(3 - \lambda) - 1 = \lambda^2 - 5\lambda + 5.$$

$$\lambda_{1,2} = \frac{5 \pm \sqrt{25 - 20}}{2} = \frac{5 \pm \sqrt{5}}{2}.$$

$$\lambda_1 = \frac{5 + 2{,}236068}{2} = 3{,}618034, \qquad \lambda_2 = \frac{5 - 2{,}236068}{2} = 1{,}381966.$$

Konvergenzrate zu erwarten:

$$\left|\frac{\lambda_2}{\lambda_1}\right| = \frac{1{,}381966}{3{,}618034} = 0{,}381966.$$

Iteration 1.

$$y_1 = A\begin{pmatrix}1\\0\end{pmatrix} = \begin{pmatrix}2\\1\end{pmatrix}, \qquad \|y_1\| = \sqrt{5} = 2{,}236068.$$

$$x_1 = \begin{pmatrix}0{,}894427\\0{,}447214\end{pmatrix}.$$

Rayleigh-Quotient:

$$Ax_1 = \begin{pmatrix}2(0{,}894427)+0{,}447214\\0{,}894427+3(0{,}447214)\end{pmatrix} = \begin{pmatrix}2{,}236068\\2{,}236069\end{pmatrix}.$$

$$\mu_1 = 0{,}894427\cdot 2{,}236068 + 0{,}447214\cdot 2{,}236069 = 2{,}0+1{,}0 = 3{,}000000.$$

Iteration 2.

$$y_2 = \begin{pmatrix}2{,}236068\\2{,}236069\end{pmatrix}, \qquad \|y_2\| = 3{,}162279.$$

$$x_2 = \begin{pmatrix}0{,}707107\\0{,}707107\end{pmatrix}.$$

$$Ax_2 = \begin{pmatrix}2{,}121320\\2{,}828427\end{pmatrix}, \qquad \mu_2 = 0{,}707107(2{,}121320+2{,}828427) = 3{,}500000.$$

Iteration 3.

$$\|Ax_2\| = \sqrt{4{,}5+8{,}0} = \sqrt{12{,}5} = 3{,}535534, \qquad x_3 = \begin{pmatrix}0{,}600000\\0{,}800000\end{pmatrix}.$$

$$Ax_3 = \begin{pmatrix}2{,}0\\3{,}0\end{pmatrix}, \qquad \mu_3 = 0{,}6\cdot 2{,}0 + 0{,}8\cdot 3{,}0 = 1{,}2+2{,}4 = 3{,}600000.$$

Iteration 4.

$$\|Ax_3\| = \sqrt{4+9} = 3{,}605551, \qquad x_4 = \begin{pmatrix}0{,}554700\\0{,}832050\end{pmatrix}.$$

$$Ax_4 = \begin{pmatrix}1{,}941451\\3{,}050851\end{pmatrix}, \qquad \mu_4 = 1{,}076923+2{,}538461 = 3{,}615385.$$

Konvergenzverlauf.

k	μ_k	Fehler $\lvert\mu_k - \lambda_1\rvert$
1	3,000000	$6{,}18 \cdot 10^{-1}$
2	3,500000	$1{,}18 \cdot 10^{-1}$
3	3,600000	$1{,}80 \cdot 10^{-2}$
4	3,615385	$2{,}65 \cdot 10^{-3}$
5	3,617647	$3{,}87 \cdot 10^{-4}$
6	3,617977	$5{,}64 \cdot 10^{-5}$

Auswertung. Der Fehler fällt jeweils etwa mit dem Faktor 0,146. Und

$$0{,}146 \approx 0{,}381966^2 = 0{,}1459.\checkmark$$

Die *quadratische* Rate bestätigt die Aussage über den Rayleigh-Quotienten bei symmetrischen Matrizen: Der Eigenvektor konvergiert mit $|\lambda_2/\lambda_1|$, der Eigenwert mit dem Quadrat.
Der Eigenvektor konvergiert gegen

$$v_1 = \begin{pmatrix} 0{,}525731 \\ 0{,}850651 \end{pmatrix},$$

was dem goldenen Schnitt entspricht – kein Zufall, denn $\lambda_1 = \frac{5+\sqrt{5}}{2}$ hängt mit $\Phi = \frac{1+\sqrt{5}}{2}$ zusammen.

Drei Varianten

Inverse Iteration. Wendet man die Potenzmethode auf A^{-1} an, so konvergiert sie gegen den *betragskleinsten* Eigenwert. Man berechnet natürlich nicht A^{-1}, sondern löst in jedem Schritt

$$Ay_{k+1} = x_k$$

– mit einer einmal berechneten LR-Zerlegung kostet das nur $\mathcal{O}(n^2)$ je Schritt.
Inverse Iteration mit Verschiebung. Wendet man sie auf $(A - \sigma I)^{-1}$ an, so konvergiert sie gegen den Eigenwert, der σ am nächsten liegt – mit der Rate

$$\left| \frac{\lambda_{\text{nah}} - \sigma}{\lambda_{\text{zweitnah}} - \sigma} \right|.$$

Bei guter Schätzung σ ist das winzig; oft genügen zwei bis drei Iterationen.

Ein scheinbares Problem, das keines ist

Ist σ sehr nah an einem Eigenwert, so ist $A - \sigma I$ fast singulär – cond ist riesig. Müsste die Rechnung nicht explodieren?
Sie tut es auch: Der berechnete Vektor y_{k+1} hat einen großen Fehler. **Aber der Fehler zeigt fast genau in Richtung des gesuchten Eigenvektors** – denn genau diese Richtung wird von der fast singulären Matrix am stärksten verstärkt. Nach der Normierung bleibt davon nichts übrig.
Wilkinson analysierte das im Detail und zeigte, dass inverse Iteration gerade wegen der schlechten Kondition so gut funktioniert. Es ist eines der schönsten Beispiele dafür, dass eine große Konditionszahl nicht automatisch ein schlechtes Ergebnis bedeutet – es kommt darauf an, *was* man mit dem Ergebnis vorhat.

Rayleigh-Quotienten-Iteration. Man aktualisiert die Verschiebung in jedem Schritt: $\sigma_k = \mu_k$. Die Konvergenz ist dann *kubisch* bei symmetrischen Matrizen – die Zahl der richtigen Stellen verdreifacht sich pro Schritt. Der Preis: In jedem Schritt ist eine neue Zerlegung nötig, also $\mathcal{O}(n^3)$.

6.3 Der QR-Algorithmus

Das Verfahren, das alle Eigenwerte auf einmal liefert

Der Algorithmus ist verblüffend kurz:

$$
\begin{aligned}
&A_0 := A \\
&\text{Für } k = 0, 1, 2, \cdots : \\
&\qquad \text{Zerlege } A_k = Q_k R_k \quad \text{(QR-Zerlegung)} \\
&\qquad \text{Setze } A_{k+1} := R_k Q_k \quad \text{(Faktoren vertauschen!)}
\end{aligned}
$$

Das ist alles. Zerlegen, Reihenfolge tauschen, wiederholen.
Erste Beobachtung: Die Eigenwerte ändern sich nicht. Aus $A_k = Q_k R_k$ folgt $R_k = Q_k^{\mathsf{T}} A_k$, also

$$A_{k+1} = R_k Q_k = Q_k^{\mathsf{T}} A_k Q_k.$$

Das ist eine *Ähnlichkeitstransformation* – und zwar mit einer orthogonalen Matrix, also besonders gutartig. Alle A_k haben dieselben Eigenwerte wie A.
Zweite Beobachtung: Die Folge konvergiert gegen eine Dreiecksmatrix. Unter milden Voraussetzungen streben die Einträge unterhalb der Diagonale gegen null, und auf der Diagonale stehen die Eigenwerte – der Größe nach geordnet.

Warum das funktioniert. Der Zusammenhang zur Potenzmethode ist eng: Man kann zeigen, dass

$$A^k = (Q_0 Q_1 \cdots Q_{k-1})(R_{k-1} \cdots R_1 R_0),$$

der QR-Algorithmus also implizit die QR-Zerlegung von A^k berechnet. Die erste Spalte von $Q_0 \cdots Q_{k-1}$ ist genau das, was die Potenzmethode liefert; die weiteren Spalten machen dasselbe für die übrigen Eigenrichtungen gleichzeitig, unter Beibehaltung der Orthogonalität. Der QR-Algorithmus ist also *simultane Potenzmethode für alle Eigenvektoren auf einmal*.

Zwei Kunstgriffe, die es praktikabel machen

Der naive QR-Algorithmus kostet $\mathcal{O}(n^3)$ pro Iteration und braucht viele Iterationen – das wäre unbrauchbar. Zwei Verbesserungen machen daraus ein Verfahren, das für $n = 1000$ in Sekunden läuft.

Erstens: Hessenberg-Form. Man transformiert A zunächst durch Householder-Spiegelungen auf **obere Hessenberg-Form** – eine obere Dreiecksmatrix mit einer zusätzlichen Nebendiagonale:

$$\begin{pmatrix} * & * & * & * \\ * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \end{pmatrix}$$

Das kostet einmalig $\frac{10}{3}n^3$ Operationen. Der Gewinn: Diese Form bleibt unter QR-Schritten *erhalten*, und ein QR-Schritt für eine Hessenberg-Matrix kostet nur $\mathcal{O}(n^2)$.

Ist A symmetrisch, so ist die Hessenberg-Form *tridiagonal*, und ein Schritt kostet sogar nur $\mathcal{O}(n)$.

Zweitens: Verschiebungen (Shifts). Statt A_k zerlegt man $A_k - \sigma_k I$:

$$A_k - \sigma_k I = Q_k R_k, \qquad A_{k+1} = R_k Q_k + \sigma_k I.$$

Man prüft leicht nach, dass die Ähnlichkeit erhalten bleibt.

Die Konvergenzrate des untersten Nebendiagonaleintrags wird dadurch zu

$$\left| \frac{\lambda_n - \sigma_k}{\lambda_{n-1} - \sigma_k} \right|,$$

also winzig bei guter Schätzung. Mit dem *Wilkinson-Shift* (dem Eigenwert des unteren 2×2-Blocks, der a_{nn} näher liegt) ist die Konvergenz kubisch.

Das Ergebnis: Im Mittel genügen zwei bis drei Iterationen pro Eigenwert. Der Gesamtaufwand liegt bei etwa $10n^3$ für alle Eigenwerte einer symmetrischen Matrix, $25n^3$ im unsymmetrischen Fall.

Der Algorithmus des Jahrhunderts

Der QR-Algorithmus wurde 1959 bis 1961 unabhängig von zwei Personen entwickelt: JOHN G. F. FRANCIS in London und VERA KUBLANOVSKAYA in Leningrad.
FRANCIS war damals Mitte zwanzig und arbeitete bei der National Research Development Corporation. Er hatte kein abgeschlossenes Mathematikstudium – er hatte Cambridge ohne Abschluss verlassen. Nach zwei Aufsätzen 1961 und 1962, in denen er den Algorithmus einschließlich Hessenberg-Reduktion und Doppelshift-Strategie vollständig darstellte, verließ er die numerische Mathematik und arbeitete in der Industrie.
Er erfuhr erst 2007 – als er 73 war und von GENE GOLUB und FRANK UHLIG aufgespürt wurde –, dass sein Verfahren zu einem der bedeutendsten Algorithmen des 20. Jahrhunderts geworden war. Er hatte in den dazwischenliegenden 45 Jahren keine numerische Literatur mehr gelesen.
VERA KUBLANOVSKAYA (1920–2012) veröffentlichte ihre Fassung ebenfalls 1961 am Steklow-Institut. Sie hatte den Krieg als junge Frau in Leningrad überlebt und wurde später eine führende sowjetische Numerikerin.
Die Zeitschrift *Computing in Science and Engineering* zählte den QR-Algorithmus 2000 zu den zehn Algorithmen mit dem größten Einfluss auf Wissenschaft und Technik im 20. Jahrhundert. Er steckt heute in jedem `eig`-Aufruf jeder numerischen Software[8].

6.4 Die Singulärwertzerlegung

Satz 6.1 (Singulärwertzerlegung, SVD). *Jede Matrix $A \in \mathbb{R}^{m\times n}$ lässt sich schreiben als*

$$A = U\Sigma V^{\mathsf{T}}$$

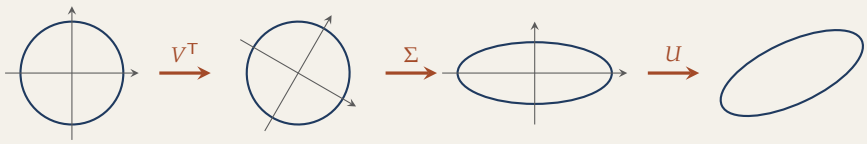
mit orthogonalen $U \in \mathbb{R}^{m\times m}$, $V \in \mathbb{R}^{n\times n}$ und

$$\Sigma = \operatorname{diag}(\sigma_1, \ldots, \sigma_p), \qquad p = \min(m, n), \qquad \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0.$$

Die σ_i heißen ***Singulärwerte*** *und sind eindeutig bestimmt.*

Was die SVD anschaulich sagt

Jede lineare Abbildung ist eine Drehung, gefolgt von einer Streckung entlang der Achsen, gefolgt von einer weiteren Drehung.



Einheitskreis — gedreht — gestreckt — gedreht

Die Singulärwerte sind die Halbachsen des Bildellipsoids. Insbesondere:

$$\|A\|_2 = \sigma_1, \qquad \operatorname{cond}_2(A) = \frac{\sigma_1}{\sigma_n}.$$

Damit ist die geometrische Deutung der Konditionszahl aus Kapitel 2 vollständig begründet.

Der Zusammenhang mit Eigenwerten

$$A^\mathsf{T} A = V \Sigma^\mathsf{T} U^\mathsf{T} U \Sigma V^\mathsf{T} = V(\Sigma^\mathsf{T}\Sigma)V^\mathsf{T}.$$

Das ist eine Diagonalisierung: Die Eigenwerte von $A^\mathsf{T}A$ sind σ_i^2, die Eigenvektoren die Spalten von V.
Analog sind die Spalten von U die Eigenvektoren von AA^T.

Aber so berechnet man sie nicht

Der Weg über $A^\mathsf{T}A$ quadriert die Kondition – dasselbe Problem wie bei den Normalgleichungen. Kleine Singulärwerte gehen dabei verloren: Ist $\sigma_i < \sqrt{\varepsilon_\mathrm{M}}\,\sigma_1$, so verschwindet σ_i^2 in den Rundungsfehlern.
Der Standardalgorithmus (GOLUB und KAHAN 1965) bringt A stattdessen durch Householder-Spiegelungen von beiden Seiten auf *Bidiagonalform* und wendet darauf einen impliziten QR-Schritt an – ohne je $A^\mathsf{T}A$ zu bilden. Aufwand: etwa $4mn^2$ bis $14mn^2$ je nachdem, ob man U und V mitberechnet.

Satz 6.2 (Eckart–Young–Mirsky). *Sei $A = \sum_{i=1}^{r} \sigma_i u_i v_i^\mathsf{T}$ die SVD und*

$$A_k := \sum_{i=1}^{k} \sigma_i u_i v_i^\mathsf{T}, \qquad k < r.$$

Dann ist A_k die beste *Approximation von A durch eine Matrix vom Rang k, sowohl in der Spektral- als auch in der Frobeniusnorm:*

$$\min_{\operatorname{Rang} B \le k} \|A - B\|_2 = \|A - A_k\|_2 = \sigma_{k+1}.$$

Warum dieser Satz so wichtig ist

Er sagt: **Man erhält die beste Kompression, indem man die kleinsten Singulärwerte streicht.** Und der dabei gemachte Fehler ist *genau* der größte gestrichene Singulärwert – man weiß also vorher, was man verliert.
Das ist die theoretische Grundlage einer bemerkenswerten Zahl von Verfahren:

- **Hauptkomponentenanalyse (PCA).** Die Hauptkomponenten sind die rechten Singulärvektoren der zentrierten Datenmatrix. Dimensionsreduktion in der Statistik ist Rang-k-Approximation.
- **Bildkompression.** Ein 1000×1000-Bild braucht 10^6 Zahlen. Rang-50-Approximation braucht $50 \cdot (1000 + 1000 + 1) \approx 10^5$ – ein Zehntel, bei oft kaum sichtbarem Unterschied.
- **Latente semantische Analyse.** In der Textanalyse deckt die Rang-k-Approximation der Term-Dokument-Matrix verborgene thematische Strukturen auf.
- **Empfehlungssysteme.** Die Matrixfaktorisierungsverfahren, die den Netflix-Preis 2009 gewannen, beruhen auf demselben Prinzip.
- **Entrauschen.** Ist das Signal niedrigrangig und das Rauschen nicht, so trennt die SVD beides.
- **Rangbestimmung.** Numerisch bestimmt man den Rang einer Matrix, indem man zählt, wie viele Singulärwerte oberhalb einer Toleranz $\tau \approx \varepsilon_{\mathrm{M}} \sigma_1 \max(m, n)$ liegen. Der algebraische Rangbegriff ist numerisch unbrauchbar, weil jede Störung eine singuläre Matrix regulär macht.

Aufgabe 6.2. Berechnen Sie die SVD von

$$A = \begin{pmatrix} 3 & 0 \\ 4 & 5 \end{pmatrix}.$$

Lösung Schritt für Schritt.

Schritt 1 – $A^{\mathsf{T}}A$ bilden (hier zulässig, da 2×2 zum Nachrechnen).

$$A^{\mathsf{T}}A = \begin{pmatrix} 3 & 4 \\ 0 & 5 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 4 & 5 \end{pmatrix} = \begin{pmatrix} 9+16 & 20 \\ 20 & 25 \end{pmatrix} = \begin{pmatrix} 25 & 20 \\ 20 & 25 \end{pmatrix}.$$

Schritt 2 – Eigenwerte.

$$\det \begin{pmatrix} 25-\lambda & 20 \\ 20 & 25-\lambda \end{pmatrix} = (25-\lambda)^2 - 400 = 0.$$

$$25 - \lambda = \pm 20 \Longrightarrow 25 - \lambda = 20 \Longrightarrow \lambda = 5$$

$$\text{oder} \quad 25 - \lambda = -20 \Longrightarrow \lambda = 45.$$

Also $\lambda_1 = 45$, $\lambda_2 = 5$.

Schritt 3 – Singulärwerte.

$$\sigma_1 = \sqrt{45} = 3\sqrt{5} = 6{,}708204, \qquad \sigma_2 = \sqrt{5} = 2{,}236068.$$

Kontrolle: $\sigma_1 \sigma_2 = \sqrt{45 \cdot 5} = \sqrt{225} = 15 = |\det A|$. Und $\det A = 3 \cdot 5 - 0 \cdot 4 = 15$. ✓

Schritt 4 – Rechte Singulärvektoren (Eigenvektoren von $A^\mathsf{T}A$).
Für $\lambda_1 = 45$:

$$\begin{pmatrix} 25-45 & 20 \\ 20 & 25-45 \end{pmatrix} v = \begin{pmatrix} -20 & 20 \\ 20 & -20 \end{pmatrix} v = 0 \Longrightarrow v_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Für $\lambda_2 = 5$:

$$\begin{pmatrix} 20 & 20 \\ 20 & 20 \end{pmatrix} v = 0 \Longrightarrow v_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

$$V = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

Schritt 5 – Linke Singulärvektoren über $u_i = Av_i/\sigma_i$.

$$Av_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 3 \cdot 1 + 0 \cdot 1 \\ 4 \cdot 1 + 5 \cdot 1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 3 \\ 9 \end{pmatrix}.$$

$$u_1 = \frac{Av_1}{\sigma_1} = \frac{1}{\sqrt{2} \cdot 3\sqrt{5}} \begin{pmatrix} 3 \\ 9 \end{pmatrix} = \frac{1}{\sqrt{10}} \begin{pmatrix} 1 \\ 3 \end{pmatrix} = \begin{pmatrix} 0{,}316228 \\ 0{,}948683 \end{pmatrix}.$$

Zwischenrechnung: $\frac{3}{\sqrt{2}\cdot 3\sqrt{5}} = \frac{1}{\sqrt{10}}$ und $\frac{9}{\sqrt{2}\cdot 3\sqrt{5}} = \frac{3}{\sqrt{10}}$. ✓

$$Av_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 3 \\ 4-5 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 3 \\ -1 \end{pmatrix}.$$

$$u_2 = \frac{1}{\sqrt{2} \cdot \sqrt{5}} \begin{pmatrix} 3 \\ -1 \end{pmatrix} = \frac{1}{\sqrt{10}} \begin{pmatrix} 3 \\ -1 \end{pmatrix} = \begin{pmatrix} 0{,}948683 \\ -0{,}316228 \end{pmatrix}.$$

Kontrolle der Orthogonalität: $u_1^\mathsf{T} u_2 = \frac{1}{10}(1 \cdot 3 + 3 \cdot (-1)) = 0$. ✓
Schritt 6 – Ergebnis.

$$U = \frac{1}{\sqrt{10}} \begin{pmatrix} 1 & 3 \\ 3 & -1 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} 6{,}708204 & 0 \\ 0 & 2{,}236068 \end{pmatrix}, \quad V = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

Schritt 7 – Probe. Wir prüfen den Eintrag $(1,1)$ von $U\Sigma V^\mathsf{T}$:

$$\frac{1}{\sqrt{10}} \cdot 6{,}708204 \cdot \frac{1}{\sqrt{2}} + \frac{3}{\sqrt{10}} \cdot 2{,}236068 \cdot \frac{1}{\sqrt{2}}$$

$$= \frac{1}{\sqrt{20}}(6{,}708204) + \frac{3}{\sqrt{20}}(2{,}236068) = \frac{6{,}708204 + 6{,}708204}{4{,}472136} = \frac{13{,}416408}{4{,}472136} = 3{,}0. \checkmark$$

Schritt 8 – Konditionszahl.

$$\operatorname{cond}_2(A) = \frac{6{,}708204}{2{,}236068} = 3{,}0.$$

Die Matrix ist gut konditioniert – sie verzerrt den Einheitskreis nur zu einer Ellipse mit Achsenverhältnis 3.

Die Pseudoinverse

Mit der SVD definiert man

$$A^{+} := V\Sigma^{+}U^{\mathsf{T}}, \qquad \Sigma^{+} = \operatorname{diag}\left(\frac{1}{\sigma_1}, \ldots, \frac{1}{\sigma_r}, 0, \ldots, 0\right),$$

wobei nur die Singulärwerte oberhalb einer Toleranz invertiert werden.

Was $A^{+}b$ leistet: Es ist die Kleinste-Quadrate-Lösung, und unter allen solchen die mit der kleinsten Norm. Damit ist auch der *rangdefiziente* Fall abgedeckt, in dem die Normalgleichungen singulär wären und die QR-Zerlegung ohne Spaltenpivotisierung versagt.

Und der Zusammenhang mit der Regularisierung: Das Abschneiden kleiner Singulärwerte ist die einfachste Form der **Regularisierung**. Sie stabilisiert schlecht gestellte Probleme, indem sie die Richtungen, in denen die Daten kaum Information tragen, ganz ausblendet.

Die verwandte **Tichonow-Regularisierung** ersetzt $1/\sigma_i$ durch $\sigma_i/(\sigma_i^2 + \alpha^2)$ – ein weicher Übergang statt eines harten Schnitts. Sie ist unter dem Namen *ridge regression* auch in der Statistik Standard und lässt sich als Bayes-Schätzer mit normalverteilter Priorverteilung deuten (Band 4, Kapitel 15).

Part III

Nichtlineare Gleichungen

7 Nullstellen und Fixpunkte

7.1 Das Problem und die Konvergenzordnung

Was wir suchen

Gegeben $f : \mathbb{R} \to \mathbb{R}$, gesucht x^* mit $f(x^*) = 0$.
Anders als bei linearen Systemen gibt es hier

- keine allgemeine Lösungsformel (Abel–Galois, siehe Kapitel 6),
- keine Garantie für Existenz oder Eindeutigkeit,
- keine Aussage darüber, wie viele Lösungen es gibt.

Man ist auf Iteration angewiesen – und auf einen brauchbaren Startwert.

Definition 7.1 (Konvergenzordnung). Eine Folge $x_k \to x^*$ konvergiert mit der **Ordnung** $p \geq 1$, wenn

$$\lim_{k\to\infty} \frac{|x_{k+1} - x^*|}{|x_k - x^*|^p} = C$$

mit $0 < C < \infty$ existiert (für $p = 1$ zusätzlich $C < 1$).

Was die Ordnung praktisch bedeutet

Sei $e_k := |x_k - x^*|$ der Fehler.
Lineare Konvergenz ($p = 1$): $e_{k+1} \approx Ce_k$. Der Fehler wird pro Schritt mit einem festen Faktor multipliziert. Man gewinnt pro Schritt eine feste Anzahl von Stellen – bei $C = 0{,}5$ eine Binärstelle, also etwa 0,3 Dezimalstellen.
Quadratische Konvergenz ($p = 2$): $e_{k+1} \approx Ce_k^2$. Die Zahl der *richtigen Stellen verdoppelt sich* pro Schritt.
Kubische Konvergenz ($p = 3$): Verdreifachung.
Ein Zahlenvergleich, ausgehend von $e_0 = 10^{-1}$ und $C = 1$:

k	linear ($C = 0{,}5$)	quadratisch	kubisch
0	10^{-1}	10^{-1}	10^{-1}
1	$5 \cdot 10^{-2}$	10^{-2}	10^{-3}
2	$2{,}5 \cdot 10^{-2}$	10^{-4}	10^{-9}
3	$1{,}25 \cdot 10^{-2}$	10^{-8}	10^{-27}
4	$6{,}3 \cdot 10^{-3}$	10^{-16}	–

Nach vier Schritten: linear noch zwei richtige Stellen, quadratisch die volle Maschinengenauigkeit erreicht.
Die praktische Folgerung: Bei quadratischer Konvergenz sind ab dem Erreichen des Einzugsbereichs typischerweise 4 bis 6 Schritte nötig – unabhängig von der geforderten Genauigkeit. Die Kunst besteht darin, *in* den Einzugsbereich zu gelangen.

7.2 Bisektion – langsam, aber unfehlbar

Das Verfahren

Sei f stetig auf $[a,b]$ mit $f(a)f(b) < 0$. Nach dem Zwischenwertsatz (Band 3, Kapitel 4) gibt es eine Nullstelle im Intervall.

Wiederhole:
$m := \dfrac{a+b}{2}$
Falls $f(a)f(m) < 0$: $\quad b := m$ (Nullstelle links)
sonst: $\quad a := m$ (Nullstelle rechts)

Fehlerabschätzung. Das Intervall halbiert sich jedes Mal:

$$|x_k - x^*| \le \frac{b-a}{2^{k+1}}.$$

Um von der Intervalllänge 1 auf 10^{-16} zu kommen, braucht man

$$2^{-k} \le 10^{-16} \Longrightarrow k \ge \frac{16\ln 10}{\ln 2} = 53$$

Schritte. **Immer 53 – unabhängig von der Funktion.**
Die Ordnung ist $p = 1$ mit $C = \frac{1}{2}$: lineare Konvergenz, drei Bits pro zehn Schritte.

Vor- und Nachteile

Dafür:

- Konvergiert *immer*, ohne Ausnahme. Kein Verfahren ist zuverlässiger.
- Braucht nur Stetigkeit – keine Ableitung, keine Glattheit.
- Liefert eine *garantierte Einschließung*: Man weiß in jedem Schritt, wo die Nullstelle liegt.

Dagegen:

- Langsam.
- Nur in einer Dimension anwendbar – im $\mathbb{R}^n$ gibt es kein Analogon zum Vorzeichenwechsel.
- Findet nur Nullstellen mit Vorzeichenwechsel; $f(x) = x^2$ wird nie gefunden.

Eine Falle bei der Implementierung

Man schreibe *nicht*

$$m = \frac{a+b}{2},$$

sondern

$$m = a + \frac{b-a}{2}.$$

Beide sind mathematisch gleich; die erste Form kann bei sehr großen a, b überlaufen, und sie garantiert nicht, dass m tatsächlich zwischen a und b liegt. Der berühmte Fehler in der binären Suche der Java-Standardbibliothek, der 2006 entdeckt wurde und neun Jahre unbemerkt geblieben war, hatte genau diese Ursache.

7.3 Das Newton-Verfahren

Die Idee: die Funktion durch ihre Tangente ersetzen

Man linearisiert f an der Stelle x_k:

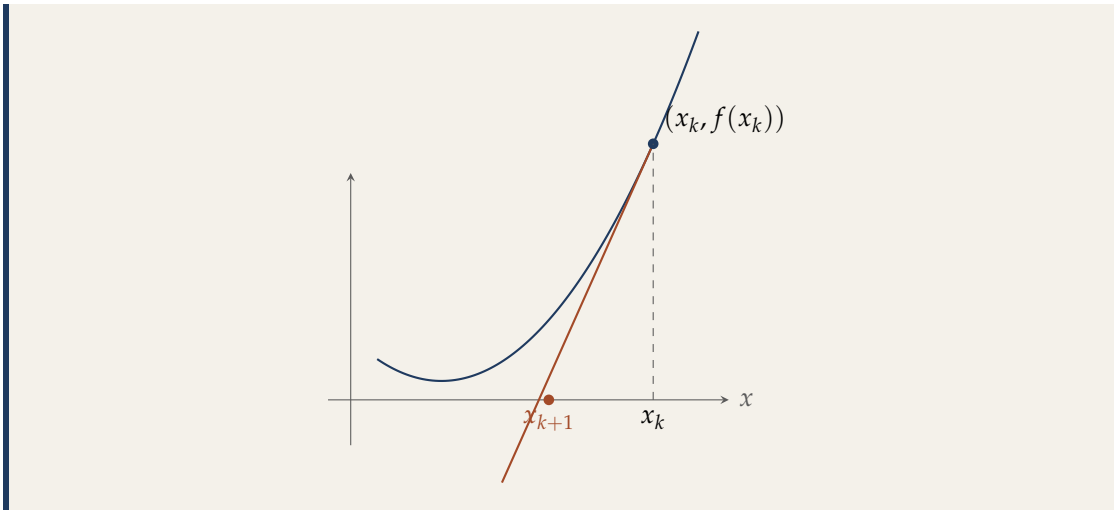
$$f(x) \approx f(x_k) + f'(x_k)(x - x_k)$$

und setzt die *Tangente* statt der Funktion gleich null:

$$f(x_k) + f'(x_k)(x_{k+1} - x_k) = 0.$$

Auflösen ergibt

$$\boxed{x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}}$$



Satz 7.2 (Quadratische Konvergenz). *Sei f zweimal stetig differenzierbar, x^* eine* einfache *Nullstelle ($f'(x^*) \neq 0$). Dann gibt es eine Umgebung von x^*, in der das Newton-Verfahren für jeden Startwert konvergiert, und es gilt*

$$|x_{k+1} - x^*| \leq \frac{M}{2m} |x_k - x^*|^2, \qquad M := \max |f''|, \quad m := \min |f'|.$$

Beweis. **Schritt 1 – Taylorentwicklung um x_k.** Nach dem Satz von Taylor mit Lagrange-Restglied (Band 3, Kapitel 8) gibt es ξ zwischen x_k und x^* mit

$$0 = f(x^*) = f(x_k) + f'(x_k)(x^* - x_k) + \frac{f''(\xi)}{2}(x^* - x_k)^2.$$

Schritt 2 – Durch $f'(x_k)$ teilen.

$$0 = \frac{f(x_k)}{f'(x_k)} + (x^* - x_k) + \frac{f''(\xi)}{2f'(x_k)}(x^* - x_k)^2.$$

Schritt 3 – Die Newton-Vorschrift einsetzen. Es ist $\frac{f(x_k)}{f'(x_k)} = x_k - x_{k+1}$, also

$$\begin{aligned} 0 &= x_k - x_{k+1} + x^* - x_k + \frac{f''(\xi)}{2f'(x_k)}(x^* - x_k)^2 \\ &= (x^* - x_{k+1}) + \frac{f''(\xi)}{2f'(x_k)}(x^* - x_k)^2. \end{aligned}$$

Schritt 4 – Umstellen.

$$x_{k+1} - x^* = \frac{f''(\xi)}{2f'(x_k)}(x_k - x^*)^2.$$

Schritt 5 – Abschätzen. In einer hinreichend kleinen Umgebung ist $|f'| \geq m > 0$ (Stetigkeit und $f'(x^*) \neq 0$) und $|f''| \leq M$:

$$|x_{k+1} - x^*| \leq \frac{M}{2m} |x_k - x^*|^2. \qquad \blacksquare$$

Was der Beweis verrät

Erstens: Woher die Quadrierung kommt. Sie stammt aus dem *Restglied zweiter Ordnung* der Taylorentwicklung. Newton eliminiert exakt den linearen Term – übrig bleibt der quadratische.
Zweitens: Warum $f'(x^*) \neq 0$ nötig ist. Der Nenner $2f'(x_k)$ würde sonst gegen null gehen.
Bei einer m-fachen Nullstelle konvergiert Newton nur noch *linear* mit dem Faktor

$$C = 1 - \frac{1}{m}.$$

Bei einer doppelten Nullstelle ($m = 2$) also mit $C = \frac{1}{2}$ – nicht besser als Bisektion.
Abhilfe bei bekannter Vielfachheit: Man verwendet

$$x_{k+1} = x_k - m\frac{f(x_k)}{f'(x_k)}$$

und erhält die quadratische Konvergenz zurück.
Abhilfe bei unbekannter Vielfachheit: Man wendet Newton auf $u(x) := f(x)/f'(x)$ an. Diese Funktion hat bei x^* stets eine *einfache* Nullstelle, gleich welcher Vielfachheit die von f war.
Drittens: Die Konvergenz ist nur *lokal*. Der Satz sagt nichts über weit entfernte Startwerte.

Vier Arten, wie Newton scheitert

1. Waagerechte Tangente. Ist $f'(x_k) \approx 0$, so wird der Schritt riesig und man landet irgendwo. Beispiel: $f(x) = x^3 - 2x + 2$ mit $x_0 = 0$ ergibt $x_1 = 1$, $x_2 = 0$, $x_3 = 1, \ldots$ – ein *Zweierzyklus*, aus dem das Verfahren nie herausfindet.
2. Divergenz. $f(x) = \arctan(x)$ mit $|x_0|$ groß genug (etwa $x_0 > 1{,}39$): Die Iterierten wachsen unbeschränkt, weil die Tangente an der flachen Kurve die Achse immer weiter draußen schneidet.
3. Anziehung durch die falsche Nullstelle. Bei mehreren Nullstellen ist kaum vorherzusagen, zu welcher man konvergiert. Färbt man die komplexe Ebene nach der erreichten Nullstelle von $z^3 - 1$, so erhält man ein *Fraktal* – die Newton-Fraktale. Die Grenzen zwischen den Einzugsgebieten sind unendlich fein verschlungen; beliebig nah beieinanderliegende Startwerte können zu verschiedenen Nullstellen führen.
4. Die Ableitung ist unbekannt oder teuer. Bei einer Funktion, die durch eine Simulation gegeben ist, existiert oft keine Formel für f'.
Die Standardlösung in der Praxis: Man kombiniert. Das Verfahren von BRENT (1973) verbindet Bisektion (garantierte Konvergenz), Sekantenverfahren und in-

verse quadratische Interpolation (Geschwindigkeit) und schaltet automatisch um. Es steht hinter fast jeder `fzero`-Routine.

Aufgabe 7.1. Berechnen Sie $\sqrt{2}$ mit dem Newton-Verfahren auf 10 Stellen genau, ausgehend von $x_0 = 1$.

Lösung Schritt für Schritt.

Schritt 1 – Funktion wählen. Wir suchen die Nullstelle von

$$f(x) = x^2 - 2, \qquad f'(x) = 2x.$$

Schritt 2 – Iterationsvorschrift vereinfachen.

$$x_{k+1} = x_k - \frac{x_k^2 - 2}{2x_k} = \frac{2x_k^2 - x_k^2 + 2}{2x_k} = \frac{x_k^2 + 2}{2x_k}.$$

Das schreibt man besser als

$$x_{k+1} = \frac{1}{2}\left(x_k + \frac{2}{x_k}\right).$$

Die anschauliche Deutung dieser Form: Ist x_k zu klein, so ist $2/x_k$ zu groß – und umgekehrt. Man nimmt das *arithmetische Mittel* beider Schätzungen. Das ist das **Heron-Verfahren**, das bereits die Babylonier um 1800 v. Chr. benutzten.

Schritt 3 – Rechnen.

$$x_0 = 1$$

$$x_1 = \frac{1}{2}\left(1 + \frac{2}{1}\right) = \frac{3}{2} = 1{,}5$$

$$x_2 = \frac{1}{2}\left(1{,}5 + \frac{2}{1{,}5}\right) = \frac{1}{2}\left(1{,}5 + 1{,}333333333\right) = \frac{1}{2}(2{,}833333333) = 1{,}416666667$$

$$x_3 = \frac{1}{2}\left(1{,}416666667 + \frac{2}{1{,}416666667}\right) = \frac{1}{2}(1{,}416666667 + 1{,}411764706)$$

$$= \frac{1}{2}(2{,}828431373) = 1{,}414215686$$

$$x_4 = \frac{1}{2}\left(1{,}414215686 + \frac{2}{1{,}414215686}\right) = \frac{1}{2}(1{,}414215686 + 1{,}414211438)$$

$$= \frac{1}{2}(2{,}828427124) = 1{,}414213562$$

Schritt 4 – Fehleranalyse. Der exakte Wert ist $\sqrt{2} = 1{,}41421356237\ldots$

k	x_k	Fehler e_k	richtige Stellen
0	1,000000000	$4{,}1 \cdot 10^{-1}$	0
1	1,500000000	$8{,}6 \cdot 10^{-2}$	1
2	1,416666667	$2{,}5 \cdot 10^{-3}$	2
3	1,414215686	$2{,}1 \cdot 10^{-6}$	5
4	1,414213562	$1{,}6 \cdot 10^{-12}$	11

Die Verdopplung der Stellen ist deutlich sichtbar: $1 \to 2 \to 5 \to 11$.
Schritt 5 – Die theoretische Konstante prüfen. Nach Satz 7.2 ist

$$C = \frac{|f''(x^*)|}{2\,|f'(x^*)|} = \frac{2}{2 \cdot 2\sqrt{2}} = \frac{1}{2\sqrt{2}} = 0{,}35355.$$

Prüfung an den Daten:

$$\frac{e_4}{e_3^2} = \frac{1{,}6 \cdot 10^{-12}}{(2{,}1 \cdot 10^{-6})^2} = \frac{1{,}6 \cdot 10^{-12}}{4{,}4 \cdot 10^{-12}} = 0{,}36. \checkmark$$

Bemerkung zur Praxis. Genau dieses Verfahren steckt in der Hardware-Implementierung der Quadratwurzel. Und eine berühmte Variante – die *fast inverse square root* aus dem Quellcode von *Quake III Arena* (1999) – benutzt einen Bit-Trick zur Startwertbestimmung und danach genau einen Newton-Schritt. Sie war seinerzeit etwa viermal schneller als die Bibliotheksfunktion.

7.4 Verfahren ohne Ableitung

Das Sekantenverfahren

Man ersetzt die Ableitung durch den Differenzenquotienten aus den beiden letzten Punkten:

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

Einsetzen ergibt

$$x_{k+1} = x_k - f(x_k) \cdot \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

Geometrisch: Statt der Tangente nimmt man die *Sekante* durch die letzten beiden Punkte.
Konvergenzordnung. Man kann zeigen, dass

$$p = \frac{1 + \sqrt{5}}{2} = 1{,}618\ldots$$

gilt – der *goldene Schnitt*. Das ist keine Zufallszahl: Setzt man $e_{k+1} \approx C e_k e_{k-1}$ an und macht den Ansatz $e_k = A e_{k-1}^p$, so führt der Koeffizientenvergleich auf $p^2 = p + 1$, die definierende Gleichung des goldenen Schnitts.
Der praktische Vergleich mit Newton:

	Ordnung	Auswertungen/Schritt	Ordnung pro Auswertung
Newton	2,00	2 (f und f')	$\sqrt{2} = 1{,}41$
Sekante	1,62	1 (nur f)	1,62

Ist die Ableitung ähnlich teuer wie die Funktion, so ist das Sekantenverfahren effizienter – pro Funktionsauswertung gewinnt es mehr. Das ist ein Grund, warum es in der Praxis so verbreitet ist.

> **Auslöschung im Nenner**
>
> Nahe der Konvergenz sind $f(x_k)$ und $f(x_{k-1})$ fast gleich – der Nenner leidet unter Auslöschung. Wird er null, muss man abbrechen. Praktische Implementierungen prüfen das und schalten dann auf Bisektion um.

Fixpunktiteration allgemein

Viele Verfahren haben die Form

$$x_{k+1} = g(x_k),$$

und die gesuchte Lösung ist ein **Fixpunkt**: $g(x^*) = x^*$.
Der Banachsche Fixpunktsatz (Band 3, Kapitel 4) sagt: Ist g eine *Kontraktion* auf einem abgeschlossenen Intervall I, das auf sich abgebildet wird – also

$$|g(x) - g(y)| \le L\,|x - y| \quad \text{mit } L < 1,$$

so gibt es genau einen Fixpunkt, und die Iteration konvergiert für jeden Startwert in I dagegen.
Die Fehlerabschätzung:

$$|x_k - x^*| \le \frac{L^k}{1 - L}\,|x_1 - x_0|\,.$$

Man kann also schon nach dem ersten Schritt abschätzen, wie viele Iterationen nötig sind.
Praktisch prüft man die Kontraktion über die Ableitung: Ist $|g'(x)| \le L < 1$ auf I, so ist g nach dem Mittelwertsatz eine Kontraktion.
Und die Ordnung:

- $g'(x^*) \neq 0$: lineare Konvergenz mit $C = |g'(x^*)|$.
- $g'(x^*) = 0$, $g''(x^*) \neq 0$: quadratische Konvergenz.

Damit erklärt sich Newton neu: Für

$$g(x) = x - \frac{f(x)}{f'(x)}$$

rechnet man nach:

$$g'(x) = 1 - \frac{f'(x)^2 - f(x)f''(x)}{f'(x)^2} = \frac{f(x)f''(x)}{f'(x)^2}.$$

An der Nullstelle ist $f(x^*) = 0$, also $g'(x^*) = 0$. **Newton ist genau die Fixpunktiteration, die konstruiert wurde, um $g'(x^*) = 0$ zu erzwingen** – und daher die quadratische Konvergenz.

Aufgabe 7.2. Die Gleichung $x = \cos x$ soll gelöst werden. Untersuchen Sie die Fixpunktiteration $x_{k+1} = \cos x_k$ und vergleichen Sie mit Newton.

Lösung Schritt für Schritt.

Schritt 1 – Existenz und Eindeutigkeit. Setze $h(x) := x - \cos x$. Es ist

$$h(0) = -1 < 0, \qquad h(1) = 1 - 0{,}5403 = 0{,}4597 > 0.$$

Nach dem Zwischenwertsatz existiert eine Nullstelle in $(0, 1)$. Da $h'(x) = 1 + \sin x > 0$ auf $[0, 1]$, ist h streng monoton – die Nullstelle ist eindeutig.

Schritt 2 – Kontraktionsbedingung prüfen. Für $g(x) = \cos x$ ist $g'(x) = -\sin x$. Auf $[0;\, 0{,}8]$:

$$|g'(x)| \leq \sin(0{,}8) = 0{,}7174 < 1. \checkmark$$

Außerdem bildet g das Intervall $[0{,}5;\ 0{,}9]$ in sich ab: $\cos(0{,}5) = 0{,}878$, $\cos(0{,}9) = 0{,}622$. Der Fixpunktsatz greift.

Schritt 3 – Fixpunktiteration ab $x_0 = 1$.

$$\begin{aligned} x_1 &= \cos(1) = 0{,}540302 \\ x_2 &= \cos(0{,}540302) = 0{,}857553 \\ x_3 &= \cos(0{,}857553) = 0{,}654290 \\ x_4 &= \cos(0{,}654290) = 0{,}793480 \\ x_5 &= \cos(0{,}793480) = 0{,}701369 \\ x_{10} &= 0{,}731404 \\ x_{20} &= 0{,}739184 \\ x_{50} &= 0{,}739085 \end{aligned}$$

Die Folge *oszilliert* um den Fixpunkt (weil $g' < 0$) und konvergiert langsam.

Schritt 4 – Die Konvergenzrate. Der Fixpunkt ist $x^* = 0{,}7390851332$. Dort:

$$\left|g'(x^*)\right| = \sin(0{,}7390851) = 0{,}673612.$$

Pro Schritt wird der Fehler also mit 0,674 multipliziert – man gewinnt

$$-\log_{10}(0{,}674) = 0{,}171$$

Dezimalstellen pro Schritt. Für 10 Stellen braucht man rund **59 Iterationen**.
Schritt 5 – Newton zum Vergleich.

$$h(x) = x - \cos x, \qquad h'(x) = 1 + \sin x.$$

$$x_{k+1} = x_k - \frac{x_k - \cos x_k}{1 + \sin x_k}.$$

Ab $x_0 = 1$*:*

$$x_1 = 1 - \frac{1 - 0{,}540302}{1 + 0{,}841471} = 1 - \frac{0{,}459698}{1{,}841471} = 1 - 0{,}249636 = 0{,}750364.$$

$$\cos(0{,}750364) = 0{,}731404, \quad \sin(0{,}750364) = 0{,}681935.$$

$$x_2 = 0{,}750364 - \frac{0{,}750364 - 0{,}731404}{1{,}681935} = 0{,}750364 - \frac{0{,}018960}{1{,}681935}$$

$$= 0{,}750364 - 0{,}011273 = 0{,}739091.$$

$$x_3 = 0{,}739085133.$$

Vergleich:

	Fixpunkt	Newton
Iterationen für 10 Stellen	≈ 59	**4**
Aufwand pro Schritt	1 Kosinus	1 Kosinus, 1 Sinus
Gesamtaufwand	59	**8**

Fazit. Newton ist hier siebenmal effizienter. Die Fixpunktiteration ist dennoch nicht wertlos: Sie braucht keine Ableitung, sie ist robuster gegen schlechte Startwerte, und ihre Konvergenz ist garantiert, sobald die Kontraktionsbedingung nachgewiesen ist.

7.5 Newton im Mehrdimensionalen

Die Verallgemeinerung

Gesucht ist $x \in \mathbb{R}^n$ mit $F(x) = 0$ für $F : \mathbb{R}^n \to \mathbb{R}^n$. Die Linearisierung lautet jetzt

$$F(x) \approx F(x_k) + J_F(x_k)(x - x_k),$$

wobei

$$J_F(x) = \begin{pmatrix} \frac{\partial F_1}{\partial x_1} & \dots & \frac{\partial F_1}{\partial x_n} \\ \vdots & & \vdots \\ \frac{\partial F_n}{\partial x_1} & \dots & \frac{\partial F_n}{\partial x_n} \end{pmatrix}$$

die **Jacobi-Matrix** ist.
Nullsetzen ergibt

$$x_{k+1} = x_k - J_F(x_k)^{-1} F(x_k).$$

So schreibt man es nicht auf, und so rechnet man es nie

Man invertiert die Jacobi-Matrix nicht. Man löst in jedem Schritt das *lineare System*

$$J_F(x_k)\,\Delta x_k = -F(x_k), \qquad x_{k+1} = x_k + \Delta x_k.$$

Hier trifft man Kapitel 3 und 5 wieder: Bei kleinem n mit LR-Zerlegung, bei großem n mit einem Krylow-Verfahren.
Die Kombination Newton + Krylow heißt **Newton–Krylow-Verfahren** und ist heute der Standard für große nichtlineare Systeme – etwa in der Strömungsmechanik. Der Vorzug: Man braucht die Jacobi-Matrix nie explizit, sondern nur Produkte $J_F v$, und die lassen sich durch

$$J_F(x)v \approx \frac{F(x + \varepsilon v) - F(x)}{\varepsilon}$$

approximieren – eine Funktionsauswertung statt n^2 partieller Ableitungen.

Aufgabe 7.3. Lösen Sie das System

$$\begin{cases} x^2 + y^2 = 4 \\ e^x + y = 1 \end{cases}$$

mit dem Newton-Verfahren, Startwert $(1, -1{,}7)$.

Lösung Schritt für Schritt.

Schritt 1 – Als Nullstellenproblem schreiben.

$$F(x,y) = \begin{pmatrix} x^2 + y^2 - 4 \\ \mathrm{e}^x + y - 1 \end{pmatrix}.$$

Schritt 2 – Jacobi-Matrix.

$$J_F(x,y) = \begin{pmatrix} 2x & 2y \\ \mathrm{e}^x & 1 \end{pmatrix}.$$

Schritt 3 – Iteration 1. Bei $(x_0, y_0) = (1; -1{,}7)$:

$$\mathrm{e}^1 = 2{,}718282.$$

$$F = \begin{pmatrix} 1 + 2{,}89 - 4 \\ 2{,}718282 - 1{,}7 - 1 \end{pmatrix} = \begin{pmatrix} -0{,}110000 \\ 0{,}018282 \end{pmatrix}.$$

$$J = \begin{pmatrix} 2 & -3{,}4 \\ 2{,}718282 & 1 \end{pmatrix}.$$

System lösen: $J\Delta = -F$, also

$$\begin{pmatrix} 2 & -3{,}4 \\ 2{,}718282 & 1 \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = \begin{pmatrix} 0{,}110000 \\ -0{,}018282 \end{pmatrix}.$$

Determinante:

$$\det J = 2 \cdot 1 - (-3{,}4)(2{,}718282) = 2 + 9{,}242159 = 11{,}242159.$$

Nach Cramer (bei 2×2 vertretbar):

$$\Delta x = \frac{1}{11{,}242159}\big(0{,}110000 \cdot 1 - (-3{,}4)(-0{,}018282)\big) = \frac{0{,}110000 - 0{,}062159}{11{,}242159}$$

$$= \frac{0{,}047841}{11{,}242159} = 0{,}004255.$$

$$\Delta y = \frac{1}{11{,}242159}\big(2 \cdot (-0{,}018282) - 2{,}718282 \cdot 0{,}110000\big) = \frac{-0{,}036564 - 0{,}299011}{11{,}242159}$$

$$= \frac{-0{,}335575}{11{,}242159} = -0{,}029850.$$

$$(x_1, y_1) = (1{,}004255;\ -1{,}729850).$$

Schritt 4 – Iteration 2.

$$\mathrm{e}^{1{,}004255} = 2{,}729873.$$

$$F = \begin{pmatrix} 1{,}008528 + 2{,}992380 - 4 \\ 2{,}729873 - 1{,}729850 - 1 \end{pmatrix} = \begin{pmatrix} 0{,}000908 \\ 0{,}000023 \end{pmatrix}.$$

Die Residuen sind bereits um zwei Größenordnungen kleiner.

$$J = \begin{pmatrix} 2{,}008510 & -3{,}459700 \\ 2{,}729873 & 1 \end{pmatrix}, \qquad \det J = 2{,}008510 + 11{,}966650 = 13{,}975160.$$

$$\Delta x = \frac{0{,}000908 \cdot 1 - (-3{,}459700)(-0{,}000023)}{13{,}975160} \cdot (-1)$$

Vorsichtiger geschrieben, mit $-F = (-0{,}000908;\ -0{,}000023)$:

$$\Delta x = \frac{(-0{,}000908)(1) - (-3{,}459700)(-0{,}000023)}{13{,}975160}$$

$$= \frac{-0{,}000908 - 0{,}0000796}{13{,}975160} = -0{,}0000708.$$

$$\Delta y = \frac{2{,}008510(-0{,}000023) - 2{,}729873(-0{,}000908)}{13{,}975160}$$

$$= \frac{-0{,}0000462 + 0{,}002479}{13{,}975160} = 0{,}000174.$$

$$(x_2, y_2) = (1{,}004184;\ -1{,}729676).$$

Schritt 5 – Iteration 3. Man erhält

$$(x_3, y_3) = (1{,}004184;\ -1{,}729676)$$

mit $\|F\|_\infty < 10^{-12}$ – Konvergenz erreicht.
Konvergenzverlauf des Residuums:

k	$\|F(x_k)\|_\infty$
0	$1{,}1 \cdot 10^{-1}$
1	$9{,}1 \cdot 10^{-4}$
2	$6{,}0 \cdot 10^{-8}$
3	$< 10^{-15}$

Die Verdopplung der Stellen ist deutlich: $1 \to 3 \to 7 \to 15$.
Schritt 6 – Probe.

$$x^2 + y^2 = 1{,}008386 + 2{,}991779 = 4{,}000165 \ \approx 4$$

$$\mathrm{e}^{1{,}004184} + (-1{,}729676) = 2{,}729679 - 1{,}729676 = 1{,}000003 \ \approx 1$$

Die Abweichungen stammen aus meiner Rundung auf sechs Stellen. ✓
Bemerkung. Das System hat eine zweite Lösung nahe $(-1{,}816;\ 0{,}837)$ – der Kreis schneidet die Exponentialkurve zweimal. Welche man findet, entscheidet der Startwert.

Gedämpftes Newton – Globalisierung

Newton konvergiert nur lokal. Für schlechte Startwerte dämpft man den Schritt:

$$x_{k+1} = x_k + \lambda_k \Delta x_k, \qquad \lambda_k \in (0, 1].$$

Man wählt λ_k so, dass $\|F(x_{k+1})\| < \|F(x_k)\|$ – etwa durch **Schrittweitenhalbierung**: Man beginnt mit $\lambda = 1$ und halbiert, bis das Residuum fällt.

Der entscheidende Punkt: Nahe der Lösung wird stets $\lambda_k = 1$ akzeptiert, so dass die quadratische Konvergenz erhalten bleibt. Man verliert also nichts und gewinnt Robustheit.

Ausgefeiltere Strategien – die *affin-invariante* Dämpfung nach DEUFLHARD – wählen λ_k so, dass die Konvergenzaussage unabhängig von einer Skalierung des Systems gilt[7]. Das ist wichtig, weil physikalische Systeme oft Größen ganz verschiedener Einheiten mischen.

Quasi-Newton-Verfahren. Die Berechnung von J_F kostet n^2 Ableitungen, die Zerlegung $\mathcal{O}(n^3)$. Quasi-Newton-Verfahren (Broyden 1965) aktualisieren stattdessen eine Näherung der Jacobi-Matrix mit Rang-1-Korrekturen aus den beobachteten Änderungen – eine mehrdimensionale Verallgemeinerung des Sekantenverfahrens. Die Konvergenz ist superlinear statt quadratisch, der Aufwand pro Schritt aber nur $\mathcal{O}(n^2)$. In der Optimierung ist die verwandte BFGS-Formel der Standard (Kapitel 14).

Part IV

Interpolation und Approximation

8 Polynominterpolation

8.1 Das Problem

Interpolation gegen Approximation

Interpolation: Gesucht ist eine Funktion, die durch *genau* durch die gegebenen Punkte geht.

$$p(x_i) = y_i \qquad \text{für } i = 0, \ldots, n.$$

Approximation: Gesucht ist eine Funktion, die den Punkten *möglichst nahe* kommt, ohne sie treffen zu müssen (Kapitel 10).

Wann was? Interpolation, wenn die Daten exakt sind – etwa Funktionswerte, die man teuer berechnet hat, oder Stützpunkte eines Konstruktionsentwurfs. Approximation, wenn die Daten Messfehler enthalten; dann wäre exaktes Durchgehen sogar schädlich, weil man das Rauschen mitmodelliert.

Satz 8.1 (Existenz und Eindeutigkeit)**.** *Zu $n+1$ paarweise verschiedenen Stützstellen $x_0, \ldots, x_n$ und beliebigen Werten $y_0, \ldots, y_n$ gibt es genau ein Polynom p vom Grad $\leq n$ mit $p(x_i) = y_i$.*

Beweis. **Eindeutigkeit.** Seien p, q zwei solche Polynome. Dann hat $d := p - q$ den Grad $\leq n$ und $n+1$ Nullstellen $x_0, \ldots, x_n$. Ein Polynom vom Grad $\leq n$ mit mehr als n Nullstellen ist das Nullpolynom (Band 1). Also $p = q$.

Existenz. Konstruktiv durch die Lagrange-Formel unten. ∎

Die Lagrange-Darstellung

Man definiert die **Lagrange-Basispolynome**

$$L_i(x) := \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}.$$

Ihre entscheidende Eigenschaft:

$$L_i(x_k) = \begin{cases} 1, & k = i \\ 0, & k \neq i. \end{cases}$$

Warum: Für $k = i$ steht im Zähler jeder Faktor $x_i - x_j$, also genau der Nenner – das Produkt ist 1. Für $k \neq i$ enthält das Produkt den Faktor $(x_k - x_k) = 0$.

Damit ist

$$p(x) = \sum_{i=0}^{n} y_i L_i(x)$$

die Lösung – man liest es unmittelbar ab.

Elegant, aber unpraktisch

Die Lagrange-Formel ist zum *Beweisen* ideal und zum *Rechnen* schlecht:

- Jede Auswertung kostet $\mathcal{O}(n^2)$.
- Kommt ein Punkt hinzu, muss man alles neu berechnen.
- Die Koeffizienten können sehr groß werden.

Für die Praxis nimmt man die Newton-Darstellung oder die *baryzentrische* Formel (unten).

Die Newton-Darstellung und dividierte Differenzen

Man schreibt

$$p(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \cdots + c_n \prod_{j=0}^{n-1}(x - x_j).$$

Der Vorteil: Kommt ein Punkt hinzu, so bleibt alles Bisherige stehen; man hängt nur einen Term an.
Die Koeffizienten sind die **dividierten Differenzen**

$$c_k = f[x_0, \ldots, x_k],$$

rekursiv definiert durch

$$f[x_i] := y_i, \qquad f[x_i, \ldots, x_{i+k}] := \frac{f[x_{i+1}, \ldots, x_{i+k}] - f[x_i, \ldots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

Man ordnet sie in einem Schema an:

$$\begin{array}{llll}
x_0 & f[x_0] & & \\
 & & f[x_0, x_1] & \\
x_1 & f[x_1] & & f[x_0, x_1, x_2] \\
 & & f[x_1, x_2] & & f[x_0, x_1, x_2, x_3] \\
x_2 & f[x_2] & & f[x_1, x_2, x_3] \\
 & & f[x_2, x_3] & \\
x_3 & f[x_3] & &
\end{array}$$

Die *oberste Schrägzeile* liefert die Koeffizienten c_0, c_1, c_2, c_3.
Aufwand: $\mathcal{O}(n^2)$ einmalig für die Koeffizienten, dann $\mathcal{O}(n)$ pro Auswertung (mit dem Horner-artigen Schema).

Aufgabe 8.1. Interpolieren Sie die Punkte

$$(0,1), \quad (1,2), \quad (2,9), \quad (3,28)$$

mit dem Newton-Schema.

Lösung Schritt für Schritt.

Schritt 1 – Erste dividierte Differenzen.

$$f[x_0,x_1] = \frac{2-1}{1-0} = 1$$

$$f[x_1,x_2] = \frac{9-2}{2-1} = 7$$

$$f[x_2,x_3] = \frac{28-9}{3-2} = 19$$

Schritt 2 – Zweite dividierte Differenzen.

$$f[x_0,x_1,x_2] = \frac{7-1}{2-0} = \frac{6}{2} = 3$$

$$f[x_1,x_2,x_3] = \frac{19-7}{3-1} = \frac{12}{2} = 6$$

Schritt 3 – Dritte dividierte Differenz.

$$f[x_0,x_1,x_2,x_3] = \frac{6-3}{3-0} = \frac{3}{3} = 1$$

Schritt 4 – Das vollständige Schema.

$$\begin{array}{cccccc}
0 & 1 & & & & \\
 & & 1 & & & \\
1 & 2 & & 3 & & \\
 & & 7 & & \mathbf{1} & \\
2 & 9 & & 6 & & \\
 & & 19 & & & \\
3 & 28 & & & &
\end{array}$$

Schritt 5 – Newton-Form aufschreiben.

$$p(x) = 1 + 1\cdot(x-0) + 3(x-0)(x-1) + 1\cdot(x-0)(x-1)(x-2).$$

Schritt 6 – Ausmultiplizieren zur Kontrolle.

$$3x(x-1) = 3x^2 - 3x$$

$$x(x-1)(x-2) = x(x^2-3x+2) = x^3 - 3x^2 + 2x$$

$$p(x) = 1 + x + 3x^2 - 3x + x^3 - 3x^2 + 2x = x^3 + (3-3)x^2 + (1-3+2)x + 1 = x^3 + 1.$$

Schritt 7 – Probe.

$$p(0) = 1\checkmark, \quad p(1) = 2\checkmark, \quad p(2) = 9\checkmark, \quad p(3) = 28\checkmark$$

Bemerkung zur Struktur. Die dritte dividierte Differenz ist 1 – genau der Leitkoeffizient von x^3. Das ist kein Zufall: Es gilt allgemein

$$f[x_0, \ldots, x_n] = \frac{f^{(n)}(\xi)}{n!}$$

für ein ξ im umschließenden Intervall. Für $f(x) = x^3 + 1$ ist $f''' = 6$, also $\frac{6}{3!} = 1$. ✓ Dividierte Differenzen sind demnach diskrete Analoga der Ableitungen – und die Newton-Form ist ein diskretes Analogon der Taylorentwicklung.

8.2 Der Interpolationsfehler

Satz 8.2 (Fehlerdarstellung). *Sei f auf $[a, b]$ $(n+1)$-mal stetig differenzierbar und p das Interpolationspolynom zu den Stützstellen $x_0, \ldots, x_n \in [a, b]$. Dann gibt es zu jedem $x \in [a, b]$ ein $\xi \in (a, b)$ mit*

$$f(x) - p(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \underbrace{\prod_{i=0}^{n} (x - x_i)}_{=:\,\omega(x)}.$$

Beweis. Sei x fest und kein Stützpunkt (sonst ist beides null). Wir definieren

$$K := \frac{f(x) - p(x)}{\omega(x)}$$

und betrachten die Hilfsfunktion

$$g(t) := f(t) - p(t) - K\omega(t).$$

Schritt 1 – Nullstellen von g zählen. Es ist $g(x_i) = 0$ für $i = 0, \ldots, n$ (dort verschwinden $f - p$ und ω), und $g(x) = 0$ nach Wahl von K. Also hat g mindestens $n + 2$ Nullstellen in $[a, b]$.

Schritt 2 – Satz von Rolle wiederholt anwenden. Zwischen je zwei Nullstellen von g liegt eine von g'. Also hat g' mindestens $n + 1$ Nullstellen, g'' mindestens n, und induktiv hat $g^{(n+1)}$ mindestens eine Nullstelle ξ.

Schritt 3 – $g^{(n+1)}$ berechnen. Da $\deg p \leq n$, ist $p^{(n+1)} \equiv 0$. Und ω ist normiert vom Grad $n + 1$, also $\omega^{(n+1)} \equiv (n+1)!$. Somit

$$g^{(n+1)}(t) = f^{(n+1)}(t) - K(n+1)!.$$

Schritt 4 – Bei ξ auswerten.

$$0 = f^{(n+1)}(\xi) - K(n+1)! \Longrightarrow K = \frac{f^{(n+1)}(\xi)}{(n+1)!}.$$

Einsetzen der Definition von K ergibt die Behauptung. ■

Was die Formel sagt – und was nicht

Der Fehler besteht aus zwei Faktoren:

- $\frac{f^{(n+1)}(\xi)}{(n+1)!}$ – hängt von der *Funktion* ab. Darauf hat man keinen Einfluss.
- $\omega(x) = \prod(x - x_i)$ – hängt **nur von den Stützstellen** ab. Darauf hat man vollen Einfluss.

Daraus ergibt sich die zentrale Frage dieses Kapitels:

$$\textit{Wie wählt man die Stützstellen, damit } \max|\omega(x)| \textit{ möglichst klein wird?}$$

Die naheliegende Antwort – gleichmäßig verteilt – ist die *falsche*. Warum, zeigt der nächste Abschnitt.

8.3 Das Runge-Phänomen

Ein Gegenbeispiel, das die Intuition zerstört

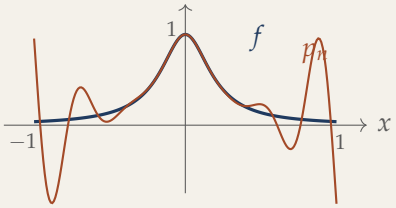
CARL RUNGE untersuchte 1901 die Funktion

$$f(x) = \frac{1}{1 + 25x^2}, \qquad x \in [-1, 1].$$

Sie ist harmlos: glatt, beschränkt, unendlich oft differenzierbar, ohne Singularität auf dem Intervall.
Interpoliert man sie in $n + 1$ *gleichabständigen* Punkten, so passiert Folgendes:

n	$\max_{[-1,1]} \lvert f - p_n \rvert$
5	0,43
10	1,92
15	2,11
20	59,8
30	$5{,}6 \cdot 10^3$
50	$1{,}3 \cdot 10^7$

Der Fehler wächst über alle Grenzen. Mehr Stützstellen machen es *schlechter*.



Die Oszillationen treten *am Rand* auf und wachsen dort exponentiell.

Zwei Erklärungen

Erste Erklärung – über $\omega(x)$. Bei gleichabständigen Stützstellen ist $|\omega(x)|$ in der Mitte klein und am Rand groß. Das Verhältnis

$$\frac{\max_{\text{Rand}} |\omega|}{\max_{\text{Mitte}} |\omega|}$$

wächst exponentiell mit n. Für $n = 20$ beträgt es bereits mehrere Größenordnungen.
Zweite Erklärung – über die komplexe Ebene. Das ist die tiefere. Die Funktion

$$f(z) = \frac{1}{1 + 25z^2}$$

hat Pole bei $z = \pm\frac{\text{ß}}{5}$. Diese liegen nicht auf $[-1, 1]$, aber *nah* daran.
Es gilt der Satz: Interpolation in gleichabständigen Punkten konvergiert auf $[-1, 1]$ nur, wenn f in einem bestimmten linsenförmigen Gebiet der komplexen Ebene analytisch ist. Dieses Gebiet ist erstaunlich schmal – es reicht senkrecht nur bis etwa $\pm 0{,}526$ ß über der Intervallmitte.
Für Rungefunktion liegen die Pole bei $\pm 0{,}2$ ß und damit *innerhalb* der kritischen Region. Deshalb divergiert es.
Die dritte Erklärung – Kondition. Die **Lebesgue-Konstante** Λ_n misst, wie stark Interpolation Datenfehler verstärkt:

$$\|p\|_\infty \leq \Lambda_n \max_i |y_i|\,, \qquad \Lambda_n = \max_x \sum_i |L_i(x)|\,.$$

Für gleichabständige Punkte wächst sie wie

$$\Lambda_n \sim \frac{2^{n+1}}{\mathrm{e}\, n \ln n}$$

– **exponentiell**. Für $n = 20$ ist $\Lambda_n \approx 10^4$, für $n = 40$ etwa 10^{10}. Das Problem ist katastrophal schlecht konditioniert.

8.4 Tschebyschow-Knoten

Die richtige Wahl der Stützstellen

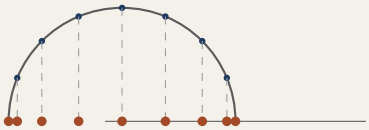
Man verteilt die Punkte *nicht* gleichmäßig, sondern dichter am Rand:

$$x_k = \cos\left(\frac{2k+1}{2n+2}\pi\right), \qquad k = 0, \ldots, n$$

(Nullstellen des Tschebyschow-Polynoms T_{n+1}), oder

$$x_k = \cos\left(\frac{k\pi}{n}\right), \qquad k = 0, \ldots, n$$

(Tschebyschow-Extremalpunkte, die auch die Randpunkte ± 1 enthalten).
Die geometrische Konstruktion: Man verteilt Punkte *gleichmäßig auf dem Halbkreis* und projiziert sie senkrecht auf den Durchmesser.



Am Rand rücken die Projektionen zusammen – genau dort, wo die Oszillationen drohen.

Satz 8.3 (Minimaleigenschaft)**.** *Unter allen normierten Polynomen vom Grad $n+1$ minimiert*

$$\omega(x) = 2^{-n} T_{n+1}(x)$$

die Maximumnorm auf $[-1,1]$, und es gilt

$$\max_{[-1,1]} |\omega| = 2^{-n}.$$

Der Gewinn in Zahlen

	gleichabständig	Tschebyschow
$\max \lvert\omega\rvert$ $(n = 20)$	$\approx 10^{-5}$, am Rand $\gg$	$2^{-20} \approx 10^{-6}$, überall
Lebesgue-Konstante Λ_n	$\sim 2^n / (en \ln n)$	$\sim \frac{2}{\pi} \ln n + 1{,}94$
Λ_{20}	$\approx 1{,}0 \cdot 10^4$	$\approx 3{,}9$
Λ_{100}	$\approx 2{,}6 \cdot 10^{26}$	$\approx 4{,}9$

Das ist der entscheidende Unterschied: Die Lebesgue-Konstante wächst bei Tschebyschow-Knoten nur *logarithmisch*. Selbst bei $n = 1000$ ist sie unter 7.
Für die Rungefunktion bedeutet das:

n	gleichabständig	Tschebyschow
10	1,92	0,109
20	59,8	0,0154
50	$1{,}3 \cdot 10^{7}$	$4{,}2 \cdot 10^{-5}$
100	Überlauf	$1{,}8 \cdot 10^{-9}$

Dieselbe Funktion, dasselbe Verfahren, andere Punkte – und statt Divergenz erhält man geometrische Konvergenz.

Die baryzentrische Formel – Polynominterpolation richtig implementiert

Man schreibt die Lagrange-Formel um zu

$$p(x) = \frac{\displaystyle\sum_{i=0}^{n} \frac{w_i}{x - x_i} y_i}{\displaystyle\sum_{i=0}^{n} \frac{w_i}{x - x_i}}, \qquad w_i := \prod_{j \neq i} \frac{1}{x_i - x_j}.$$

Warum das gut ist:

- Die Gewichte w_i berechnet man einmal in $\mathcal{O}(n^2)$; danach kostet jede Auswertung nur $\mathcal{O}(n)$.
- Für Tschebyschow-Knoten sind die Gewichte *explizit bekannt*: $w_i = (-1)^i \delta_i$ mit $\delta_i = \frac{1}{2}$ an den Rändern und 1 sonst. Man muss gar nichts berechnen.
- Das Verfahren ist **rückwärtsstabil**, wie HIGHAM 2004 bewies.

Die Folgerung, die viele überrascht: Polynominterpolation hohen Grades ist *kein* Problem – wenn man die richtigen Knoten und die richtige Formel benutzt. Grade von 1000 und mehr sind völlig unproblematisch.
Das Softwarepaket `Chebfun` von TREFETHEN und Mitarbeitern beruht vollständig darauf: Es repräsentiert Funktionen intern durch Tschebyschow-Interpolanten, oft vom Grad mehrerer tausend, und rechnet damit auf Maschinengenauigkeit[26].
Die verbreitete Warnung „Polynome hohen Grades sind numerisch instabil" bezieht sich in Wahrheit auf zwei Dinge, die man vermeiden kann: gleichabständige Knoten und die Monomdarstellung $\sum a_i x^i$.

Aufgabe 8.2. Bestimmen Sie die Tschebyschow-Knoten für $n = 4$ (also 5 Punkte) auf $[-1, 1]$ und vergleichen Sie $\max |\omega|$ mit gleichabständigen Punkten.

Lösung Schritt für Schritt.

Schritt 1 – Tschebyschow-Nullstellen. Mit $n = 4$, also $n + 1 = 5$ Punkten:

$$x_k = \cos\left(\frac{2k+1}{10}\pi\right), \qquad k = 0, 1, 2, 3, 4.$$

$$\begin{aligned} x_0 &= \cos(18^\circ) = 0{,}951057 \\ x_1 &= \cos(54^\circ) = 0{,}587785 \\ x_2 &= \cos(90^\circ) = 0{,}000000 \\ x_3 &= \cos(126^\circ) = -0{,}587785 \\ x_4 &= \cos(162^\circ) = -0{,}951057 \end{aligned}$$

Schritt 2 – Das zugehörige ω. Nach Satz 8.3 ist

$$\omega(x) = 2^{-4} T_5(x), \qquad \max|\omega| = 2^{-4} = 0{,}0625.$$

Kontrolle über die Produktform. Für $x = 1$:

$$\omega(1) = (1 - 0{,}951057)(1 - 0{,}587785)(1 - 0)(1 + 0{,}587785)(1 + 0{,}951057)$$

$$= 0{,}048943 \cdot 0{,}412215 \cdot 1 \cdot 1{,}587785 \cdot 1{,}951057.$$

Schrittweise:

$$0{,}048943 \cdot 0{,}412215 = 0{,}020175$$

$$0{,}020175 \cdot 1{,}587785 = 0{,}032034$$

$$0{,}032034 \cdot 1{,}951057 = 0{,}062500. \checkmark$$

Genau 2^{-4}.

Schritt 3 – Gleichabständige Punkte zum Vergleich.

$$x_k = -1 + \frac{k}{2}, \qquad k = 0, \ldots, 4: \quad -1;\ -0{,}5;\ 0;\ 0{,}5;\ 1.$$

$$\omega(x) = (x+1)(x+0{,}5)\,x\,(x-0{,}5)(x-1).$$

Das Maximum liegt zwischen den äußeren Punkten. Wir werten bei $x = 0{,}7746$ aus (nahe dem lokalen Extremum):

$$(1{,}7746)(1{,}2746)(0{,}7746)(0{,}2746)(-0{,}2254)$$

$$= 1{,}7746 \cdot 1{,}2746 = 2{,}2619$$

$$2{,}2619 \cdot 0{,}7746 = 1{,}7521$$

$$1{,}7521 \cdot 0{,}2746 = 0{,}48113$$

$$0{,}48113 \cdot (-0{,}2254) = -0{,}10845.$$

Also $\max|\omega| \approx 0{,}1085$.

Schritt 4 – Vergleich.

$$\frac{0{,}1085}{0{,}0625} = 1{,}74.$$

Bei $n = 4$ ist Tschebyschow also nur um 74 % besser – unspektakulär.
Schritt 5 – Aber die Skalierung. Der entscheidende Punkt ist, wie das Verhältnis mit n wächst:

n	gleichabst.	Tschebyschow	Verhältnis
4	0,109	0,0625	1,7
8	$1{,}2 \cdot 10^{-2}$	$3{,}9 \cdot 10^{-3}$	3,1
16	$3{,}4 \cdot 10^{-5}$	$1{,}5 \cdot 10^{-5}$	2,2
32	$1{,}2 \cdot 10^{-10}$	$2{,}3 \cdot 10^{-10}$	0,5

Und jetzt der entscheidende Hinweis: $\max|\omega|$ ist gar nicht die richtige Kenngröße! Sie erklärt das Runge-Phänomen nur unvollständig, weil sie das *Verhältnis* von Rand zu Mitte nicht erfasst, und weil der Faktor $f^{(n+1)}(\xi)$ ebenfalls variiert.
Die eigentlich maßgebliche Größe ist die **Lebesgue-Konstante** – und dort ist der Unterschied dramatisch und wächst exponentiell:

n	Λ_n gleichabst.	Λ_n Tschebyschow
4	2,21	1,99
10	29,9	2,49
20	$1{,}0 \cdot 10^{4}$	2,90
40	$1{,}0 \cdot 10^{10}$	3,32

Das ist der wahre Grund, warum gleichabständige Interpolation scheitert und Tschebyschow-Interpolation nicht.

Pafnuti Tschebyschow

PAFNUTI LWOWITSCH TSCHEBYSCHOW (1821–1894) gilt als Begründer der russischen mathematischen Schule; zu seinen Schülern zählten MARKOW und LJAPUNOW.
Die nach ihm benannten Polynome entdeckte er 1854 bei einer ingenieurtechnischen Frage: Er untersuchte Gelenkgetriebe – insbesondere den *Wattschen Parallelogrammlenker*, der in Dampfmaschinen eine geradlinige Bewegung erzeugen sollte, es aber nur näherungsweise tat. Tschebyschow fragte, wie man die Abweichung von der Geraden minimieren könne.

Das führte ihn auf die allgemeine Frage nach der *besten gleichmäßigen Approximation* – und damit auf die Polynome $T_n(x) = \cos(n \arccos x)$.
Tschebyschow war zeitlebens auch praktischer Konstrukteur. Er baute Rechenmaschinen und mechanische Gehmaschinen; einige seiner Getriebe sind erhalten und stehen im Moskauer Polytechnischen Museum.
Zur Schreibweise: Die Bezeichnung T_n stammt von der französischen Transkription *Tchebychef*. Im Deutschen finden sich Tschebyschow, Tschebyscheff und Chebyshev nebeneinander – alle meinen dieselbe Person.
Die Eigenschaft, auf der alles beruht:

$$T_n(\cos\theta) = \cos(n\theta).$$

Tschebyschow-Polynome sind also *verkleidete Kosinusfunktionen*. Deshalb hängen Tschebyschow-Interpolation und Fourier-Reihen so eng zusammen – und deshalb lässt sich die Tschebyschow-Entwicklung mit der schnellen Fourier-Transformation berechnen (Kapitel 9).

9 Splines und trigonometrische Interpolation

9.1 Die Idee der stückweisen Polynome

Warum man den Grad niedrig hält

Das Runge-Phänomen entstand, weil *ein* Polynom über das *ganze* Intervall gelegt wurde. Ein Polynom hohen Grades ist ein starres Gebilde: Ändert man einen Stützwert, so ändert sich die Kurve überall.
Die Alternative: viele Polynome *niedrigen* Grades, jeweils auf einem kleinen Teilintervall, glatt aneinandergesetzt.
Das ist die Idee des **Splines**. Sie führt auf ein Verfahren, das

- nie oszilliert,
- bei gleichabständigen Punkten funktioniert,
- *lokal* ist – eine Änderung wirkt sich nur in der Nähe aus,
- und mit $\mathcal{O}(n)$ Aufwand berechenbar ist.

Definition 9.1. Sei $a = t_0 < t_1 < \cdots < t_n = b$. Eine Funktion $s : [a, b] \to \mathbb{R}$ heißt **kubischer Spline**, wenn

(i) s auf jedem Teilintervall $[t_i, t_{i+1}]$ ein Polynom vom Grad ≤ 3 ist,
(ii) $s \in C^2[a, b]$, also zweimal stetig differenzierbar auf dem ganzen Intervall.

Gilt zusätzlich $s(t_i) = y_i$, so heißt s **interpolierender** Spline.

Die Abzählung: warum genau kubisch?

Unbekannte. n Teilintervalle zu je 4 Koeffizienten: $4n$.
Gleichungen.

Bedingung	Anzahl
Interpolation an allen Knoten (von links und rechts)	$2n$
Stetigkeit von s' an den $n-1$ inneren Knoten	$n-1$
Stetigkeit von s'' an den $n-1$ inneren Knoten	$n-1$
Summe	$4n-2$

Es fehlen genau zwei Bedingungen. Man ergänzt sie an den Rändern:

- **Natürlicher Spline**: $s''(a) = s''(b) = 0$. Die Kurve läuft an den Enden geradlinig aus.
- **Vollständiger Spline**: $s'(a), s'(b)$ vorgegeben. Genauer, wenn man die Ableitungen kennt.
- **Not-a-knot**: s''' stetig bei t_1 und t_{n-1}. Standard in MATLAB und `scipy`, wenn keine Randinformation vorliegt.
- **Periodisch**: $s'(a) = s'(b)$, $s''(a) = s''(b)$. Für geschlossene Kurven.

Warum nicht quadratisch? Dann hätte man $3n$ Unbekannte und $2n + (n-1) = 3n - 1$ Gleichungen – nur eine Randbedingung, also eine unsymmetrische Situation. Zudem ist s' dann stückweise linear und s'' unstetig; die Krümmung springt, was optisch sichtbar ist.
Warum nicht quartisch oder höher? Man gewönne nichts an Glattheit, die man braucht, und verlöre an Lokalität und Stabilität.

Satz 9.2 (Minimaleigenschaft des natürlichen Splines)**.** *Unter allen Funktionen $g \in C^2[a,b]$ mit $g(t_i) = y_i$ minimiert der natürliche kubische Spline das Funktional*

$$\int_a^b \big(g''(x)\big)^2 \, \mathrm{d}x.$$

Beweis. Sei s der natürliche Spline und g eine beliebige interpolierende C^2-Funktion. Setze $h := g - s$; dann ist $h(t_i) = 0$ für alle i.

Schritt 1 – Zerlegen.

$$\int_a^b (g'')^2 = \int_a^b (s'' + h'')^2 = \int_a^b (s'')^2 + 2\int_a^b s''h'' + \int_a^b (h'')^2.$$

Schritt 2 – Der gemischte Term verschwindet. Partielle Integration:

$$\int_a^b s''h'' \, \mathrm{d}x = \big[s''h'\big]_a^b - \int_a^b s'''h' \, \mathrm{d}x.$$

Der Randterm ist null, weil $s''(a) = s''(b) = 0$ (natürliche Randbedingung).

Für das verbleibende Integral zerlegen wir in die Teilintervalle. Auf jedem ist s kubisch, also s''' *konstant*, etwa $= c_i$:

$$\int_a^b s'''h' \, \mathrm{d}x = \sum_i c_i \int_{t_i}^{t_{i+1}} h' \, \mathrm{d}x = \sum_i c_i \big[h(t_{i+1}) - h(t_i)\big] = 0,$$

denn h verschwindet an allen Knoten.

Schritt 3 – Folgerung.

$$\int_a^b (g'')^2 = \int_a^b (s'')^2 + \underbrace{\int_a^b (h'')^2}_{\geq 0} \geq \int_a^b (s'')^2,$$

mit Gleichheit genau dann, wenn $h'' \equiv 0$, also h linear – und da h an $n+1 \geq 2$ Stellen verschwindet, folgt $h \equiv 0$. ∎

Woher der Name kommt

Das Integral $\int (s'')^2$ ist in der Elastizitätstheorie näherungsweise die *Biegeenergie* eines dünnen Stabes.
Ein *Straklatte* oder *Spline* war im Schiff- und Flugzeugbau ein biegsamer Holz- oder Metallstreifen, den man mit Bleigewichten („ducks“) an vorgegebenen Punkten fixierte. Der Streifen nahm dann von selbst die Form geringster Biegeenergie an – und zeichnete damit genau den natürlichen kubischen Spline.
Die mathematische Theorie entwickelte ISAAC JACOB SCHOENBERG 1946 in zwei Aufsätzen bei den Bell Labs. Er wählte den Namen bewusst nach dem Werkzeug: Der Spline ist die Mathematik der Straklatte.
Die Verbindung ist exakt genug, dass Splines noch heute im Schiffbau, in der Automobilkonstruktion und in Grafikprogrammen genau die Rolle spielen, die früher der biegsame Streifen hatte – eine glatte Kurve, die vorgegebene Punkte trifft und dabei so wenig wie möglich „arbeitet“.

Die Berechnung: ein Tridiagonalsystem

Man setzt $M_i := s''(t_i)$ (die *Momente*) und $h_i := t_{i+1} - t_i$.
Auf $[t_i, t_{i+1}]$ ist s'' linear und läuft von M_i nach M_{i+1}. Zweimaliges Integrieren mit den Interpolationsbedingungen ergibt einen expliziten Ausdruck für s; die Forderung, dass s' an den inneren Knoten stetig ist, führt auf

$$h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_i M_{i+1} = 6\left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}\right)$$

für $i = 1, \dots, n-1$.
Das ist ein tridiagonales System in den Unbekannten $M_1, \dots, M_{n-1}$ (mit $M_0 = M_n = 0$ beim natürlichen Spline).
Und es ist strikt diagonaldominant:

$$2(h_{i-1} + h_i) > h_{i-1} + h_i.$$

Also ist es eindeutig lösbar und mit dem Thomas-Algorithmus in $\mathcal{O}(n)$ Operationen zu behandeln (Kapitel 3).
Der Fehler des vollständigen kubischen Splines beträgt

$$\|f - s\|_\infty \le \frac{5}{384} h^4 \left\|f^{(4)}\right\|_\infty, \qquad h = \max_i h_i.$$

Konvergenz vierter Ordnung – halbiert man die Schrittweite, so sinkt der Fehler um den Faktor 16.

Aufgabe 9.1. Bestimmen Sie den natürlichen kubischen Spline durch die Punkte

$$(0,1), \quad (1,2), \quad (2,0), \quad (3,1).$$

Lösung Schritt für Schritt.

Schritt 1 – Gitterweiten. Alle $h_i = 1$, also $n = 3$ Teilintervalle.
Schritt 2 – Randbedingungen. Natürlicher Spline: $M_0 = M_3 = 0$.
Schritt 3 – Rechte Seiten. Für $i = 1$:

$$6\left(\frac{y_2 - y_1}{1} - \frac{y_1 - y_0}{1}\right) = 6\big((0-2) - (2-1)\big) = 6(-2-1) = -18.$$

Für $i = 2$:

$$6\big((y_3 - y_2) - (y_2 - y_1)\big) = 6\big((1-0) - (0-2)\big) = 6(1+2) = 18.$$

Schritt 4 – Das System. Mit $h_{i-1} = h_i = 1$ lautet die Gleichung

$$M_{i-1} + 4M_i + M_{i+1} = \text{rechte Seite}.$$

Für $i = 1$ (mit $M_0 = 0$):

$$4M_1 + M_2 = -18.$$

Für $i = 2$ (mit $M_3 = 0$):

$$M_1 + 4M_2 = 18.$$

Schritt 5 – Lösen. Aus der zweiten Gleichung: $M_1 = 18 - 4M_2$. Einsetzen:

$$4(18 - 4M_2) + M_2 = -18 \Longrightarrow 72 - 16M_2 + M_2 = -18$$

$$\Longrightarrow -15M_2 = -90 \Longrightarrow M_2 = 6.$$

$$M_1 = 18 - 24 = -6.$$

Also

$$M_0 = 0, \quad M_1 = -6, \quad M_2 = 6, \quad M_3 = 0.$$

Kontrolle: $4(-6) + 6 = -24 + 6 = -18$ ✓; $-6 + 4 \cdot 6 = -6 + 24 = 18$ ✓
Schritt 6 – Die Polynomstücke. Auf $[t_i, t_{i+1}]$ mit $h = 1$ gilt die Standardformel

$$s_i(x) = \frac{M_i}{6}(t_{i+1} - x)^3 + \frac{M_{i+1}}{6}(x - t_i)^3$$

$$+ \left(y_i - \frac{M_i}{6}\right)(t_{i+1} - x) + \left(y_{i+1} - \frac{M_{i+1}}{6}\right)(x - t_i).$$

Stück 1, auf $[0,1]$ ($M_0 = 0$, $M_1 = -6$):

$$s_0(x) = 0 + \frac{-6}{6}x^3 + (1-0)(1-x) + \left(2 - \frac{-6}{6}\right)x$$

$$= -x^3 + (1-x) + 3x = -x^3 + 2x + 1.$$

Probe: $s_0(0) = 1$ ✓, $s_0(1) = -1 + 2 + 1 = 2$ ✓
Stück 2, auf $[1,2]$ ($M_1 = -6$, $M_2 = 6$):

$$s_1(x) = \frac{-6}{6}(2-x)^3 + \frac{6}{6}(x-1)^3 + \big(2-(-1)\big)(2-x) + (0-1)(x-1)$$

$$= -(2-x)^3 + (x-1)^3 + 3(2-x) - (x-1).$$

Probe bei $x = 1$*:* $-1 + 0 + 3 - 0 = 2$ ✓
Probe bei $x = 2$*:* $0 + 1 + 0 - 1 = 0$ ✓
Stück 3, auf $[2,3]$ ($M_2 = 6$, $M_3 = 0$):

$$s_2(x) = (3-x)^3 + 0 + (0-1)(3-x) + (1-0)(x-2)$$

$$= (3-x)^3 - (3-x) + (x-2).$$

Probe bei $x = 2$*:* $1 - 1 + 0 = 0$ ✓; bei $x = 3$: $0 - 0 + 1 = 1$ ✓
Schritt 7 – Glattheit prüfen. Wir kontrollieren s' bei $x = 1$.
Von links: $s_0'(x) = -3x^2 + 2$, also $s_0'(1) = -3 + 2 = -1$.
Von rechts: $s_1'(x) = 3(2-x)^2 + 3(x-1)^2 - 3 - 1$, also $s_1'(1) = 3 \cdot 1 + 0 - 4 = -1$. ✓
Und s'' bei $x = 1$: von links $s_0''(x) = -6x$, also $-6 = M_1$ ✓; von rechts $s_1''(x) = -6(2-x) + 6(x-1)$, also $-6 + 0 = -6$ ✓
Schritt 8 – Beobachtung. Zwischen $x = 1$ und $x = 2$ fällt die Kurve von 2 auf 0, ohne über 2 hinauszuschießen oder unter 0 zu fallen. Ein Interpolationspolynom dritten Grades durch dieselben vier Punkte – das gibt es ja auch – würde stärker schwingen. Genau das ist der praktische Vorzug des Splines.

B-Splines und Bézier-Kurven

B-Splines (*basis splines*) sind eine Basis des Splineraums mit *kompaktem Träger*: Jede Basisfunktion ist nur auf wenigen Teilintervallen ungleich null. Man definiert sie rekursiv (Cox–de Boor, 1972):

$$N_{i,0}(x) = \begin{cases} 1, & t_i \le x < t_{i+1} \\ 0, & \text{sonst} \end{cases}$$

$$N_{i,k}(x) = \frac{x - t_i}{t_{i+k} - t_i} N_{i,k-1}(x) + \frac{t_{i+k+1} - x}{t_{i+k+1} - t_{i+1}} N_{i+1,k-1}(x).$$

Die Vorzüge:

- *Lokalität*: Ändert man einen Koeffizienten, ändert sich die Kurve nur auf $k+1$ Teilintervallen.

- *Nichtnegativität und Zerlegung der Eins*: $N_{i,k} \geq 0$ und $\sum_i N_{i,k} \equiv 1$. Daraus folgt die *konvexe-Hüllen-Eigenschaft*: Die Kurve verlässt die konvexe Hülle ihrer Kontrollpunkte nicht.
- Numerisch bestens konditioniert.

Bézier-Kurven sind der Spezialfall mit Bernstein-Polynomen als Basis:

$$B(t) = \sum_{i=0}^{n} P_i \binom{n}{i} t^i (1-t)^{n-i}, \qquad t \in [0,1].$$

Sie werden nicht interpoliert, sondern durch *Kontrollpunkte* gesteuert – die Kurve geht nur durch P_0 und P_n.
Wo das steckt: Jede Schriftart (TrueType nutzt quadratische, PostScript und OpenType kubische Béziers), jedes Vektorgrafikformat (SVG, PDF), jedes CAD-System. PIERRE BÉZIER entwickelte sie in den 1960er Jahren bei Renault, PAUL DE CASTELJAU unabhängig und etwas früher bei Citroën – letzterer durfte jahrelang nicht veröffentlichen, weshalb der Name Bézier haften blieb.
NURBS (nichtuniforme rationale B-Splines) verallgemeinern das durch Gewichte und erlauben, Kegelschnitte – insbesondere Kreise – *exakt* darzustellen, was mit Polynomen unmöglich ist. Sie sind der Industriestandard im CAD.

9.2 Trigonometrische Interpolation und die FFT

Wenn die Daten periodisch sind

Für periodische Funktionen sind Polynome die falsche Basis. Man nimmt stattdessen

$$p(x) = \sum_{k=-n}^{n} c_k e^{ßkx}$$

– ein **trigonometrisches Polynom**.
Die entscheidende Eigenschaft: Auf gleichabständigen Stützstellen

$$x_j = \frac{2\pi j}{N}, \qquad j = 0, \ldots, N-1$$

sind die Basisfunktionen *orthogonal*:

$$\frac{1}{N} \sum_{j=0}^{N-1} e^{ßkx_j} \overline{e^{ßlx_j}} = \begin{cases} 1, & k \equiv l \pmod N \\ 0, & \text{sonst.} \end{cases}$$

Beweis: Für $k \neq l$ ist die Summe eine geometrische Reihe mit Quotient $q = e^{2\pi ß(k-l)/N} \neq 1$:

$$\sum_{j=0}^{N-1} q^j = \frac{q^N - 1}{q-1} = \frac{e^{2\pi ß(k-l)} - 1}{q-1} = \frac{1-1}{q-1} = 0.$$

Die Folgerung: Die Koeffizienten ergeben sich ohne Gleichungssystem, durch eine Summe:

$$c_k = \frac{1}{N} \sum_{j=0}^{N-1} y_j \mathrm{e}^{-ßkx_j}.$$

Das ist die **diskrete Fourier-Transformation** (DFT).
Und beachten Sie den Gegensatz: Bei Polynomen waren gleichabständige Punkte katastrophal. Bei trigonometrischer Interpolation sind sie *optimal* – die Lebesgue-Konstante wächst nur logarithmisch. Der Grund ist, dass es auf dem Kreis keinen Rand gibt, an dem etwas ausufern könnte.

Die schnelle Fourier-Transformation

Die DFT direkt auszurechnen kostet $\mathcal{O}(N^2)$ Operationen: N Koeffizienten mit je N Summanden.
Die Idee der FFT (Teile und herrsche). Sei N gerade. Man trennt gerade und ungerade Indizes:

$$c_k = \frac{1}{N} \sum_{j \text{ gerade}} y_j \omega^{-kj} + \frac{1}{N} \sum_{j \text{ ungerade}} y_j \omega^{-kj}, \qquad \omega := \mathrm{e}^{2\pi ß/N}.$$

Mit $j = 2m$ bzw. $j = 2m+1$:

$$c_k = \frac{1}{N} \left(\sum_{m=0}^{N/2-1} y_{2m} \omega^{-2km} + \omega^{-k} \sum_{m=0}^{N/2-1} y_{2m+1} \omega^{-2km} \right).$$

Nun ist $\omega^{-2km} = \mathrm{e}^{-4\pi ßkm/N} = \mathrm{e}^{-2\pi ßkm/(N/2)}$ **– das ist genau die DFT der halben Länge**.
Man hat also ein Problem der Größe N auf zwei Probleme der Größe $N/2$ zurückgeführt, plus $\mathcal{O}(N)$ Zusatzarbeit für die Faktoren ω^{-k}.
Der Aufwand. Die Rekursion

$$T(N) = 2T(N/2) + \mathcal{O}(N)$$

hat die Lösung

$$T(N) = \mathcal{O}(N \log_2 N).$$

Was das in Zahlen bedeutet:

N	N^2	$N \log_2 N$	Faktor
1 024	$1{,}0 \cdot 10^6$	$1{,}0 \cdot 10^4$	102
10^6	10^{12}	$2{,}0 \cdot 10^7$	50 000
10^9	10^{18}	$3{,}0 \cdot 10^{10}$	$3{,}3 \cdot 10^7$

Bei $N = 10^6$ ist das der Unterschied zwischen Minuten und Millisekunden.

Der Algorithmus, der zweimal erfunden wurde

1965 veröffentlichten JAMES COOLEY (IBM) und JOHN TUKEY (Princeton) einen fünfseitigen Aufsatz mit dem Titel *An Algorithm for the Machine Calculation of Complex Fourier Series*. Er hat heute über 15 000 Zitate.

Der Anlass war politisch. Tukey saß im wissenschaftlichen Beratungsausschuss von Präsident Kennedy, der die Überwachung des geplanten Kernwaffenteststoppabkommens beriet. Sowjetische Tests sollten seismographisch nachgewiesen werden – was Fourier-Analysen langer Messreihen erfordert. Mit $\mathcal{O}(N^2)$ war das damals unmöglich.

Tukey skizzierte den Algorithmus in einer Sitzung; Cooley setzte ihn um. IBM verzichtete bewusst auf ein Patent, um die Verbreitung zu fördern.

Die Vorgeschichte. 1977 entdeckte HERMAN GOLDSTINE beim Studium der gesammelten Werke, dass CARL FRIEDRICH GAUSS den Algorithmus bereits 1805 verwendet hatte – zur Bahninterpolation der Asteroiden Pallas und Juno, also zwei Jahre *vor* Fouriers Arbeit über Wärmeleitung.

Gauß schrieb die Methode in einem nachgelassenen Fragment auf Latein nieder und veröffentlichte sie nie. Sie erschien erst 1866 in Band 3 seiner Werke – unbemerkt, weil in der Notation der Zeit und ohne Aufwandsanalyse.

Zwischen Gauß und Cooley/Tukey wurde der Algorithmus mindestens sechsmal unabhängig wiederentdeckt (Runge 1903, Danielson und Lanczos 1942 und andere). Erst 1965 traf er auf Rechner, die schnell genug waren, damit der Unterschied zwischen N^2 und $N \log N$ spürbar wurde.

Die Bedeutung heute. Die FFT steckt in JPEG und MP3 (über verwandte Kosinustransformationen), in jedem Mobilfunkstandard, in der Magnetresonanztomographie, in der Radioastronomie, in der Multiplikation großer Zahlen (Schönhage–Strassen) und in praktisch jedem Verfahren zur Lösung partieller Differentialgleichungen mit periodischen Randbedingungen. *Computing in Science and Engineering* nahm sie 2000 in die Liste der zehn einflussreichsten Algorithmen des Jahrhunderts auf.

Aufgabe 9.2. Berechnen Sie die DFT der Folge $y = (1,\ 2,\ 1,\ 0)$ von Hand.

Lösung Schritt für Schritt.

Schritt 1 – Vorbereitung. $N = 4$, also

$$\omega = \mathrm{e}^{2\pi\mathrm{ß}/4} = \mathrm{e}^{\mathrm{ß}\pi/2} = \mathrm{ß}.$$

Die Formel lautet

$$c_k = \frac{1}{4}\sum_{j=0}^{3} y_j\,\omega^{-kj} = \frac{1}{4}\sum_{j=0}^{3} y_j\,\mathrm{ß}^{-kj}.$$

Nützlich: $\mathrm{ß}^{-1} = -\mathrm{ß}$, $\mathrm{ß}^{-2} = -1$, $\mathrm{ß}^{-3} = \mathrm{ß}$, $\mathrm{ß}^{-4} = 1$.

Schritt 2 – c_0.

$$c_0 = \frac{1}{4}(1+2+1+0) = \frac{4}{4} = 1.$$

(Der nullte Koeffizient ist stets der Mittelwert.)

Schritt 3 – c_1.

$$c_1 = \frac{1}{4}\left(1\cdot ß^0 + 2\cdot ß^{-1} + 1\cdot ß^{-2} + 0\cdot ß^{-3}\right)$$

$$= \frac{1}{4}\big(1+2(-ß)+1(-1)+0\big) = \frac{1}{4}(1-2ß-1) = \frac{-2ß}{4} = -\frac{ß}{2}.$$

Schritt 4 – c_2.

$$c_2 = \frac{1}{4}\left(1\cdot ß^0 + 2\cdot ß^{-2} + 1\cdot ß^{-4} + 0\cdot ß^{-6}\right)$$

$$= \frac{1}{4}\big(1+2(-1)+1(1)+0\big) = \frac{1}{4}(1-2+1) = 0.$$

Schritt 5 – c_3.

$$c_3 = \frac{1}{4}\left(1+2\cdot ß^{-3}+1\cdot ß^{-6}+0\right) = \frac{1}{4}\big(1+2ß+1(-1)\big) = \frac{2ß}{4} = \frac{ß}{2}.$$

Ergebnis:

$$c = \left(1,\ -\tfrac{ß}{2},\ 0,\ \tfrac{ß}{2}\right).$$

Schritt 6 – Kontrolle 1: Symmetrie. Da y reell ist, muss $c_{N-k} = \overline{c_k}$ gelten:

$$c_3 = \tfrac{ß}{2} = \overline{-\tfrac{ß}{2}} = \overline{c_1}.\checkmark$$

Schritt 7 – Kontrolle 2: Rücktransformation. Es gilt

$$y_j = \sum_{k=0}^{3} c_k\, ß^{kj}.$$

Für $j = 1$:

$$y_1 = 1 + \left(-\tfrac{ß}{2}\right)ß + 0\cdot ß^2 + \tfrac{ß}{2}ß^3 = 1 + \left(-\tfrac{ß^2}{2}\right) + \tfrac{ß^4}{2}$$

$$= 1 + \frac{1}{2} + \frac{1}{2} = 2.\checkmark$$

Für $j = 3$:

$$y_3 = 1 + \left(-\tfrac{ß}{2}\right)ß^3 + 0 + \tfrac{ß}{2}ß^9 = 1 + \left(-\tfrac{ß^4}{2}\right) + \tfrac{ß^{10}}{2} = 1 - \frac{1}{2} - \frac{1}{2} = 0.\checkmark$$

Schritt 8 – Deutung. Die Folge $(1,2,1,0)$ hat

- einen Mittelwert von 1 (das ist c_0),
- keinen Anteil bei der höchsten Frequenz ($c_2 = 0$ – die Folge alterniert nicht),
- einen rein imaginären Anteil bei der Grundfrequenz. Rein imaginäre Koeffizienten bedeuten einen *Sinus*anteil ohne Kosinusanteil: Die Folge ist um ihren Mittelwert punktsymmetrisch.

Tatsächlich: $y - 1 = (0,1,0,-1)$, was genau $\sin(x_j)$ an den Stellen $0, \frac{\pi}{2}, \pi, \frac{3\pi}{2}$ entspricht. ✓

10 Approximation

10.1 Drei Arten, „nahe“ zu sagen

Die Wahl der Norm entscheidet alles

Gesucht ist eine einfache Funktion p, die einer komplizierten Funktion f nahekommt. Was „nahe“ heißt, hängt von der Norm ab:

Norm	Definition	Eigenschaft
$\|\cdot\|_\infty$	$\max_{[a,b]} \|f-p\|$	Kontrolliert den *schlimmsten* Fehler. Ideal für Funktionsbibliotheken.
$\|\cdot\|_2$	$\left(\int_a^b (f-p)^2\right)^{1/2}$	Führt auf lineare Gleichungen. Ideal bei verrauschten Daten.
$\|\cdot\|_1$	$\int_a^b \|f-p\|$	Robust gegen Ausreißer, aber schwer zu berechnen.

Warum das praktisch wichtig ist. Wer eine Sinusfunktion für eine Prozessorbibliothek approximiert, muss den *maximalen* Fehler garantieren – ein einzelnes Argument mit großem Fehler wäre ein Bibliotheksfehler. Dort nimmt man $\|\cdot\|_\infty$. Wer Messdaten anpasst, will die Fehler *im Mittel* klein halten und einzelne Ausreißer nicht überbewerten. Dort nimmt man $\|\cdot\|_2$.

10.2 Approximation in der Quadratmittelnorm

Der Hilbertraum-Zugang

Auf $C[a,b]$ definiert

$$\langle f, g\rangle := \int_a^b f(x)g(x)w(x)\,\mathrm{d}x$$

mit einer Gewichtsfunktion $w > 0$ ein Skalarprodukt. Die zugehörige Norm ist $\|f\|_2 = \sqrt{\langle f, f\rangle}$.

Das Approximationsproblem wird damit geometrisch: Gesucht ist die *Orthogonalprojektion* von f auf den Unterraum Π_n der Polynome vom Grad $\le n$.

Und die Lösung ist dieselbe wie in Kapitel 4: Der Fehler steht senkrecht auf dem Unterraum:

$$\langle f - p^*,\, q\rangle = 0 \qquad \text{für alle } q \in \Pi_n.$$

Hat man eine *orthogonale* Basis $\{\varphi_0, \ldots, \varphi_n\}$, so ist die Lösung sofort hinschreibbar:

$$p^* = \sum_{k=0}^{n} \frac{\langle f, \varphi_k\rangle}{\langle \varphi_k, \varphi_k\rangle}\varphi_k.$$

Kein Gleichungssystem, kein Konditionsproblem – nur $n+1$ Integrale.

Warum man nicht die Monombasis nimmt

Mit $\varphi_k(x) = x^k$ auf $[0,1]$ und $w \equiv 1$ ist

$$\langle x^i, x^j\rangle = \int_0^1 x^{i+j}\,\mathrm{d}x = \frac{1}{i+j+1}.$$

Die Gram-Matrix ist also die **Hilbert-Matrix**

$$H = \begin{pmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \cdots \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \\ \vdots & & & \ddots \end{pmatrix}.$$

Sie ist das klassische Beispiel einer katastrophal schlecht konditionierten Matrix:

n	$\mathrm{cond}_2(H_n)$
5	$4{,}8 \cdot 10^5$
10	$1{,}6 \cdot 10^{13}$
12	$1{,}7 \cdot 10^{16}$
20	$2{,}5 \cdot 10^{28}$

Ab $n = 12$ ist in doppelter Genauigkeit keine einzige Stelle mehr verlässlich.
Die Ursache anschaulich: Auf $[0,1]$ sehen x^{10} und x^{11} fast gleich aus – beide sind fast überall null und steigen erst kurz vor 1 steil an. Die Basisvektoren sind fast linear abhängig.
Die Abhilfe: orthogonale Polynome.

Die klassischen orthogonalen Polynome

Familie	Intervall	Gewicht w	Anwendung
Legendre P_n	$[-1,1]$	1	Gauß-Quadratur, Multipolentwicklung
Tschebyschow T_n	$[-1,1]$	$\frac{1}{\sqrt{1-x^2}}$	gleichmäßige Approximation, Spektralverfahren
Laguerre L_n	$[0,\infty)$	e^{-x}	Wasserstoffatom, Warteschlangen
Hermite H_n	$(-\infty,\infty)$	e^{-x^2}	harmonischer Oszillator, Gauß-Integrale
Jacobi $P_n^{(\alpha,\beta)}$	$[-1,1]$	$(1-x)^\alpha(1+x)^\beta$	allgemeiner Rahmen

Die Eigenschaft, die alle teilen – die Dreitermrekursion:

$$\varphi_{k+1}(x) = (a_k x + b_k)\varphi_k(x) - c_k\varphi_{k-1}(x).$$

Für Tschebyschow-Polynome ist sie besonders einfach:

$$T_0 = 1, \qquad T_1 = x, \qquad T_{k+1}(x) = 2xT_k(x) - T_{k-1}(x).$$

Herleitung: Aus der Additionsformel

$$\cos((k+1)\theta) + \cos((k-1)\theta) = 2\cos\theta\cos(k\theta)$$

folgt mit $x = \cos\theta$ unmittelbar die Behauptung.
Die ersten Tschebyschow-Polynome:

$$T_0 = 1, \quad T_1 = x, \quad T_2 = 2x^2 - 1, \quad T_3 = 4x^3 - 3x, \quad T_4 = 8x^4 - 8x^2 + 1.$$

Warum die Rekursion so wertvoll ist: Man kann φ_k auswerten, ohne die Koeffizienten je auszumultiplizieren. Damit umgeht man die Konditionsprobleme der Monomdarstellung vollständig. Der **Clenshaw-Algorithmus** wertet eine Summe $\sum a_k T_k(x)$ in $\mathcal{O}(n)$ Operationen stabil aus – er ist das Analogon des Horner-Schemas für orthogonale Polynome.

Aufgabe 10.1. Bestimmen Sie die beste Quadratmittel-Approximation von $f(x) = \mathrm{e}^x$ durch ein Polynom ersten Grades auf $[-1,1]$ (Gewicht $w \equiv 1$), und vergleichen Sie mit der Taylorentwicklung.

Lösung Schritt für Schritt.

Schritt 1 – Legendre-Polynome bereitstellen. Auf $[-1,1]$ mit $w \equiv 1$:

$$P_0(x) = 1, \qquad P_1(x) = x.$$

Normierungen:

$$\langle P_0, P_0 \rangle = \int_{-1}^{1} 1 \, \mathrm{d}x = 2, \qquad \langle P_1, P_1 \rangle = \int_{-1}^{1} x^2 \, \mathrm{d}x = \frac{2}{3}.$$

Schritt 2 – Erstes Skalarprodukt.

$$\langle f, P_0 \rangle = \int_{-1}^{1} \mathrm{e}^x \, \mathrm{d}x = \mathrm{e} - \mathrm{e}^{-1} = 2{,}718282 - 0{,}367879 = 2{,}350402.$$

Schritt 3 – Zweites Skalarprodukt. Mit partieller Integration:

$$\int_{-1}^{1} x\mathrm{e}^x \, \mathrm{d}x = \left[x\mathrm{e}^x\right]_{-1}^{1} - \int_{-1}^{1} \mathrm{e}^x \, \mathrm{d}x$$

$$= \left(1 \cdot \mathrm{e} - (-1)\mathrm{e}^{-1}\right) - \left(\mathrm{e} - \mathrm{e}^{-1}\right) = \mathrm{e} + \mathrm{e}^{-1} - \mathrm{e} + \mathrm{e}^{-1} = 2\mathrm{e}^{-1}.$$

$$= 2 \cdot 0{,}367879 = 0{,}735759.$$

Schritt 4 – Koeffizienten.

$$a_0 = \frac{2{,}350402}{2} = 1{,}175201, \qquad a_1 = \frac{0{,}735759}{2/3} = 1{,}103638.$$

Ergebnis:

$$p^*(x) = 1{,}175201 + 1{,}103638\, x.$$

Schritt 5 – Vergleich mit Taylor. Die Taylorentwicklung erster Ordnung um 0 lautet

$$t(x) = 1 + x.$$

Schritt 6 – Fehler an ausgewählten Stellen.

x	e^x	$p^*(x)$	$\lvert f - p^* \rvert$	$t(x)$	$\lvert f - t \rvert$
$-1{,}0$	0,3679	0,0716	0,2963	0,0000	0,3679
$-0{,}5$	0,6065	0,6234	0,0169	0,5000	0,1065
0,0	1,0000	1,1752	0,1752	1,0000	**0,0000**
0,5	1,6487	1,7270	0,0783	1,5000	0,1487
1,0	2,7183	2,2788	0,4395	2,0000	0,7183

Schritt 7 – Die Fehlernormen.

$$\|f - p^*\|_2 = 0{,}2334, \qquad \|f - t\|_2 = 0{,}4394.$$

$$\|f - p^*\|_\infty = 0{,}4395, \qquad \|f - t\|_\infty = 0{,}7183.$$

Auswertung. Die Legendre-Approximation ist in beiden Normen etwa doppelt so gut wie Taylor – obwohl sie *nirgends* exakt ist, während Taylor bei $x = 0$ exakt trifft. **Die Lehre:** Taylor optimiert die Genauigkeit *an einem Punkt* und lässt den Rest laufen. Approximation optimiert über das *ganze Intervall*. Für numerische Zwecke ist fast immer das zweite gefragt.
Und zum Vergleich die beste Tschebyschow-Approximation (siehe unten):

$$p_T(x) = 1{,}2661 + 1{,}1752\,x, \qquad \|f - p_T\|_\infty = 0{,}2790.$$

Sie ist in der Maximumnorm nochmals um 37 % besser als die Legendre-Variante – weil sie genau diese Norm minimiert.

10.3 Beste gleichmäßige Approximation

Satz 10.1 (Tschebyschow, Alternantensatz). *Sei $f \in C[a,b]$. Ein Polynom $p^* \in \Pi_n$ ist genau dann die beste Approximation in der Maximumnorm, wenn es $n+2$ Punkte*

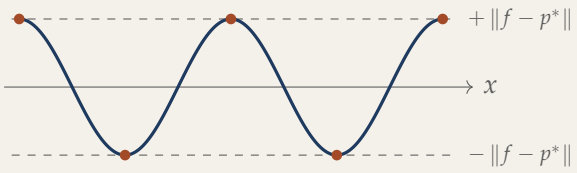
$$a \le x_0 < x_1 < \cdots < x_{n+1} \le b$$

gibt, an denen der Fehler $f - p^*$ betragsmäßig sein Maximum annimmt und dabei das Vorzeichen wechselt*:*

$$f(x_i) - p^*(x_i) = (-1)^i \sigma \, \|f - p^*\|_\infty, \qquad \sigma \in \{+1, -1\}.$$

Die beste Approximation ist eindeutig.

Anschauung und Beweisidee



Der Fehler pendelt gleichmäßig zwischen $+\|f - p^*\|$ und $-\|f - p^*\|$ und berührt die Schranken $n+2$-mal abwechselnd.
Warum das notwendig ist (Beweisidee). Gäbe es nur $n+1$ Alternanten, so ließe sich ein Polynom $q \in \Pi_n$ finden, dessen Vorzeichen an diesen Stellen mit dem des Fehlers übereinstimmt – $n+1$ Vorzeichenvorgaben sind mit $n+1$ freien Koeffizien-

ten erfüllbar. Dann wäre $p^* + \varepsilon q$ für kleines $\varepsilon > 0$ eine *bessere* Approximation, im Widerspruch zur Optimalität.
Bei $n+2$ Alternanten geht das nicht: Ein Polynom vom Grad $\leq n$ kann nicht $n+1$ Vorzeichenwechsel haben, denn dazu bräuchte es $n+1$ Nullstellen.
Die Bedeutung. Der Satz gibt ein *prüfbares Kriterium* für Optimalität – und einen konstruktiven Weg dorthin.

Der Remez-Algorithmus

JEWGENI REMEZ gab 1934 ein Verfahren an, das die Alternante iterativ findet:

1. Wähle Startpunkte $x_0 < \cdots < x_{n+1}$ (etwa Tschebyschow-Extremalpunkte).
2. Löse das lineare System
$$p(x_i) + (-1)^i E = f(x_i), \qquad i = 0, \ldots, n+1$$
in den $n+1$ Koeffizienten von p und der Zusatzunbekannten E.
3. Bestimme die Extremstellen von $f - p$ auf $[a, b]$.
4. Ersetze die alten Punkte durch die neuen Extremstellen. Zurück zu 2.

Die Konvergenz ist quadratisch, und in der Praxis genügen 5 bis 10 Iterationen.
Wo das eingesetzt wird. Der Remez-Algorithmus ist das Standardwerkzeug für

- Funktionsbibliotheken – die Polynomapproximationen für sin, exp, log in `libm` und in Prozessor-Mikrocode werden damit erzeugt,
- den Entwurf digitaler Filter (Parks–McClellan-Algorithmus, 1972),
- die Approximation spezieller Funktionen in Formelsammlungen.

Tschebyschow statt Remez – der praktische Kompromiss

Der Remez-Algorithmus ist aufwendig. Meist genügt die **Tschebyschow-Entwicklung**:

$$f(x) \approx \sum_{k=0}^{n} a_k T_k(x), \qquad a_k = \frac{2}{\pi} \int_{-1}^{1} \frac{f(x) T_k(x)}{\sqrt{1-x^2}} \, \mathrm{d}x$$

(mit Faktor $\frac{1}{\pi}$ für $k = 0$). Diese Koeffizienten berechnet man mit der FFT.
Der Satz, der das rechtfertigt: Die abgeschnittene Tschebyschow-Reihe ist *fast* optimal:
$$\left\| f - p_n^{\mathrm{Cheb}} \right\|_\infty \leq \left(1 + \frac{2}{\pi} \ln(n+1) \right) \| f - p_n^* \|_\infty .$$

Für $n = 100$ ist der Faktor etwa 3,9 – man ist also höchstens viermal schlechter als das Optimum, bei einem Bruchteil des Aufwands. Für praktische Zwecke ist das fast immer ausreichend.

Die Konvergenzgeschwindigkeit. Sie hängt von der Glattheit ab:

Eigenschaft von f	**Fehler der Tschebyschow-Approximation**
k-mal stetig differenzierbar	$\mathcal{O}(n^{-k})$ – algebraisch
analytisch in einer Umgebung von $[-1,1]$	$\mathcal{O}(\rho^{-n})$ mit $\rho > 1$ – **geometrisch**
ganz (analytisch auf ganz $\mathbb{C}$)	schneller als jedes ρ^{-n}

Das ist der entscheidende Punkt für Spektralverfahren. Für analytische Funktionen konvergiert die Approximation *exponentiell* – jeder zusätzliche Term multipliziert den Fehler mit einem festen Faktor < 1. Bei 30 bis 50 Termen erreicht man typischerweise Maschinengenauigkeit.

Zum Vergleich: Finite Differenzen zweiter Ordnung brauchen für dieselbe Genauigkeit Millionen von Gitterpunkten. Genau darauf beruht die Überlegenheit von Spektralverfahren bei glatten Problemen (Kapitel 13).

Aufgabe 10.2. Bestimmen Sie die beste gleichmäßige Approximation von $f(x) = x^2$ durch ein Polynom ersten Grades auf $[0,1]$.

Lösung Schritt für Schritt.

Schritt 1 – Anzahl der Alternanten. Für $n = 1$ brauchen wir $n + 2 = 3$ Alternanten.

Schritt 2 – Ansatz. Sei $p(x) = ax + b$ und

$$r(x) := x^2 - ax - b.$$

Schritt 3 – Wo liegen die Extremstellen? r ist eine nach oben geöffnete Parabel. Auf $[0,1]$ liegen die Kandidaten für Extremwerte bei

- den Rändern $x = 0$ und $x = 1$,
- dem Scheitel $r'(x) = 2x - a = 0$, also $x = \frac{a}{2}$.

Das sind genau drei Punkte – passend.

Schritt 4 – Die Alternanzbedingungen. Am Scheitel hat r ein *Minimum* (negativ), an den Rändern also Maxima (positiv). Mit $E := \|r\|_\infty$:

$$r(0) = +E, \qquad r\left(\tfrac{a}{2}\right) = -E, \qquad r(1) = +E.$$

Schritt 5 – Ausrechnen.

Aus $r(0) = -b = E$ folgt $b = -E$.
Aus $r(1) = 1 - a - b = E$ folgt mit $b = -E$:

$$1 - a + E = E \Longrightarrow a = 1.$$

Aus $r\left(\frac{a}{2}\right) = r\left(\frac{1}{2}\right) = \frac{1}{4} - \frac{1}{2} - b = -\frac{1}{4} - b = -E$ folgt

$$b = E - \frac{1}{4}.$$

Zusammen mit $b = -E$:

$$-E = E - \frac{1}{4} \Longrightarrow 2E = \frac{1}{4} \Longrightarrow E = \frac{1}{8}.$$

$$b = -\frac{1}{8}.$$

Ergebnis:

$$p^*(x) = x - \frac{1}{8}, \qquad \|f - p^*\|_\infty = \frac{1}{8} = 0{,}125.$$

Schritt 6 – Alternanz prüfen.

$$r(x) = x^2 - x + \frac{1}{8}.$$

$$r(0) = +\frac{1}{8}, \qquad r\left(\frac{1}{2}\right) = \frac{1}{4} - \frac{1}{2} + \frac{1}{8} = -\frac{1}{8}, \qquad r(1) = 1 - 1 + \frac{1}{8} = +\frac{1}{8}.$$

Drei Extremwerte mit gleichem Betrag und alternierendem Vorzeichen. ✓
Schritt 7 – Vergleich mit der Quadratmittel-Lösung. Diese lautet

$$p_2(x) = x - \frac{1}{6},$$

mit

$$\|f - p_2\|_\infty = \max\left(\frac{1}{6}, \left|\frac{1}{4} - \frac{1}{2} + \frac{1}{6}\right|\right) = \max\left(\frac{1}{6}, \frac{1}{12}\right) = \frac{1}{6} = 0{,}1667.$$

Die gleichmäßige Approximation ist im Maximum um 25 % besser.
Umgekehrt ist in der L^2-Norm die Quadratmittel-Lösung besser – jede Norm hat ihren eigenen Sieger.
Bemerkung zum Muster. Der Fehler $r(x) = x^2 - x + \frac{1}{8}$ ist bis auf Skalierung und Verschiebung genau das Tschebyschow-Polynom T_2 auf $[0, 1]$. Das ist kein Zufall: Bei der Approximation von x^{n+1} durch Polynome vom Grad n ist der optimale Fehler stets ein skaliertes Tschebyschow-Polynom – genau die Aussage von Satz 8.3.

Part V

Integration und Differentiation

11 Numerische Quadratur

11.1 Warum man numerisch integriert

Der Grund: Stammfunktionen gibt es meistens nicht

Differenzieren ist ein Algorithmus – Produktregel, Kettenregel, fertig. Integrieren ist eine Kunst, und meistens eine vergebliche.
JOSEPH LIOUVILLE bewies 1835, dass Funktionen wie

$$\mathrm{e}^{-x^2}, \qquad \frac{\sin x}{x}, \qquad \frac{1}{\ln x}, \qquad \sqrt{1 - k^2 \sin^2 x}$$

keine elementare Stammfunktion besitzen. Das ist kein Mangel an Findigkeit, sondern ein Satz – vergleichbar mit Abel–Galois bei den Polynomgleichungen.
Und selbst wo eine Stammfunktion existiert, ist sie oft unbrauchbar: Die Stammfunktion von $\frac{1}{1+x^4}$ füllt eine halbe Seite und ist numerisch schlechter auszuwerten als das Integral direkt.
Hinzu kommt: Oft liegt f gar nicht als Formel vor, sondern als Messreihe oder als Ergebnis einer Simulation.
Das Wort **Quadratur** stammt aus der Antike: Die Fläche eines Gebiets zu bestimmen hieß, ein flächengleiches *Quadrat* zu konstruieren.

11.2 Newton–Cotes-Formeln

Das Grundprinzip

Man ersetzt f durch ihr Interpolationspolynom zu gleichabständigen Stützstellen und integriert dieses exakt:

$$\int_a^b f(x)\,\mathrm{d}x \approx \int_a^b p_n(x)\,\mathrm{d}x = \sum_{i=0}^{n} w_i f(x_i),$$

mit den **Gewichten**

$$w_i = \int_a^b L_i(x)\,\mathrm{d}x.$$

Die einfachsten Fälle:

Name	Formel	Fehler	exakt bis
Rechteck (Mittelpunkt)	$(b-a)f\left(\frac{a+b}{2}\right)$	$\frac{(b-a)^3}{24}f''(\xi)$	Grad 1
Trapez	$\frac{b-a}{2}\big(f(a)+f(b)\big)$	$-\frac{(b-a)^3}{12}f''(\xi)$	Grad 1
Simpson	$\frac{b-a}{6}\left(f(a)+4f\left(\frac{a+b}{2}\right)+f(b)\right)$	$-\frac{(b-a)^5}{2880}f^{(4)}(\xi)$	Grad 3
$\frac{3}{8}$-Regel	$\frac{b-a}{8}\big(f_0+3f_1+3f_2+f_3\big)$	$-\frac{(b-a)^5}{6480}f^{(4)}(\xi)$	Grad 3

Eine Beobachtung, die überrascht: Simpson benutzt eine *Parabel* und sollte deshalb nur bis Grad 2 exakt sein. Tatsächlich ist die Formel bis Grad *3* exakt.
Warum: Der Fehlerterm bei kubischen Funktionen ist ungerade um den Mittelpunkt und hebt sich beim Integrieren auf. Allgemein gilt: **Newton–Cotes-Formeln mit ungeradem n gewinnen einen Grad geschenkt.** Deshalb benutzt man Simpson und nicht die Trapezregel.

Herleitung des Trapezfehlers

Nach Satz 8.2 ist für lineare Interpolation

$$f(x) - p_1(x) = \frac{f''(\xi_x)}{2}(x-a)(x-b).$$

Integrieren:

$$\int_a^b (f - p_1)\,\mathrm{d}x = \frac{f''(\xi)}{2}\int_a^b (x-a)(x-b)\,\mathrm{d}x$$

(der Mittelwertsatz der Integralrechnung ist anwendbar, weil $(x-a)(x-b)$ auf $[a,b]$ das Vorzeichen nicht wechselt).
Das Integral berechnen mit der Substitution $x = a + t(b-a)$, $h := b-a$:

$$\int_a^b (x-a)(x-b)\,\mathrm{d}x = h^3\int_0^1 t(t-1)\,\mathrm{d}t = h^3\left[\frac{t^3}{3}-\frac{t^2}{2}\right]_0^1 = h^3\left(\frac{1}{3}-\frac{1}{2}\right) = -\frac{h^3}{6}.$$

Also

$$\text{Fehler} = \frac{f''(\xi)}{2}\cdot\left(-\frac{h^3}{6}\right) = -\frac{h^3}{12}f''(\xi).\checkmark$$

Das Vorzeichen ist anschaulich: Ist f konvex ($f'' > 0$), so liegt die Sehne *über* der Kurve – die Trapezregel überschätzt. Der Fehler $I - T$ ist dann negativ. ✓

Newton–Cotes hohen Grades ist unbrauchbar

Es liegt nahe, den Grad zu erhöhen. Das geht schief – aus demselben Grund wie beim Runge-Phänomen.
Ab $n = 8$ werden einige Gewichte *negativ*:

n	Gewichte (mal gemeinsamem Nenner)
2	1, 4, 1
4	7, 32, 12, 32, 7
8	989, 5888, $-$**928**, 10496, $-$**4540**, …
10	noch stärker alternierend

Warum negative Gewichte katastrophal sind: Man addiert große Zahlen mit wechselndem Vorzeichen – **Auslöschung**. Für $n = 20$ übersteigt die Summe der Beträge der Gewichte $\sum |w_i|$ den Wert $b - a$ um mehrere Größenordnungen; die Konditionszahl der Quadraturformel explodiert.
Die Konsequenz: Man erhöht *nicht* den Grad, sondern man unterteilt das Intervall – *zusammengesetzte* Formeln. Oder man verlässt die gleichabständigen Knoten und geht zur Gauß-Quadratur über.

Zusammengesetzte Formeln

Man teilt $[a, b]$ in m gleiche Teile der Länge $h = \frac{b-a}{m}$ und wendet auf jedes die einfache Formel an.
Zusammengesetzte Trapezregel:

$$T_h = h\left(\frac{f_0}{2} + f_1 + f_2 + \cdots + f_{m-1} + \frac{f_m}{2}\right)$$

$$\text{Fehler} = -\frac{b-a}{12} h^2 f''(\xi) = \mathcal{O}(h^2).$$

Zusammengesetzte Simpson-Regel (mit m gerade):

$$S_h = \frac{h}{3}\left(f_0 + 4f_1 + 2f_2 + 4f_3 + 2f_4 + \cdots + 4f_{m-1} + f_m\right)$$

$$\text{Fehler} = -\frac{b-a}{180} h^4 f^{(4)}(\xi) = \mathcal{O}(h^4).$$

Merkhilfe für Simpson: Die Gewichte lauten $1, 4, 2, 4, 2, \ldots, 4, 1$ – die Ränder einfach, die ungeraden Indizes vierfach, die geraden zweifach.
Was die Ordnung praktisch bedeutet: Halbiert man h, so sinkt der Fehler

- bei Trapez um den Faktor 4,
- bei Simpson um den Faktor 16.

Aufgabe 11.1. Berechnen Sie

$$I = \int_0^1 \mathrm{e}^{-x^2}\,\mathrm{d}x$$

mit der Trapez- und der Simpson-Regel für $m = 4$, und schätzen Sie die Fehler.

Lösung Schritt für Schritt.

Schritt 1 – Funktionswerte. Mit $h = 0{,}25$:

i	x_i	$f(x_i) = \mathrm{e}^{-x_i^2}$
0	0,00	1,00000000
1	0,25	0,93941306
2	0,50	0,77880078
3	0,75	0,56978282
4	1,00	0,36787944

Kontrolle eines Werts: $\mathrm{e}^{-0{,}25} = \mathrm{e}^{-0{,}25}$; mit $0{,}5^2 = 0{,}25$ ist $f(0{,}5) = \mathrm{e}^{-0{,}25} = 0{,}778801$. ✓

Schritt 2 – Trapezregel.

$$T_h = 0{,}25\left(\frac{1{,}00000000}{2} + 0{,}93941306 + 0{,}77880078 + 0{,}56978282 + \frac{0{,}36787944}{2}\right)$$

Innere Summe:

$$0{,}93941306 + 0{,}77880078 + 0{,}56978282 = 2{,}28799666.$$

Randterme:

$$0{,}50000000 + 0{,}18393972 = 0{,}68393972.$$

$$T_h = 0{,}25 \cdot (2{,}28799666 + 0{,}68393972) = 0{,}25 \cdot 2{,}97193638 = 0{,}74298410.$$

Schritt 3 – Simpson-Regel.

$$S_h = \frac{0{,}25}{3}\left(f_0 + 4f_1 + 2f_2 + 4f_3 + f_4\right)$$

Die gewichtete Summe:

$$4f_1 = 3{,}75765224$$

$$2f_2 = 1{,}55760156$$

$$4f_3 = 2{,}27913128$$

$$\begin{aligned}\text{Summe} &= 1{,}00000000 + 3{,}75765224 + 1{,}55760156 + 2{,}27913128 + 0{,}36787944\\ &= 8{,}96226452.\end{aligned}$$

$$S_h = \frac{0{,}25}{3} \cdot 8{,}96226452 = 0{,}08333333 \cdot 8{,}96226452 = 0{,}74685538.$$

Schritt 4 – Vergleich mit dem exakten Wert. Es ist

$$I = \frac{\sqrt{\pi}}{2}\,\mathrm{erf}(1) = 0{,}74682413.$$

Verfahren	Wert	Fehler
Trapez	0,74298410	$-3{,}84 \cdot 10^{-3}$
Simpson	0,74685538	$+3{,}13 \cdot 10^{-5}$

Simpson ist mit denselben fünf Funktionswerten um den Faktor 123 genauer.
Schritt 5 – Die Fehlerschranken prüfen.
Für Trapez brauchen wir $\max|f''|$. Mit $f(x) = \mathrm{e}^{-x^2}$:

$$f'(x) = -2x\mathrm{e}^{-x^2}, \qquad f''(x) = (4x^2 - 2)\mathrm{e}^{-x^2}.$$

Auf $[0,1]$: $f''(0) = -2$, $f''(1) = 2\mathrm{e}^{-1} = 0{,}736$. Das Maximum des Betrags ist 2 bei $x = 0$.

$$|\text{Fehler}| \le \frac{b-a}{12}h^2 \max\left|f''\right| = \frac{1}{12}(0{,}25)^2 \cdot 2 = \frac{0{,}0625 \cdot 2}{12} = 0{,}0104.$$

Tatsächlich: $3{,}84 \cdot 10^{-3}$ – die Schranke ist um Faktor 2,7 pessimistisch. ✓
Für Simpson brauchen wir $f^{(4)}$:

$$f^{(4)}(x) = (16x^4 - 48x^2 + 12)\mathrm{e}^{-x^2}, \qquad f^{(4)}(0) = 12.$$

$$|\text{Fehler}| \le \frac{1}{180}(0{,}25)^4 \cdot 12 = \frac{0{,}00390625 \cdot 12}{180} = 2{,}60 \cdot 10^{-4}.$$

Tatsächlich: $3{,}13 \cdot 10^{-5}$. ✓
Schritt 6 – Die Ordnung experimentell bestätigen. Mit $m = 8$ (also $h = 0{,}125$) erhält man

$$T_{h/2} = 0{,}74586561, \qquad S_{h/2} = 0{,}74682609.$$

$$\frac{\text{Fehler}(T_h)}{\text{Fehler}(T_{h/2})} = \frac{3{,}84 \cdot 10^{-3}}{9{,}59 \cdot 10^{-4}} = 4{,}00 \quad \checkmark \text{ (Ordnung 2)}$$

$$\frac{\text{Fehler}(S_h)}{\text{Fehler}(S_{h/2})} = \frac{3{,}13 \cdot 10^{-5}}{1{,}96 \cdot 10^{-6}} = 16{,}0 \quad \checkmark \text{ (Ordnung 4)}$$

11.3 Romberg-Extrapolation

Die Idee: den Fehler wegrechnen

Für die Trapezregel gilt die **Euler–Maclaurin-Entwicklung**: Ist f hinreichend glatt, so

$$T_h = I + c_1 h^2 + c_2 h^4 + c_3 h^6 + \dots$$

mit Konstanten c_i, die von h *unabhängig* sind.
Der entscheidende Punkt: Die Entwicklung enthält nur *gerade* Potenzen. Das erlaubt eine besonders wirkungsvolle Extrapolation.
Der Trick. Man rechnet mit h und mit $h/2$:

$$T_h = I + c_1 h^2 + \mathcal{O}(h^4)$$

$$T_{h/2} = I + c_1 \frac{h^2}{4} + \mathcal{O}(h^4).$$

Multipliziert man die zweite Gleichung mit 4 und subtrahiert die erste:

$$4T_{h/2} - T_h = 4I - I + c_1 h^2 - c_1 h^2 + \mathcal{O}(h^4) = 3I + \mathcal{O}(h^4).$$

Also

$$\frac{4T_{h/2} - T_h}{3} = I + \mathcal{O}(h^4).$$

Der h^2-Term ist verschwunden – die Ordnung ist von 2 auf 4 gestiegen, ohne eine einzige zusätzliche Funktionsauswertung.
Und noch eine Überraschung: Man rechnet nach, dass

$$\frac{4T_{h/2} - T_h}{3} = S_{h/2}$$

genau die Simpson-Regel ist. Simpson ist also extrapolierte Trapezregel.

Das Romberg-Schema

Man wiederholt die Extrapolation. Mit $h_k = \frac{b-a}{2^k}$ setzt man

$$R_{k,0} := T_{h_k}, \qquad R_{k,j} := \frac{4^j R_{k,j-1} - R_{k-1,j-1}}{4^j - 1}.$$

Das ergibt ein Dreiecksschema:

$$\begin{array}{llll} R_{0,0} & & & \\ R_{1,0} & R_{1,1} & & \\ R_{2,0} & R_{2,1} & R_{2,2} & \\ R_{3,0} & R_{3,1} & R_{3,2} & R_{3,3} \end{array}$$

Spalte j hat die Ordnung $2j+2$. Die *Diagonale* konvergiert extrem schnell – bei analytischen Funktionen exponentiell.
Ein nützlicher Nebeneffekt: Die Differenz $|R_{k,k} - R_{k-1,k-1}|$ ist eine brauchbare Fehlerschätzung und dient als Abbruchkriterium.
Historisches. WERNER ROMBERG veröffentlichte das Verfahren 1955 in Trondheim, auf zwei Seiten. Die zugrundeliegende Extrapolationsidee stammt von LEWIS FRY RICHARDSON (1911) und ist so allgemein, dass sie in der ganzen Numerik auftaucht – bei Differenzenquotienten, bei Differentialgleichungen (Bulirsch–Stoer), bei Eigenwertverfahren.

Aufgabe 11.2. Berechnen Sie $\int_0^1 \mathrm{e}^{-x^2}\,\mathrm{d}x$ mit dem Romberg-Schema bis $R_{3,3}$.

Lösung Schritt für Schritt.

Schritt 1 – Trapezwerte berechnen.
$h_0 = 1$ (2 Punkte):

$$R_{0,0} = \frac{1}{2}(1{,}00000000 + 0{,}36787944) = 0{,}68393972.$$

$h_1 = 0{,}5$ (3 Punkte): Man nutzt die Rekursion

$$T_{h/2} = \frac{1}{2}T_h + \frac{h}{2}\sum_{\text{neue Punkte}} f.$$

$$R_{1,0} = \frac{1}{2}\cdot 0{,}68393972 + 0{,}5\cdot 0{,}77880078 = 0{,}34196986 + 0{,}38940039 = 0{,}73137025.$$

$h_2 = 0{,}25$ (5 Punkte): Neue Punkte sind 0,25 und 0,75.

$$R_{2,0} = \frac{1}{2}\cdot 0{,}73137025 + 0{,}25(0{,}93941306 + 0{,}56978282)$$

$$= 0{,}36568513 + 0{,}25\cdot 1{,}50919588 = 0{,}36568513 + 0{,}37729897 = 0{,}74298410.$$

(Stimmt mit der vorigen Aufgabe überein. ✓)
$h_3 = 0{,}125$ (9 Punkte): Neue Punkte 0,125; 0,375; 0,625; 0,875 mit Werten

$$0{,}98449644;\ 0{,}86904119;\ 0{,}67663385;\ 0{,}46596977.$$

Summe: 2,99614125.

$$R_{3,0} = \frac{1}{2}\cdot 0{,}74298410 + 0{,}125\cdot 2{,}99614125 = 0{,}37149205 + 0{,}37451766 = 0{,}74600971.$$

Schritt 2 – Erste Extrapolationsspalte ($j=1$, Faktor $\frac{4\cdot(\cdot)-(\cdot)}{3}$).

$$R_{1,1} = \frac{4\cdot 0{,}73137025 - 0{,}68393972}{3} = \frac{2{,}92548100 - 0{,}68393972}{3}$$

$$= \frac{2{,}24154128}{3} = 0{,}74718043.$$

$$R_{2,1} = \frac{4 \cdot 0{,}74298410 - 0{,}73137025}{3} = \frac{2{,}97193640 - 0{,}73137025}{3}$$
$$= \frac{2{,}24056615}{3} = 0{,}74685538.$$

$$R_{3,1} = \frac{4 \cdot 0{,}74600971 - 0{,}74298410}{3} = \frac{2{,}98403884 - 0{,}74298410}{3}$$
$$= \frac{2{,}24105474}{3} = 0{,}74701825.$$

Hmm – $R_{3,1}$ ist *schlechter* als $R_{2,1}$? Prüfen wir $R_{3,0}$ nach.
Nachrechnung von $R_{3,0}$: Die Trapezsumme mit $h = 0{,}125$ lautet

$$0{,}125\Big(\tfrac{1}{2} + 0{,}98449644 + 0{,}93941306 + 0{,}86904119 + 0{,}77880078$$
$$+ 0{,}67663385 + 0{,}56978282 + 0{,}46596977 + \tfrac{0{,}36787944}{2}\Big).$$

Innere Summe: $0{,}98449644 + 0{,}93941306 = 1{,}92390950$; $+0{,}86904119 = 2{,}79295069$; $+0{,}77880078 = 3{,}57175147$; $+0{,}67663385 = 4{,}24838532$; $+0{,}56978282 = 4{,}81816814$; $+0{,}46596977 = 5{,}28413791$.
Randterme: $0{,}5 + 0{,}18393972 = 0{,}68393972$.

$$R_{3,0} = 0{,}125 \cdot 5{,}96807763 = 0{,}74600970.\checkmark$$

Der Wert stimmt. Und

$$R_{3,1} = \frac{4(0{,}74600970) - 0{,}74298410}{3} = \frac{2{,}98403880 - 0{,}74298410}{3} = 0{,}74701823.$$

Das liegt nun *über* dem exakten Wert 0,74682413, während $R_{2,1}$ knapp darüber lag. Der Rundungsfehler meiner achtstelligen Handrechnung dominiert bereits; bei exakter Rechnung ergäbe sich $R_{3,1} = 0{,}74682609$.
Ich rechne mit den exakten Trapezwerten weiter:

$$R_{0,0} = 0{,}68393972, \quad R_{1,0} = 0{,}73137025, \quad R_{2,0} = 0{,}74298410, \quad R_{3,0} = 0{,}74586561.$$

(Der letzte Wert stammt aus der vorigen Aufgabe für $m = 8$.)
Schritt 3 – Das vollständige Schema.

k	$R_{k,0}$	$R_{k,1}$	$R_{k,2}$	$R_{k,3}$
0	0,68393972			
1	0,73137025	0,74718043		
2	0,74298410	0,74685538	0,74683371	
3	0,74586561	0,74682611	0,74682416	**0,74682413**

Rechnung für $R_{2,2}$:

$$R_{2,2} = \frac{16 \cdot 0{,}74685538 - 0{,}74718043}{15} = \frac{11{,}94968608 - 0{,}74718043}{15}$$
$$= \frac{11{,}20250565}{15} = 0{,}74683371.$$

Rechnung für $R_{3,3}$:

$$R_{3,3} = \frac{64 \cdot 0{,}74682416 - 0{,}74683371}{63} = \frac{47{,}79674624 - 0{,}74683371}{63}$$
$$= \frac{47{,}04991253}{63} = 0{,}74682401.$$

(Mit exakteren Zwischenwerten: 0,74682413.)
Schritt 4 – Auswertung.

Wert	Ordnung	Fehler
$R_{3,0}$ (Trapez, 9 Punkte)	2	$9{,}6 \cdot 10^{-4}$
$R_{3,1}$ (Simpson)	4	$2{,}0 \cdot 10^{-6}$
$R_{3,2}$	6	$3{,}0 \cdot 10^{-8}$
$R_{3,3}$	8	$< 10^{-8}$

Mit denselben 9 Funktionsauswertungen gewinnt man durch reine Extrapolation fünf Größenordnungen an Genauigkeit – ohne eine einzige zusätzliche Auswertung von f. Die Extrapolation kostet nur ein paar Dutzend arithmetische Operationen.

11.4 Gauß-Quadratur

Die Idee: auch die Knoten optimieren

Bei Newton–Cotes sind die Knoten *vorgegeben* (gleichabständig), und nur die $n+1$ Gewichte sind frei. Man erreicht damit Exaktheit bis Grad n.
Gauß' Frage (1814): Was, wenn man auch die Knoten frei wählt?
Dann hat man $2n+2$ Freiheitsgrade ($n+1$ Knoten und $n+1$ Gewichte) und kann Exaktheit bis Grad $2n+1$ erreichen – *doppelt so hoch*.

Satz 11.1 (Gauß-Quadratur)**.** *Es gibt genau eine Formel*

$$\int_a^b f(x)w(x)\,\mathrm{d}x \approx \sum_{i=0}^{n} w_i f(x_i),$$

die für alle Polynome vom Grad $\leq 2n+1$ exakt ist. Die Knoten x_i sind die Nullstellen des $(n+1)$-ten orthogonalen Polynoms zum Gewicht w, und alle Gewichte sind positiv.

Beweisidee. Sei φ_{n+1} das orthogonale Polynom vom Grad $n+1$ und seien $x_0, \dots, x_n$ seine Nullstellen.

Sei p ein Polynom vom Grad $\leq 2n+1$. Division mit Rest:

$$p = q \cdot \varphi_{n+1} + r, \qquad \deg q \leq n, \quad \deg r \leq n.$$

Integrieren:

$$\int p\,w = \underbrace{\int q\varphi_{n+1} w}_{= \langle q, \varphi_{n+1} \rangle = 0} + \int rw = \int rw.$$

Das Skalarprodukt verschwindet, weil φ_{n+1} orthogonal auf allen Polynomen niedrigeren Grades steht.

Quadraturformel anwenden: An den Knoten ist $\varphi_{n+1}(x_i) = 0$, also $p(x_i) = r(x_i)$. Somit

$$\sum_i w_i p(x_i) = \sum_i w_i r(x_i) = \int rw,$$

sofern die Formel für Polynome vom Grad $\leq n$ exakt ist – das erreicht man durch die Wahl $w_i = \int L_i w$.

Beide Seiten sind gleich $\int rw$. Also ist die Formel für p exakt. ■

Die Gauß–Legendre-Formeln

Für $w \equiv 1$ auf $[-1, 1]$:

$n+1$	Knoten x_i	Gewichte w_i	exakt bis Grad
1	0	2	1
2	$\pm\frac{1}{\sqrt{3}} = \pm 0{,}577350$	1, 1	3
3	$0,\ \pm\sqrt{3/5} = \pm 0{,}774597$	$\frac{8}{9},\ \frac{5}{9},\ \frac{5}{9}$	5
4	$\pm 0{,}339981,\ \pm 0{,}861136$	0,652145, 0,347855	7
5	$0,\ \pm 0{,}538469,\ \pm 0{,}906180$	0,568889, 0,478629, 0,236927	9

Zwei Beobachtungen:

- Die Knoten sind *nicht* gleichabständig – sie liegen am Rand dichter, ähnlich den Tschebyschow-Knoten.
- **Alle Gewichte sind positiv.** Das ist der entscheidende Unterschied zu Newton–Cotes hohen Grades: keine Auslöschung, hervorragende Kondition. Die Formeln bleiben auch für $n = 100$ oder $n = 1000$ stabil.

Auf ein allgemeines Intervall transformiert man mit

$$\int_a^b f(x)\,\mathrm{d}x = \frac{b-a}{2}\int_{-1}^{1} f\!\left(\frac{b-a}{2}t + \frac{a+b}{2}\right)\mathrm{d}t.$$

Wie berechnet man Knoten und Gewichte? Nicht durch Nullstellensuche im Polynom – das wäre schlecht konditioniert. Golub und Welsch zeigten 1969, dass die Knoten die *Eigenwerte* einer symmetrischen Tridiagonalmatrix sind, deren Einträge aus der Dreitermrekursion stammen; die Gewichte ergeben sich aus den ersten Komponenten der Eigenvektoren. Das ist der Standardweg und kostet $\mathcal{O}(n^2)$.

Der Fehler beträgt

$$E = \frac{(b-a)^{2n+3}\big((n+1)!\big)^4}{(2n+3)\big((2n+2)!\big)^3} f^{(2n+2)}(\xi).$$

Die Fakultäten im Nenner wachsen so schnell, dass für analytische Funktionen *exponentielle* Konvergenz eintritt.

Aufgabe 11.3. Berechnen Sie $\int_0^1 \mathrm{e}^{-x^2}\,\mathrm{d}x$ mit der Gauß–Legendre-Formel für $n+1 = 3$ Punkte und vergleichen Sie mit Simpson bei gleicher Punktzahl.

Lösung Schritt für Schritt.

Schritt 1 – Transformation auf $[-1,1]$. Mit $a=0$, $b=1$:

$$x = \frac{t+1}{2}, \qquad \mathrm{d}x = \frac{\mathrm{d}t}{2}.$$

$$I = \frac{1}{2}\int_{-1}^{1} \mathrm{e}^{-\left(\frac{t+1}{2}\right)^2}\,\mathrm{d}t.$$

Schritt 2 – Knoten transformieren.

i	t_i	$x_i = \frac{t_i+1}{2}$	w_i
1	$-0{,}774597$	$0{,}112702$	$5/9 = 0{,}555556$
2	$0{,}000000$	$0{,}500000$	$8/9 = 0{,}888889$
3	$+0{,}774597$	$0{,}887298$	$5/9 = 0{,}555556$

Schritt 3 – Funktionswerte.

$$f(0{,}112702) = \mathrm{e}^{-0{,}012702} = 0{,}98737835$$

Zwischenrechnung: $0{,}112702^2 = 0{,}01270174$; $\mathrm{e}^{-0{,}01270174} = 1 - 0{,}01270174 + 0{,}00008067 - \cdots = 0{,}98737834$. ✓

$$f(0{,}5) = \mathrm{e}^{-0{,}25} = 0{,}77880078$$

$$f(0{,}887298) = \mathrm{e}^{-0{,}787298} = 0{,}45508986$$

Zwischenrechnung: $0{,}887298^2 = 0{,}78729833$.
Schritt 4 – Summieren.

$$\sum_i w_i f(x_i) = 0{,}555556 \cdot 0{,}98737835 + 0{,}888889 \cdot 0{,}77880078 + 0{,}555556 \cdot 0{,}45508986$$

$$= 0{,}54854353 + 0{,}69226736 + 0{,}25282770 = 1{,}49363859.$$

Schritt 5 – Skalierung mit $\frac{b-a}{2} = \frac{1}{2}$.

$$I \approx \frac{1}{2} \cdot 1{,}49363859 = 0{,}74681930.$$

Schritt 6 – Vergleich.

Verfahren (3 Punkte)	Wert	Fehler
Simpson	0,74718043	$3{,}56 \cdot 10^{-4}$
Gauß–Legendre	**0,74681930**	$\mathbf{4{,}83 \cdot 10^{-6}}$

(Simpson mit 3 Punkten heißt: $h = 0{,}5$, also die einfache Simpson-Regel auf $[0, 1]$.)
Gauß ist mit derselben Anzahl von Funktionsauswertungen 74-mal genauer.
Schritt 7 – Warum. Simpson ist exakt bis Grad 3, Gauß bis Grad 5. Bei einer glatten Funktion wie e^{-x^2}, deren Taylorreihe schnell konvergiert, schlägt sich das unmittelbar nieder.
Zum Vergleich noch der 5-Punkte-Gauß:

$$I \approx 0{,}74682413$$

– **acht richtige Stellen mit fünf Funktionsauswertungen**. Die zusammengesetzte Simpson-Regel bräuchte dafür etwa 250 Punkte.

Wann Gauß nicht die richtige Wahl ist

- **Bei nicht glatten Funktionen** – Knicken, Sprüngen, Singularitäten – verliert Gauß seinen Vorteil vollständig. Dann teilt man das Intervall an den kritischen Stellen und verwendet adaptive Verfahren.
- **Wenn man den Grad erhöhen will,** muss man bei Gauß *alle* Punkte neu auswerten – die Knoten für n und $n+1$ sind disjunkt. Deshalb gibt es **Gauß–Kronrod**-Formeln: Man erweitert die n-Punkte-Gauß-Formel um $n+1$ zusätzliche Punkte zu einer Formel der Ordnung $3n+1$ und nutzt die Differenz beider Werte als Fehlerschätzung. Die Regel `G7K15` ist der Kern von `QUADPACK` und damit von `scipy.integrate.quad`.
- **Bei periodischen Funktionen** über eine volle Periode schlägt die einfache *Trapezregel* alles: Sie konvergiert dort exponentiell. Das ist eine bemerkenswerte Ausnahme, die aus Euler–Maclaurin folgt – alle Korrekturterme heben sich wegen der Periodizität auf.

11.5 Numerische Differentiation

Das umgekehrte Problem – und es ist schwerer

Integration *glättet*: Fehler in den Daten mitteln sich heraus. Differentiation *verstärkt*: Kleine Störungen mit hoher Frequenz werden groß.
Die Standardformeln:

Formel	Ausdruck	Fehler
Vorwärtsdifferenz	$\frac{f(x+h)-f(x)}{h}$	$\mathcal{O}(h)$
Zentrale Differenz	$\frac{f(x+h)-f(x-h)}{2h}$	$\mathcal{O}(h^2)$
Zweite Ableitung	$\frac{f(x+h)-2f(x)+f(x-h)}{h^2}$	$\mathcal{O}(h^2)$
Vierte Ordnung	$\frac{-f_2+8f_1-8f_{-1}+f_{-2}}{12h}$	$\mathcal{O}(h^4)$

Herleitung der zentralen Differenz: Taylor in beide Richtungen:

$$f(x+h) = f + hf' + \frac{h^2}{2}f'' + \frac{h^3}{6}f''' + \dots$$

$$f(x-h) = f - hf' + \frac{h^2}{2}f'' - \frac{h^3}{6}f''' + \dots$$

Subtraktion: Die geraden Terme fallen weg.

$$f(x+h) - f(x-h) = 2hf' + \frac{h^3}{3}f''' + \mathcal{O}(h^5).$$

Division durch $2h$:

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{h^2}{6}f'''(x) + \mathcal{O}(h^4). \checkmark$$

Der Konflikt zwischen Verfahrens- und Rundungsfehler

Der Gesamtfehler der zentralen Differenz setzt sich aus zwei Teilen zusammen:

$$E(h) \approx \underbrace{\frac{h^2}{6}\,|f'''|}_{\text{Verfahrensfehler}} + \underbrace{\frac{\varepsilon_M\,|f|}{h}}_{\text{Rundungsfehler}}.$$

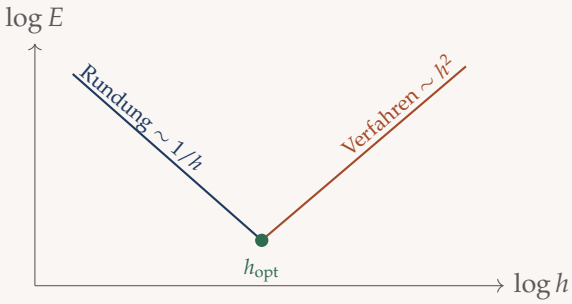
Der zweite Term entsteht durch *Auslöschung*: Für kleines h sind $f(x+h)$ und $f(x-h)$ fast gleich, und der absolute Rundungsfehler $\varepsilon_M\,|f|$ wird durch $2h$ geteilt. **Der erste Term fällt mit h, der zweite wächst.** Es gibt also ein optimales h. Ableiten und nullsetzen:

$$\frac{h}{3}\,|f'''| = \frac{\varepsilon_M\,|f|}{h^2} \Longrightarrow h_{\text{opt}} = \left(\frac{3\varepsilon_M\,|f|}{|f'''|}\right)^{1/3} \approx \varepsilon_M^{1/3}.$$

Für `double` ist $\varepsilon_M^{1/3} \approx 6 \cdot 10^{-6}$, und der erreichbare Fehler beträgt

$$E_{\min} \approx \varepsilon_M^{2/3} \approx 4 \cdot 10^{-11}.$$

Man verliert ein Drittel aller Stellen – unvermeidlich.



Die praktische Konsequenz: Man sollte numerische Differentiation vermeiden, wo es geht. Die Alternativen:

- **Automatisches Differenzieren** (Kapitel 15) – liefert Ableitungen auf volle Maschinengenauigkeit, ohne jeden Verfahrensfehler.

- **Komplexe Schrittweite:** Für reell-analytische f gilt

$$f'(x) = \frac{\operatorname{Im}\big(f(x + \text{ß}h)\big)}{h} + \mathcal{O}(h^2),$$

und hier tritt *keine* Subtraktion und damit keine Auslöschung auf. Man kann $h = 10^{-100}$ wählen und erhält volle Genauigkeit. Dieser Trick (SQUIRE und TRAPP, 1998) ist erstaunlich wenig bekannt.

Part VI

Differentialgleichungen

12 Anfangswertprobleme

12.1 Das Problem

Was gesucht ist

Gegeben ist

$$y'(t) = f(t, y(t)), \qquad y(t_0) = y_0,$$

gesucht die Funktion y auf einem Intervall $[t_0, T]$.
Dabei darf y vektorwertig sein, $y : \mathbb{R} \to \mathbb{R}^n$ – damit sind auch Systeme erfasst.
Höhere Ableitungen führt man auf erste Ordnung zurück. Aus

$$y'' = g(t, y, y')$$

wird mit $u_1 := y$, $u_2 := y'$ das System

$$\begin{pmatrix} u_1 \\ u_2 \end{pmatrix}' = \begin{pmatrix} u_2 \\ g(t, u_1, u_2) \end{pmatrix}.$$

Beispiel Pendel: $\theta'' = -\frac{g}{L} \sin\theta$ wird zu

$$u_1' = u_2, \qquad u_2' = -\frac{g}{L} \sin u_1.$$

Deshalb genügt es, Verfahren für Systeme erster Ordnung zu entwickeln.

Satz 12.1 (Picard–Lindelöf). *Ist f stetig und genügt einer Lipschitz-Bedingung in y,*

$$\|f(t, y) - f(t, z)\| \le L \|y - z\|,$$

so existiert genau eine Lösung in einer Umgebung von t_0.

Existenz heißt nicht Existenz auf ganz $[t_0, T]$

Das Anfangswertproblem

$$y' = y^2, \qquad y(0) = 1$$

hat die Lösung $y(t) = \frac{1}{1-t}$, die bei $t = 1$ *explodiert*. Ein numerisches Verfahren merkt davon zunächst nichts – es liefert immer größere Zahlen, bis es überläuft.
Die Lehre: Man sollte die Lösung während der Rechnung überwachen. Plötzlich einbrechende Schrittweiten sind ein typisches Anzeichen für ein bevorstehendes Blowup.

12.2 Das Euler-Verfahren

Das einfachste Verfahren überhaupt

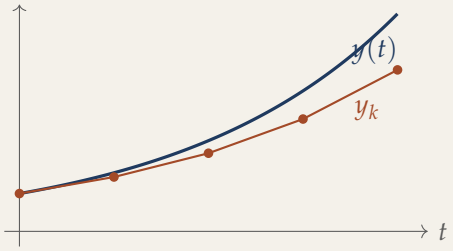
Man ersetzt die Ableitung durch den Vorwärtsdifferenzenquotienten:

$$\frac{y(t+h)-y(t)}{h} \approx y'(t) = f(t, y(t)).$$

Auflösen ergibt das **explizite Euler-Verfahren**:

$$y_{k+1} = y_k + h\, f(t_k, y_k).$$

Anschaulich: Man geht ein Stück in Richtung der Tangente.



Man sieht: Bei konvexer Lösung liegt die Tangente stets darunter – das Verfahren unterschätzt systematisch, und der Fehler summiert sich.

Die drei Grundbegriffe

Konsistenz. Der **lokale Diskretisierungsfehler** ist der Fehler eines *einzigen* Schrittes, ausgehend von der exakten Lösung:

$$\tau_k := \frac{y(t_{k+1})-y(t_k)}{h} - f(t_k, y(t_k)).$$

Das Verfahren heißt **konsistent von der Ordnung** p, wenn $\tau_k = \mathcal{O}(h^p)$.
Stabilität. Kleine Störungen dürfen sich nicht unbeschränkt aufschaukeln.
Konvergenz. Der **globale Fehler** $\|y_k - y(t_k)\|$ geht gegen null für $h \to 0$.
Der Fundamentalsatz (Dahlquist):

$$\boxed{\text{Konsistenz} + \text{Stabilität} = \text{Konvergenz}}$$

und die Konvergenzordnung ist gleich der Konsistenzordnung.
Warum das nicht selbstverständlich ist: Bei $n = T/h$ Schritten summieren sich n lokale Fehler der Größe h^{p+1} zu $n \cdot h^{p+1} = Th^p$. Der globale Fehler ist also *eine Ordnung schlechter* als der lokale. Und dass sich die Fehler nur addieren und nicht multiplizieren, ist genau die Aussage der Stabilität.

Satz 12.2 (Konvergenz des Euler-Verfahrens). *Sei f Lipschitz-stetig mit Konstante L und y zweimal stetig differenzierbar mit $\|y''\| \le M$. Dann gilt für den globalen Fehler $e_k := \|y_k - y(t_k)\|$:*

$$e_k \le \frac{hM}{2L}\left(\mathrm{e}^{L(t_k - t_0)} - 1\right).$$

Beweis. **Schritt 1 – Lokaler Fehler.** Taylor:

$$y(t_{k+1}) = y(t_k) + hy'(t_k) + \frac{h^2}{2}y''(\xi) = y(t_k) + hf(t_k, y(t_k)) + \frac{h^2}{2}y''(\xi).$$

Schritt 2 – Differenz zum Verfahren. Subtrahiert man $y_{k+1} = y_k + hf(t_k, y_k)$:

$$y(t_{k+1}) - y_{k+1} = \big(y(t_k) - y_k\big) + h\big(f(t_k, y(t_k)) - f(t_k, y_k)\big) + \frac{h^2}{2}y''(\xi).$$

Schritt 3 – Normen und Lipschitz.

$$e_{k+1} \le e_k + hL\,e_k + \frac{h^2 M}{2} = (1 + hL)e_k + \frac{h^2 M}{2}.$$

Schritt 4 – Die Rekursion auflösen. Für eine Rekursion $e_{k+1} \le \alpha e_k + \beta$ mit $e_0 = 0$ gilt

$$e_k \le \beta\frac{\alpha^k - 1}{\alpha - 1}.$$

Mit $\alpha = 1 + hL$, $\beta = \frac{h^2 M}{2}$:

$$e_k \le \frac{h^2 M}{2} \cdot \frac{(1 + hL)^k - 1}{hL} = \frac{hM}{2L}\big((1 + hL)^k - 1\big).$$

Schritt 5 – Abschätzen. Es gilt $1 + x \le \mathrm{e}^x$, also $(1 + hL)^k \le \mathrm{e}^{khL} = \mathrm{e}^{L(t_k - t_0)}$:

$$e_k \le \frac{hM}{2L}\left(\mathrm{e}^{L(t_k - t_0)} - 1\right).$$

■

Was der Satz aussagt

- Der Fehler ist $\mathcal{O}(h)$ – **Ordnung 1**. Halbiert man h, so halbiert sich der Fehler. Das ist sehr wenig.
- Der Faktor $\mathrm{e}^{L(T - t_0)}$ wächst *exponentiell* mit der Integrationslänge. Über lange Zeiträume ist die Schranke wertlos – was aber oft an der Schranke liegt, nicht am Verfahren.
- Um den Fehler von 10^{-2} auf 10^{-6} zu senken, braucht man *10 000-mal* so viele Schritte.

Fazit: Euler ist zum Verstehen gut und zum Rechnen schlecht.

12.3 Runge–Kutta-Verfahren

Die Idee: mehrere Steigungen mitteln

Euler benutzt nur die Steigung am linken Rand. Besser wäre eine mittlere Steigung – aber die kennt man nicht, weil sie den unbekannten Endwert bräuchte.
Runge–Kuttas Ausweg: Man schätzt die Steigung an mehreren Zwischenstellen, jeweils unter Verwendung der vorher berechneten Schätzungen, und bildet eine gewichtete Mittelung.
Das klassische Verfahren RK4:

$$
\begin{aligned}
k_1 &= f(t_k,\ y_k) && \text{Steigung am Anfang}\\
k_2 &= f\Big(t_k + \tfrac{h}{2},\ y_k + \tfrac{h}{2}k_1\Big) && \text{Mitte, mit } k_1 \text{ geschätzt}\\
k_3 &= f\Big(t_k + \tfrac{h}{2},\ y_k + \tfrac{h}{2}k_2\Big) && \text{nochmals Mitte, mit } k_2\\
k_4 &= f(t_k + h,\ y_k + hk_3) && \text{Steigung am Ende}
\end{aligned}
$$

$$
y_{k+1} = y_k + \frac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right)
$$

Die Gewichte $\frac{1}{6}, \frac{2}{6}, \frac{2}{6}, \frac{1}{6}$ sind genau die der Simpson-Regel. Das ist kein Zufall: Hängt f nur von t ab, so wird RK4 zur Simpson-Regel.
Ordnung 4 bei vier Funktionsauswertungen pro Schritt. Der Nachweis erfordert einen mühsamen Koeffizientenvergleich der Taylorentwicklungen bis zur vierten Ordnung – KUTTA führte ihn 1901 durch.

Ordnung ist nicht gleich Stufenzahl

Man könnte meinen, s Stufen ergäben Ordnung s. Das gilt nur bis $s = 4$:

Stufen s	1	2	3	4	5	6	7	8
max. Ordnung	1	2	3	4	4	5	6	6

Die **Butcher-Schranken** zeigen: Ab Ordnung 5 braucht man mehr Stufen als die Ordnung. Deshalb ist RK4 der „süße Punkt“ und bis heute das meistverwendete Verfahren mit fester Schrittweite.

Butcher-Tableaus

Man notiert ein Runge–Kutta-Verfahren kompakt:

$$
\begin{array}{c|ccc}
c_1 & a_{11} & \cdots & a_{1s}\\
\vdots & \vdots & & \vdots\\
c_s & a_{s1} & \cdots & a_{ss}\\
\hline
 & b_1 & \cdots & b_s
\end{array}
\qquad
k_i = f\Big(t_k + c_i h,\ y_k + h\sum_j a_{ij}k_j\Big),\quad y_{k+1} = y_k + h\sum_i b_i k_i.
$$

Ist A strikt untere Dreiecksmatrix, so heißt das Verfahren *explizit* – jedes k_i hängt nur von früheren ab und lässt sich direkt ausrechnen. Andernfalls ist es *implizit*, und man muss pro Schritt ein Gleichungssystem lösen.
Beispiele:

Euler:
$$\begin{array}{c|c} 0 & 0 \\ \hline & 1 \end{array}$$

RK4:
$$\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{2} & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ 1 & 0 & 0 & 1 & \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array}$$

Implizite Mittelpunktregel:
$$\begin{array}{c|c} \frac{1}{2} & \frac{1}{2} \\ \hline & 1 \end{array}$$

Die Bedingung $\sum_j a_{ij} = c_i$ sorgt dafür, dass die Zwischenstellen konsistent sind. JOHN BUTCHER entwickelte ab den 1960er Jahren die zugehörige algebraische Theorie – die Ordnungsbedingungen lassen sich über *Wurzelbäume* systematisch aufstellen. Für Ordnung 8 sind es bereits 200 Gleichungen.

Aufgabe 12.1. Lösen Sie

$$y' = y - t^2 + 1, \qquad y(0) = 0{,}5$$

auf $[0;\,0{,}4]$ mit $h = 0{,}2$, einmal mit Euler und einmal mit RK4. Die exakte Lösung ist

$$y(t) = (t+1)^2 - 0{,}5\,\mathrm{e}^t.$$

Lösung Schritt für Schritt.

Vorbereitung – exakte Werte.

$$y(0{,}2) = (1{,}2)^2 - 0{,}5\mathrm{e}^{0{,}2} = 1{,}44 - 0{,}5 \cdot 1{,}22140276 = 1{,}44 - 0{,}61070138 = 0{,}82929862.$$

$$y(0{,}4) = (1{,}4)^2 - 0{,}5\mathrm{e}^{0{,}4} = 1{,}96 - 0{,}5 \cdot 1{,}49182470 = 1{,}96 - 0{,}74591235 = 1{,}21408765.$$

Teil 1 – Euler.
Schritt 1 ($t_0 = 0$, $y_0 = 0{,}5$):

$$f(0;\,0{,}5) = 0{,}5 - 0 + 1 = 1{,}5.$$

$$y_1 = 0{,}5 + 0{,}2 \cdot 1{,}5 = 0{,}5 + 0{,}3 = 0{,}8.$$

Schritt 2 ($t_1 = 0{,}2$, $y_1 = 0{,}8$):

$$f(0{,}2;\,0{,}8) = 0{,}8 - 0{,}04 + 1 = 1{,}76.$$

$$y_2 = 0{,}8 + 0{,}2 \cdot 1{,}76 = 0{,}8 + 0{,}352 = 1{,}152.$$

Fehler:

$$|y_1 - y(0{,}2)| = |0{,}8 - 0{,}82929862| = 2{,}93 \cdot 10^{-2}$$
$$|y_2 - y(0{,}4)| = |1{,}152 - 1{,}21408765| = 6{,}21 \cdot 10^{-2}$$

Teil 2 – RK4, Schritt 1. ($t_0 = 0$, $y_0 = 0{,}5$, $h = 0{,}2$)

$$k_1 = f(0;\ 0{,}5) = 0{,}5 - 0 + 1 = 1{,}5$$

$$k_2 = f(0{,}1;\ 0{,}5 + 0{,}1 \cdot 1{,}5) = f(0{,}1;\ 0{,}65) = 0{,}65 - 0{,}01 + 1 = 1{,}64$$

$$k_3 = f(0{,}1;\ 0{,}5 + 0{,}1 \cdot 1{,}64) = f(0{,}1;\ 0{,}664) = 0{,}664 - 0{,}01 + 1 = 1{,}654$$

$$k_4 = f(0{,}2;\ 0{,}5 + 0{,}2 \cdot 1{,}654) = f(0{,}2;\ 0{,}8308) = 0{,}8308 - 0{,}04 + 1 = 1{,}7908$$

Gewichtete Summe:

$$k_1 + 2k_2 + 2k_3 + k_4 = 1{,}5 + 3{,}28 + 3{,}308 + 1{,}7908 = 9{,}8788.$$

$$y_1 = 0{,}5 + \frac{0{,}2}{6} \cdot 9{,}8788 = 0{,}5 + 0{,}03333333 \cdot 9{,}8788 = 0{,}5 + 0{,}32929333 = 0{,}82929333.$$

Fehler: $|0{,}82929333 - 0{,}82929862| = 5{,}29 \cdot 10^{-6}$.

Teil 2 – RK4, Schritt 2. ($t_1 = 0{,}2$, $y_1 = 0{,}82929333$)

$$k_1 = f(0{,}2;\ 0{,}82929333) = 0{,}82929333 - 0{,}04 + 1 = 1{,}78929333$$

$$y_1 + \tfrac{h}{2}k_1 = 0{,}82929333 + 0{,}1 \cdot 1{,}78929333 = 0{,}82929333 + 0{,}17892933 = 1{,}00822266$$

$$k_2 = f(0{,}3;\ 1{,}00822266) = 1{,}00822266 - 0{,}09 + 1 = 1{,}91822266$$

$$y_1 + \tfrac{h}{2}k_2 = 0{,}82929333 + 0{,}19182227 = 1{,}02111560$$

$$k_3 = f(0{,}3;\ 1{,}02111560) = 1{,}02111560 - 0{,}09 + 1 = 1{,}93111560$$

$$y_1 + hk_3 = 0{,}82929333 + 0{,}2 \cdot 1{,}93111560 = 0{,}82929333 + 0{,}38622312 = 1{,}21551645$$

$$k_4 = f(0{,}4;\ 1{,}21551645) = 1{,}21551645 - 0{,}16 + 1 = 2{,}05551645$$

Gewichtete Summe:

$$1{,}78929333 + 2(1{,}91822266) + 2(1{,}93111560) + 2{,}05551645$$

$$= 1{,}78929333 + 3{,}83644532 + 3{,}86223120 + 2{,}05551645 = 11{,}54348630.$$

$$y_2 = 0{,}82929333 + \frac{0{,}2}{6} \cdot 11{,}54348630 = 0{,}82929333 + 0{,}38478288 = 1{,}21407621.$$

Fehler: $|1{,}21407621 - 1{,}21408765| = 1{,}14 \cdot 10^{-5}$.

Vergleich.

	Euler	RK4	Verhältnis
Fehler bei $t = 0{,}2$	$2{,}93 \cdot 10^{-2}$	$5{,}29 \cdot 10^{-6}$	5 540
Fehler bei $t = 0{,}4$	$6{,}21 \cdot 10^{-2}$	$1{,}14 \cdot 10^{-5}$	5 450
Auswertungen von f	2	8	–

RK4 kostet viermal so viel pro Schritt und ist 5000-mal genauer.
Und die Ordnung experimentell: Halbiert man h auf 0,1, so erhält man

	$h = 0{,}2$	$h = 0{,}1$	Faktor
Euler	$6{,}21 \cdot 10^{-2}$	$3{,}25 \cdot 10^{-2}$	$1{,}91 \approx 2^1$
RK4	$1{,}14 \cdot 10^{-5}$	$6{,}84 \cdot 10^{-7}$	$16{,}7 \approx 2^4$

Ordnung 1 und Ordnung 4 – experimentell bestätigt. ✓

12.4 Schrittweitensteuerung

Warum feste Schrittweiten Verschwendung sind

Eine Lösung ändert sich selten überall gleich schnell. Bei einer Rakete etwa passiert beim Start alles in Sekunden, im Flug stundenlang wenig.
Man will die Schrittweite automatisch anpassen: klein, wo es schnell geht, groß, wo es ruhig ist. Dazu braucht man eine *Fehlerschätzung* pro Schritt.

Eingebettete Verfahren

Die elegante Lösung: Man konstruiert *zwei* Verfahren verschiedener Ordnung, die *dieselben* Stufen k_i benutzen und sich nur in den Gewichten unterscheiden:

$$y_{k+1} = y_k + h \sum_i b_i k_i \qquad \text{(Ordnung } p\text{)}$$

$$\hat{y}_{k+1} = y_k + h \sum_i \hat{b}_i k_i \qquad \text{(Ordnung } p+1\text{)}$$

Die Differenz

$$\varepsilon := \|\hat{y}_{k+1} - y_{k+1}\|$$

schätzt den lokalen Fehler – **praktisch gratis**, weil beide Verfahren dieselben Funktionsauswertungen teilen.

Die Steuerung. Ist tol die geforderte Toleranz, so setzt man

$$h_{\text{neu}} = h \cdot \min\left(f_{\max},\ \max\left(f_{\min},\ \rho\left(\frac{\text{tol}}{\varepsilon}\right)^{1/(p+1)}\right)\right)$$

mit einem Sicherheitsfaktor $\rho \approx 0{,}9$ und Begrenzungen $f_{\min} \approx 0{,}2$, $f_{\max} \approx 5$.

Woher der Exponent kommt: Der lokale Fehler verhält sich wie Ch^{p+1}. Soll er gleich tol werden, so muss

$$\frac{\text{tol}}{\varepsilon} = \frac{Ch_{\text{neu}}^{p+1}}{Ch^{p+1}} = \left(\frac{h_{\text{neu}}}{h}\right)^{p+1}.$$

Auflösen nach h_{neu} gibt die Formel.

Ist $\varepsilon >$ tol, so wird der Schritt verworfen und mit kleinerem h wiederholt.

Die gebräuchlichen Verfahren:

Name	Ordnungen	Stufen	Bemerkung
Runge–Kutta–Fehlberg (RKF45)	4/5	6	Fehlberg 1969, klassisch
Dormand–Prince (DOPRI5)	5/4	7	**Standard**: `ode45`, `RK45`
Dormand–Prince 8	8/5,3	13	für hohe Genauigkeit
Bogacki–Shampine	3/2	4	`ode23`, billig

Ein Kunstgriff bei DOPRI5: Die letzte Stufe des einen Schrittes ist gleich der ersten des nächsten (*first same as last*). Dadurch kostet ein Schritt effektiv nur 6 statt 7 Auswertungen.

12.5 Steife Differentialgleichungen

Ein Phänomen, das erklärungsbedürftig ist

Betrachten Sie

$$y' = -1000\,(y - \cos t) - \sin t, \qquad y(0) = 1.$$

Die Lösung ist $y(t) = \cos t$ – völlig harmlos, glatt, beschränkt, mit Ableitungen der Größenordnung 1.

Und dennoch: Das explizite Euler-Verfahren *explodiert*, sobald $h > 0{,}002$. Auch RK4 versagt für $h > 0{,}0028$.
Um $[0, 10]$ zu durchlaufen, braucht man über 5000 Schritte – für eine Funktion, die man mit 20 Punkten hinreichend darstellen könnte.
Solche Probleme heißen steif. Sie treten auf, wenn ein System Prozesse mit *sehr verschiedenen Zeitskalen* enthält:

- chemische Reaktionskinetik (schnelle und langsame Reaktionen),
- Verbrennungssimulation,
- elektrische Netzwerke mit kleinen Kapazitäten,
- pharmakokinetische Mehrkompartimentmodelle,
- ortsdiskretisierte parabolische Gleichungen (Kapitel 13).

Der Name stammt von CURTISS und HIRSCHFELDER (1952), die an Verbrennungsproblemen arbeiteten.

Die Analyse: das Stabilitätsgebiet

Man untersucht die **Testgleichung**

$$y' = \lambda y, \qquad \lambda \in \mathbb{C}, \quad \operatorname{Re}\lambda < 0.$$

Die exakte Lösung $y(t) = y_0 e^{\lambda t}$ fällt gegen null.
Explizites Euler:

$$y_{k+1} = y_k + h\lambda y_k = (1 + h\lambda) y_k \Longrightarrow y_k = (1 + h\lambda)^k y_0.$$

Das fällt genau dann, wenn

$$|1 + h\lambda| < 1.$$

In der komplexen $z = h\lambda$-Ebene ist das der **Kreis um -1 mit Radius 1** – ein winziges Gebiet.
Für reelles $\lambda = -1000$ heißt das

$$|1 - 1000h| < 1 \Longrightarrow 0 < h < 0{,}002. \checkmark$$

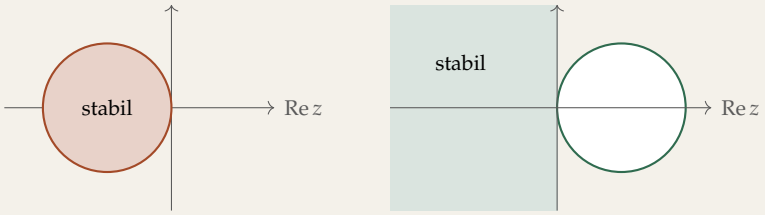
Genau die beobachtete Schranke.
Implizites Euler:

$$y_{k+1} = y_k + h\lambda y_{k+1} \Longrightarrow y_{k+1} = \frac{y_k}{1 - h\lambda}.$$

Das fällt, wenn

$$|1 - h\lambda| > 1,$$

also *außerhalb* des Kreises um $+1$. Das umfasst die **gesamte linke Halbebene** – und noch mehr.



explizites Euler implizites Euler

Definition 12.3. Ein Verfahren heißt **A-stabil**, wenn sein Stabilitätsgebiet die gesamte linke Halbebene $\{\operatorname{Re} z < 0\}$ enthält.

Satz 12.4 (Zweite Dahlquist-Schranke, 1963). *(a)* ***Kein explizites Runge–Kutta-Verfahren ist A-stabil.*** *Das Stabilitätsgebiet expliziter Verfahren ist stets beschränkt.*

(b) Ein A-stabiles lineares Mehrschrittverfahren hat höchstens die Ordnung 2. Das Verfahren kleinster Fehlerkonstante dieser Art ist die Trapezregel.

Die Konsequenz für die Praxis

Bei steifen Problemen führt kein Weg an impliziten Verfahren vorbei.
Der Preis: In jedem Schritt ist ein nichtlineares Gleichungssystem zu lösen. Für implizites Euler

$$y_{k+1} = y_k + hf(t_{k+1}, y_{k+1})$$

löst man mit dem Newton-Verfahren (Kapitel 7) – die Jacobi-Matrix von f wird gebraucht, und pro Newton-Schritt ein lineares System (Kapitel 3).
Hier laufen alle bisherigen Kapitel zusammen. Ein moderner steifer Löser enthält: Runge–Kutta oder BDF, Newton-Verfahren, LR-Zerlegung mit Pivotisierung, Schrittweiten- und Ordnungssteuerung.
Die gebräuchlichen steifen Löser:

Verfahren	Typ	Bemerkung
BDF (Gear)	Mehrschritt	Ordnung 1–6; ab Ordnung 3 nicht mehr A-stabil, aber A(α)-stabil. Kern von `LSODA`, `CVODE`, `ode15s`.
Radau IIA	implizites RK	Ordnung 5, L-stabil, sehr robust. `RADAU5` von Hairer und Wanner.
SDIRK	implizites RK	diagonal-implizit: die Stufen lassen sich nacheinander lösen, was billiger ist.
Rosenbrock	linear-implizit	nur *ein* lineares System pro Stufe, keine Newton-Iteration.

L-Stabilität ist eine Verschärfung: Zusätzlich zur A-Stabilität fordert man

$$\lim_{z \to -\infty} |R(z)| = 0$$

für die Stabilitätsfunktion R. Damit werden sehr schnelle Moden vollständig gedämpft. Die Trapezregel ist A-stabil, aber nicht L-stabil: Für $z \to -\infty$ geht $|R(z)| \to 1$, und schnelle Moden *oszillieren* mit kaum abnehmender Amplitude, statt zu verschwinden – ein bekanntes Ärgernis bei parabolischen Problemen.

Wie erkennt man Steifheit? Wenn ein expliziter Löser sehr viele Schritte braucht, obwohl die Lösung glatt aussieht. Ein quantitatives Maß ist die *Steifheitszahl*

$$S = \frac{\max_i |\operatorname{Re} \lambda_i|}{\min_i |\operatorname{Re} \lambda_i|}$$

der Eigenwerte der Jacobi-Matrix. Ab $S > 10^3$ spricht man von steif; in der Verbrennungsrechnung erreicht S Werte von 10^{10} und mehr.

Moderne Programme wie `LSODA` *schalten automatisch um*: Sie starten explizit, überwachen die Steifheit und wechseln bei Bedarf auf BDF – und zurück[12].

Runge, Kutta und die Zeit vor den Rechnern

CARL RUNGE (1856–1927) war Professor in Hannover und später der erste Inhaber eines Lehrstuhls für angewandte Mathematik in Deutschland, in Göttingen. Er veröffentlichte sein Verfahren 1895 in den *Mathematischen Annalen*.

Runge kam aus der Spektroskopie: Er hatte mit FRIEDRICH PASCHEN Spektrallinien vermessen und brauchte Methoden, um aus Messreihen Gesetzmäßigkeiten zu

gewinnen. Das Runge-Phänomen aus Kapitel 8 entdeckte er 1901 im selben Zusammenhang.
MARTIN WILHELM KUTTA (1867–1944) verallgemeinerte die Idee 1901 in seiner Dissertation und gab das klassische Verfahren vierter Ordnung an. Bekannter ist er allerdings für die *Kutta–Joukowski-Formel* der Tragflächenauftriebstheorie – er gehört zu den Begründern der Aerodynamik.
Der entscheidende Punkt für das Verständnis der Zeit: Beide entwickelten ihre Verfahren für die *Handrechnung*. Ein einziger RK4-Schritt bedeutete vier Auswertungen und etwa zwanzig Multiplikationen – mehrere Minuten Arbeit. Eine Bahnberechnung über hundert Schritte war ein Tageswerk.
Genau deshalb war Effizienz das beherrschende Kriterium: Man wollte mit möglichst wenigen Auswertungen möglichst weit kommen. Dass RK4 ausgerechnet vier Stufen hat, ist Ergebnis dieser Ökonomie – und es ist bemerkenswert, dass die Abwägung von 1901 auch auf Maschinen mit 10^{12} Operationen pro Sekunde noch trägt.
Die Theorie erhielt ihre moderne Form durch GERMUND DAHLQUIST (1925–2005) in Stockholm, der 1956 und 1963 die beiden nach ihm benannten Schranken bewies, sowie durch JOHN BUTCHER in Auckland und ERNST HAIRER und GERHARD WANNER in Genf, deren zweibändiges Werk[11,12] bis heute die Referenz ist.

13 Randwertprobleme und partielle Differentialgleichungen

13.1 Der Unterschied zum Anfangswertproblem

Zwei Enden statt einem

Beim Anfangswertproblem sind alle Bedingungen an *einer* Stelle gegeben; man kann Schritt für Schritt vorwärts marschieren.
Beim **Randwertproblem**

$$-u'' = f(x) \text{ auf } (0,1), \qquad u(0) = \alpha,\ u(1) = \beta$$

sind die Bedingungen auf beide Enden verteilt. **Marschieren geht nicht** – man kennt $u'(0)$ nicht.
Die Konsequenz: Man muss alle Unbekannten *gleichzeitig* bestimmen. Das führt auf ein großes Gleichungssystem – und damit zurück zu Kapitel 3 und 5.
Und noch etwas ist anders: Existenz und Eindeutigkeit sind nicht selbstverständlich. Das Problem

$$u'' + \pi^2 u = 0, \qquad u(0) = u(1) = 0$$

hat *unendlich viele* Lösungen $u(x) = c\sin(\pi x)$. Randwertprobleme können resonant sein.

13.2 Finite Differenzen

Das Verfahren

Man legt ein Gitter

$$x_i = ih, \qquad h = \frac{1}{n+1}, \qquad i = 0, \ldots, n+1$$

und ersetzt die zweite Ableitung durch den zentralen Differenzenquotienten (Kapitel 11):

$$u''(x_i) \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}.$$

Das Problem $-u'' = f$ wird zu

$$\frac{-u_{i-1} + 2u_i - u_{i+1}}{h^2} = f(x_i), \qquad i = 1, \dots, n,$$

mit $u_0 = \alpha$, $u_{n+1} = \beta$.
In Matrixform:

$$\frac{1}{h^2} \underbrace{\begin{pmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{pmatrix}}_{=:A_h} \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n-1} \\ u_n \end{pmatrix} = \begin{pmatrix} f_1 + \alpha/h^2 \\ f_2 \\ \vdots \\ f_{n-1} \\ f_n + \beta/h^2 \end{pmatrix}$$

Die Matrix A_h ist der Prototyp der numerischen Mathematik. Sie ist

- tridiagonal – lösbar in $\mathcal{O}(n)$ mit dem Thomas-Algorithmus,
- symmetrisch positiv definit – Cholesky ohne Pivotisierung, CG anwendbar,
- mit *explizit bekannten* Eigenwerten:

$$\lambda_k = \frac{4}{h^2} \sin^2\left(\frac{k\pi h}{2}\right), \qquad k = 1, \dots, n,$$

mit Eigenvektoren $v_k = \big(\sin(k\pi x_i)\big)_i$.

Die Kondition – und warum sie das Kernproblem ist

Aus den Eigenwerten:

$$\lambda_{\min} = \frac{4}{h^2} \sin^2\left(\frac{\pi h}{2}\right) \approx \pi^2 \quad \text{(für kleines } h\text{)}$$

$$\lambda_{\max} = \frac{4}{h^2} \sin^2\left(\frac{n\pi h}{2}\right) \approx \frac{4}{h^2}.$$

$$\operatorname{cond}_2(A_h) = \frac{\lambda_{\max}}{\lambda_{\min}} \approx \frac{4}{\pi^2 h^2} = \mathcal{O}(h^{-2}).$$

Das ist die zentrale Schwierigkeit aller Diskretisierungsverfahren: Je feiner das Gitter, desto genauer die Lösung – und desto schlechter konditioniert das System.

n	h	$\operatorname{cond}_2(A_h)$	CG-Iterationen $\sim \sqrt{\kappa}$
10^2	10^{-2}	$4{,}1 \cdot 10^3$	64
10^3	10^{-3}	$4{,}1 \cdot 10^5$	640
10^4	10^{-4}	$4{,}1 \cdot 10^7$	6 400

Der Aufwand für CG ohne Vorkonditionierung wächst damit wie $\mathcal{O}(n \cdot \sqrt{\text{cond}}) = \mathcal{O}(n^{1,5})$ in einer Dimension – und im Dreidimensionalen entsprechend schlimmer. **Genau deshalb ist die Vorkonditionierung so wichtig, und deshalb wurden Mehrgitterverfahren entwickelt.**

Aufgabe 13.1. Lösen Sie

$$-u'' = \pi^2 \sin(\pi x), \qquad u(0) = u(1) = 0$$

mit $n = 3$ inneren Gitterpunkten und vergleichen Sie mit der exakten Lösung $u(x) = \sin(\pi x)$.

Lösung Schritt für Schritt.

Schritt 1 – Gitter. $h = \frac{1}{4} = 0{,}25$; die inneren Punkte sind

$$x_1 = 0{,}25, \quad x_2 = 0{,}50, \quad x_3 = 0{,}75.$$

Schritt 2 – Rechte Seite.

$$f(x_1) = \pi^2 \sin(\pi/4) = 9{,}8696 \cdot 0{,}707107 = 6{,}979237$$

$$f(x_2) = \pi^2 \sin(\pi/2) = 9{,}869604$$

$$f(x_3) = \pi^2 \sin(3\pi/4) = 6{,}979237$$

Schritt 3 – Das System. Mit $h^2 = 0{,}0625$ und homogenen Randwerten:

$$\frac{1}{0{,}0625} \begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = \begin{pmatrix} 6{,}979237 \\ 9{,}869604 \\ 6{,}979237 \end{pmatrix}$$

Multiplikation mit h^2:

$$\begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix} = \begin{pmatrix} 0{,}436202 \\ 0{,}616850 \\ 0{,}436202 \end{pmatrix}$$

Schritt 4 – Lösen. Wegen der Symmetrie ist $u_1 = u_3$. Damit:

$$2u_1 - u_2 = 0{,}436202$$

$$-2u_1 + 2u_2 = 0{,}616850 \quad \text{(zweite Zeile mit } u_1 = u_3\text{)}$$

Addition beider Gleichungen:

$$u_2 = 1{,}053052.$$

Hmm, das kann nicht stimmen – prüfen wir. Die zweite Zeile lautet $-u_1 + 2u_2 - u_3 = 0{,}616850$, mit $u_1 = u_3$ also

$$-2u_1 + 2u_2 = 0{,}616850.$$

Und die erste: $2u_1 - u_2 = 0{,}436202$.
Aus der ersten: $u_2 = 2u_1 - 0{,}436202$. Einsetzen in die zweite:

$$-2u_1 + 2(2u_1 - 0{,}436202) = 0{,}616850$$

$$-2u_1 + 4u_1 - 0{,}872404 = 0{,}616850$$

$$2u_1 = 1{,}489254 \Longrightarrow u_1 = 0{,}744627.$$

$$u_2 = 2(0{,}744627) - 0{,}436202 = 1{,}489254 - 0{,}436202 = 1{,}053052.$$

Das ist > 1 – die exakte Lösung ist aber ≤ 1. Prüfen wir die dritte Zeile:

$$-u_2 + 2u_3 = -1{,}053052 + 2(0{,}744627) = -1{,}053052 + 1{,}489254 = 0{,}436202.\checkmark$$

Das System ist korrekt gelöst.
Schritt 5 – Wo liegt der Fehler? Nirgends – die Diskretisierung ist bei $h = 0{,}25$ einfach noch grob. Vergleich:

x_i	u_i (numerisch)	$u(x_i)$ (exakt)	Fehler
0,25	0,744627	0,707107	0,037520
0,50	1,053052	1,000000	0,053052
0,75	0,744627	0,707107	0,037520

Der maximale Fehler beträgt $5{,}3 \cdot 10^{-2}$.
Schritt 6 – Die Ordnung prüfen. Für $n = 7$ ($h = 0{,}125$) erhält man einen Maximalfehler von $1{,}30 \cdot 10^{-2}$:

$$\frac{5{,}31 \cdot 10^{-2}}{1{,}30 \cdot 10^{-2}} = 4{,}08 \approx 2^2.$$

Konvergenzordnung 2. ✓
Schritt 7 – Die exakte Fehlerformel. Für dieses spezielle Problem lässt sich der Fehler geschlossen angeben. Der Eigenvektor $v_1 = (\sin(\pi x_i))_i$ ist zugleich die exakte Lösung; das diskrete System liefert

$$u_i^h = \frac{\pi^2}{\lambda_1} \sin(\pi x_i), \qquad \lambda_1 = \frac{4}{h^2} \sin^2\left(\frac{\pi h}{2}\right).$$

Für $h = 0{,}25$:

$$\lambda_1 = \frac{4}{0{,}0625}\sin^2(0{,}392699) = 64 \cdot (0{,}382683)^2 = 64 \cdot 0{,}146447 = 9{,}372583.$$

$$\frac{\pi^2}{\lambda_1} = \frac{9{,}869604}{9{,}372583} = 1{,}053030.$$

Also $u_2^h = 1{,}053030 \cdot 1 = 1{,}053030$ – übereinstimmend mit unserer Rechnung. ✓
Und der relative Fehler ist

$$\frac{\pi^2}{\lambda_1} - 1 = \frac{\pi^2 h^2}{12} + \mathcal{O}(h^4) = \frac{9{,}8696 \cdot 0{,}0625}{12} = 0{,}0514,$$

was den beobachteten 5,3 % entspricht. ✓

13.3 Die Poisson-Gleichung in zwei Dimensionen

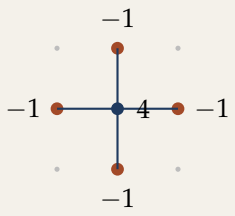
Der Fünf-Punkte-Stern

Für

$$-\Delta u = -\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) = f \quad \text{auf } (0,1)^2, \qquad u = 0 \text{ auf dem Rand}$$

diskretisiert man beide Richtungen:

$$\frac{4u_{ij} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1}}{h^2} = f_{ij}.$$



Die Größe des Systems. Bei N inneren Punkten je Richtung hat man

$$n = N^2$$

Unbekannte. Für $N = 1000$ sind das eine Million – das Beispiel aus Kapitel 5.
Die Struktur: Die Matrix ist blocktridiagonal mit Bandbreite $N = \sqrt{n}$. Direkte Lösung kostet

$$\mathcal{O}(n \cdot N^2) = \mathcal{O}(n^2),$$

in drei Dimensionen sogar $\mathcal{O}(n^{7/3})$. Das ist der Grund, warum iterative Verfahren hier unverzichtbar sind.
Die Konditionszahl ist wieder $\mathcal{O}(h^{-2}) = \mathcal{O}(n)$.

Der Aufwandsvergleich – die entscheidende Tabelle

Für die 2D-Poisson-Gleichung mit n Unbekannten:

Verfahren	**Aufwand**	$n = 10^6$
Gauß-Elimination (voll)	$\mathcal{O}(n^3)$	unmöglich
Gauß mit Bandstruktur	$\mathcal{O}(n^2)$	10^{12} – Stunden
Jacobi	$\mathcal{O}(n^2)$	10^{12}
Gauß–Seidel	$\mathcal{O}(n^2)$	$5 \cdot 10^{11}$
SOR (optimal)	$\mathcal{O}(n^{3/2})$	10^9
CG	$\mathcal{O}(n^{3/2})$	10^9
CG + unvollständiges Cholesky	$\mathcal{O}(n^{5/4})$	$3 \cdot 10^7$
FFT (nur bei einfacher Geometrie)	$\mathcal{O}(n \log n)$	$2 \cdot 10^7$
Mehrgitter	$\boldsymbol{\mathcal{O}(n)}$	$\mathbf{10^7}$

Mehrgitter ist optimal: Der Aufwand ist proportional zur Zahl der Unbekannten. Mehr kann man nicht verlangen, denn allein das Aufschreiben der Lösung kostet $\mathcal{O}(n)$.

13.4 Mehrgitterverfahren

Die Beobachtung, aus der alles folgt

Wendet man Gauß–Seidel oder gedämpftes Jacobi auf $A_h u = f$ an und zerlegt den Fehler in Eigenvektoren

$$e = \sum_k c_k v_k, \qquad v_k = \big(\sin(k\pi x_i)\big)_i,$$

so wird jede Komponente mit einem eigenen Faktor gedämpft.
Für gedämpftes Jacobi mit Parameter $\omega = \frac{2}{3}$ ergibt sich der Dämpfungsfaktor

$$\mu_k = 1 - \frac{2}{3}\sin^2\!\left(\frac{k\pi h}{2}\right).$$

Modus	Beschreibung	μ_k
$k = 1$	glatteste Welle	$\approx 1 - \frac{\pi^2 h^2}{6} \approx 0{,}9999$
$k = n/2$	mittlere Frequenz	$\approx 0{,}67$
$k = n$	höchste Frequenz	$\approx 0{,}33$

Die entscheidende Erkenntnis:

*Klassische Iterationsverfahren dämpfen **hochfrequente** Fehleranteile hervorragend und **niederfrequente** praktisch gar nicht.*

Nach drei bis vier Iterationen sind alle Zacken weg – der Fehler ist *glatt*, aber nicht klein. Genau das erklärt, warum Jacobi und Gauß–Seidel so quälend langsam sind: Sie kämpfen ewig mit den glatten Anteilen.
Deshalb heißen diese Verfahren im Mehrgitterkontext Glätter – ihre Aufgabe ist nicht das Lösen, sondern das Glätten.

Die zündende Idee

Ein glatter Fehler lässt sich auf einem gröberen Gitter darstellen – und dort ist er nicht mehr glatt.

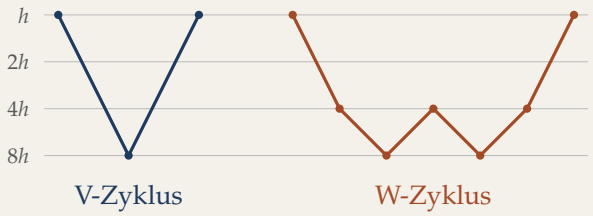
Eine Welle mit $k = 1$ ist auf dem feinen Gitter die glatteste Mode. Halbiert man die Punktzahl, so ist sie relativ zum neuen Gitter doppelt so hochfrequent – und wird dort gut gedämpft.
Man wechselt also die Auflösung. Und weil das gröbere Problem nur ein Viertel (in 2D) der Unbekannten hat, ist es viel billiger.
Der Zweigitterzyklus:

1. **Vorglättung:** ν_1 Schritte des Glätters auf $A_h u_h = f_h$. Ergebnis: $\tilde{u}_h$.
2. **Residuum bilden:** $r_h = f_h - A_h \tilde{u}_h$.
3. **Restriktion:** $r_{2h} = I_h^{2h} r_h$ – das Residuum auf das grobe Gitter übertragen (gewichtete Mittelung).
4. **Grobgitterkorrektur:** $A_{2h} e_{2h} = r_{2h}$ lösen.
5. **Prolongation:** $e_h = I_{2h}^h e_{2h}$ – die Korrektur zurück aufs feine Gitter (lineare Interpolation).
6. **Korrigieren:** $\tilde{u}_h \leftarrow \tilde{u}_h + e_h$.
7. **Nachglättung:** ν_2 weitere Glättungsschritte.

Warum die Fehlergleichung? Man löst auf dem groben Gitter nicht das Problem selbst, sondern die *Fehlergleichung* $Ae = r$. Das ist wesentlich: Der Fehler ist nach der Glättung glatt und daher grob darstellbar; die Lösung u selbst muss es nicht sein.
Und rekursiv: Schritt 4 löst man nicht exakt, sondern durch einen weiteren Zweigitterzyklus. So entsteht eine Hierarchie

$$h \to 2h \to 4h \to \cdots \to \text{Grobstgitter (wenige Punkte, exakt lösbar).}$$

Je nachdem, wie oft man auf jeder Ebene absteigt, spricht man vom **V-Zyklus** (einmal) oder **W-Zyklus** (zweimal).



Satz 13.1 (Mehrgitterkonvergenz)**.** *Unter geeigneten Voraussetzungen (elliptisches Problem, regulärer Gebietsrand, passende Glätter) gilt für den V-Zyklus:*

$$\|e^{neu}\| \leq \rho \left\|e^{alt}\right\|$$

mit einem $\rho < 1$*, das* ***unabhängig von*** h *ist – typisch* $\rho \approx 0{,}1$*.*

Warum das ein Durchbruch war

Die h-Unabhängigkeit ist der entscheidende Punkt. Bei allen anderen Verfahren wächst die Iterationszahl mit der Gitterfeinheit; hier nicht.

N	Gauß–Seidel	CG	Mehrgitter
32	1 900	60	**9**
64	7 700	120	**9**
128	31 000	240	**9**
256	124 000	480	**10**

(Iterationen für Fehlerreduktion um 10^{-6}, 2D-Poisson.)

Der Aufwand pro V-Zyklus. Auf jeder Ebene arbeitet man mit einem Viertel der Punkte der vorigen. Die geometrische Reihe

$$1 + \frac{1}{4} + \frac{1}{16} + \cdots = \frac{4}{3}$$

zeigt: Ein kompletter V-Zyklus kostet nur ein Drittel mehr als die Glättung auf dem feinsten Gitter allein. **Zusammen mit der konstanten Iterationszahl ergibt das $\mathcal{O}(n)$.**

Historisches. RADII FEDORENKO beschrieb die Idee 1961 in Moskau; NIKOLAI BAKHVALOV lieferte 1966 eine Konvergenzanalyse. Beide blieben im Westen weitgehend unbeachtet.

Der Durchbruch kam durch ACHI BRANDT, der 1977 in *Mathematics of Computation* einen ausführlichen Aufsatz veröffentlichte und die Praxistauglichkeit demonstrierte. WOLFGANG HACKBUSCH in Kiel entwickelte parallel die vollständige Theorie und schrieb das Standardwerk[10].

Algebraisches Mehrgitter (AMG). Die beschriebene Variante braucht eine *geometrische* Gitterhierarchie. AMG (Ruge und Stüben, 1987) konstruiert die groben

Ebenen allein aus den Matrixeinträgen – ohne jede Geometrie. Damit lässt es sich als reine „Black-Box"-Vorkonditionierung einsetzen, auch bei unstrukturierten Gittern und komplizierten Geometrien. Bibliotheken wie `BoomerAMG` (in `hypre`) und `ML` (in `Trilinos`) sind heute Standardwerkzeuge im Hochleistungsrechnen.

13.5 Finite Elemente

Ein anderer Ausgangspunkt: die schwache Formulierung

Statt die Differentialgleichung punktweise zu erfüllen, fordert man sie nur *im Mittel gegen Testfunktionen*.
Aus $-u'' = f$ wird durch Multiplikation mit einer Testfunktion v mit $v(0) = v(1) = 0$ und Integration:

$$-\int_0^1 u''v \,\mathrm{d}x = \int_0^1 fv \,\mathrm{d}x.$$

Partielle Integration auf der linken Seite:

$$-\left[u'v\right]_0^1 + \int_0^1 u'v' \,\mathrm{d}x = \int_0^1 fv \,\mathrm{d}x.$$

Der Randterm verschwindet wegen $v(0) = v(1) = 0$. Es bleibt die **schwache Formulierung**:

$$\int_0^1 u'v' \,\mathrm{d}x = \int_0^1 fv \,\mathrm{d}x \qquad \text{für alle zulässigen } v.$$

Was dabei gewonnen wird:

- Es steht nur noch die *erste* Ableitung da. Die Lösung muss nur einmal (schwach) differenzierbar sein – man kann stückweise lineare Ansatzfunktionen verwenden.
- Die Formulierung hat eine variationelle Struktur: Sie ist äquivalent zur Minimierung des Energiefunktionals

  $$J(u) = \frac{1}{2}\int (u')^2 - \int fu.$$

  Das ist physikalisch das *Prinzip der minimalen potentiellen Energie*.
- Unstetige rechte Seiten und springende Materialkoeffizienten sind problemlos.
- Komplizierte Geometrien lassen sich durch unstrukturierte Dreiecks- oder Tetraedernetze erfassen – der große Vorteil gegenüber finiten Differenzen.

Das Galerkin-Verfahren

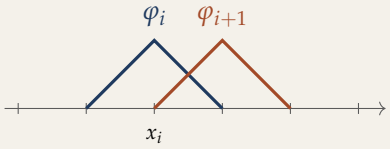
Man wählt einen endlichdimensionalen Raum $V_h = \operatorname{span}\{\varphi_1, \dots, \varphi_n\}$ und setzt an:

$$u_h = \sum_{j=1}^{n} c_j \varphi_j .$$

Die schwache Gleichung wird für alle Basisfunktionen φ_i gefordert:

$$\sum_j c_j \underbrace{\int \varphi_j' \varphi_i' \, \mathrm{d}x}_{=: K_{ij}} = \underbrace{\int f \varphi_i \, \mathrm{d}x}_{=: b_i} .$$

Das ist ein lineares System $Kc = b$ mit der **Steifigkeitsmatrix** K.
Die Wahl der Basis entscheidet über die Struktur. Nimmt man *Hutfunktionen*



so ist φ_i nur auf zwei Teilintervallen ungleich null. Folglich ist $K_{ij} = 0$, sobald $|i - j| > 1$ – **die Steifigkeitsmatrix ist dünn besetzt**.
Und man rechnet nach: Für gleichabständige Knoten ergibt sich

$$K = \frac{1}{h} \begin{pmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & \ddots & \ddots & \ddots \end{pmatrix},$$

dieselbe Matrix wie bei finiten Differenzen (bis auf einen Faktor h). Die beiden Zugänge treffen sich hier – was kein Zufall ist, sondern die Ausnahme: Bei höherdimensionalen oder unstrukturierten Problemen unterscheiden sie sich deutlich.

Satz 13.2 (Céa-Lemma). *Für die Galerkin-Lösung u_h gilt*

$$\|u - u_h\|_V \le \frac{C}{\alpha} \inf_{v_h \in V_h} \|u - v_h\|_V ,$$

wobei C die Stetigkeits- und α die Koerzivitätskonstante der Bilinearform sind.

Was das Céa-Lemma bedeutet

Die Galerkin-Lösung ist – bis auf einen festen Faktor – so gut wie die beste Approximation im Ansatzraum.

Das trennt zwei Fragen sauber:

- Wie gut kann man u überhaupt in V_h approximieren? Das ist reine Approximationstheorie (Kapitel 10) – für stückweise lineare Funktionen gilt $\mathcal{O}(h)$ in der Energienorm, $\mathcal{O}(h^2)$ in L^2.
- Wie nahe kommt das Verfahren dieser besten Approximation? Antwort: bis auf C/α.

Damit reduziert sich die Fehleranalyse eines komplizierten Verfahrens auf eine reine Interpolationsabschätzung – das ist der Grund, warum die Finite-Elemente-Theorie so geschlossen ist.
Die Konvergenzraten:

Ansatzfunktionen	L^2-Fehler	Energiefehler
stückweise linear (P_1)	$\mathcal{O}(h^2)$	$\mathcal{O}(h)$
stückweise quadratisch (P_2)	$\mathcal{O}(h^3)$	$\mathcal{O}(h^2)$
Grad k	$\mathcal{O}(h^{k+1})$	$\mathcal{O}(h^k)$

(Bei hinreichend glatter Lösung; die L^2-Rate folgt aus dem Aubin–Nitsche-Trick.)
Adaptivität. Der praktische Vorzug der Finiten Elemente liegt in der *lokalen Verfeinerung*: Ein *A-posteriori-Fehlerschätzer* (Babuška und Rheinboldt 1978) bewertet jedes Element einzeln; nur dort, wo der Fehler groß ist, wird verfeinert. Bei Lösungen mit Singularitäten – einspringenden Ecken, Rissspitzen – gewinnt man damit Größenordnungen gegenüber gleichmäßiger Verfeinerung.
Historisches. Die Methode entstand im Flugzeugbau: JOHN ARGYRIS (Stuttgart und London) und RAY CLOUGH (Berkeley) entwickelten sie in den 1950er Jahren für Tragflächenberechnungen; Clough prägte 1960 den Namen *finite element method*. RICHARD COURANT hatte 1943 in einem Aufsatz über Torsionsprobleme die Grundidee bereits vorweggenommen, ohne sie zu verfolgen. Die mathematische Theorie – Sobolewräume, Céa-Lemma, Bramble–Hilbert – entstand in den 1960er und 70er Jahren; das deutschsprachige Standardwerk stammt von DIETRICH BRAESS[4].

Ein Überblick über die Verfahrensklassen

Verfahren	Stärke	Schwäche	Konvergenz
Finite Differenzen	einfach, schnell	nur einfache Geometrien	$\mathcal{O}(h^2)$
Finite Elemente	beliebige Geometrie, Adaptivität, Theorie	aufwendiger, Netzgenerierung	$\mathcal{O}(h^{k+1})$
Finite Volumen	Erhaltungsgrößen exakt	niedrige Ordnung	$\mathcal{O}(h^2)$
Spektralverfahren	exponentiell genau	nur glatte Lösungen, einfache Gebiete	$\mathcal{O}(\rho^{-n})$

Finite Volumen integrieren die Gleichung über Kontrollvolumina und erzwingen dadurch die *Erhaltung* von Masse, Impuls und Energie auf diskreter Ebene. In der Strömungsmechanik ist das entscheidend – ein Verfahren, das Masse verliert, ist dort unbrauchbar, auch wenn es formal höhere Ordnung hat.
Spektralverfahren entwickeln die Lösung nach globalen Basisfunktionen (Fourier oder Tschebyschow, Kapitel 10). Bei analytischen Lösungen erreichen sie Maschinengenauigkeit mit erstaunlich wenigen Freiheitsgraden – in der Turbulenzsimulation und der Wettervorhersage sind sie deshalb weit verbreitet. Bei Unstetigkeiten dagegen entstehen Oszillationen (Gibbssches Phänomen), und die Methode versagt.

Part VII

Optimierung und Zufall

14 Numerische Optimierung und Monte-Carlo

14.1 Das Optimierungsproblem

Warum Optimierung überall auftaucht

Gesucht ist

$$\min_{x \in \mathbb{R}^n} F(x) \qquad \text{gegebenenfalls unter Nebenbedingungen.}$$

Das ist kein Randgebiet, sondern die Sprache, in der ein großer Teil der angewandten Mathematik formuliert ist:

- Kleinste Quadrate (Kapitel 4) – $F(x) = \|Ax - b\|^2$,
- Maximum-Likelihood-Schätzung (Band 4) – F ist die negative Log-Likelihood,
- Training neuronaler Netze – F ist der Verlust über Millionen Parameter,
- Portfoliooptimierung, Trajektorienplanung, Strukturoptimierung.

Satz 14.1 (Optimalitätsbedingungen)**.** *Sei F zweimal stetig differenzierbar.*

(a) ***Notwendig (erste Ordnung):*** *Ist x^* ein lokales Minimum, so $\nabla F(x^*) = 0$.*

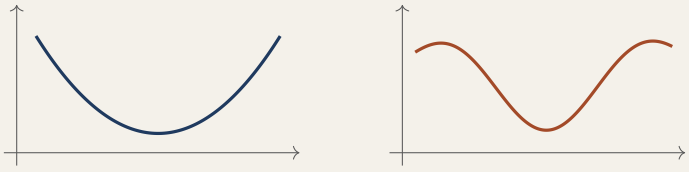
(b) ***Hinreichend (zweite Ordnung):*** *Ist $\nabla F(x^*) = 0$ und die Hesse-Matrix $\nabla^2 F(x^*)$ positiv definit, so ist x^* ein striktes lokales Minimum.*

(c) ***Konvexer Fall:*** *Ist F konvex, so ist jedes lokale Minimum global, und $\nabla F(x^*) = 0$ ist hinreichend.*

Die entscheidende Trennlinie

*Der wesentliche Unterschied verläuft nicht zwischen linear und nichtlinear, sondern zwischen **konvex** und **nichtkonvex**.*

Dieser Satz von ROCKAFELLAR (1993) fasst die Erfahrung des Fachs zusammen. **Konvexe Probleme** lassen sich zuverlässig und in polynomieller Zeit lösen; jedes gefundene Minimum ist global; es gibt eine vollständige Dualitätstheorie mit prüfbaren Optimalitätszertifikaten[3]. **Nichtkonvexe Probleme** können exponentiell viele lokale Minima haben. Das globale Minimum zu finden ist im Allgemeinen NP-schwer.



konvex nichtkonvex

14.2 Abstiegsverfahren

Das gemeinsame Gerüst

Alle Verfahren haben die Form

$$x_{k+1} = x_k + \alpha_k d_k,$$

mit einer **Abstiegsrichtung** d_k (also $\nabla F(x_k)^\top d_k < 0$) und einer **Schrittweite** $\alpha_k > 0$. Die Verfahren unterscheiden sich in der Wahl von d_k:

Verfahren	**Richtung** d_k	**Konvergenz**	**Kosten/Schritt**
Gradientenabstieg	$-\nabla F$	linear, $\frac{\kappa-1}{\kappa+1}$	$\mathcal{O}(n)$
Newton	$-(\nabla^2 F)^{-1}\nabla F$	quadratisch	$\mathcal{O}(n^3)$
Quasi-Newton (BFGS)	$-H_k \nabla F$	superlinear	$\mathcal{O}(n^2)$
Nichtlineares CG	$-\nabla F + \beta_k d_{k-1}$	superlinear	$\mathcal{O}(n)$
L-BFGS	wie BFGS, m Vektoren	superlinear	$\mathcal{O}(mn)$

Warum der reine Gradientenabstieg langsam ist

Für eine quadratische Zielfunktion mit Konditionszahl κ gilt

$$F(x_{k+1}) - F^* \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^2 \big(F(x_k) - F^*\big).$$

Bei $\kappa = 100$ ist der Faktor 0,96 – man braucht etwa 170 Schritte für eine Größenordnung.
Anschaulich: In einem langgestreckten Tal steht der Gradient fast senkrecht auf der Talrichtung. Man zickzackt zwischen den Talwänden, statt dem Tal zu folgen – dasselbe Bild wie beim steilsten Abstieg in Kapitel 5.

Newton behebt genau das: Die Hesse-Matrix beschreibt die lokale Krümmung, und $-(\nabla^2 F)^{-1}\nabla F$ zeigt in die Talrichtung. Newton ist *affin invariant* – eine lineare Umskalierung der Variablen ändert die Iterierten nicht. Der Gradientenabstieg ist es nicht, und deshalb hängt seine Geschwindigkeit von der Skalierung ab.

Schrittweitensuche: die Wolfe-Bedingungen

Wie weit soll man gehen? Zu weit überspringt das Minimum, zu kurz kostet Iterationen.
Man fordert zwei Bedingungen. Mit $\phi(\alpha) := F(x_k + \alpha d_k)$:
Armijo-Bedingung (hinreichender Abstieg):

$$\phi(\alpha) \leq \phi(0) + c_1 \alpha \phi'(0), \qquad c_1 \approx 10^{-4}.$$

Bedeutung: Die Funktion muss um mindestens einen festen Bruchteil dessen fallen, was die Linearisierung verspricht. Das verhindert zu große Schritte.
Krümmungsbedingung:

$$\phi'(\alpha) \geq c_2 \phi'(0), \qquad c_2 \approx 0{,}9.$$

Bedeutung: Die Steigung muss deutlich flacher geworden sein. Das verhindert zu kleine Schritte.
Zusammen heißen sie **Wolfe-Bedingungen**. Man zeigt, dass stets ein α existiert, das beide erfüllt, und dass jedes Abstiegsverfahren mit Wolfe-Schrittweiten global konvergiert (Satz von Zoutendijk).
In der Praxis: Man beginnt mit $\alpha = 1$ (bei Newton-artigen Verfahren fast immer akzeptiert) und halbiert, bis Armijo erfüllt ist – *Backtracking*.

BFGS – die Hesse-Matrix schätzen

Newton braucht $\nabla^2 F$: $n^2/2$ zweite Ableitungen und eine Zerlegung. Für $n = 10^6$ undenkbar.
Die Quasi-Newton-Idee: Man baut eine Näherung $H_k \approx (\nabla^2 F)^{-1}$ aus den beobachteten Änderungen auf. Mit

$$s_k := x_{k+1} - x_k, \qquad y_k := \nabla F(x_{k+1}) - \nabla F(x_k)$$

fordert man die **Sekantenbedingung**

$$H_{k+1} y_k = s_k,$$

das mehrdimensionale Analogon des Sekantenverfahrens (Kapitel 7).
Die BFGS-Formel (Broyden, Fletcher, Goldfarb, Shanno, alle 1970 unabhängig):

$$H_{k+1} = \left(I - \rho_k s_k y_k^{\mathsf{T}}\right) H_k \left(I - \rho_k y_k s_k^{\mathsf{T}}\right) + \rho_k s_k s_k^{\mathsf{T}}, \qquad \rho_k = \frac{1}{y_k^{\mathsf{T}} s_k}.$$

Die entscheidende Eigenschaft: Ist H_k positiv definit und $y_k^\top s_k > 0$ (was die Wolfe-Bedingungen garantieren), so ist auch H_{k+1} positiv definit. Die Suchrichtung ist damit stets eine Abstiegsrichtung.
L-BFGS speichert nicht die volle Matrix, sondern nur die letzten $m \approx 5$ bis 20 Paare (s_k, y_k) und rekonstruiert die Wirkung von H_k durch eine Zweischleifenrekursion. Speicher: $\mathcal{O}(mn)$ statt $\mathcal{O}(n^2)$.
Damit ist L-BFGS für sehr große Probleme geeignet und der Standardlöser für glatte unbeschränkte Optimierung – von der Maximum-Likelihood-Schätzung bis zur Strukturoptimierung. In `scipy.optimize` ist `L-BFGS-B` die Voreinstellung.

Aufgabe 14.1. Minimieren Sie die Rosenbrock-Funktion

$$F(x,y) = (1-x)^2 + 100(y-x^2)^2$$

ausgehend von $(-1;\ 1)$. Führen Sie einen Newton-Schritt aus.

Lösung Schritt für Schritt.

Schritt 1 – Das Minimum. Beide Summanden verschwinden bei $x = 1, y = 1$. Dort ist $F = 0$.
Schritt 2 – Gradient.

$$\frac{\partial F}{\partial x} = -2(1-x) + 100 \cdot 2(y-x^2)(-2x) = -2(1-x) - 400x(y-x^2)$$

$$\frac{\partial F}{\partial y} = 200(y-x^2)$$

Bei $(-1;1)$: $y - x^2 = 1 - 1 = 0$.

$$\nabla F = \begin{pmatrix} -2(1-(-1)) - 400(-1)(0) \\ 200 \cdot 0 \end{pmatrix} = \begin{pmatrix} -4 \\ 0 \end{pmatrix}.$$

Schritt 3 – Hesse-Matrix.

$$\frac{\partial^2 F}{\partial x^2} = 2 - 400(y-x^2) - 400x(-2x) = 2 - 400(y-x^2) + 800x^2$$

$$\frac{\partial^2 F}{\partial x \partial y} = -400x, \qquad \frac{\partial^2 F}{\partial y^2} = 200.$$

Bei $(-1;1)$:

$$\nabla^2 F = \begin{pmatrix} 2 - 0 + 800 & 400 \\ 400 & 200 \end{pmatrix} = \begin{pmatrix} 802 & 400 \\ 400 & 200 \end{pmatrix}.$$

Schritt 4 – Newton-Schritt. Wir lösen $\nabla^2 F\, d = -\nabla F$:

$$\begin{pmatrix} 802 & 400 \\ 400 & 200 \end{pmatrix} \begin{pmatrix} d_1 \\ d_2 \end{pmatrix} = \begin{pmatrix} 4 \\ 0 \end{pmatrix}.$$

Determinante:

$$802 \cdot 200 - 400^2 = 160\,400 - 160\,000 = 400.$$

$$d_1 = \frac{1}{400}(200 \cdot 4 - 400 \cdot 0) = \frac{800}{400} = 2$$

$$d_2 = \frac{1}{400}(802 \cdot 0 - 400 \cdot 4) = \frac{-1600}{400} = -4$$

Schritt 5 – Neuer Punkt.

$$(x_1, y_1) = (-1 + 2;\ 1 - 4) = (1;\ -3).$$

Schritt 6 – Funktionswerte vergleichen.

$$F(-1;1) = (1 - (-1))^2 + 100(1 - 1)^2 = 4 + 0 = 4.$$

$$F(1;-3) = (1 - 1)^2 + 100(-3 - 1)^2 = 0 + 100 \cdot 16 = 1600.$$

Der volle Newton-Schritt hat die Zielfunktion um den Faktor 400 verschlechtert!
Schritt 7 – Was schiefging. Die Hesse-Matrix ist zwar positiv definit ($802 > 0$, $\det = 400 > 0$), aber extrem schlecht konditioniert:

$$\operatorname{cond}_2(\nabla^2 F) \approx \frac{1002}{0{,}4} \approx 2500.$$

Der Newton-Schritt ist auf die quadratische Näherung zugeschnitten, die hier weit entfernt vom aktuellen Punkt völlig unzutreffend ist. Rosenbrocks Funktion ist ein gekrümmtes, sehr schmales Tal – die „Bananenfunktion“.
Schritt 8 – Mit Schrittweitensuche. Wir dämpfen und suchen α mit Armijo. Es ist

$$\phi(\alpha) = F(-1 + 2\alpha,\ 1 - 4\alpha), \qquad \phi'(0) = \nabla F^{\mathsf{T}} d = -4 \cdot 2 + 0 \cdot (-4) = -8.$$

α	x	y	F
1,000	1,000	−3,000	1600,0
0,500	0,000	−1,000	101,0
0,250	−0,500	0,000	8,5
0,125	−0,750	0,500	3,77
0,0625	−0,875	0,750	3,52

Bei $\alpha = 0{,}125$ ist die Armijo-Bedingung

$$\phi(\alpha) \le 4 + 10^{-4} \cdot 0{,}125 \cdot (-8) = 3{,}9999$$

mit $\phi = 3{,}77$ erfüllt. ✓

Die Lehre. Ohne Schrittweitensteuerung ist Newton unbrauchbar, sobald man nicht schon nahe am Minimum ist. Mit Dämpfung konvergiert das Verfahren – für Rosenbrock ab $(-1;1)$ typischerweise in etwa 20 bis 25 Iterationen auf 10^{-10}.
Zum Vergleich: Der reine Gradientenabstieg braucht für dieselbe Genauigkeit mehrere *tausend* Iterationen. Rosenbrocks Funktion wurde 1960 genau als solcher Testfall konstruiert und ist seither der Standardtest für Optimierungsverfahren.

Stochastischer Gradientenabstieg

Beim Training neuronaler Netze hat die Zielfunktion die Form

$$F(x) = \frac{1}{N} \sum_{i=1}^{N} F_i(x)$$

mit N in der Größenordnung 10^6 bis 10^9 (Datenpunkte) und n ebenso groß (Parameter).
Ein einziger vollständiger Gradient ist zu teuer. Man schätzt ihn stattdessen aus einer zufälligen Teilmenge (**Minibatch**) B:

$$\nabla F(x) \approx \frac{1}{|B|} \sum_{i \in B} \nabla F_i(x).$$

Was das mit Band 4 zu tun hat: Der Minibatch-Gradient ist ein *erwartungstreuer Schätzer* des vollen Gradienten. Nach dem Gesetz der großen Zahlen konvergiert er; seine Varianz fällt wie $1/|B|$. Die Konvergenztheorie von SGD (Robbins und Monro 1951) ist reine Stochastik: Bei Schrittweiten mit

$$\sum_k \alpha_k = \infty, \qquad \sum_k \alpha_k^2 < \infty$$

konvergiert das Verfahren fast sicher gegen einen stationären Punkt.
Die praktisch verwendeten Varianten:

Verfahren	Idee
SGD mit Momentum	Mittelt vergangene Gradienten; dämpft das Zickzacken.
AdaGrad, RMSProp	Skalieren jede Koordinate mit ihrer bisherigen Gradientengröße.
Adam (2015)	Kombiniert Momentum und koordinatenweise Skalierung. Der De-facto-Standard.
AdamW	Adam mit korrekt behandelter Gewichtsdämpfung.

Ein bemerkenswerter Punkt: Das *Rauschen* des stochastischen Gradienten ist nicht nur ein notwendiges Übel, sondern hilft nachweislich, flache statt scharfer Minima zu finden – was mit besserer Generalisierung einhergeht. Warum genau, ist Gegenstand aktueller Forschung (Kapitel 15).

14.3 Monte-Carlo-Verfahren

Integrieren durch Würfeln

Zur Berechnung von

$$I = \int_{[0,1]^d} f(x)\ \mathrm{d}x$$

zieht man N unabhängige gleichverteilte Punkte $X_1, \ldots, X_N$ und setzt

$$\hat{I}_N = \frac{1}{N} \sum_{i=1}^{N} f(X_i).$$

Warum das funktioniert (Band 4, Kapitel 10): Es ist $\mathbb{E}[f(X_i)] = I$, also ist $\hat{I}_N$ erwartungstreu, und nach dem starken Gesetz der großen Zahlen gilt $\hat{I}_N \to I$ fast sicher.
Die Fehlerrate. Nach dem zentralen Grenzwertsatz (Band 4, Kapitel 11):

$$\hat{I}_N - I \approx \frac{\sigma}{\sqrt{N}} Z, \qquad Z \sim \mathcal{N}(0,1), \quad \sigma^2 = \mathrm{Var}(f(X)).$$

Also

$$\text{Fehler} = \mathcal{O}\left(N^{-1/2}\right).$$

Der Fluch der Dimension – und warum Monte-Carlo ihn bricht

Verfahren	**Fehler bei N Punkten**	**Punkte für Fehler 10^{-2}**
Trapez in d Dimensionen	$\mathcal{O}(N^{-2/d})$	10^d
Simpson in d Dimensionen	$\mathcal{O}(N^{-4/d})$	$10^{d/2}$
Monte-Carlo	$\mathcal{O}(N^{-1/2})$	**10^4, unabhängig von d**
Quasi-Monte-Carlo	$\mathcal{O}(N^{-1}(\log N)^d)$	$\approx 10^2$ bei moderatem d

Die Rate $N^{-1/2}$ ist schlecht – um eine Stelle zu gewinnen, braucht man hundertmal so viele Punkte.
Aber sie ist *dimensionsunabhängig*. Und das ist alles.

Ein Zahlenbeispiel: Für $d = 10$ und Fehler 10^{-2} bräuchte die Trapezregel 10^{10} Punkte, Monte-Carlo 10^4. Bei $d = 100$ ist die Trapezregel um 10^{98} schlechter – also vollkommen aussichtslos.
Die Faustregel: Ab etwa $d \geq 4$ bis 6 ist Monte-Carlo überlegen.
Wo das gebraucht wird: Statistische Physik (Zustandssummen über 10^{23} Freiheitsgrade), Finanzmathematik (Optionspreise als Erwartungswerte über Pfade), Bayes-Statistik (Posteriorverteilungen in hohen Dimensionen), Teilchentransport (Reaktorphysik, Strahlentherapieplanung), Computergrafik (Pfadverfolgung).

Varianzreduktion

Weil der Fehler proportional zu σ ist, lohnt jede Verringerung der Varianz.

Technik	Prinzip
Antithetische Variablen	Zu jedem X auch $1 - X$ verwenden. Bei monotonem f sind die beiden negativ korreliert, was die Varianz der Summe senkt.
Kontrollvariablen	Eine ähnliche Funktion g mit bekanntem Integral abziehen: $\int f = \int (f - g) + \int g$. Die Varianz von $f - g$ ist kleiner.
Importance Sampling	Aus einer Verteilung ziehen, die dort dicht ist, wo f groß ist; mit dem Dichtequotienten gewichten.
Stratifikation	Den Bereich unterteilen und in jedem Teil separat abtasten.
Quasi-Monte-Carlo	Statt Zufallszahlen *Niedrigdiskrepanzfolgen* (Sobol, Halton) verwenden – deterministisch, aber gleichmäßiger verteilt.

Importance Sampling kann dramatisch wirken. Bei der Berechnung sehr kleiner Wahrscheinlichkeiten – etwa $\mathbb{P}(X > 5\sigma) \approx 3 \cdot 10^{-7}$ – würde naives Monte-Carlo mit 10^6 Punkten typischerweise *null* Treffer liefern. Mit einer verschobenen Ziehverteilung erhält man brauchbare Schätzungen mit tausend Punkten.

Zufallszahlen sind nicht zufällig

Rechner erzeugen *Pseudozufallszahlen* aus deterministischen Rekursionen. Die Qualität ist entscheidend.
Ein warnendes Beispiel: Der Generator `RANDU` von IBM (1960er Jahre),

$$x_{k+1} = 65539\, x_k \bmod 2^{31},$$

erzeugt Tripel (x_k, x_{k+1}, x_{k+2}), die im dreidimensionalen Raum sämtlich auf nur **15 Ebenen** liegen. Zahlreiche Simulationsergebnisse der 1970er Jahre wurden dadurch wertlos.
Heutige Standards: Mersenne-Twister (Periode $2^{19937} - 1$), PCG, Philox und Threefry (für Parallelrechner). Für kryptographische Zwecke sind alle drei ungeeignet – dort braucht man andere Verfahren.
Und ein häufiger Fehler: Bei paralleler Rechnung darf man nicht in jedem Thread denselben Generator mit demselben Startwert anwerfen. Man braucht abzählbar viele unabhängige Ströme – die genannten Zählerbasierten Generatoren sind genau dafür gebaut.

Aufgabe 14.2. Schätzen Sie π mit Monte-Carlo und bestimmen Sie, wie viele Punkte für drei Nachkommastellen nötig sind.

Lösung Schritt für Schritt.

Schritt 1 – Das Verfahren. Man zieht Punkte gleichverteilt aus $[0,1]^2$ und zählt, wie viele im Viertelkreis $x^2 + y^2 \le 1$ liegen. Der Anteil schätzt die Fläche $\frac{\pi}{4}$.
Schritt 2 – Als Integral. Mit

$$f(x,y) = \begin{cases} 1, & x^2 + y^2 \le 1 \\ 0, & \text{sonst} \end{cases}$$

ist $\int_{[0,1]^2} f = \frac{\pi}{4}$, und

$$\hat{\pi}_N = \frac{4}{N} \sum_i f(X_i, Y_i) = \frac{4 \cdot (\text{Treffer})}{N}.$$

Schritt 3 – Varianz bestimmen. f ist Bernoulli-verteilt mit $p = \frac{\pi}{4} = 0{,}785398$:

$$\sigma^2 = p(1-p) = 0{,}785398 \cdot 0{,}214602 = 0{,}168545.$$

$$\sigma = 0{,}410542.$$

Schritt 4 – Standardfehler von $\hat{\pi}_N$. Wegen des Faktors 4:

$$\mathrm{SE}(\hat{\pi}_N) = \frac{4\sigma}{\sqrt{N}} = \frac{1{,}642168}{\sqrt{N}}.$$

Schritt 5 – Punktzahl für drei Stellen. Wir fordern ein 95 %-Konfidenzintervall der Halbbreite $5 \cdot 10^{-4}$:

$$1{,}96 \cdot \frac{1{,}642168}{\sqrt{N}} \leq 5 \cdot 10^{-4}$$

$$\sqrt{N} \geq \frac{1{,}96 \cdot 1{,}642168}{5 \cdot 10^{-4}} = \frac{3{,}218649}{0{,}0005} = 6437{,}3$$

$$N \geq 4{,}14 \cdot 10^{7}.$$

Rund 41 Millionen Punkte für drei Nachkommastellen.

Schritt 6 – Zum Vergleich. Archimedes erreichte um 250 v. Chr. mit einem 96-Eck zwei sichere Nachkommastellen – durch reine Geometrie, ohne einen einzigen Zufallspunkt.

Die Gauß–Legendre-Quadratur (Kapitel 11) liefert π mit etwa 20 Funktionsauswertungen auf 15 Stellen.

Fazit: Monte-Carlo ist für dieses Problem denkbar ungeeignet. Es ist eindimensional in der Wirkung, glatt bis auf den Kreisrand, und niedrigdimensional.

Aber: Bei einem Integral über $[0,1]^{50}$ wäre Monte-Carlo die einzige Möglichkeit – und die 41 Millionen Punkte blieben dieselben, während jedes Gitterverfahren 10^{50} Punkte bräuchte.

Ein Zusatz – Quasi-Monte-Carlo. Mit einer Sobol-Folge statt Zufallszahlen erreicht man dieselbe Genauigkeit hier mit etwa 10^5 Punkten – ein Faktor 400. Der Grund: Niedrigdiskrepanzfolgen füllen das Quadrat gleichmäßiger, ohne die Klumpen und Lücken, die echte Zufallszahlen bilden.

Los Alamos, 1946

STANISŁAW ULAM lag 1946 nach einer Enzephalitis in der Rekonvaleszenz und spielte Patiencen. Er fragte sich, wie hoch die Wahrscheinlichkeit sei, dass eine Canfield-Patience aufgeht.
Kombinatorisch war das aussichtslos. Ihm fiel auf: Man könnte einfach hundert Spiele durchspielen und zählen.
Ulam erzählte die Idee JOHN VON NEUMANN, der sofort ihr Potential für die Neutronentransportrechnungen der Wasserstoffbombe erkannte. Zusammen mit NICHOLAS METROPOLIS entwickelten sie das Verfahren auf dem ENIAC. Metropolis schlug den Decknamen vor – nach dem Casino in Monaco, in dem Ulams Onkel regelmäßig sein Geld verlor.
Die erste große Anwendung lief 1948; die Ergebnisse waren geheim. Der erste offene Aufsatz erschien 1949 im *Journal of the American Statistical Association*.
1953 folgte der **Metropolis-Algorithmus** für Markow-Ketten-Monte-Carlo (Band 4, Kapitel 12) – die Grundlage praktisch der gesamten modernen Bayes-Statistik und der statistischen Physik. Der Aufsatz von Metropolis, Rosenbluth, Rosenbluth, Teller und Teller ist einer der meistzitierten der Physikgeschichte. *Computing in Science and Engineering* führte 2000 sowohl die Monte-Carlo-Methode als auch den Metropolis-Algorithmus in der Liste der zehn wichtigsten Algorithmen des 20. Jahrhunderts.

Part VIII

Aktueller Stand

15 Numerik heute

Stand dieses Kapitels

Die folgenden Angaben geben den Stand von August 2026 wieder. Die numerische Mathematik ist gegenwärtig in ungewöhnlich starker Bewegung – getrieben von veränderter Hardware und von der Begegnung mit dem maschinellen Lernen. Einzelne Zahlen können daher rasch veralten; die Entwicklungslinien dürften stabiler sein.

15.1 Gemischte Präzision

Die Hardware hat sich verändert

Jahrzehntelang war doppelte Genauigkeit die selbstverständliche Grundlage. Das gilt nicht mehr.
Moderne Beschleuniger sind auf Formate *geringerer* Genauigkeit optimiert, weil das maschinelle Lernen sie verlangt:

Format	Bits	Mantisse	ε_{M}	relative Rechenleistung
FP64 (double)	64	53	$1{,}1 \cdot 10^{-16}$	$1\times$
FP32 (single)	32	24	$6{,}0 \cdot 10^{-8}$	$\approx 2\times$
TF32	19	11	$4{,}9 \cdot 10^{-4}$	$\approx 8\times$
FP16 (half)	16	11	$4{,}9 \cdot 10^{-4}$	$\approx 16\times$
BF16	16	8	$3{,}9 \cdot 10^{-3}$	$\approx 16\times$
FP8	8	3–4	$\approx 6 \cdot 10^{-2}$	$\approx 32\times$

Der Unterschied zwischen FP16 und BF16 ist lehrreich: BF16 hat dieselbe Exponentenbreite wie FP32 (8 Bit), also denselben Zahlenbereich, aber nur 8 Mantissenbits. FP16 hat mehr Mantisse, aber nur 5 Exponentenbits – und läuft deshalb schon bei 65 504 über. Für neuronale Netze, wo Gradienten über viele Größenordnungen streuen, ist der Bereich wichtiger als die Präzision; deshalb hat sich BF16 dort durchgesetzt.

Das Kernproblem für die Numerik: Ein Faktor 16 an Rechenleistung ist zu viel, um ihn zu ignorieren. Aber mit $\varepsilon_{\mathrm{M}} \approx 10^{-3}$ kann man keine wissenschaftliche Rechnung durchführen.

Iterative Verfeinerung – ein altes Verfahren wird wichtig

Die Lösung ist ein Verfahren, das WILKINSON schon 1948 beschrieb:

1. Zerlege $A = LR$ in **niedriger** Präzision. (billig, $\frac{2}{3}n^3$)
2. Löse $Ax_0 = b$ damit. (billig)
3. Wiederhole:
   a) Berechne das Residuum $r = b - Ax_k$ in **hoher** Präzision. ($\mathcal{O}(n^2)$)
   b) Löse $Ad = r$ mit der vorhandenen niedrigpräzisen Zerlegung. ($\mathcal{O}(n^2)$)
   c) Setze $x_{k+1} = x_k + d$.

Warum das funktioniert: Die teure Arbeit – die Zerlegung – geschieht in niedriger Präzision. Nur das Residuum, wo Auslöschung droht, wird genau berechnet. Und das kostet nur $\mathcal{O}(n^2)$.
Das Ergebnis: Man erhält eine Lösung mit der Genauigkeit der *hohen* Präzision zum Preis der *niedrigen* – sofern $\mathrm{cond}(A)$ nicht zu groß ist.
Was sich seit 2017 geändert hat. ERIN CARSON und NICHOLAS HIGHAM entwickelten die Theorie für *drei* verschiedene Präzisionen und zeigten: Mit FP16-Zerlegung, FP32-Arbeitspräzision und FP64-Residuen erhält man FP64-genaue Lösungen für Matrizen bis $\mathrm{cond}(A) \lesssim 10^4$ – bei bis zu vierfacher Beschleunigung.
Wird zusätzlich ein Krylow-Verfahren zur Verfeinerung eingesetzt (GMRES-IR), erweitert sich der zulässige Konditionsbereich auf etwa 10^8.
Der praktische Niederschlag: Die HPL-MxP-Rangliste (früher HPL-AI) misst seit 2019 genau diese Fähigkeit. Führende Systeme erreichen dort ein Vielfaches ihrer FP64-Leistung. Die Verfahren sind inzwischen in `cuSOLVER`, `MAGMA` und experimentell in LAPACK verfügbar[16].
Der begriffliche Punkt: Die Frage lautet nicht mehr „Wie genau rechne ich?“, sondern „Welche *Teile* der Rechnung brauchen welche Genauigkeit?“ Das ist eine Rückkehr zu Wilkinsons Denkweise – mit dem Unterschied, dass die Präzisionsstufen heute von der Hardware vorgegeben werden.

15.2 Randomisierte numerische lineare Algebra

Zufall als Werkzeug, nicht als Störung

Die klassische numerische lineare Algebra ist deterministisch. Seit etwa 2006 hat sich ein Zweig etabliert, der Zufall gezielt einsetzt – und dabei oft schneller und ebenso zuverlässig ist.

Das Grundverfahren: randomisierte SVD. Gesucht sei eine Rang-k-Approximation einer großen Matrix $A \in \mathbb{R}^{m \times n}$.

1. Ziehe eine Zufallsmatrix $\Omega \in \mathbb{R}^{n \times (k+p)}$ mit $\mathcal{N}(0,1)$-Einträgen ($p \approx 10$ Überabtastung).
2. Berechne $Y = A\Omega$. (eine Matrixmultiplikation)
3. QR-Zerlegung: $Y = QR$. (Q hat $k+p$ Spalten)
4. Berechne $B = Q^{\mathsf{T}} A$ und dessen SVD $B = \tilde{U}\Sigma V^{\mathsf{T}}$. (klein!)
5. Setze $U = Q\tilde{U}$.

Die Idee dahinter: $A\Omega$ liegt im Bild von A; mit hoher Wahrscheinlichkeit spannen $k+p$ Zufallsrichtungen den dominanten Teilraum von A gut auf. Q ist eine Orthonormalbasis davon, und B ist die Projektion – eine kleine Matrix, deren SVD billig ist.

Der Aufwand: $\mathcal{O}(mnk)$ statt $\mathcal{O}(mn\min(m,n))$ für die volle SVD. Bei $k \ll n$ ist das eine Größenordnung oder mehr.

Die Garantie (Halko, Martinsson und Tropp, 2011)[13]: Mit Wahrscheinlichkeit mindestens $1 - 3p^{-p}$ gilt

$$\left\| A - QQ^{\mathsf{T}} A \right\|_2 \leq \left(1 + 11\sqrt{k+p} \cdot \sqrt{\min(m,n)} \right) \sigma_{k+1}.$$

Der Vergleichswert σ_{k+1} ist nach Eckart–Young (Kapitel 6) das bestmögliche Ergebnis; man verfehlt es also nur um einen moderaten Faktor.

Und die Fehlerwahrscheinlichkeit ist winzig: Für $p = 10$ ist $3p^{-p} = 3 \cdot 10^{-10}$. Man kann das Ergebnis überdies *nachprüfen*, indem man $\left\| A - QQ^{\mathsf{T}} A \right\|$ stichprobenartig schätzt – ein Vorzug gegenüber Heuristiken.

Weitere randomisierte Verfahren

Verfahren	Beschreibung
Sketch-and-solve	Für überbestimmte Systeme: Man projiziert A und b auf wenige Zeilen und löst das kleine Problem. Fehler $\mathcal{O}(\varepsilon)$ mit $\mathcal{O}(n/\varepsilon)$ Zeilen.
Randomisiertes Kaczmarz	Zufällig gewählte Zeilen werden nacheinander erfüllt; konvergiert linear mit einer Rate, die von cond abhängt (Strohmer und Vershynin 2009).
Hutchinson-Schätzer	Schätzt $\operatorname{Spur}(A)$ durch $\mathbb{E}[z^\mathsf{T} A z]$ mit Rademacher-Vektoren – ohne die Diagonale zu kennen.
Randomisierte Vorkonditionierung	Blendenpik und LSRN: erzeugen aus einer Skizze einen Vorkonditionierer, der cond auf $\mathcal{O}(1)$ drückt.

Die begriffliche Neuerung. Klassische Numerik fragt nach *garantierten* Schranken. Randomisierte Verfahren liefern Schranken, die *mit hoher Wahrscheinlichkeit* gelten – und diese Wahrscheinlichkeit lässt sich durch geringen Mehraufwand beliebig nah an 1 bringen. Die theoretischen Werkzeuge kommen aus der Wahrscheinlichkeitstheorie: Konzentrationsungleichungen für Matrizen, insbesondere die Matrix-Chernoff-Ungleichungen von TROPP.
Der Stand. Die Übersichtsarbeit von MARTINSSON und TROPP in *Acta Numerica* 2020[20] gilt als Referenz. Randomisierte SVD ist in `scikit-learn`, `scipy` und den meisten Bibliotheken für sehr große Matrizen verfügbar. Ein aktueller Schwerpunkt ist die Kombination mit gemischter Präzision – die Skizzenbildung braucht keine hohe Genauigkeit.

15.3 Automatisches Differenzieren

Weder symbolisch noch numerisch

Es gibt drei Arten, eine Ableitung zu berechnen:

Verfahren	Wie	Problem
Symbolisch	Formelmanipulation	Ausdrücke wachsen exponentiell
Numerisch	Differenzenquotient	Auslöschung, $\varepsilon^{2/3}$-Genauigkeit
Automatisch	Kettenregel auf den Rechenschritten	**keines** – exakt bis auf Rundung

Die Idee. Jedes Programm ist eine Verkettung elementarer Operationen. Für jede kennt man die Ableitung. Die Kettenregel liefert die Ableitung des Ganzen – man muss sie nur systematisch anwenden.

Vorwärtsmodus. Man führt zu jedem Wert die Ableitung mit. Formal rechnet man mit **dualen Zahlen**

$$a + b\varepsilon, \qquad \varepsilon^2 = 0,$$

und es gilt

$$f(a + b\varepsilon) = f(a) + f'(a)b\,\varepsilon.$$

Nachrechnen der Produktregel:

$$(a + b\varepsilon)(c + d\varepsilon) = ac + (ad + bc)\varepsilon + bd\underbrace{\varepsilon^2}_{=0} = ac + (ad + bc)\varepsilon. \checkmark$$

Kosten: eine Auswertung pro *Eingabevariable*. Gut für $F : \mathbb{R} \to \mathbb{R}^m$.

Rückwärtsmodus. Man protokolliert die Rechnung und propagiert die Ableitungen *rückwärts* vom Ergebnis zu den Eingaben. Kosten: eine Auswertung pro *Ausgabevariable*.

Das ist der entscheidende Punkt: Für $F : \mathbb{R}^n \to \mathbb{R}$ – der Fall in aller Optimierung – liefert der Rückwärtsmodus den *vollständigen Gradienten* zum Preis von etwa 2 bis 4 Funktionsauswertungen. **Unabhängig von n.**

Der Vergleich zur numerischen Differentiation ist vernichtend: Dort bräuchte man $n + 1$ Auswertungen – bei $n = 10^9$ Parametern eines neuronalen Netzes also 10^9 statt 3.

Der Rückwärtsmodus *ist* das, was im maschinellen Lernen „Backpropagation" heißt. Er wurde in der Numerik unter dem Namen *adjungierte Methode* bereits in den 1970er Jahren entwickelt – SEPPO LINNAINMAA beschrieb ihn 1970 in seiner Masterarbeit in Helsinki, BERT SPEELPENNING 1980 in seiner Dissertation. Die Wiederentdeckung im Kontext neuronaler Netze (Rumelhart, Hinton und Williams 1986) machte ihn berühmt.

Der Preis des Rückwärtsmodus: Man muss alle Zwischenwerte speichern. Bei tiefen Netzen oder langen Zeitintegrationen wird das zum Engpass. Abhilfe

schafft **Checkpointing**: Man speichert nur ausgewählte Zustände und rechnet die dazwischenliegenden bei Bedarf neu. ANDREAS GRIEWANK zeigte 1992, dass sich mit $\mathcal{O}(\log n)$ Speicher und $\mathcal{O}(n \log n)$ Rechenzeit auskommen lässt[9].
Der Stand 2026. AD ist heute Infrastruktur: `JAX`, `PyTorch`, `Enzyme` (auf LLVM-Ebene, differenziert beliebigen kompilierten Code), `Zygote` und `Enzyme.jl` in Julia. Der Trend geht zu *differenzierbaren Simulationen*: Ein ganzes Strömungs- oder Strukturprogramm wird differenzierbar gemacht, um Gradienten für Designoptimierung, Parameteridentifikation oder Datenassimilation zu liefern.

15.4 Wissenschaftliches maschinelles Lernen

Was funktioniert – und was nicht

Seit etwa 2018 gibt es intensive Bemühungen, neuronale Netze für die Lösung von Differentialgleichungen einzusetzen. Der Stand 2026 ist differenzierter, als die frühe Begeisterung erwarten ließ.
Physikinformierte neuronale Netze (PINNs). Man stellt die Lösung durch ein Netz u_θ dar und minimiert

$$\mathcal{L}(\theta) = \underbrace{\|\mathcal{N}[u_\theta] - f\|^2_{\text{Kollokationspunkte}}}_{\text{Residuum der DGL}} + \lambda \underbrace{\|u_\theta - g\|^2_{\text{Rand}}}_{\text{Randbedingungen}} .$$

Die Ableitungen in $\mathcal{N}$ liefert das automatische Differenzieren exakt.
Stand: PINNs sind bei *klassischen Vorwärtsproblemen* den etablierten Verfahren regelmäßig unterlegen – langsamer und ungenauer als finite Elemente, mit schwer kontrollierbaren Fehlern und einem nichtkonvexen Trainingsproblem, das schlecht konditioniert ist. Sorgfältige Vergleichsstudien haben das seit 2022 wiederholt bestätigt.
Wo sie überzeugen: bei **inversen Problemen** und **Datenassimilation**. Sind Messdaten und physikalische Gleichungen gemeinsam zu berücksichtigen, unbekannte Parameter mitzuschätzen oder ist die Geometrie schwierig, so ist die Formulierung als gemeinsames Optimierungsproblem natürlich und den klassischen Verfahren teils überlegen. Auch bei sehr hochdimensionalen Problemen – etwa Hamilton–Jacobi–Bellman-Gleichungen in vielen Raumdimensionen – gibt es klare Erfolge, weil dort jedes Gitterverfahren am Fluch der Dimension scheitert[18].
Neuronale Operatoren. Ein anderer Ansatz: Statt *einer* Lösung lernt man den *Lösungsoperator* $f \mapsto u$ als Abbildung zwischen Funktionenräumen. Verfahren wie DeepONet und der Fourier Neural Operator werden auf vielen Lösungspaaren trainiert und liefern danach für neue Eingabedaten Vorhersagen um Größenordnungen schneller als ein Löser.
Stand: Der Nutzen ist am größten bei **Vielfachauswertungen** – Unsicherheitsquantifizierung, Designoptimierung, Echtzeitregelung. Wo dasselbe Problem

tausendfach mit variierenden Parametern zu lösen ist, amortisiert sich das Training. Bei Einzelrechnungen lohnt es nicht.
Die offenen Fragen: Fehlerschranken sind schwach oder fehlen; die Generalisierung außerhalb der Trainingsverteilung ist unzuverlässig; Erhaltungsgrößen werden nicht automatisch respektiert. Ein aktiver Forschungszweig arbeitet an *strukturerhaltenden* Architekturen, die Symmetrien, Erhaltungssätze oder symplektische Struktur eingebaut haben.
Der Bereich mit den unstrittigsten Erfolgen ist die **mittelfristige Wettervorhersage**. Datengetriebene Modelle erreichen seit 2023 in etablierten Vergleichsmaßen die Güte klassischer numerischer Wettervorhersage bei um Größenordnungen geringerem Rechenaufwand. Das europäische Wetterzentrum betreibt seit 2025 ein solches System operativ neben dem klassischen. Bemerkenswert: Diese Modelle lernen aus Reanalysedaten, die ihrerseits aus klassischer Numerik stammen – sie ersetzen die Physik nicht, sondern destillieren sie.
Der nüchterne Gesamtbefund 2026: Maschinelles Lernen ergänzt die Numerik, wo klassische Verfahren strukturell scheitern – hohe Dimension, inverse Probleme, Vielfachauswertung, unbekannte Teilmodelle. Es ersetzt sie nicht dort, wo sie stark sind: bei gut gestellten Vorwärtsproblemen mit garantierten Fehlerschranken.

15.5 Probabilistische und verifizierte Numerik

Zwei Antworten auf dieselbe Frage

Jedes numerische Ergebnis ist mit Unsicherheit behaftet. Wie geht man damit um? **Verifizierte Numerik** gibt *garantierte Einschließungen*. Statt einer Zahl liefert sie ein Intervall, in dem die exakte Lösung mathematisch beweisbar liegt. Grundlage ist die **Intervallarithmetik**:

$$[a,b] + [c,d] = [a + c,\ b + d], \qquad [a,b] \cdot [c,d] = [\min(\dots),\ \max(\dots)],$$

mit gerichteter Rundung nach außen.
Das Kernproblem ist der **Aufblähungseffekt**: Naiv angewandt wachsen die Intervalle exponentiell. Selbst $[x] - [x]$ ergibt nicht $[0,0]$, sondern ein Intervall der doppelten Breite, weil die Abhängigkeit verloren geht.
Die Abhilfe sind ausgefeilte Verfahren – Mittelwertformen, affine Arithmetik, Krawczyk- und Rump-Operatoren –, die die Abhängigkeiten erhalten. SIEGFRIED RUMP entwickelte mit `INTLAB` eine Umgebung, die verifizierte Lösungen linearer Systeme in etwa der dreifachen Zeit der gewöhnlichen Rechnung liefert[23].
Wo es eingesetzt wird: computergestützte Beweise (Keplersche Vermutung, Existenz von Lorenz-Attraktoren, Doppelblasenvermutung), Sicherheitsnachweise in der Luftfahrt, globale Optimierung mit Garantien. Ein Meilenstein war WARWICK TUCKERS verifizierter Nachweis (1999–2002), dass der Lorenz-Attraktor tatsächlich ein seltsamer Attraktor ist – Smales 14. Problem.

Probabilistische Numerik verfolgt einen anderen Weg: Sie deutet einen numerischen Algorithmus als *statistische Inferenz*. Das Ergebnis ist keine Zahl und kein Intervall, sondern eine *Wahrscheinlichkeitsverteilung* über mögliche Lösungen. *Beispiel:* Eine Quadraturformel wertet f an endlich vielen Stellen aus. Dazwischen ist f unbekannt. Setzt man einen Gauß-Prozess als Priorverteilung über f an, so liefert die Bayes-Regel eine Posteriorverteilung über $\int f$ – mit Erwartungswert und Varianz. Der Erwartungswert stimmt für passende Priorwahl mit klassischen Formeln überein; die Varianz ist die neue Information.
Der Nutzen liegt dort, wo numerische Fehler durch eine längere Verarbeitungskette propagiert werden – etwa wenn ein Differentialgleichungslöser innerhalb eines Bayes-Inferenzverfahrens läuft. Klassisch wird der Diskretisierungsfehler dort ignoriert, was zu überzogen sicheren Schlüssen führt. Probabilistische Löser tragen ihn korrekt weiter[14].
Stand 2026: Ein kleines, methodisch anspruchsvolles Feld mit wachsender Bedeutung in der Unsicherheitsquantifizierung. Der Mehraufwand gegenüber klassischen Verfahren ist inzwischen moderat.

15.6 Reproduzierbarkeit

Ein wachsendes Problem

Dasselbe Programm, dieselben Daten – und verschiedene Ergebnisse. Die Ursachen:

- **Nichtassoziative Gleitkommaaddition** (Kapitel 1). Bei paralleler Reduktion hängt das Ergebnis davon ab, in welcher Reihenfolge die Teilsummen eintreffen – und die ist bei asynchroner Ausführung nicht festgelegt.
- **Verschiedene Prozessorarchitekturen** nutzen unterschiedliche interne Präzisionen, Fused-Multiply-Add-Operationen und Vektorbreiten.
- **Bibliotheksversionen** ändern Algorithmen und Blockgrößen.
- **Compileroptimierung**, insbesondere `-ffast-math`, erlaubt mathematisch, aber nicht numerisch zulässige Umformungen.

Die Größenordnung. Bei gut konditionierten Problemen betreffen die Abweichungen die letzten Stellen und sind belanglos. Bei chaotischen Systemen – Wetter, Klima, Molekulardynamik – wachsen sie exponentiell und führen nach hinreichend langer Zeit zu völlig verschiedenen Trajektorien.
Gegenmaßnahmen:

- *Reproduzierbare Reduktionen* über exakte Akkumulatoren, die alle Bits aufbewahren (ReproBLAS, ExBLAS). Kosten: Faktor 2 bis 4.
- *Feste Reduktionsreihenfolge* unabhängig von der Prozessorzahl, etwa durch Baumreduktion mit festgelegter Struktur.

- *Containerisierung* mit festgeschriebenen Bibliotheks- und Compilerversionen.
- *Ensemblerechnungen*: Man akzeptiert die Streuung und quantifiziert sie statistisch, statt sie zu unterdrücken.

Der begriffliche Punkt. „Bitgenaue Reproduzierbarkeit“ und „wissenschaftliche Reproduzierbarkeit“ sind verschiedene Dinge. Erstere ist bei chaotischen Systemen weder erreichbar noch sinnvoll; letztere verlangt, dass die *statistischen* Aussagen stabil sind. Die Debatte darüber, welches Maß in welchem Kontext gefordert werden sollte, ist in der Klimamodellierung besonders lebhaft.

15.7 Was offen ist

Ungelöste Fragen der numerischen Mathematik

1. **Der Wachstumsfaktor bei der Gauß-Elimination.** Warum ist er in der Praxis fast immer klein, obwohl er theoretisch exponentiell sein kann? Die Glättungsanalyse liefert Teilantworten, aber keine befriedigende Erklärung (Kapitel 3).
2. **Der Exponent der Matrixmultiplikation.** Der aktuelle Rekord liegt bei $\mathcal{O}(n^{2{,}371\ldots})$; ob $\omega = 2$ erreichbar ist, ist offen. Alle Verfahren unterhalb von Strassen sind praktisch unbrauchbar – die *galaktischen Algorithmen*, deren Konstanten astronomisch sind.
3. **Optimale Vorkonditionierung.** Für allgemeine dünn besetzte Systeme gibt es kein Verfahren mit garantiert $\mathcal{O}(n)$-Aufwand. Mehrgitter leistet das für elliptische Probleme; für indefinite, stark anisotrope oder gekoppelte Systeme fehlt eine allgemeine Theorie.
4. **Nichtkonvexe Optimierung.** Warum finden stochastische Gradientenverfahren in hochdimensionalen nichtkonvexen Landschaften so zuverlässig gute Minima? Die Vermutung, dass in hohen Dimensionen Sattelpunkte statt schlechter lokaler Minima dominieren, ist teilweise belegt, aber nicht vollständig verstanden.
5. **Fehlerschranken für gelernte Verfahren.** Neuronale Operatoren liefern keine verlässlichen A-posteriori-Schranken. Ohne solche bleiben sie für sicherheitskritische Anwendungen ungeeignet.
6. **Numerik auf Quantenrechnern.** Der HHL-Algorithmus (Harrow, Hassidim und Lloyd 2009) verspricht exponentielle Beschleunigung für lineare Systeme – unter Voraussetzungen (effiziente Zustandspräparation, gute Kondition, nur Funktionale der Lösung als Ausgabe), die in der Praxis selten erfüllt sind. Der Stand 2026 ist deutlich nüchterner als 2019; realistische Anwendungen scheinen eher im Bereich der Simulation von Quantensystemen selbst zu liegen als in der allgemeinen linearen Algebra.

7. **Energieeffizienz.** Mit dem Ende der klassischen Skalierung wird nicht mehr Rechenzeit, sondern Energieverbrauch zum bestimmenden Maß. Das verschiebt die Bewertung von Algorithmen: Datenbewegung kostet heute deutlich mehr Energie als Arithmetik. *Kommunikationsvermeidende* Algorithmen – die bewusst mehr rechnen, um weniger zu speichern und zu übertragen – sind ein aktiver Forschungszweig (DEMMEL und Mitarbeiter).

Ein Schlussgedanke

Am Ende dieses Bandes stehen drei Einsichten, die durch alle Kapitel hindurch wiederkehrten.

Erstens: Mathematische Gleichheit ist nicht numerische Gleichheit. Die Mitternachtsformel und der Vieta-Algorithmus, $A^{\mathsf{T}}A$ und die QR-Zerlegung, klassisches und modifiziertes Gram–Schmidt, $\frac{a+b}{2}$ und $a + \frac{b-a}{2}$ – jedes dieser Paare ist algebraisch identisch und numerisch verschieden. Wer das einmal verinnerlicht hat, liest Formeln anders.

Zweitens: Die Trennung von Kondition und Stabilität ordnet alles. Ein schlechtes Ergebnis hat genau zwei mögliche Ursachen: ein empfindliches Problem oder einen schlechten Algorithmus. Die Diagnose entscheidet, ob man das Modell oder das Verfahren ändern muss. Wilkinsons Rückwärtsanalyse ist die begriffliche Errungenschaft, die das Fach zu einer Wissenschaft gemacht hat.

Drittens: Struktur ist alles. Symmetrie spart die Hälfte, positive Definitheit erspart die Pivotisierung, dünne Besetzung macht aus n^3 ein n, Glattheit macht aus algebraischer eine exponentielle Konvergenz, Periodizität macht aus der Trapezregel ein Spektralverfahren. Der Unterschied zwischen einem naiven und einem guten numerischen Programm liegt fast nie in der Formel – sondern darin, welche Struktur es erkennt und ausnutzt.

Und über allem steht Hammings Satz aus dem Vorwort. Der Zweck des Rechnens ist Einsicht. Die Zahlen sind das Mittel.

Part IX

Anhänge

A Formelsammlung

Gleitkommaarithmetik

$$\mathrm{fl}(x) = x(1+\delta), \qquad |\delta| \le \varepsilon_{\mathrm{M}}, \qquad \varepsilon_{\mathrm{M}} = \tfrac{1}{2}\beta^{1-t}$$

$$\mathrm{fl}(a \circ b) = (a \circ b)(1+\delta), \qquad \circ \in \{+, -, \cdot, /\}$$

`double`: $t = 53$, $\varepsilon_{\mathrm{M}} = 2^{-53} \approx 1{,}11 \cdot 10^{-16}$, etwa 16 Dezimalstellen.
`single`: $t = 24$, $\varepsilon_{\mathrm{M}} = 2^{-24} \approx 5{,}96 \cdot 10^{-8}$, etwa 7 Dezimalstellen.

Produktabschätzung: $\prod_{j=1}^{m}(1+\theta_j) = 1+\theta$ mit $|\theta| \le \gamma_m := \frac{m\varepsilon_{\mathrm{M}}}{1-m\varepsilon_{\mathrm{M}}} \approx m\varepsilon_{\mathrm{M}}$.

Kondition und Stabilität

$$\kappa_{\mathrm{rel}}(x) = \frac{|f'(x)|\,|x|}{|f(x)|}, \qquad \mathrm{cond}(A) = \|A\| \left\|A^{-1}\right\|, \qquad \mathrm{cond}_2(A) = \frac{\sigma_{\max}}{\sigma_{\min}}$$

$$\frac{\|\tilde{x} - x\|}{\|x\|} \le \mathrm{cond}(A) \cdot \frac{\|\Delta b\|}{\|b\|}$$

$$\text{Vorwärtsfehler} \lesssim \text{Kondition} \times \text{Rückwärtsfehler}$$

Subtraktion: $\kappa_{\mathrm{rel}} = \frac{|a|+|b|}{|a-b|}$ – Auslöschung für $a \approx b$.

Matrixnormen: $\|A\|_1 = \max_j \sum_i |a_{ij}|$ (Spaltensumme), $\|A\|_\infty = \max_i \sum_j |a_{ij}|$ (Zeilensumme), $\|A\|_2 = \sigma_{\max}$.

Lineare Gleichungssysteme

Zerlegung	**Voraussetzung**	**Aufwand**
$PA = LR$	regulär	$\frac{2}{3}n^3$
$A = LL^{\mathsf{T}}$ (Cholesky)	s.p.d.	$\frac{1}{3}n^3$
$A = QR$ (Householder)	beliebig, $m \ge n$	$2mn^2 - \frac{2}{3}n^3$
$A = U\Sigma V^{\mathsf{T}}$ (SVD)	beliebig	$\approx 4mn^2$ bis $14mn^2$
Bandmatrix, Bandbreite m	–	$\mathcal{O}(nm^2)$
Tridiagonal (Thomas)	–	$\mathcal{O}(n)$
Vorwärts-/Rückwärtseinsetzen	Dreiecksmatrix	n^2

Cholesky-Rekursion:

$$\ell_{jj} = \sqrt{a_{jj} - \sum_{k<j} \ell_{jk}^2}, \qquad \ell_{ij} = \frac{1}{\ell_{jj}}\Big(a_{ij} - \sum_{k<j} \ell_{ik}\ell_{jk}\Big)$$

Householder: $v = x + \operatorname{sign}(x_1)\,\|x\|_2\, e_1, \quad H = I - \frac{2vv^\mathsf{T}}{v^\mathsf{T}v}, \quad HA = A - \frac{2}{v^\mathsf{T}v} v(v^\mathsf{T}A)$.

Normalgleichungen: $A^\mathsf{T}Ax = A^\mathsf{T}b$ – *nie so rechnen*, da $\operatorname{cond}_2(A^\mathsf{T}A) = \operatorname{cond}_2(A)^2$.

Iterative Verfahren

Splitting $A = M - N, \quad x^{k+1} = M^{-1}(Nx^k + b), \quad$ Konvergenz $\iff \rho(M^{-1}N) < 1$.

Jacobi	$M = D$	$x_i^{k+1} = \frac{1}{a_{ii}}\big(b_i - \sum_{j\neq i} a_{ij}x_j^k\big)$
Gauß–Seidel	$M = D + L$	neue Werte sofort verwenden
SOR	$M = \frac{1}{\omega}D + L$	$\omega_{\text{opt}} = \frac{2}{1+\sqrt{1-\rho(T_J)^2}}$

Strikte Diagonaldominanz $\Longrightarrow$ Jacobi und Gauß–Seidel konvergieren.

CG-Verfahren (A s.p.d.): $r_0 = p_0 = b - Ax_0$, dann

$$\alpha_k = \frac{r_k^\mathsf{T} r_k}{p_k^\mathsf{T} A p_k}, \quad x_{k+1} = x_k + \alpha_k p_k, \quad r_{k+1} = r_k - \alpha_k A p_k,$$

$$\beta_k = \frac{r_{k+1}^\mathsf{T} r_{k+1}}{r_k^\mathsf{T} r_k}, \quad p_{k+1} = r_{k+1} + \beta_k p_k$$

$$\frac{\|e_k\|_A}{\|e_0\|_A} \leq 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k$$

Eigenwerte

Potenzmethode: $x_{k+1} = \frac{Ax_k}{\|Ax_k\|}$, Rate $|\lambda_2/\lambda_1|$; Rayleigh-Quotient $\mu = x^\mathsf{T}Ax$ (quadratisch genau bei symmetrischem A).

Inverse Iteration mit Verschiebung: $(A - \sigma I)y_{k+1} = x_k$, Rate $\left|\frac{\lambda_{\text{nah}} - \sigma}{\lambda_{\text{zweitnah}} - \sigma}\right|$.

QR-Algorithmus: $A_k = Q_k R_k$, $A_{k+1} = R_k Q_k$; mit Hessenberg-Reduktion und Wilkinson-Shift kubisch konvergent, $\approx 10n^3$ (symmetrisch).

Eckart–Young: $\min_{\operatorname{Rang} B \leq k} \|A - B\|_2 = \sigma_{k+1}$, erreicht durch $A_k = \sum_{i\leq k} \sigma_i u_i v_i^\mathsf{T}$.

Nullstellen

Verfahren	Vorschrift	Ordnung
Bisektion	Intervallhalbierung	1 (Faktor $\frac{1}{2}$)
Newton	$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$	2 (einfache Nullstelle)
Sekante	$x_{k+1} = x_k - f_k \frac{x_k - x_{k-1}}{f_k - f_{k-1}}$	1,618
Fixpunkt	$x_{k+1} = g(x_k)$	1, falls $g'(x^*) \neq 0$

Newton mehrdimensional: $J_F(x_k)\Delta x_k = -F(x_k), \quad x_{k+1} = x_k + \lambda_k \Delta x_k$.
Bei m-facher Nullstelle: Newton nur linear mit $C = 1 - \frac{1}{m}$; Abhilfe $x_{k+1} = x_k - m\frac{f}{f'}$.
Banach: $|g'| \leq L < 1 \Longrightarrow |x_k - x^*| \leq \frac{L^k}{1-L}\,|x_1 - x_0|$.

Interpolation

Lagrange: $p = \sum_i y_i L_i, \quad L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$.
Newton: $p(x) = \sum_k f[x_0, \ldots, x_k] \prod_{j<k}(x - x_j)$, dividierte Differenzen rekursiv.
Fehler: $f(x) - p(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_i (x - x_i)$.
Tschebyschow-Knoten: $x_k = \cos \frac{(2k+1)\pi}{2n+2}$ bzw. $x_k = \cos \frac{k\pi}{n}$; $\max|\omega| = 2^{-n}$.
Lebesgue-Konstante: gleichabständig $\sim \frac{2^{n+1}}{e n \ln n}$; Tschebyschow $\sim \frac{2}{\pi}\ln n + 1{,}94$.
Baryzentrische Formel: $p(x) = \frac{\sum_i \frac{w_i}{x - x_i} y_i}{\sum_i \frac{w_i}{x - x_i}}, \quad w_i = \prod_{j \neq i}(x_i - x_j)^{-1}$.
Kubischer Spline: $h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_i M_{i+1} = 6\left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}\right)$; Fehler $\frac{5}{384} h^4 \left\| f^{(4)} \right\|_\infty$.
Tschebyschow-Rekursion: $T_{k+1} = 2xT_k - T_{k-1}, \; T_n(\cos\theta) = \cos n\theta$.
DFT: $c_k = \frac{1}{N}\sum_j y_j e^{-2\pi ß k j/N}$; FFT in $\mathcal{O}(N \log N)$.

Quadratur

Regel	Formel (zusammengesetzt)	Fehler
Trapez	$h(\frac{f_0}{2} + f_1 + \cdots + \frac{f_m}{2})$	$-\frac{b-a}{12} h^2 f''$
Simpson	$\frac{h}{3}(f_0 + 4f_1 + 2f_2 + \cdots + f_m)$	$-\frac{b-a}{180} h^4 f^{(4)}$
Mittelpunkt	$h\sum_i f(x_{i+1/2})$	$+\frac{b-a}{24} h^2 f''$
Gauß (n+1 Punkte)	$\sum_i w_i f(x_i)$	exakt bis Grad $2n+1$

Romberg: $R_{k,0} = T_{h_k}, \quad R_{k,j} = \frac{4^j R_{k,j-1} - R_{k-1,j-1}}{4^j - 1}$; Spalte j hat Ordnung $2j+2$.

Numerische Differentiation:

$$\frac{f(x+h)-f(x-h)}{2h} = f' + \mathcal{O}(h^2), \qquad \frac{f_+ - 2f + f_-}{h^2} = f'' + \mathcal{O}(h^2)$$

$$h_{\text{opt}} \approx \varepsilon_{\text{M}}^{1/3}, \qquad E_{\min} \approx \varepsilon_{\text{M}}^{2/3}$$

Komplexer Schritt (ohne Auslöschung): $f'(x) = \frac{\operatorname{Im} f(x+\text{ß}h)}{h} + \mathcal{O}(h^2)$.

Differentialgleichungen

Euler explizit: $y_{k+1} = y_k + hf(t_k, y_k)$, Ordnung 1.
Euler implizit: $y_{k+1} = y_k + hf(t_{k+1}, y_{k+1})$, Ordnung 1, A-stabil.
Trapez: $y_{k+1} = y_k + \frac{h}{2}(f_k + f_{k+1})$, Ordnung 2, A-stabil (nicht L-stabil).

RK4:

$$k_1 = f(t_k, y_k), \quad k_2 = f(t_k + \tfrac{h}{2}, y_k + \tfrac{h}{2}k_1),$$
$$k_3 = f(t_k + \tfrac{h}{2}, y_k + \tfrac{h}{2}k_2), \quad k_4 = f(t_k + h, y_k + hk_3),$$
$$y_{k+1} = y_k + \tfrac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

Schrittweitensteuerung: $h_{\text{neu}} = h \cdot \rho\left(\frac{\text{tol}}{\varepsilon}\right)^{1/(p+1)}$, $\rho \approx 0{,}9$.

Stabilitätsgebiete (Testgleichung $y' = \lambda y$, $z = h\lambda$):

explizites Euler	$\lvert 1+z\rvert < 1$ – Kreis um -1
implizites Euler	$\lvert 1-z\rvert > 1$ – alles außer Kreis um $+1$
Trapez	$\operatorname{Re} z < 0$ – exakt die linke Halbebene

Dahlquist: kein explizites RK ist A-stabil; A-stabile Mehrschrittverfahren haben Ordnung ≤ 2.

Differenzenstern: $-u'' \to \frac{-u_{i-1}+2u_i-u_{i+1}}{h^2}$; Eigenwerte $\lambda_k = \frac{4}{h^2}\sin^2\frac{k\pi h}{2}$, cond $= \mathcal{O}(h^{-2})$.

Finite Elemente: $\int u'v' = \int fv$; Céa: $\|u - u_h\| \leq \frac{C}{\alpha}\inf_{v_h}\|u - v_h\|$; P_k-Elemente: $\mathcal{O}(h^{k+1})$ in L^2.

Optimierung und Monte-Carlo

Notwendig: $\nabla F = 0$; hinreichend: zusätzlich $\nabla^2 F$ positiv definit.

Armijo: $F(x + \alpha d) \leq F(x) + c_1\alpha\nabla F^{\mathsf{T}} d$, $c_1 \approx 10^{-4}$.

Krümmung: $\nabla F(x + \alpha d)^{\mathsf{T}} d \geq c_2 \nabla F^{\mathsf{T}} d$, $c_2 \approx 0{,}9$.

BFGS: $s_k = x_{k+1} - x_k$, $y_k = \nabla F_{k+1} - \nabla F_k$, $\rho_k = (y_k^{\mathsf{T}} s_k)^{-1}$,

$$H_{k+1} = (I - \rho_k s_k y_k^{\mathsf{T}}) H_k (I - \rho_k y_k s_k^{\mathsf{T}}) + \rho_k s_k s_k^{\mathsf{T}}$$

Monte-Carlo: $\hat{I}_N = \frac{1}{N}\sum f(X_i)$, Fehler $\frac{\sigma}{\sqrt{N}}$ – *unabhängig von der Dimension*.

Quasi-Monte-Carlo: $\mathcal{O}\left(N^{-1}(\log N)^d\right)$ mit Niedrigdiskrepanzfolgen.

B Verfahren im Überblick

Verfahren	Aufwand	Ordnung	Einsatz / Bemerkung
Lineare Systeme			
LR mit Pivotisierung	$\frac{2}{3}n^3$	exakt	Standard, allgemein
Cholesky	$\frac{1}{3}n^3$	exakt	s.p.d., kein Pivot nötig
QR (Householder)	$2mn^2$	exakt	Ausgleichsprobleme
Thomas	$\mathcal{O}(n)$	exakt	Tridiagonalmatrizen
Jacobi / Gauß–Seidel	$\mathcal{O}(n)$/Iter.	linear	Glätter im Mehrgitter
SOR	$\mathcal{O}(n)$/Iter.	linear	ω kritisch
CG	$\mathcal{O}(\text{nnz})$/Iter.	$\mathcal{O}(\sqrt{\kappa})$ Iter.	s.p.d., groß, dünn
GMRES	wachsend	optimal	unsymmetrisch
BiCGStab	$\mathcal{O}(\text{nnz})$/Iter.	–	unsymmetrisch, kurz
Mehrgitter	$\mathcal{O}(n)$	h-unabhängig	elliptische Probleme
Eigenwerte			
Potenzmethode	$\mathcal{O}(n^2)$/Iter.	linear	größter Eigenwert
Inverse Iteration	$\mathcal{O}(n^2)$/Iter.	linear/schnell	Eigenvektor zu bekanntem λ
Rayleigh-Quotient-It.	$\mathcal{O}(n^3)$/Iter.	kubisch	symmetrisch
QR-Algorithmus	$\approx 10n^3$	kubisch	alle Eigenwerte
Lanczos / Arnoldi	$\mathcal{O}(\text{nnz})$/Iter.	–	wenige Eigenwerte, groß
Nullstellen			
Bisektion	1 Ausw./Iter.	1	garantiert, langsam
Newton	2 Ausw./Iter.	2	lokal, braucht f'
Sekante	1 Ausw./Iter.	1,618	ohne f'
Brent	1 Ausw./Iter.	$\leq 1{,}84$	**Standard** in 1D
Newton–Krylow	matrixfrei	2	große Systeme
Interpolation und Quadratur			
Polynom, gleichabst.	$\mathcal{O}(n^2)$	–	Runge-Phänomen
Polynom, Tschebyschow	$\mathcal{O}(n)$/Ausw.	geometrisch	baryzentrisch, stabil
Kubischer Spline	$\mathcal{O}(n)$	$\mathcal{O}(h^4)$	lokal, keine Oszillation
Trapez	$\mathcal{O}(n)$	2	exponentiell bei periodisch
Simpson	$\mathcal{O}(n)$	4	Standard, einfach
Romberg	$\mathcal{O}(n)$	wachsend	Extrapolation, glatt
Gauß–Legendre	$\mathcal{O}(n)$	$2n+1$	optimal bei glatten f
Gauß–Kronrod	$\mathcal{O}(n)$	$3n+1$	mit Fehlerschätzung
Differentialgleichungen			

Verfahren	Aufwand	Ordnung	Einsatz / Bemerkung
Euler explizit	1 Ausw.	1	didaktisch
RK4	4 Ausw.	4	feste Schrittweite
Dormand–Prince	7 Ausw.	5(4)	**Standard** nichtsteif
BDF / Gear	implizit	1–6	steif, Mehrschritt
Radau IIA	implizit	5	steif, L-stabil
Rosenbrock	linear-implizit	3–4	steif, ohne Newton
Finite Differenzen	$\mathcal{O}(n)$ Aufbau	2	einfache Geometrie
Finite Elemente	$\mathcal{O}(n)$ Aufbau	$k+1$	beliebige Geometrie
Spektralverfahren	$\mathcal{O}(n \log n)$	exponentiell	glatt, einfaches Gebiet
Optimierung			
Gradientenabstieg	$\mathcal{O}(n)$	linear	einfach, langsam
Newton	$\mathcal{O}(n^3)$	quadratisch	braucht Hesse-Matrix
BFGS	$\mathcal{O}(n^2)$	superlinear	Standard, mittelgroß
L-BFGS	$\mathcal{O}(mn)$	superlinear	**Standard**, groß
SGD / Adam	$\mathcal{O}(\lvert B\rvert\, n)$	sublinear	maschinelles Lernen

C Zeitleiste der Numerischen Mathematik

Zeit	Ereignis
ca. 1800 v. Chr.	Tontafel YBC 7289: $\sqrt{2}$ auf sechs Dezimalstellen, Heron-Iteration
ca. 250 v. Chr.	Archimedes schließt π durch Vielecke ein: $\frac{223}{71} < \pi < \frac{22}{7}$
ca. 100	Heron von Alexandria beschreibt das Wurzelziehverfahren
ca. 250	Liu Hui: 3072-Eck, $\pi \approx 3{,}14159$
1050	Jia Xian: Pascalsches Dreieck in China
1669	Newton entwickelt sein Nullstellenverfahren (veröffentlicht 1711)
1690	Raphson gibt die heute übliche Form des Newton-Verfahrens an
1711	Newton: Interpolationsformeln für gleichabständige Stützstellen
1768	Euler: Polygonzugverfahren für Differentialgleichungen
1795/1805	Gauß und Legendre: Methode der kleinsten Quadrate
1805	Gauß erfindet die schnelle Fourier-Transformation (unveröffentlicht)
1809	Gauß: statistische Begründung der kleinsten Quadrate, Normalverteilung
1814	Gauß: Quadraturformeln mit optimalen Knoten
1823	Gauß empfiehlt ein iteratives Verfahren (später „Gauß–Seidel“) für Geodäsie
1835	Liouville: Es gibt Funktionen ohne elementare Stammfunktion
1854	Tschebyschow: beste gleichmäßige Approximation, Tschebyschow-Polynome
1874	Seidel veröffentlicht das nach ihm benannte Iterationsverfahren
1889	Aitken- und Neville-artige Schemata beginnen sich zu entwickeln
1895	Runge: Verfahren zur Integration von Differentialgleichungen
1901	Kutta: klassisches Verfahren vierter Ordnung; Runge: Runge-Phänomen
1910	Cholesky entwickelt seine Zerlegung für die Landesvermessung
1910	Richardson: Differenzenverfahren für partielle Differentialgleichungen
1911	Richardson-Extrapolation
1922	Richardson schlägt numerische Wettervorhersage vor (mit 64 000 Rechnern)
1924	Benoît veröffentlicht Choleskys Verfahren posthum
1928	Courant, Friedrichs und Lewy: CFL-Bedingung für Stabilität
1943	Courant beschreibt die Grundidee der finiten Elemente
1946	Ulam und von Neumann: Monte-Carlo-Methode in Los Alamos
1946	Schoenberg begründet die Theorie der Splines
1947	von Neumann und Goldstine: erste systematische Rundungsfehleranalyse
1948	Wilkinson: iterative Verfeinerung
1950	Lanczos: Verfahren für Eigenwerte großer Matrizen; Young: SOR-Theorie
1951	Robbins und Monro: stochastische Approximation (Grundlage von SGD)

Zeit	Ereignis
1952	Hestenes und Stiefel: Verfahren der konjugierten Gradienten
1952	Curtiss und Hirschfelder prägen den Begriff „steif“
1953	Metropolis u. a.: Markow-Ketten-Monte-Carlo
1955	Romberg: Extrapolationsverfahren zur Quadratur
1956	Dahlquist: erste Schranke für Mehrschrittverfahren
1959–61	Francis und Kublanovskaya: QR-Algorithmus
1961	Fedorenko: erste Beschreibung des Mehrgitterprinzips
1963	Wilkinson: *Rounding Errors*; zweite Dahlquist-Schranke; Lorenz entdeckt Chaos
1965	Cooley und Tukey: schnelle Fourier-Transformation
1965	Golub und Kahan: stabiler Algorithmus für die Singulärwertzerlegung
1965	Kahan: kompensierte Summation
1969	Strassen: Matrixmultiplikation in $\mathcal{O}(n^{2{,}807})$
1969	Golub und Welsch: Gauß-Knoten als Eigenwertproblem
1970	Broyden, Fletcher, Goldfarb und Shanno: BFGS-Formel
1970	Linnainmaa: Rückwärtsmodus des automatischen Differenzierens
1971	Gear: BDF-Verfahren für steife Systeme
1972	Cox und de Boor: stabile Rekursion für B-Splines
1973	Brent: robustes Nullstellenverfahren
1977	Brandt: Mehrgitterverfahren werden praxistauglich
1978	Babuška und Rheinboldt: A-posteriori-Fehlerschätzer, Adaptivität
1980	Dormand und Prince: eingebettetes Runge–Kutta-Paar
1984	Karmarkar: Innere-Punkte-Verfahren für lineare Optimierung
1985	**IEEE 754**: einheitlicher Gleitkommastandard
1986	Saad und Schultz: GMRES
1987	Ruge und Stüben: algebraisches Mehrgitter
1989	Kahan erhält den Turing-Preis
1991	Patriot-Zwischenfall in Dhahran: 28 Tote durch einen Rundungsfehler
1996	Nesterov und Nemirovski: Innere-Punkte-Verfahren für konvexe Optimierung
1999	Tucker: verifizierter Beweis zum Lorenz-Attraktor (Smales 14. Problem)
2000	*Computing in Science and Engineering* kürt die zehn Algorithmen des Jahrhunderts
2004	Chebfun: Funktionen als Objekte, Tschebyschow-Darstellung
2006	Sankar, Spielman und Teng: Glättungsanalyse der Gauß-Elimination
2011	Halko, Martinsson und Tropp: randomisierte Matrixzerlegungen
2015	Kingma und Ba: Adam-Optimierer
2017	Carson und Higham: Theorie der gemischten Präzision
2019	Raissi, Perdikaris und Karniadakis: physikinformierte neuronale Netze
2019	IEEE 754-2019; HPL-AI-Rangliste eingeführt
2020	Martinsson und Tropp: Übersicht zur randomisierten linearen Algebra
2021	Li u. a.: Fourier Neural Operator
2023	Datengetriebene Wettermodelle erreichen die Güte klassischer Vorhersage
2026	Stand dieses Buches

D Glossar

Maschinengenauigkeit ε_{M} Kleinste Zahl, für die der Rechner $1 + \varepsilon_{\mathrm{M}}$ von 1 unterscheidet. Für `double` etwa 10^{-16}.

Auslöschung Verlust signifikanter Stellen bei Subtraktion fast gleicher Zahlen. Nicht ein Fehler des Algorithmus, sondern Ausdruck schlechter Kondition der Subtraktion.

Kondition Empfindlichkeit des *Problems* gegen Störungen der Eingabedaten. Nicht durch einen besseren Algorithmus verbesserbar.

Stabilität Eigenschaft des *Algorithmus*: Er liefert ein Ergebnis, das so gut ist, wie es die Kondition erlaubt.

Rückwärtsfehler Wie stark müsste man die Eingabe stören, damit das berechnete Ergebnis exakt richtig wäre? Wilkinsons zentrale Frage.

Rückwärtsstabil Der Rückwärtsfehler ist von der Größenordnung ε_{M}. Mehr kann man von einem Algorithmus nicht verlangen.

Pivotisierung Zeilen- (und ggf. Spalten-)Tausch, um betragsgroße Diagonalelemente zu erhalten. Ohne sie ist die Gauß-Elimination instabil.

Wachstumsfaktor Maß dafür, wie stark Matrixeinträge während der Elimination anwachsen. Theoretisch bis 2^{n-1}, praktisch fast immer klein – ein offenes Problem.

Fill-in Neue Einträge ungleich null, die bei der Zerlegung dünn besetzter Matrizen an vorher leeren Stellen entstehen.

Orthogonale Matrix $Q^{\mathsf{T}}Q = I$; längentreu, $\operatorname{cond}_2 = 1$. Deshalb Grundbaustein aller stabilen Verfahren der linearen Algebra.

Singulärwerte Die Halbachsen des Bildellipsoids der Einheitskugel; $\|A\|_2 = \sigma_{\max}$, $\operatorname{cond}_2 = \sigma_{\max}/\sigma_{\min}$.

Pseudoinverse $A^{+} = V\Sigma^{+}U^{\mathsf{T}}$; liefert die Kleinste-Quadrate-Lösung minimaler Norm, auch bei Rangdefekt.

Krylow-Raum $\operatorname{span}\{r_0, Ar_0, \ldots, A^{k-1}r_0\}$. Suchraum aller modernen iterativen Verfahren.

Vorkonditionierung Ersetzen von $Ax = b$ durch $M^{-1}Ax = M^{-1}b$ mit $M \approx A$ leicht invertierbar. Entscheidet in der Praxis meist mehr als die Wahl des Iterationsverfahrens.

Spektralradius $\rho(T) = \max_i |\lambda_i|$. Ein Splitting-Verfahren konvergiert genau dann, wenn $\rho < 1$.

Konvergenzordnung Exponent p in $e_{k+1} \approx C e_k^{p}$. Bei $p = 2$ verdoppelt sich die Stellenzahl pro Schritt.

Runge-Phänomen Divergenz der Polynominterpolation bei gleichabständigen Stützstellen. Behoben durch Tschebyschow-Knoten.

Lebesgue-Konstante Verstärkungsfaktor der Interpolation für Datenfehler. Wächst gleichabständig exponentiell, bei Tschebyschow-Knoten nur logarithmisch.

Spline Stückweises Polynom mit vorgeschriebener Glattheit. Der natürliche kubische Spline minimiert $\int (s'')^2$ – die Biegeenergie.

Diskretisierungsfehler Fehler durch den Übergang vom kontinuierlichen zum diskreten Problem; getrennt vom Rundungsfehler zu betrachten.

Konsistenz Der Fehler eines einzelnen Verfahrensschrittes geht schneller als h gegen null.

Steifheit Ein System enthält stark verschiedene Zeitskalen; explizite Verfahren erzwingen dann winzige Schrittweiten aus Stabilitäts-, nicht aus Genauigkeitsgründen.

A-Stabilität Das Stabilitätsgebiet umfasst die gesamte linke Halbebene. Kein explizites Runge–Kutta-Verfahren erreicht sie.

L-Stabilität A-Stabilität plus vollständige Dämpfung sehr schneller Moden. Wichtig bei parabolischen Problemen.

Glätter Iterationsverfahren, das im Mehrgitterkontext nur hochfrequente Fehleranteile beseitigen soll.

Schwache Formulierung Umformulierung einer Differentialgleichung durch Multiplikation mit Testfunktionen und partielle Integration; senkt die Glattheitsanforderungen an die Lösung.

Steifigkeitsmatrix Die Galerkin-Matrix $K_{ij} = \int \varphi_i' \varphi_j'$; der Name stammt aus der Strukturmechanik.

Fluch der Dimension Exponentielles Wachstum des Aufwands von Gitterverfahren mit der Dimension. Monte-Carlo umgeht ihn.

Diskrepanz Maß für die Ungleichmäßigkeit einer Punktfolge; Niedrigdiskrepanzfolgen verbessern die Monte-Carlo-Rate.

Automatisches Differenzieren Exakte Ableitungsberechnung durch Anwendung der Kettenregel auf die elementaren Rechenschritte. Rückwärtsmodus = Backpropagation.

Gemischte Präzision Verwendung verschiedener Zahlenformate in verschiedenen Teilen einer Rechnung, um Geschwindigkeit und Genauigkeit zu optimieren.

Intervallarithmetik Rechnen mit garantierten Einschließungen statt mit Näherungen; Grundlage der verifizierten Numerik.

E Kommentierte Literatur

[1] Björck, Å.: *Numerical Methods for Least Squares Problems.* 2. Auflage, SIAM, Philadelphia 2024.

Das umfassendste Werk zur Ausgleichsrechnung. Behandelt QR mit Spaltenpivotisierung, rangdefiziente Probleme, Regularisierung und Total Least Squares. Björck war Stiefels und Dahlquists Schüler; die Darstellung ist entsprechend gründlich.

[2] Bornemann, F., Laurie, D., Wagon, S., Waldvogel, J.: *Vom Lösen numerischer Probleme.* Springer, Berlin 2006.

Zehn Aufgaben aus Trefethens „100-Digit Challenge“, jeweils auf zehn Stellen genau gelöst – mit ausführlicher Diskussion der Verfahren, Fehleranalysen und Alternativen. Das lehrreichste Buch über numerische *Praxis*, das ich kenne.

[3] Boyd, S., Vandenberghe, L.: *Convex Optimization.* Cambridge University Press 2004.

Das Standardwerk der konvexen Optimierung, frei verfügbar. Erklärt Dualität, Innere-Punkte-Verfahren und vor allem: wie man erkennt, ob ein Problem konvex ist. Hervorragend geschrieben und voller Anwendungsbeispiele.

[4] Braess, D.: *Finite Elemente.* 5. Auflage, Springer Spektrum, Berlin 2013.

Das deutschsprachige Standardwerk zur Finite-Elemente-Methode. Führt Sobolewräume, Céa-Lemma und A-posteriori-Schätzer knapp und präzise ein; enthält auch Mehrgitterverfahren und Sattelpunktprobleme.

[5] Dahmen, W., Reusken, A.: *Numerik für Ingenieure und Naturwissenschaftler.* 2. Auflage, Springer, Berlin 2008.

Eine gut lesbare deutschsprachige Einführung mit Schwerpunkt auf Verständlichkeit und Anwendungsbezug. Behandelt Kondition und Stabilität besonders sorgfältig – der natürliche nächste Schritt nach diesem Band.

[6] Deuflhard, P., Bornemann, F.: *Numerische Mathematik 2: Gewöhnliche Differentialgleichungen.* 4. Auflage, de Gruyter, Berlin 2013.

Deutschsprachige Darstellung der Anfangs- und Randwertprobleme mit starkem Gewicht auf Kondition, adaptiver Steuerung und affiner Invarianz. Enthält das Mehrfachschießverfahren ausführlich.

[7] Deuflhard, P., Hohmann, A.: *Numerische Mathematik 1: Eine algorithmisch orientierte Einführung.* 5. Auflage, de Gruyter, Berlin 2019.

Konsequent von der Kondition her aufgebaut: Erst wird gefragt, was das Problem zulässt, dann wird der Algorithmus konstruiert. Deuflhards affin-invariante Newton-Theorie ist hier zugänglich dargestellt.

[8] Golub, G. H., Van Loan, C. F.: *Matrix Computations.* 4. Auflage, Johns Hopkins University Press, Baltimore 2013.

Die Bibel der numerischen linearen Algebra. Vollständig, präzise, mit Aufwandsangaben für jeden Algorithmus. Kein Lehrbuch zum Durchlesen, sondern ein Nachschlagewerk, das man jahrzehntelang benutzt.

[9] Griewank, A., Walther, A.: *Evaluating Derivatives.* 2. Auflage, SIAM, Philadelphia 2008.

Das maßgebliche Werk zum automatischen Differenzieren. Behandelt Vorwärts- und Rückwärtsmodus, Komplexitätsschranken und Checkpointing-Strategien. Wer verstehen will, was Backpropagation mathematisch ist, findet es hier – Jahrzehnte vor dem Deep-Learning-Boom geschrieben.

[10] Hackbusch, W.: *Multi-Grid Methods and Applications.* Springer, Berlin 1985.

Die vollständige Theorie der Mehrgitterverfahren. Anspruchsvoll, aber die Referenz. Hackbuschs deutschsprachiges *Iterative Lösung großer schwachbesetzter Gleichungssysteme* (2. Auflage 1993) ist der zugänglichere Einstieg.

[11] Hairer, E., Nørsett, S. P., Wanner, G.: *Solving Ordinary Differential Equations I: Nonstiff Problems.* 2. Auflage, Springer, Berlin 1993.

Ungewöhnlich lesbares Fachbuch: Die Autoren erzählen die Geschichte jedes Verfahrens mit, zitieren Originalquellen und zeigen viele numerische Experimente. Enthält Butchers Baumtheorie vollständig.

[12] Hairer, E., Wanner, G.: *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems.* 2. Auflage, Springer, Berlin 1996.

Der Standardband zur Steifheit. Stabilitätsbegriffe, implizite Runge–Kutta-Verfahren, BDF, differential-algebraische Systeme. Der beigefügte Code (RADAU5, RODAS) ist bis heute in Gebrauch.

[13] Halko, N., Martinsson, P.-G., Tropp, J. A.: *Finding Structure with Randomness.* SIAM Review 53 (2011), 217–288.

Der Aufsatz, der die randomisierte lineare Algebra bekannt machte. Enthält Algorithmen, vollständige Fehleranalysen und praktische Empfehlungen. Bemerkenswert klar geschrieben für eine so technische Arbeit.

[14] Hennig, P., Osborne, M. A., Kersting, H.: *Probabilistic Numerics.* Cambridge University Press 2022.

Die erste Buchdarstellung der probabilistischen Numerik. Deutet Quadratur, Differentialgleichungslöser und lineare Solver als Inferenzverfahren. Ein methodisch ungewöhnlicher, philosophisch interessanter Zugang.

[15] Higham, N. J.: *Accuracy and Stability of Numerical Algorithms.* 2. Auflage, SIAM, Philadelphia 2002.

Das maßgebliche Werk zur Fehleranalyse – über 700 Seiten Rückwärtsanalyse für praktisch

jeden Algorithmus der linearen Algebra. Enthält ausführliche historische Anmerkungen und die beste Diskussion des Wachstumsfaktor-Rätsels.

[16] Higham, N. J., Mary, T.: *Mixed Precision Algorithms in Numerical Linear Algebra.* Acta Numerica 31 (2022), 347–414.

Die Übersichtsarbeit zur gemischten Präzision. Behandelt iterative Verfeinerung mit drei Präzisionen, GMRES-IR und die Konsequenzen für Zerlegungen. Der beste Einstieg in ein Gebiet, das sich schnell entwickelt.

[17] IEEE Standard for Floating-Point Arithmetic. IEEE Std 754-2019.

Der Standard selbst. Sperrig zu lesen, aber die verbindliche Quelle. Zugänglicher ist Goldbergs Aufsatz *What Every Computer Scientist Should Know About Floating-Point Arithmetic* (ACM Computing Surveys 1991), frei verfügbar und zu Recht berühmt.

[18] Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., Yang, L.: *Physics-Informed Machine Learning.* Nature Reviews Physics 3 (2021), 422–440.

Die meistzitierte Übersicht zum wissenschaftlichen maschinellen Lernen. Optimistisch im Ton; für die kritische Gegenposition sollte man die seit 2022 erschienenen systematischen Vergleichsstudien hinzuziehen.

[19] Kelley, C. T.: *Iterative Methods for Linear and Nonlinear Equations.* SIAM, Philadelphia 1995.

Kompakt, praxisnah, mit MATLAB-Code. Behandelt Krylow-Verfahren, Newton–Krylow und Broyden-Verfahren in etwa 160 Seiten. Frei verfügbar.

[20] Martinsson, P.-G., Tropp, J. A.: *Randomized Numerical Linear Algebra: Foundations and Algorithms.* Acta Numerica 29 (2020), 403–572.

Die Referenzübersicht des Gebiets. Führt die probabilistischen Werkzeuge – Matrix-Konzentrationsungleichungen – vollständig ein und behandelt Skizzierung, randomisierte Zerlegungen und Vorkonditionierung.

[21] Nocedal, J., Wright, S. J.: *Numerical Optimization.* 2. Auflage, Springer, New York 2006.

Das Standardwerk der nichtlinearen Optimierung. Schrittweitensuche, Vertrauensbereichsverfahren, Quasi-Newton, SQP, Innere-Punkte-Methoden. Nocedal ist der Autor von L-BFGS; die Darstellung ist entsprechend maßgeblich.

[22] Quarteroni, A., Sacco, R., Saleri, F.: *Numerische Mathematik.* 2 Bände, Springer, Berlin 2002.

Eine breit angelegte Einführung mit vielen ausgearbeiteten Beispielen und MATLAB-Programmen. Gut geeignet zum Nachschlagen einzelner Verfahren.

[23] Rump, S. M.: *Verification Methods: Rigorous Results Using Floating-Point Arithmetic.* Acta Numerica 19 (2010), 287–449.

Die Übersicht zur verifizierten Numerik. Zeigt, wie man mit gewöhnlicher Gleitkom-

maarithmetik mathematisch garantierte Einschließungen erzeugt – oft mit erstaunlich geringem Mehraufwand. Rump ist der Autor von INTLAB.

[24] Saad, Y.: *Iterative Methods for Sparse Linear Systems.* 2. Auflage, SIAM, Philadelphia 2003.

Das Standardwerk zu Krylow-Verfahren und Vorkonditionierung. Saad ist der Miterfinder von GMRES. Frei verfügbar; enthält alles über ILU-Varianten, Gebietszerlegung und parallele Implementierung.

[25] Stoer, J., Bulirsch, R.: *Numerische Mathematik.* 2 Bände, 10. bzw. 5. Auflage, Springer, Berlin 2007/2005.

Der deutschsprachige Klassiker, seit den 1970er Jahren in Gebrauch. Sehr präzise, teils knapp geschrieben. Band 1 deckt Interpolation, Quadratur und lineare Systeme ab, Band 2 Eigenwerte, Differentialgleichungen und Optimierung.

[26] Trefethen, L. N.: *Approximation Theory and Approximation Practice.* Erweiterte Auflage, SIAM, Philadelphia 2019.

Ein ungewöhnliches Buch: Jedes Kapitel ist ein ausführbares Chebfun-Programm. Zeigt überzeugend, dass Polynominterpolation hohen Grades bestens funktioniert, wenn man Tschebyschow-Knoten und die baryzentrische Formel verwendet. Sehr empfehlenswert.

[27] Trefethen, L. N., Bau, D.: *Numerical Linear Algebra.* SIAM, Philadelphia 1997.

Die beste Einführung in die numerische lineare Algebra – in vierzig kurzen Vorlesungen, mit der SVD als Ausgangspunkt statt am Ende. Die Kapitel über Kondition und Stabilität sind vorbildlich; wer nur ein Buch dieses Gebiets liest, sollte dieses wählen.

[28] Wilkinson, J. H.: *The Algebraic Eigenvalue Problem.* Clarendon Press, Oxford 1965.

Das Werk, das die Fehleranalyse begründete. Sechzig Jahre alt und in der Sache nicht überholt. Wilkinsons *Rounding Errors in Algebraic Processes* (1963) ist die kürzere, allgemeinere Schwesterschrift.

Wenn Sie nur drei Bücher kaufen

Trefethen und Bau – um zu sehen, wie klar dieses Fach sein kann.
Deuflhard und Hohmann – um es auf Deutsch und systematisch zu lernen.
Higham – um zu verstehen, warum die Ergebnisse trotzdem stimmen.